Technical Reference Manual for PentaWoundPt2mm

Revised October 13th, 2026

Build unversioned

Contents

List of Figures **17**

List of Tables **19**

I System **23**

1 About this manual **24**
1.1 Scope and audience . . . 24
1.2 Organisation of this manual . . . 24
1.3 Related documents . . . 24
1.4 Conventions . . . 25
1.5 Register access types . . . 26
1.6 The anatomy of a register description . . . 26
1.7 Callouts . . . 27
1.8 Glossary of terms and abbreviations . . . 28

2 Overview **30**
2.1 Features . . . 30
2.2 Block diagram . . . 31
2.3 Chip configuration . . . 32

3 Package and pins **37**
3.1 GPIO pins and alternate functions . . . 42

4 Memory map **45**

5 Multi-core architecture **48**
5.1 Harts and Roles . . . 48
5.2 The Shared Window . . . 49
5.2.1 Arbitration and Stalling . . . 50
5.3 Boot and Tile Loading . . . 52
5.3.1 Harvested (field-powered) boot . . . 54
5.4 Synchronization . . . 55

6 Reset, clocks and power **57**
6.1 Reset . . . 57
6.2 Boot mode selection . . . 57
6.3 Clock tree . . . 57
6.3.1 Clock sources . . . 57
6.3.2 Roots and dividers . . . 58
6.3.3 Clock state at reset and after boot . . . 59
6.4 Power domains . . . 59
6.4.1 Sequencing . . . 60

6.4.2 Cold gating . . . 60
6.4.3 Memory block power . . . 60
6.4.4 Hart sleep . . . 60
6.4.5 Boot gate and field power . . . 60

II Processor 62

7 VestaRV core 63
7.1 Instruction set . . . 63
7.2 Registers and calling convention . . . 63
7.3 Hart identities . . . 64
7.4 Control and status registers . . . 64
7.4.1 Unimplemented CSRs and illegal accesses . . . 65

8 Privileged Architecture 67
8.1 Machine Information Registers . . . 67
8.2 The Legacy Trap Mechanism . . . 68
8.3 Selecting a Delivery Mode: `mtrapctl` . . . 68
8.4 The Machine Trap Registers . . . 69
8.5 Trap Entry, Trap Return, and Causes . . . 70
8.6 Waiting for an Interrupt . . . 71

9 Interrupts and exceptions 72
9.1 Exceptions . . . 78
9.2 Interrupt vector table . . . 78
9.3 Interrupt entry, exit and recursion . . . 78
9.4 Sleep mode and interrupt wake-up . . . 79

10 Core performance and instruction timing 80
10.1 Instruction timing . . . 80
10.2 Compressed instructions and fetch alignment . . . 81
10.3 Measured CPI . . . 82
10.4 Measuring application code . . . 82
10.5 Measurement methodology . . . 83

11 Debug 84
11.1 Overview . . . 84
11.2 Block diagram . . . 84
11.3 Functional description . . . 84
11.3.1 JTAG . . . 84
11.3.2 The RISC-V debug stack . . . 87
11.3.3 Debug port . . . 87
11.3.4 Debug Transport Module . . . 88
11.3.5 Debug Module . . . 89
11.3.6 Debug mode in the hart . . . 92
11.3.7 Halt groups and unavailable harts . . . 93
11.3.8 Debug program page . . . 94
11.3.9 DMI register map, field by field . . . 94

11.3.10 DMDATA0: Abstract data register 0, datacount = 1 . . . 95
11.3.11 DMCONTROL: Debug Module control . . . 95
11.3.12 DMSTATUS: Debug Module status, read only . . . 96
11.3.13 DMHARTINFO: Hart information, read only . . . 97
11.3.14 DMHALTSUM1: Halt summary 1, read only . . . 97
11.3.15 DMABSTRACTCS: Abstract control and status . . . 97
11.3.16 DMCOMMAND: Abstract command . . . 98
11.3.17 DMABSTRACTAUTO: Abstract command autoexec . . . 98
11.3.18 DMPROGBUF0: Program buffer word 0 . . . 99
11.3.19 DMPROGBUF1: Program buffer word 1 . . . 99
11.3.20 DMCS2: Halt-group assignment of the selected hart . . . 99
11.3.21 DMSBCS: System Bus Access control and status . . . 99
11.3.22 DMHALTSUM0: Halt summary 0, read only . . . 100
11.4 Interrupts and events . . . 100
11.5 Programming . . . 100
11.5.1 Halt, read a register, resume . . . 100
11.5.2 Connecting a debugger . . . 102
11.6 Usage notes . . . 102

III Peripherals 104

12 General-purpose input and output (GPIOx) 105
12.1 Overview . . . 105
12.2 Instances and signals . . . 105
12.3 Functional description . . . 110
12.3.1 GPIO mode . . . 110
12.3.2 Set, clear and toggle registers . . . 110
12.3.3 Alternate-function mode . . . 111
12.3.4 Relocatable peripheral inputs . . . 111
12.3.5 Pin interrupts . . . 111
12.3.6 Event-fabric tasks . . . 111
12.4 Interrupts and events . . . 112
12.5 Programming . . . 112
12.5.1 Configuring a pin as an output . . . 112
12.5.2 Routing a peripheral output to an alternate pin . . . 112
12.5.3 Enabling a pin interrupt . . . 112
12.6 Usage notes . . . 113
12.7 Registers . . . 113
12.7.1 PxIN: GPIO read pin register . . . 113
12.7.2 PxOUT: GPIO output drive register . . . 114
12.7.3 PxOUTS: GPIO output drive set register . . . 114
12.7.4 PxOUTC: GPIO output drive clear register . . . 114
12.7.5 PxOUTT: GPIO output drive toggle register . . . 114
12.7.6 PxDIR: GPIO pin direction register . . . 115
12.7.7 PxIF: GPIO interrupt flag register . . . 115
12.7.8 PxIES: GPIO interrupt edge select register . . . 115
12.7.9 PxIE: GPIO interrupt enable register . . . 116

12.7.10 PxSEL: GPIO peripheral select register . . . 116
12.7.11 PxREN: GPIO resistor enable register . . . 116
12.7.12 PxAFS: GPIO alternate function select register . . . 116
12.7.13 PxTASK: GPIO event-fabric task pin-select register . . . 117

13 Serial peripheral interface (SPIx) 118
13.1 Overview . . . 118
13.2 Instances and signals . . . 118
13.3 Block diagram . . . 119
13.4 Functional description . . . 120
13.4.1 Transfer . . . 120
13.4.2 Bit and byte order . . . 120
13.4.3 Baud clock . . . 122
13.4.4 Transmit queue . . . 122
13.4.5 Status flags . . . 122
13.4.6 Flash extended memory . . . 123
13.4.7 Boot use of SPI0 . . . 123
13.5 Interrupts and events . . . 123
13.6 Programming . . . 123
13.6.1 Initialisation as a master . . . 123
13.6.2 Master transfer of one packet . . . 124
13.6.3 Initialisation as a slave (SPI1) . . . 124
13.6.4 Enabling the flash window . . . 124
13.7 Usage notes . . . 125
13.8 Registers . . . 125
13.8.1 SPIxCR: SPI control register . . . 126
13.8.2 SPIxSR: SPI status register . . . 127
13.8.3 SPIxTX: SPI transmit buffer register . . . 128
13.8.4 SPIxRX: SPI receive buffer register . . . 128
13.8.5 SPIxFOS: SPI Flash memory address offset . . . 128

14 Universal asynchronous receiver and transmitter (UARTx) 129
14.1 Overview . . . 129
14.2 Instances and signals . . . 129
14.3 Block diagram . . . 130
14.4 Functional description . . . 131
14.4.1 Frame . . . 131
14.4.2 Baud generation . . . 131
14.4.3 Parity . . . 131
14.4.4 Transmit queue . . . 131
14.4.5 Reception and status . . . 131
14.5 Interrupts and events . . . 132
14.6 Programming . . . 132
14.6.1 Initialisation . . . 132
14.6.2 Transmitting a byte . . . 132
14.6.3 Receiving a byte . . . 132
14.7 Usage notes . . . 132
14.8 Registers . . . 133

14.8.1 UARTxCR: UART control register . . . 133
14.8.2 UARTxSR: UART status register . . . 134
14.8.3 UARTxBR: UART baud rate register . . . 135
14.8.4 UARTxRX: UART receive buffer register . . . 135
14.8.5 UARTxTX: UART transmit buffer register . . . 135

15 Timer (TIMERx) 136
15.1 Overview . . . 136
15.2 Instances and signals . . . 136
15.3 Block diagram . . . 138
15.4 Functional description . . . 138
15.4.1 Clock source and divider . . . 138
15.4.2 Counting . . . 138
15.4.3 Overflow and rollover . . . 138
15.4.4 Output compare and PWM . . . 139
15.4.5 Input capture . . . 139
15.4.6 Status register . . . 139
15.5 Interrupts and events . . . 140
15.6 Programming . . . 140
15.6.1 Initialisation as a PWM generator . . . 140
15.6.2 Initialisation for input capture . . . 141
15.7 Usage notes . . . 141
15.8 Registers . . . 141
15.8.1 TIMxCR: Timer/Counter control register . . . 142
15.8.2 TIMxSR: Timer/Counter status register . . . 144
15.8.3 TIMxVAL: Timer/Counter value register . . . 145
15.8.4 TIMxCMP0: Timer/Counter Compare 0 threshold register . . . 145
15.8.5 TIMxCMP1: Timer/Counter Compare 1 threshold register . . . 145
15.8.6 TIMxCMP2: Timer/Counter Compare 2 threshold register . . . 145
15.8.7 TIMxCAPn (n = 0..1): Timer/Counter Capture n value register . . . 145

16 System control (SYSTEM) 147
16.1 Overview . . . 147
16.2 Instances and signals . . . 147
16.3 Functional description . . . 147
16.3.1 Clock control . . . 147
16.3.2 Memory block power . . . 148
16.3.3 Hart sleep . . . 148
16.3.4 CRC accumulator . . . 148
16.3.5 Watchdog . . . 148
16.4 Interrupts and events . . . 149
16.5 Programming . . . 149
16.5.1 Selecting the clock sources . . . 149
16.5.2 Computing a CRC . . . 149
16.5.3 Starting the watchdog . . . 149
16.5.4 Putting a hart to sleep . . . 149
16.6 Usage notes . . . 150
16.7 Registers . . . 150

16.7.1 SYSCLKCR: System clock control register . . . 150
16.7.2 CLKDIVCR: MCLK and SMCLK clock divider control register . . . 151
16.7.3 BLOCKPWR: Block power control register . . . 152
16.7.4 CRCDATA: CRC input data register . . . 153
16.7.5 CRCSTATE: CRC state register . . . 153
16.7.6 WDTPASS: Watchdog timer password register . . . 154
16.7.7 WDTCR: Watchdog timer control register . . . 154
16.7.8 WDTSR: Watchdog timer status register . . . 155
16.7.9 WDTVAL: Watchdog timer value register . . . 156
16.7.10 DCOnBIAS (n = 0..1): Digitally controlled oscillator n bias register . . . 156

17 Neural processing unit (NPU) 157
17.1 Overview . . . 157
17.2 Instances and signals . . . 157
17.3 Block diagram . . . 157
17.4 Functional description . . . 158
17.4.1 Staging RAM ownership . . . 158
17.4.2 Data formats . . . 158
17.4.3 Packed operand formats . . . 159
17.4.4 Datapath modes . . . 159
17.4.5 Activation functions . . . 159
17.4.6 Run time . . . 160
17.5 Interrupts and events . . . 160
17.6 Programming . . . 160
17.6.1 Running one computation . . . 160
17.7 Usage notes . . . 161
17.8 Registers . . . 161
17.8.1 NPUCR: NPU control register . . . 161
17.8.2 NPUIVSAR . . . 163
17.8.3 NPUWVSAR . . . 163
17.8.4 NPUOVSAR . . . 164
17.8.5 NPUSR: NPU status register . . . 164
17.8.6 NPUCFG1: NPU mode configuration word 1 . . . 164
17.8.7 NPUCFG2: NPU mode configuration word 2 . . . 165

18 Power control (PWRCTRL) 166
18.1 Overview . . . 166
18.2 Instances and signals . . . 166
18.3 Functional description . . . 166
18.3.1 Gate sequencer . . . 166
18.3.2 Task wake . . . 167
18.3.3 Boot gate . . . 167
18.4 Interrupts and events . . . 167
18.5 Programming . . . 168
18.5.1 Gating tile h . . . 168
18.5.2 Waking tile h . . . 168
18.5.3 Arming the boot gate from software . . . 168
18.5.4 Supercap duty-cycled service loop . . . 168

18.6 Usage notes . . . 168
18.7 Registers . . . 169
18.7.1 PWRCR: Power gate control . . . 169
18.7.2 PWRSR: Power sequencer state, one read-only nibble per hart (bits 4h+3:4h) . . . 169
18.7.3 PWRWAKE: Boot gate and wake source control . . . 170
18.7.4 PWRSTS . . . 170
18.7.5 TASKWKM: Event-fabric task-wake mask, one bit per gateable tile hart . . . 171

19 Quad serial peripheral interface (QSPIx) 172
19.1 Overview . . . 172
19.2 Instances and signals . . . 172
19.3 Functional description . . . 173
19.3.1 QSPI Pins . . . 173
19.3.2 Transaction Structure . . . 173
19.4 Interrupts and events . . . 173
19.5 Programming . . . 173
19.6 Usage notes . . . 174
19.7 Registers . . . 174
19.7.1 QSPIxCR: QSPI control register . . . 174
19.7.2 QSPIxCMD: QSPI command register . . . 175
19.7.3 QSPIxADR: QSPI transaction address . . . 176
19.7.4 QSPIxTX: QSPI transmit data . . . 176
19.7.5 QSPIxRX: QSPI receive data . . . 176
19.7.6 QSPIxSR: QSPI status register . . . 176

20 Inter-integrated circuit interface (I2Cx) 178
20.1 Overview . . . 178
20.2 Instances and signals . . . 178
20.3 Functional description . . . 180
20.3.1 Wire protocol . . . 180
20.3.2 Master operation . . . 181
20.3.3 Slave operation . . . 181
20.3.4 Clocking . . . 181
20.4 Interrupts and events . . . 181
20.5 Programming . . . 181
20.5.1 Initialisation as a master . . . 181
20.5.2 Initialisation as a slave . . . 182
20.5.3 Master write of one byte . . . 182
20.6 Usage notes . . . 183
20.7 Registers . . . 183
20.7.1 I2CxCR: I2C control register . . . 183
20.7.2 I2CxFCR: I2C flow control register . . . 185
20.7.3 I2CxSR: I2C status register . . . 186
20.7.4 I2CxMTX: I2C master transmit register . . . 188
20.7.5 I2CxMRX: I2C master receive register . . . 188
20.7.6 I2CxSTX: I2C slave transmit register . . . 189
20.7.7 I2CxSRX: I2C slave receive register . . . 189
20.7.8 I2CxAR: I2C this slave address register . . . 189

20.7.9 I2CxAMR: I2C this slave address mask register . . . 189

21 Core-local interruptor (CLINT) 191
21.1 Overview . . . 191
21.2 Instances and signals . . . 191
21.3 Functional description . . . 191
21.3.1 Software interrupts . . . 191
21.3.2 Timer interrupts . . . 192
21.3.3 Clocking . . . 192
21.3.4 Address decoding . . . 192
21.4 Interrupts and events . . . 192
21.5 Programming . . . 192
21.5.1 Reading the counter . . . 192
21.5.2 Arming a timer interrupt for hart *h* . . . 193
21.5.3 Sending a software interrupt to hart *h* . . . 193
21.6 Usage notes . . . 193
21.7 Registers . . . 193
21.7.1 MSIPn (n = 0..4): Hart n software interrupt (IPI) register . . . 194
21.7.2 MTIMEL: Machine time counter, lower 32 bits . . . 194
21.7.3 MTIMEH: Machine time counter, upper 32 bits . . . 194
21.7.4 MTIMECMPnL (n = 0..4): Hart n timer compare register, lower 32 bits . . . 194
21.7.5 MTIMECMPnH (n = 0..4): Hart n timer compare register, upper 32 bits . . . 194

22 Hardware mutexes (MUTEX) 195
22.1 Overview . . . 195
22.2 Instances and signals . . . 195
22.3 Functional description . . . 195
22.3.1 Claim and release . . . 195
22.3.2 Address decoding . . . 196
22.4 Interrupts and events . . . 196
22.5 Programming . . . 196
22.5.1 Claiming mutex *n* . . . 196
22.5.2 Releasing mutex *n* . . . 196
22.6 Usage notes . . . 197
22.7 Registers . . . 197
22.7.1 MUTEXn (n = 0..15): Hardware mutex n . . . 197

23 Improved inter-integrated circuit interface (I3Cx) 198
23.1 Overview . . . 198
23.2 Instances and signals . . . 198
23.3 Functional description . . . 199
23.3.1 I3C Pins . . . 199
23.3.2 Dynamic Address Assignment and In-Band Interrupts . . . 199
23.4 Interrupts and events . . . 200
23.5 Programming . . . 200
23.6 Usage notes . . . 200
23.7 Registers . . . 200
23.7.1 I3CxCR: I3C control register . . . 201

23.7.2 I3CxCMD: I3C command register . . . 202
23.7.3 I3CxTX: I3C transmit data . . . 203
23.7.4 I3CxRX: I3C receive data . . . 203
23.7.5 I3CxSR: I3C status register . . . 203
23.7.6 I3CxDAT: Device Address Table window . . . 205
23.7.7 I3CxDATPID: Provisional ID (low 32 bits) of the DAT entry selected by I3CIDX . . . 205
23.7.8 I3CxDATINFO . . . 205
23.7.9 I3CxIBI: In-band interrupt capture . . . 206

24 Near-field communication interface (NFCx) 207
24.1 Overview . . . 207
24.2 Instances and signals . . . 207
24.3 Functional description . . . 208
24.3.1 Analog front-end boundary . . . 208
24.3.2 Transaction engine . . . 208
24.3.3 Indexed windows . . . 208
24.3.4 Protocol timing . . . 208
24.4 Interrupts and events . . . 209
24.5 Programming . . . 209
24.5.1 Provisioning and arming the tag . . . 209
24.5.2 Servicing a frame delivered to firmware . . . 209
24.6 Usage notes . . . 209
24.7 Registers . . . 210
24.7.1 NFCxCR: NFC control register . . . 210
24.7.2 NFCxSR: NFC status register . . . 211
24.7.3 NFCxUID: Provisioned single-size 4-byte tag UID, used by bit-frame anticollision . . . 212
24.7.4 NFCxCFG . . . 213
24.7.5 NFCxTIM . . . 213
24.7.6 NFCxRXST . . . 213
24.7.7 NFCxIDX . . . 214
24.7.8 NFCxDATA: The byte at NFCIDX in the window selected by NFCIDXSEL . . . 214
24.7.9 NFCxTXCTL . . . 214
24.7.10 NFCxDBG: Debug telemetry / bench cross-checks: frame counters . . . 215

25 Real-time clock (RTCx) 216
25.1 Overview . . . 216
25.2 Instances and signals . . . 216
25.3 Functional description . . . 216
25.3.1 Always-On Clock Domain . . . 216
25.3.2 Reading the Time Coherently . . . 217
25.4 Interrupts and events . . . 217
25.5 Programming . . . 217
25.6 Usage notes . . . 217
25.7 Registers . . . 218
25.7.1 RTCxCR: RTC control register . . . 218
25.7.2 RTCxSEC: Wall-clock seconds . . . 219
25.7.3 RTCxSUB: Subsecond prescaler, 0 to 32767 (2^15 - 1) . . . 219
25.7.4 RTCxALM: Alarm compare value, in seconds . . . 219

25.7.5 RTCxPER . . . 219
25.7.6 RTCxSR: RTC status register . . . 220
25.7.7 RTCxTRIM: Reserved for digital fractional-prescaler / ppm calibration (deferred) . . . 220

26 Pulse-width modulator (PWMx) 221
26.1 Overview . . . 221
26.2 Instances and signals . . . 221
26.3 Functional description . . . 222
26.3.1 Buffered, Glitch-Free Update . . . 222
26.3.2 Polarity and the Safe Level . . . 222
26.3.3 The Software Fault . . . 222
26.4 Interrupts and events . . . 222
26.5 Programming . . . 223
26.6 Usage notes . . . 223
26.7 Registers . . . 223
26.7.1 PWMxCR: PWM control register . . . 223
26.7.2 PWMxPER: PWM period modulus (buffered, D9) . . . 224
26.7.3 PWMxDTY0: Channel 0 duty compare (buffered, D9) . . . 225
26.7.4 PWMxDTY1: Channel 1 duty compare (buffered, D9) . . . 225
26.7.5 PWMxDTY2: Reserved for the channel 2 duty compare (4-channel bolt-on, D16) . . . 225
26.7.6 PWMxDTY3: Reserved for the channel 3 duty compare (4-channel bolt-on, D16) . . . 225
26.7.7 PWMxPOL . . . 226
26.7.8 PWMxDT . . . 226
26.7.9 PWMxSR: PWM status register . . . 226

27 1-Wire interface (OWx) 228
27.1 Overview . . . 228
27.2 Instances and signals . . . 228
27.3 Functional description . . . 229
27.3.1 The Time Base and the Two Speeds . . . 229
27.3.2 Presence, Errors and Bus Faults . . . 229
27.4 Interrupts and events . . . 229
27.5 Programming . . . 229
27.5.1 Launching a Transaction . . . 229
27.6 Usage notes . . . 230
27.7 Registers . . . 230
27.7.1 OWxCR: 1-Wire control register . . . 230
27.7.2 OWxCMD: 1-Wire command register . . . 231
27.7.3 OWxTX: Next write byte, the write-byte (OP = 011) source, transmitted LSB-first . . . 232
27.7.4 OWxRX . . . 232
27.7.5 OWxDIV: Time-base divisor . . . 232
27.7.6 OWxSR: 1-Wire status register . . . 232
27.7.7 OWxSPU: Reserved for the strong-pullup / parasite-power stage (D15) . . . 233

28 Direct memory access controller (DMAx) 234
28.1 Overview . . . 234
28.2 Instances and signals . . . 234
28.3 Functional description . . . 235

28.3.1 Transaction Structure and Bus Behaviour . . . 235
28.3.2 Peripheral-Paced Transfers . . . 235
28.3.3 The CRC Ride-Along . . . 236
28.3.4 Errors, Aborts, and the Read-Side-Effect Guard . . . 236
28.4 Interrupts and events . . . 236
28.5 Programming . . . 237
28.6 Usage notes . . . 237
28.7 Registers . . . 237
28.7.1 DMAxCR: DMA control register . . . 238
28.7.2 DMAxSR: DMA status register . . . 239
28.7.3 DMAxC0SRC . . . 239
28.7.4 DMAxC0DST . . . 240
28.7.5 DMAxC0LEN: Channel 0 transfer length in WORDS . . . 240
28.7.6 DMAxC0CFG . . . 240
28.7.7 DMAxC1SRC . . . 241
28.7.8 DMAxC1DST . . . 241
28.7.9 DMAxC1LEN: Channel 1 transfer length in WORDS . . . 241
28.7.10 DMAxC1CFG . . . 242
28.7.11 DMAxC2SRC . . . 242
28.7.12 DMAxC2DST . . . 243
28.7.13 DMAxC2LEN: Channel 2 transfer length in WORDS . . . 243
28.7.14 DMAxC2CFG . . . 243
28.7.15 DMAxC3SRC . . . 244
28.7.16 DMAxC3DST . . . 244
28.7.17 DMAxC3LEN: Channel 3 transfer length in WORDS . . . 244
28.7.18 DMAxC3CFG . . . 245
28.7.19 DMAxCRC: CRC16-CDMA2000 (poly 0xC857) accumulator / seed, resets to 0xFFFF . . . 245
28.7.20 DMAxDESC . . . 246

29 True random number generator (TRNGx) 247
29.1 Overview . . . 247
29.1.1 The Entropy Caveat . . . 247
29.2 Instances and signals . . . 247
29.3 Functional description . . . 248
29.3.1 Enabling, Configuring, and Draining the Harvest . . . 248
29.3.2 Health Test . . . 248
29.3.3 Clocking . . . 248
29.4 Interrupts and events . . . 249
29.5 Programming . . . 249
29.6 Usage notes . . . 249
29.7 Registers . . . 249
29.7.1 TRNGxCR: TRNG control register . . . 250
29.7.2 TRNGxSR: TRNG status register . . . 250
29.7.3 TRNGxDR: Entropy data register . . . 251
29.7.4 TRNGxHT: Health-test control/diagnostic register . . . 251

30 I2C target (I2CTx) 253
30.1 Overview . . . 253
30.2 Instances and signals . . . 253
30.3 Functional description . . . 254
30.3.1 Address Match, Direction, and Framing . . . 254
30.3.2 Receive, Transmit, and Clock Stretching . . . 254
30.3.3 Bus Speed and the MCLK Ratio . . . 254
30.4 Interrupts and events . . . 254
30.5 Programming . . . 255
30.6 Usage notes . . . 255
30.7 Registers . . . 255
30.7.1 I2CTxCR: I2C target control register . . . 255
30.7.2 I2CTxSR: I2C target status register . . . 256
30.7.3 I2CTxTX: Transmit byte buffer . . . 258
30.7.4 I2CTxRX . . . 258
30.7.5 I2CTxWDG: SCL-low watchdog timeout . . . 258

31 Event fabric (EVFAB) 259
31.1 Overview . . . 259
31.2 Instances and signals . . . 259
31.3 Functional description . . . 260
31.3.1 Producers: the Event Lines . . . 260
31.3.2 Consumers: the Task Lines . . . 261
31.3.3 Channel Enable, Status, and Injection . . . 262
31.4 Interrupts and events . . . 263
31.5 Programming . . . 263
31.5.1 Driver Contract . . . 263
31.5.2 Worked Example: An Autonomous Sensing Chain . . . 263
31.6 Usage notes . . . 264
31.6.1 Shared Access on PentaWoundPt2mm . . . 264
31.7 Registers . . . 264
31.7.1 EVFCR: Event fabric control register . . . 265
31.7.2 EVFSR: Event fabric status register . . . 265
31.7.3 EVFIE: Reserved interrupt-enable register . . . 266
31.7.4 EVFCAP: Capability register (read-only constant) . . . 266
31.7.5 EVFCHEN: Channel enable register . . . 266
31.7.6 EVFCHENSET: Channel enable SET alias . . . 267
31.7.7 EVFCHENCLR: Channel enable CLEAR alias . . . 267
31.7.8 EVFCHTRIG: Channel trigger injection register . . . 267
31.7.9 EVFFIRED: Sticky channel-fired flags . . . 267
31.7.10 EVFOVR: Sticky channel-overrun flags . . . 268
31.7.11 EVFEVSTAT: Sticky raw-event record . . . 268
31.7.12 EVFEVTRIG: Raw event injection register . . . 268
31.7.13 EVFGPIOMASK: GPIO0 edge-path mask . . . 269
31.7.14 EVFCH0CFG: Channel 0 configuration . . . 269
31.7.15 EVFCH1CFG: Channel 1 configuration . . . 270
31.7.16 EVFCH2CFG: Channel 2 configuration . . . 271
31.7.17 EVFCH3CFG: Channel 3 configuration . . . 271

31.7.18 EVFCH4CFG: Channel 4 configuration . . . 272
31.7.19 EVFCH5CFG: Channel 5 configuration . . . 273
31.7.20 EVFCH6CFG: Channel 6 configuration . . . 274
31.7.21 EVFCH7CFG: Channel 7 configuration . . . 275
31.7.22 EVFCH8CFG: Reserved channel-8 configuration slot . . . 275
31.7.23 EVFCH9CFG: Reserved channel-9 configuration slot . . . 275
31.7.24 EVFCH10CFG: Reserved channel-10 configuration slot . . . 276
31.7.25 EVFCH11CFG: Reserved channel-11 configuration slot . . . 276
31.7.26 EVFCH12CFG: Reserved channel-12 configuration slot . . . 276
31.7.27 EVFCH13CFG: Reserved channel-13 configuration slot . . . 276
31.7.28 EVFCH14CFG: Reserved channel-14 configuration slot . . . 276
31.7.29 EVFCH15CFG: Reserved channel-15 configuration slot . . . 277

32 AFEx 278
32.1 Overview . . . 278
32.2 Instances and signals . . . 278
32.3 Functional description . . . 279
32.3.1 Control surface . . . 279
32.3.2 Converter sequencer . . . 279
32.3.3 Result FIFO . . . 279
32.3.4 Simultaneous sampling and the swap engine . . . 279
32.4 Interrupts and events . . . 280
32.5 Programming . . . 280
32.5.1 Power-up . . . 280
32.5.2 One conversion . . . 280
32.5.3 Continuous record . . . 280
32.5.4 Square-wave impedance record . . . 280
32.6 Usage notes . . . 280
32.7 Registers . . . 281
32.7.1 AFExCR: Control . . . 281
32.7.2 AFExSR: Status . . . 282
32.7.3 AFExDATA: Result FIFO head, side-effect-free . . . 283
32.7.4 AFExTIA . . . 283
32.7.5 AFExDACVP: Potential-program DAC (A_Dac_Vp, 610.35 uV per code) and its enable 283
32.7.6 AFExDACVCM: Common-mode DAC (A_Dac_Vcm, 610.35 uV per code) and its enable 283
32.7.7 AFExBIAS: Local bias generator trim . . . 284
32.7.8 AFExMUX: Converter input and analog test-port selects . . . 284
32.7.9 AFExSWAP . . . 284
32.7.10 AFExBIASG0 . . . 284
32.7.11 AFExBIASG1 . . . 285
32.7.12 AFExBIASG2 . . . 285
32.7.13 AFExBIASG3 . . . 286
32.7.14 AFExBIASGCR: Shared bias generator mode, enables and trim . . . 286

33 Interrupt router (IRQROUTER) 288
33.1 Overview . . . 288
33.2 Instances and signals . . . 288
33.3 Block diagram . . . 288
33.4 Functional description . . . 289
33.4.1 Routing rows . . . 289
33.4.2 Claim and complete . . . 289
33.4.3 Status words . . . 289
33.5 Interrupts and events . . . 290
33.6 Programming . . . 290
33.6.1 Routing a peripheral to hart *h* . . . 290
33.6.2 Servicing a routed interrupt . . . 290
33.7 Usage notes . . . 291
33.8 Registers . . . 291
33.8.1 HnENL (n = 0..4): Hart n interrupt routing register, vectors 31:0 . . . 292
33.8.2 HnENM (n = 0..4): Hart n interrupt routing register, vectors 63:32 . . . 292
33.8.3 HnENU (n = 0..4): Hart n interrupt routing register, vectors 95:64 . . . 292
33.8.4 HnENX (n = 0..4) . . . 292
33.8.5 CLAIM: Interrupt claim/complete register (M19) . . . 292
33.8.6 PENDL . . . 293
33.8.7 PENDM: Raw pending interrupt levels, vectors 63:32 (read-only) . . . 293
33.8.8 PENDU: Raw pending interrupt levels, vectors 95:64 (read-only) . . . 293
33.8.9 PENDX . . . 293
33.8.10 INSVCL . . . 293
33.8.11 INSVCM: Under-service flags, vectors 63:32 (read-only) . . . 294
33.8.12 INSVCU: Under-service flags, vectors 95:64 (read-only) . . . 294
33.8.13 INSVCX . . . 294

34 PINMUX 295
34.1 Overview . . . 295
34.2 Instances and signals . . . 295
34.3 Functional description . . . 295
34.3.1 The Per-Pad Menu . . . 295
34.3.2 The Pad Menus . . . 296
34.3.3 What the Pad Does . . . 296
34.3.4 The Input Path . . . 297
34.3.5 The GPIO Mirrors . . . 297
34.4 Interrupts and events . . . 297
34.5 Programming . . . 297
34.6 Usage notes . . . 298
34.7 Registers . . . 298
34.7.1 PMXCAP: Pin-mux capability register, read-only . . . 298
34.7.2 PMXIN0 . . . 299
34.7.3 PMXCFG0 . . . 299
34.7.4 PMXCFG1 . . . 300
34.7.5 PMXCFG2 . . . 300

IV Analog front end **302**

V Boot ROM and software **303**

35 ROM monitor **304**
35.1 Interpreter model . . . 304
35.2 Word list . . . 304
35.2.1 Memory Access Functions . . . 304
35.2.2 Print and Input Functions . . . 306
35.2.3 Arithmetic Functions . . . 307
35.2.4 Bitwise Logic Functions . . . 309
35.2.5 Comparison Functions . . . 310
35.2.6 Boolean Functions . . . 311
35.2.7 Stack Manipulation Functions . . . 312
35.2.8 SPI Flash R/W and Programming Functions . . . 313
35.2.9 Miscellaneous Functions . . . 314
35.3 Interpreter limits . . . 317
35.4 Defining words . . . 317
35.5 Conditionals . . . 317
35.6 Loops . . . 318

36 Boot image format **319**
36.1 Programming the flash from the ROM monitor . . . 319

A Getting started and software development **321**
A.1 First run . . . 321
A.2 Building a firmware image . . . 322
A.3 A program of your own . . . 322
A.4 Simulating the image before silicon . . . 324
A.5 Loading a program onto a chip . . . 324
A.6 Debugging on the part . . . 325

B Register index **327**

C Errata **335**

D Revision history **336**

List of Figures

2.1 PentaWoundPt2mm whole-chip diagram . . . 32

3.1 LQFP-176 pinout, top view . . . 37

4.1 PentaWoundPt2mm address space: one column for all five harts . . . 45

5.1 One read through a TCM aperture, from hart 0 to the target tile's TCM and back. . . . 50
5.2 Timing diagram of one uncontended shared-window read at the `mp_arbiter` pins. . . . 51
5.3 Timing diagram of one stalled shared-window read through a TCM aperture; the grey tick marks where an unstalled read would have completed, and the tile read (*SETTLE*, *READ*, *LATCH*, then one `mclk` of `tcm_ext_done`) runs while `s_stall` is high. . . . 52
5.4 Flow chart of boot and tile loading across the harts. . . . 53
5.5 Decision chart for choosing a synchronization primitive. . . . 55

6.1 The clock tree: the four sources, the two selects and dividers, the SMCLK gate, and the blocks each root clocks. . . . 58
6.2 The always-on domain and one switched channel tile, with the four per-tile controls from `pwr_ctrl`. . . . 59

11.1 Block diagram of the debug path, from the probe to the harts. . . . 85
11.2 State diagram of the sixteen-state JTAG test access port. . . . 86
11.3 Timing diagram of one four-bit JTAG data-register scan. . . . 86
11.4 Field layout of the 41-bit DMI data register. . . . 89
11.5 Timing diagram of one DMI transaction crossing between the two clock domains. . . . 91
11.6 Swimlane diagram of the twelve steps across the four agents that perform them. . . . 101

13.1 SPI connection diagram: `SPI0` to the boot flash on fixed pads and `SPI1`, drawn as master, to an external device. . . . 120
13.2 SPI timing for the four modes, one 8-bit frame. . . . 121
13.3 Byte order of a multi-byte frame with and without the swap. . . . 122
13.4 Bit order of a 16-bit frame. . . . 122

14.1 UART connection to an external device. . . . 130
14.2 One UART frame with the optional parity bit. . . . 131

15.1 Clock path of the `TIMERx` peripheral. . . . 138
15.2 Rollover at `TIMxCMP2` with `CMP2RST` set. . . . 139
15.3 Output compare and PWM generation on `TxCMP0`. . . . 139
15.4 Input capture: an edge on `TxCAP0` latches `TIMxVAL` into `TIMxCAP0`. . . . 140

17.1 The NPU datapath and the staging RAM it borrows. . . . 158

20.1 One I2C byte write on the wire: START, address, ACK, data byte, ACK, STOP. . . . 180

22.1 Two harts racing for the same mutex: the first read returns 0 and takes the word, the second returns the holder's marker (mhartid +1) and must retry, and the holder releases with sw x0. 196

33.1 The interrupt fabric: peripheral sources, the router's enable rows and claim/complete stage, the harts, and the CLINT pair in red because it bypasses the router. 289
33.2 Claim/complete delivery of one peripheral interrupt. 290

List of Tables

1.1 Documents and generated artifacts related to this manual. . . . 25
1.2 Register access types . . . 26

2.1 Chip configuration knobs (`make chip CONFIG=config.json`) and the values of this build. The basic knobs come first: the hart count and orchestrator, the fabric, the ISA letters, the memory sizes, the peripheral toggles and the package. The advanced block below them holds the Z-series extensions, the privileged architecture, debug, and the knobs that size a block that is already on. . . . 32
2.2 Derived geometry of this configuration (computed, not configurable) . . . 35
2.3 Per-hart-class ISA of this build. Each row is a hart class the generator emitted; the ISA string is what that class advertises in its read-only `misa` CSR. A minimal tile class is rv32iac and nothing more: the generated `TILE_ENABLE_*` constants drop M, Zb, every Z-series extension, U-mode and PMP, while A, C, the trap CSRs and debug are kept (the tiles run the shared-fabric LR/SC and AMO locking, and the Debug Module halts them). Zicntr, where the chip has it, is kept on both classes because `cycle` and `instret` exist on every hart unconditionally and the knob gates no hardware. Code that more than one class executes must be built for the narrowest row. . . . 36

3.1 Package Pins . . . 38
3.2 GPIO Pin Functions and Reset Values . . . 42
3.3 GPIO Alternate Function Selection (`PxAFS`) . . . 43

4.1 Address map (R = readable, W = writable, X = executable; unmapped rows read zero) . . . 46

6.1 Boot mode selected by the `P0.7` strap. . . . 57

7.1 General-purpose registers and their ABI roles. . . . 64
7.2 CSRs present in every configuration (partial list, Privileged Architecture 1.12 numbering). 64
7.3 CSRs present only in some configurations (partial list, Privileged Architecture 1.12 and Debug 1.0 numbering). . . . 65

8.1 Machine trap control and status registers . . . 69
8.2 Trap causes, values, and saved program counters . . . 70

9.1 Interrupt vectors . . . 72
9.2 CPU-internal interrupt causes . . . 78

10.1 Measured instruction timing, in MCLK cycles. . . . 80
10.2 Straddling fetches that survive the fetch-ahead, measured with and without the C extension. 81
10.3 Measured CPI by benchmark (timed kernel, single hart, no bus contention). . . . 82

11.1 Debug Transport Module instruction register values. . . . 88
11.2 Debug Module registers at their DMI addresses. . . . 89
11.3 `DEBUG` register summary . . . 95

12.1 `GPIOx` instances . . . 105

12.2 GPIOx signals . . . 105
12.3 GPIOx interrupt vectors . . . 109
12.4 GPIO pin states. . . . 110
12.5 GPIOx interrupt vectors; pin y uses the port's base vector plus y. . . . 112
12.6 GPIOx register summary . . . 113

13.1 SPIx instances . . . 118
13.2 SPIx signals . . . 118
13.3 SPIx interrupt vectors . . . 119
13.4 SPI mode key. . . . 120
13.5 SPIx interrupt vectors. . . . 123
13.6 SPIx register summary . . . 125

14.1 UARTx instances . . . 129
14.2 UARTx signals . . . 129
14.3 UARTx interrupt vectors . . . 130
14.4 UARTx interrupt vectors. . . . 132
14.5 UARTx register summary . . . 133

15.1 TIMERx instances . . . 136
15.2 TIMERx signals . . . 136
15.3 TIMERx interrupt vectors . . . 137
15.4 TIMER0 interrupt vectors; TIMER1 follows the same order from vector 22. . . . 140
15.5 TIMERx register summary . . . 141

16.1 SYSTEM instances . . . 147
16.2 SYSTEM interrupt vectors . . . 147
16.3 SYSTEM interrupt vector. . . . 149
16.4 SYSTEM register summary . . . 150

17.1 NPU instances . . . 157
17.2 NPU interrupt vectors . . . 157
17.3 NPU interrupt vector. . . . 160
17.4 NPU register summary . . . 161

18.1 PWRCTRL instances . . . 166
18.2 PWRCTRL register summary . . . 169

19.1 QSPIx instances . . . 172
19.2 QSPIx signals . . . 172
19.3 QSPIx interrupt vectors . . . 173
19.4 QSPIx register summary . . . 174

20.1 I2Cx instances . . . 178
20.2 I2Cx signals . . . 178
20.3 I2Cx interrupt vectors . . . 178
20.4 I2C0 interrupt vectors; I2C1 follows the same order from vector 70. . . . 182
20.5 I2Cx register summary . . . 183

21.1 CLINT instances . . . 191

21.2 CLINT interrupt vectors . . . 191
21.3 CLINT interrupt vectors. . . . 192
21.4 CLINT register summary . . . 193

22.1 MUTEX instances . . . 195
22.2 MUTEX register summary . . . 197

23.1 I3Cx instances . . . 198
23.2 I3Cx signals . . . 198
23.3 I3Cx interrupt vectors . . . 199
23.4 I3Cx register summary . . . 200

24.1 NFCx instances . . . 207
24.2 NFCx signals . . . 207
24.3 NFCx interrupt vectors . . . 208
24.4 NFC0 interrupt vectors. . . . 209
24.5 NFCx register summary . . . 210

25.1 RTCx instances . . . 216
25.2 RTCx interrupt vectors . . . 216
25.3 RTCx register summary . . . 218

26.1 PWMx instances . . . 221
26.2 PWMx signals . . . 221
26.3 PWMx interrupt vectors . . . 222
26.4 PWMx register summary . . . 223

27.1 OWx instances . . . 228
27.2 OWx signals . . . 228
27.3 OWx interrupt vectors . . . 229
27.4 OWx register summary . . . 230

28.1 DMAx instances . . . 234
28.2 DMAx interrupt vectors . . . 235
28.3 DMAx register summary . . . 237

29.1 TRNGx instances . . . 248
29.2 TRNGx interrupt vectors . . . 248
29.3 TRNGx register summary . . . 249

30.1 I2CTx instances . . . 253
30.2 I2CTx interrupt vectors . . . 254
30.3 I2CTx register summary . . . 255

31.1 EVFAB instances . . . 259
31.2 EVFEVSELn event lines . . . 260
31.3 EVFTASKSELn task lines . . . 261
31.4 EVFAB register summary . . . 264

32.1 AFEx instances . . . 278

32.2 AFEx interrupt vectors . . . 278
32.3 AFEx register summary . . . 281

33.1 IRQROUTER instances . . . 288
33.2 IRQROUTER delivery into the harts. . . . 290
33.3 IRQROUTER register summary . . . 291

34.1 PINMUX instances . . . 295
34.2 Pin-mux pad menus: the code-1 mirror and the class carried by codes 2 and up. . . . 296
34.3 The four pin-mux classes: what codes 2 and up select on a pad of each class. . . . 297
34.4 PINMUX register summary . . . 298

36.1 Boot image stream, as read by the boot ROM from flash offset 0x000000. . . . 319

A.1 Tools the build provisions on first use. . . . 321
A.2 Artifacts of one firmware target, for a package //software/NAME. . . . 322
A.3 Simulation targets, all license-free. . . . 324

B.1 Register index . . . 327

C.1 Errata. . . . 335

D.1 Revision history of this manual. . . . 336

Part I System

1 About this manual

1.1 Scope and audience

This manual is the technical reference for the PentaWoundPt2mm microcontroller. It is written for firmware developers and board designers, and it assumes familiarity with 32-bit microcontrollers, memory-mapped peripherals and the RISC-V instruction set. It describes the hardware as implemented: the address map, every peripheral and its registers, the interrupt fabric, the boot ROM and the processor core's configuration. The RISC-V specifications are normative for the instruction set and for the privileged architecture; where this manual restates them it does so only to say which parts are implemented and how. Where the implementation deviates from a specification, the deviation is stated where it applies.

Everything in this manual is generated from one chip configuration, the same one that produces the RTL, the C headers and the linker scripts (Section 2.3), so a value printed here is the value built.

1.2 Organisation of this manual

Part I, System
What is on the chip, its package and pins, the address map, the multi-core architecture, and reset, clocks and power. Read this part first.

Part II, Processor
The VestaRV core: its instruction set, control and status registers, privileged architecture, interrupts and exceptions, instruction timing, and the debug system.

Part III, Peripherals
One chapter per peripheral, each with a functional description, a programming sequence, usage notes and the complete register description.

Part IV, Analog front end
The mixed-signal front end and its characterisation.

Part V, Boot ROM and software
The ROM monitor and the boot image format the boot ROM loads.

Appendices
Getting started and software development, the register index, errata, and the revision history of this document.

1.3 Related documents

Table 1.1 lists the documents this manual relies on and the generated artifacts that accompany it. This manual does not contain electrical characteristics, absolute maximum ratings, package mechanical drawings, or errata for individual silicon revisions; those belong to the datasheet and the errata sheet of a specific silicon revision.

Table 1.1: Documents and generated artifacts related to this manual.

Document or artifact	Role
RISC-V Instruction Set Manual, Volume I: Unprivileged ISA, version 20191213, with the ratified Bit-Manipulation extensions (Zba, Zbb, Zbc, Zbs, version 1.0)	Normative for the instruction set (Chapter 7).
RISC-V Instruction Set Manual, Volume II: Privileged Architecture, version 1.12	Normative for the machine-mode control and status registers and the trap architecture (Chapters 7 and 8).
RISC-V External Debug Support, version 1.0	Normative for the debug transport and debug module (Chapter 11).
`platform/common/out/software/include/MemoryMap.h`, `core_features.h`, `periph.S`	Generated C and assembly headers: every base address, register offset, field mask and vector number in this manual under the header symbol names.
`platform/common/out/linker-scripts/memory.x`, `periph.x`	Generated linker script fragments for the memory map of this build.
`software/include/regs/castalia_regs.h` and one header per block	Register headers emitted from the SystemRDL descriptions: field masks, positions, widths, reset values and enumerated values, plus a base address and a typed pointer per instance (Section A.3).
`docs/register_browser.html`	Interactive register browser built from the same memory map.
`docs/chip_configurator.html`	Interactive editor for the chip configuration file.

1.4 Conventions

Numbers
Hexadecimal numbers carry the prefix `0x` (`0x4600`); binary numbers carry the prefix `0b` (`0b101`); decimal numbers have no prefix. A value of the form `0x8000 + 4n` is an address formula in which *n* is the array index.

Units
Capacities are binary: 1 KiB is 1024 bytes. Rates and frequencies are decimal: 115,200 baud, 24 MHz. A word is 32 bits, a half-word 16 bits and a byte 8 bits.

Registers and fields
A register is named by its peripheral template and register name, and a field by the dotted form `PERIPHERAL.REGISTER.FIELD`, for example `TIMx.TIMxCR.TEN`. In prose, register names are the template form (`PxIN`, `TIMxCR`), where `x` stands for the instance number; the instance form (`P0IN`, `TIM0CR`) appears only in the per-instance address tables and in the generated headers.

Bit ranges
Bits are numbered from 0 at the least significant end. A range is written `[h:l]` with the high bit first; `[7:4]` is four bits. A single bit is written `[n]`.

Set and clear
A bit is set when it holds 1 and cleared when it holds 0. Asserted means the active level of a signal, whatever its polarity; an active-low pin is named with a trailing `N` (`RESETN`).

Reserved
A reserved bit reads as zero and must be written as zero. No exception is raised by an access to a reserved bit or to a reserved register offset inside a peripheral's window.

Register width
Every register is described at its natural width, 8 or 32 bits, and every register occupies one 4-byte slot at a 4-byte aligned offset regardless of its width; the unused upper bytes of an 8-bit register are reserved.

Narrow accesses
The bus carries four byte-lane write strobes, so a byte or half-word store reaches the peripheral with only the addressed lanes asserted. Reads always return the full word of the addressed slot. Whether a partial write is honoured is a property of the register: an 8-bit register lives in byte lane 0, and a register described as requiring a full-word write commits only stores that assert lane 0.

1.5 Register access types

Each field in a register description carries an access type from the list in Table 1.2. The list is the set the chip generator accepts, so it cannot drift from the register descriptions.

Table 1.2: Register access types

Type	Meaning	Read effect	Write effect
`rw`	Read/write	Returns the current value.	Sets the field to the written value.
`r`	Read only	Returns the current value.	Ignored.
`r0`	Reserved, reads zero	Returns 0.	Ignored. Write 0.
`r1`	Reserved, reads one	Returns 1.	Ignored. Write 1.
`rw0`	Read, write 0 to act	Returns the current value.	A 0 in a bit position acts on that bit; a 1 leaves it unchanged.
`rw1`	Read, write 1 to act	Returns the current value.	A 1 in a bit position acts on that bit (clears a flag, or applies the register's set, clear or toggle operation); a 0 leaves it unchanged.
`w`	Write only	Not defined by this manual.	Sets the field to the written value.
`w0`	Write 0 to trigger	Not defined by this manual.	A 0 triggers the action; a 1 has no effect.
`w1`	Write 1 to trigger	Not defined by this manual.	A 1 triggers the action (a command strobe); a 0 has no effect.

1.6 The anatomy of a register description

Every register in Part III is described in the same layout. The example below is a fictitious register, `DUMMY.DUMMYCR`, chosen to show every element once; the callouts after it name each element.

DUMMYxCR: Dummy control register

Offset `0x04` **Reset** `0x00000000` **Width** 32

Controls the dummy function. One or two sentences say what the register is for and point to the functional section that explains it.

Bits	Field	Type	Reset	Description
[31:8]	-	-	-	Reserved. Reads zero; write zero.
[7:4]	DIV	rw	0x0	Clock divider. The dummy clock is the source divided by 2^{DIV}.
[3:2]	MODE	rw	0b00	Operating mode. 0b00 = DMODE_OFF: the function is off. 0b01 = DMODE_ONCE: one operation, then off. 0b10 = DMODE_CONT: continuous. 0b11 = reserved.
[1]	DIF	rw1	0	Done interrupt flag. Set by hardware when an operation completes; write 1 to clear.
[0]	DEN	rw	0	Enable. 0 = disabled, 1 = enabled. Write this bit last.

Heading
The register's template name and a short title. The heading appears in the chapter's own register list, not in the main table of contents. The `x` in the name is the instance number.

Offset, Reset, Width
The byte offset from the instance base address (the base addresses are in the chapter's instance table), the reset value of the whole register, and its width in bits.

Summary sentence
What the register is for, in one to three sentences, with a pointer to the functional description for anything longer.

Field rows
One row per field, most significant first, every bit covered. **Bits** is the range, **Field** the name used in prose and in the generated headers (with the peripheral's field prefix), **Type** the access type of Table 1.2, **Reset** the field's reset value at the field's width.

Enumerated values
Listed inside the description cell as `value = SYMBOL`: meaning. The symbol is the macro name in the generated header. Values are binary for fields of four bits or fewer and hexadecimal for wider fields.

Reserved runs
Consecutive reserved bits are collapsed into one row whose field, type and reset are a dash.

1.7 Callouts

Two kinds of callout appear in this manual, and only two. A *Note* is advisory: it adds context or a recommendation, and nothing breaks if it is skipped. A *Caution* states a constraint the hardware enforces: ignoring it causes a wrong result, a hang, or lost data.

Note: A note looks like this. It is at most a few sentences long.

Caution: A caution looks like this. Software must satisfy the condition it states, because the hardware does not check it.

1.8 Glossary of terms and abbreviations

ABI Application binary interface: the register-usage and calling convention of Table 7.1.

AF Alternate function: a peripheral signal a GPIO pin can carry instead of its plain I/O function; a pin has up to eight AF planes.

AFE Analog front end: the mixed-signal circuitry of Part IV.

CLINT Core-local interruptor: the peripheral that provides each hart's software interrupt and timer interrupt.

CPI Cycles per instruction.

CRC Cyclic redundancy check.

CSR Control and status register: a register inside the processor core reached by the `csr*` instructions.

DM Debug module: the on-chip block that halts, resumes and inspects harts on behalf of an external debugger.

DMI Debug module interface: the register bus between the debug transport module and the debug module.

DTM Debug transport module: the JTAG-side block that converts scans into DMI accesses.

EIS Electrochemical impedance spectroscopy: the swept-frequency impedance measurement the analog front end performs.

GPIO General-purpose input and output.

hart Hardware thread: one RISC-V execution context. Every hart on this chip is a separate core with its own TCM.

I^2C Inter-integrated circuit: the two-wire serial bus.

IPI Inter-processor interrupt: a CLINT software interrupt sent from one hart to another.

IRQROUTER Interrupt router: the peripheral that routes every interrupt source to any hart and provides the claim and complete registers.

ISA Instruction set architecture.

ISR Interrupt service routine.

IVT Interrupt vector table: the table of jump instructions at the base of each hart's TCM.

JTAG The IEEE 1149.1 test access port over which the debug transport operates.

MCU Microcontroller unit.

MTCMOS Multi-threshold CMOS power gating: the header switches that cut the supply of a hart tile.

MUTEX Hardware mutual-exclusion bank: the peripheral that provides atomically claimable locks.

NFC Near-field communication: the peripheral that reads a 13.56 MHz tag interface and, on a field-powered board, supplies the chip.

NPU Neural processing unit: the fixed-point matrix accelerator.

PGOOD Power good: the supply-supervisor input that releases the boot gate on a field-powered board.

PMP Physical memory protection: the RISC-V machine-mode memory protection unit, a configuration option.

POC Power-on control: the input of the I/O pad ring that holds the pads in a safe state while the I/O supply rises. It is not a boot strap.

POR Power-on reset.

PWRCTRL Power controller: the peripheral that gates hart tiles and holds the boot gate.

QSPI Quad serial peripheral interface: a four-data-line SPI variant, a configuration option.

ROM Read-only memory: the boot ROM at address `0x00000`.

SPI Serial peripheral interface.

TCM Tightly coupled memory: the private RAM of one hart, 8 KiB at `0x8000`, reached without arbitration.

TRNG True random number generator, a configuration option.

UART Universal asynchronous receiver and transmitter.

W1C Write one to clear: a flag bit that is cleared by writing 1 to it (access type `rw1`).

WARL Write any, read legal: a CSR field that accepts any written value and reads back a legal one.

XIP Execute in place: fetching instructions directly from the external SPI flash through the extended-flash window.

2 Overview

The PentaWoundPt2mm microcontroller (MCU) is a five-hart multiprocessor built from five identical VestaRV cores, a 32-bit RISC-V processor, together with a boot ROM that contains the boot loader and a ROM monitor, a private tightly coupled memory (TCM) per hart, an arbitrated shared memory window for inter-hart communication, and the peripherals listed below. The multi-core architecture is described in Chapter 5, and Figure 2.1 shows the top-level organisation of this configuration.

2.1 Features

- Chip name: PentaWoundPt2mm
- Compatible with RISC-V compiler RV32I[M][A][C][Zba][Zbb][Zbs][Zbc]
- 32-bit memory bus
- Contains 32 general purpose dual-port CPU registers
- Supports the RISC-V compressed instruction set and allows the use of the [C] compiler extension. Allowing the compiler to use the [C] extension reduces executable code size, but also takes the processor longer to fetch 32-bit long instructions that are aligned on a 2 but not 4 byte boundary from memory.
- Includes a multi-stage bit shifter in the ALU to save power and chip area for bit shift operations at the expense of speed
- Includes a fast 32-bit signed integer hardware multiplier in the ALU and allows the use of the [M] compiler extension
- Includes a medium speed 32-bit signed integer hardware divider and remainder calculator in the ALU
- Supports the RISC-V atomic instruction set (LR/SC and AMOs) and allows the use of the [A] compiler extension; on multi-hart configurations atomics are globally coherent across the shared window
- Supports the RISC-V bit-manipulation extensions Zba (address generation), Zbb (basic bit manipulation), Zbs (single-bit operations), and Zbc (carry-less multiplication)
- The read-only misa CSR (0x301) advertises the enabled instruction-set extensions at run time
- Supports maskable hardware interrupts generated by peripheral signals
- 5× VestaRV harts (cores), each with private RAM; hart ID readable via the `mhartid` CSR
- An arbitrated shared memory window with 80 KiB of shared RAM, a CLINT (inter-processor and per-hart timer interrupts), 16 hardware mutexes, and a per-hart peripheral interrupt router
- 6× 8-pin general purpose I/O (GPIO) ports with edge-triggered interrupts and per-pin multiplexed alternate functions (GPIO + up to 8 alternate functions per pin)
- 2× SPI interfaces (SPI0 provides memory-mapped access to external flash memory)
- 2× UART interfaces with hardware parity support
- 2× 32-bit timers with pulse-width modulation outputs and input capture units
- A CRC calculation engine (CRC16_CDMA2000)

- 2× internal digitally controllable oscillators
- 2× external clock pins for clock generation and accurate timing
- A windowed watchdog timer
- A neural processing unit (NPU) co-processor for hardware acceleration of machine learning tasks
- Per-tile MTCMOS power gating with hardware gate/wake sequencing (cold-boot wake)
- 1× QSPI flash controller (single/dual/quad lane; per-phase width)
- 2× I^2C interfaces (both master and slave mode)
- 1× I3C controller (SDR + legacy-I2C, dynamic address assignment, in-band interrupts)
- 1× NFC ISO 14443A tag / card-emulation engine (Miller/Manchester codec, CRC-A, anticollision, digital AFE boundary)
- 1× real-time clock (32.768 kHz always-on wall clock, one-shot alarm, periodic tick, single combined IRQ)
- 1× buffered PWM generator (2 channels, glitch-free double-buffered update, software fault trip, period-event tick, two IRQs)
- 1× 1-Wire master (reset/presence + bit/byte primitives, standard + overdrive, firmware ROM search + CRC-8, single combined IRQ)
- 1× multi-channel single-shot DMA controller (2/4 channels, peripheral-paced or mem-to-mem, CRC16 ride-along, read-side-effect guard, two IRQs)
- 1× ring-oscillator true-random-number-generator harvest engine (NRO-ring ensemble, read-consumes data register, repetition-count health test with auto-halt, single combined IRQ, bring-up-grade entropy)
- 1× hardware-autonomous I2C target (7-bit address match + mask + general call, hardware clock stretching, START/STOP framing flags, single-byte RX/TX with ready/empty status, stuck-SCL watchdog, 2 combined IRQs)
- 1× event/trigger fabric (PPI-style crossbar: 8 channels, 16 event producers, 10 task consumers, one-MCLK registered pulses, peripheral-to-peripheral chains with every hart asleep)
- 4× analog front-end sites (potentiostat control, SAR sequencer with result FIFO, simultaneous-sample trigger, excitation swap engine)
- Claim/complete peripheral interrupt delivery (PLIC-style) with any-vector-to-any-hart routing
- Pin multiplexer over 19 spare pads (per-pad function menu, high-Z at reset)

2.2 Block diagram

Figure 2.1 is the view to read first: the bus masters above the arbiter, the arbiter drawn as the single bus bar it behaves like, the memories and every peripheral family this configuration instantiates on one rank below it, and the package boundary in red with the pin groups that cross it. It is generated from the configuration, so the peripheral set and the instance counts in it are this build's. The memory system's own view of the chip, the one address column all five harts share, is Figure 4.1 in Chapter 4, and the per-pin bonding is Chapter 3.

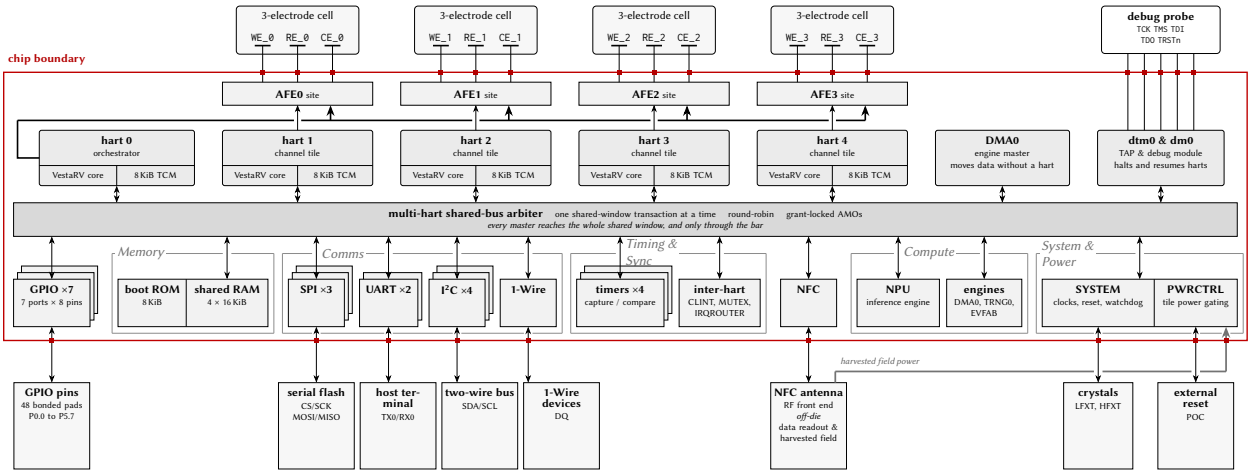


Figure 2.1: PentaWoundPt2mm whole-chip diagram: the masters above one `mp_arbiter` bus bar, every peripheral on one rank below it, and the package boundary in red.

2.3 Chip configuration

This document, the memory-map headers, the linker scripts, and the drop-in RTL are all generated from one chip configuration by `make chip`. The configuration is a small JSON document (every key optional; a missing key keeps the PentaWoundPt2mm default) passed as `make chip CONFIG=config.json`; an interactive front end for producing it ships with the generator (`docs/chip_configurator.html`). Table 2.1 lists every configuration knob together with the value used to build this document, Table 2.2 lists the geometry derived from those knobs, and Table 2.3 lists the ISA of each hart class this build emitted. The resolved configuration is also written to `config/ChipConfig.resolved.json` at every build, and `make verify [CONFIG=config.json]` proves a configuration by staging the generated RTL into a behavioural simulation environment and running the multi-core boot and ISA smoke suite against it. The core peripheral set and the pad ring are not configuration knobs; they are fixed template content, and the pad ring (Chapter 3) is derived from the generator's package model. The second-instance peripherals (`UART1`, `SPI1`, `TIMER1`, `I2C1`) and the `NPU` are individually droppable through the `peripherals` section: a dropped instance's register window reads zero, its interrupt vectors are reserved (the numbering is frozen), and its pins revert to plain GPIO.

Table 2.1: Chip configuration knobs (`make chip CONFIG=config.json`) and the values of this build. The basic knobs come first: the hart count and orchestrator, the fabric, the ISA letters, the memory sizes, the peripheral toggles and the package. The advanced block below them holds the Z-series extensions, the privileged architecture, debug, and the knobs that size a block that is already on.

Configuration key	This build	Meaning / valid values
`chipName`	`PentaWoundPt2mm`	non-empty string: renames the chip in the TRM and headers (CHIP_NAME env wins)
`numHarts`	`5`	int 1..32: hart count; hart 0 is the always-on management hart and harts 1..N-1 are the power-gateable tiles
`orchestrator`	`true`	bool: hart 0 is the always-on orchestrator; harts 1..N-1 are gateable tiles

Table 2.1 continued on next page

Table 2.1 continued from previous page

Configuration key	This build	Meaning / valid values
numMutexes	16	int 1..1024: hardware mutex bank size, the number of MUTEXn registers
registerFileDualPort	true	bool: docs-only, the register file is dual-port in the RTL either way
isa.mul	true	bool: M multiply
isa.fastMul	true	bool: docs-only, the multiplier is already single-cycle
isa.div	true	bool: M divide
isa.atomics	true	bool: A extension (LR/SC + AMO)
isa.compressed	true	bool: C extension
isa.bitmanip	true	bool: Zba/Zbb/Zbs/Zbc
isa.counters	false	bool: Zicntr march suffix only (cycle/instret always exist, time/timeh never do)
isa.counters64	false	bool: docs-only high counter halves (always present); needs isa.counters
memory.romSize	8192 (8 KiB)	int bytes, 1 KiB multiple <= 0x4000: boot ROM (0x0-0x1FFF at the shipped 8 KiB; the page runs to 0x3FFF and its tail is unmapped)
memory.tcmSizePerHart	8192 (8 KiB)	int bytes, 1 KiB multiple <= 0x4000: per-hart TCM based at 0x8000 (top = 0x8000 + size - 1; 0x9FFF at the shipped 8 KiB)
memory.sharedBulkRamSize	65536 (64 KiB)	int bytes, multiple of 0x4000: shared bulk RAM from 0x10000, one bank per 16 KiB
memory.npuStagingRamSize	16384 (16 KiB)	int bytes, 1 KiB multiple <= 0x4000: NPU staging RAM (0xC000-0xFFFF)
peripherals.npu	true	bool: false drops the NPU (slot 10 and the 0xC000 staging window read zero)
peripherals.i2c1	true	bool: false drops I2C1 (slot 15 reads zero; SDA1/SCL1 revert to plain GPIO)
peripherals.uart1	true	bool: false drops UART1 (slot 5 reads zero; TX1/RX1 revert to plain GPIO)
peripherals.spi1	true	bool: false drops SPI1 (slot 3 reads zero; its four pins revert to plain GPIO)
peripherals.timer1	true	bool: false drops TIMER1 (slot 7 reads zero; the T1 pins revert to plain GPIO)
peripherals.cqAfeStubs	false	bool: the four AFE register stubs and the EIS engine stub (slot 12, 0x4C00)
peripherals.qspi	true	bool: QSPI0 controller in slot 12 (0x4C00); needs cqAfeStubs false
peripherals.afe2	true	bool: four AFE2 sites (anatop_pixel control + SAR sequencer) at 0x6C00-0x6F00; needs cqAfeStubs false
peripherals.i3c	true	bool: I3C0 controller (MVP + DAA + IBI) at 0x6100

Table 2.1 continued on next page

Table 2.1 continued from previous page

Configuration key	This build	Meaning / valid values
peripherals.nfc	true	bool: NFC0 ISO 14443A tag engine at 0x6200 (the RF front end is off-die)
peripherals.rtc	true	bool: RTC0 32.768 kHz wall clock with alarm and tick at 0x6500
peripherals.pwm	true	bool: PWM0 two-channel buffered PWM at 0x6600 (outputs on P2.2/P2.3)
peripherals.onewire	true	bool: OW0 1-Wire master at 0x6700, open-drain DQ on P4.7
peripherals.fieldPower	true	bool: wires the field-power pins (PGOOD, harvest strap) into PWRCTRL
peripherals.dma	true	bool: DMA0 multi-channel DMA at 0x6800; adds one more arbiter master
peripherals.i2ctarget	true	bool: I2CT0 hardware I2C target at 0x6A00, sharing the I2C0 pads
peripherals.trng	true	bool: TRNG0 ring-oscillator entropy source at 0x6900 (firmware must DRBG it)
peripherals.eventFabric	true	bool: EVFAB0 event/trigger crossbar at 0x6B00 (8 channels, no IRQ vector)
peripherals.pinmux	true	bool: PINMUX spare-pad pin multiplexer at 0x7C00 (menu in config/pinmux.json)
package.model	castalia-lqfp176-pt2mm	string: package pinout model, one of the pinout models defined in the generator (QFN-44, QFN-64 quad, or LQFP-100)
package.preliminary	true	bool: prints the Preliminary note in the TRM package section
Advanced knobs		
afeTopology	per_tile	enum top_ports|per_tile: where the four AFE2 sites' analog pins leave the digital hierarchy (per_tile needs afe2 + 5 harts + orchestrator)
core.fetchAhead	true	bool: C-extension fetch-ahead (one flip-flop; removes most of the straddling-fetch stall)
isa.minimalTiles	true	bool: harts 1..N-1 drop M and B (rv32iac); hart 0 keeps the full ISA
isa.zicond	false	bool: Zicond conditional-zero ops (czero.eqz/czero.nez)
isa.zcb	false	bool: Zcb extra compressed loads/stores and ALU ops; needs isa.compressed
isa.zimop	false	bool: Zimop+Zcmop may-be-ops (mop.r/mop.rr rd<-0, c.mop.n nops)
isa.zihint	false	bool: Zihintpause+Zihintntl (PAUSE = 16-cycle arbiter-yield window; ntl.* nops)
isa.zihpm	false	bool: Zihpm performance counters 3 and 4 (four chip events)
isa.zawrs	false	bool: Zawrs wrs.nto/wrs.sto wait-on-reservation-set (needs isa.atomics)

Table 2.1 continued on next page

Table 2.1 continued from previous page

Configuration key	This build	Meaning / valid values
`isa.zabha`	`false`	bool: Zabha byte/halfword AMOs; needs isa.atomics
`isa.zacas`	`false`	bool: Zacas compare-and-swap (amocas.w/.b/.h); needs isa.atomics
`isa.zicboz`	`false`	bool: Zicboz cbo.zero cache-block zero (64-byte block)
`isa.zcmp`	`false`	bool: Zcmp compressed push/pop and register moves; needs isa.compressed
`isa.zcmt`	`false`	bool: Zcmt compressed table jump and the jvt CSR; needs isa.compressed
`isa.zbkb`	`false`	bool: Zbkb crypto bit-manipulation (pack/brev8/zip/unzip)
`isa.zbkc`	`false`	bool: Zbkc carry-less multiply (clmul/clmulh)
`isa.zbkx`	`false`	bool: Zbkx crossbar permute (xperm8/xperm4)
`isa.zkn`	`false`	bool: Zkn AES+SHA (Zknd+Zkne+Zknh)
`isa.zfinx`	`false`	bool: Zfinx single-precision floating point in the x registers
`priv.trapCsr`	`true`	bool: standard M-mode trap CSRs and delivery; firmware opts in via mtrapctl
`priv.umode`	`false`	bool: user mode (privilege register, MPP stack, mcounteren); needs priv.trapCsr
`priv.pmp`	`false`	bool: PMP (Smpmp) CSR bank and match unit; needs priv.umode
`priv.pmpEntries`	`16`	int, 8 or 16: PMP entry count when priv.pmp is true (default 16)
`debug.enable`	`true`	bool: debug mode, the Debug Module and the JTAG DTM; needs priv.trapCsr
`peripherals.dmaChannels`	`4`	int, 2 or 4: DMA0 channel count when peripherals.dma is true (default 4)
`peripherals.trngRings`	`8`	int, 4 or 8: TRNG0 ring count when peripherals.trng is true (default 8)
`peripherals.pinmuxPads`	`19`	int, 8 to 120: pmx pad count when peripherals.pinmux is true (default 19)

Table 2.2: Derived geometry of this configuration (computed, not configurable)

Derived value	This build	Notes
ISA string (march)	`rv32imac_zba_zbb_zbs_zbc`	Advertised in the read-only `misa` CSR
Shared-window address width	16 bits (words)	Arbiter/tile word-address width; the window is `0x0` to $2^{w+2} - 1$
Shared RAM banks	4×16 KiB	One SRAM macro per bank behind the arbiter

Table 2.2 continued on next page

Table 2.2 continued from previous page

Derived value	This build	Notes
Extended flash base	`0x40000`	First address decoded to the SPI-flash XIP path (hart 0 only); strictly the complement of the shared window
Interrupt vectors	`125`	Vectors 83/84 are the CLINT software/timer interrupts
CLINT `MTIME`	`0x5020`	Layout is hart-count-derived: `MSIPx` at `0x5000` + $4h$, `MTIMECMPx` from `0x5030`
Boot-loader mailbox rows	`0x10500 + 0x10*hartid`	Tile-loading rows {SRC, LEN, ENTRY} consumed by the boot ROM
Stack pointer at reset	`0xA000`	Top of each hart's private TCM, growing down
Peripheral count	`35`	Instantiated peripherals (including CLINT/MUTEX/IRQROUTER)

Table 2.3: Per-hart-class ISA of this build. Each row is a hart class the generator emitted; the ISA string is what that class advertises in its read-only `misa` CSR. A minimal tile class is rv32iac and nothing more: the generated `TILE_ENABLE_*` constants drop M, Zb, every Z-series extension, U-mode and PMP, while A, C, the trap CSRs and debug are kept (the tiles run the shared-fabric LR/SC and AMO locking, and the Debug Module halts them). Zicntr, where the chip has it, is kept on both classes because `cycle` and `instret` exist on every hart unconditionally and the knob gates no hardware. Code that more than one class executes must be built for the narrowest row.

Hart class	Harts	ISA string	Privilege features
Orchestrator (Soft core)	`0`	`rv32imac_zba_zbb_zbs_zbc`	M-mode, trap CSRs, debug
Tile (Hardened hart_tile macro)	`1-4`	`rv32iac`	M-mode, trap CSRs, debug

Every image more than one hart executes is built for the narrowest row, `rv32iac`: the boot ROM all 5 harts fetch at reset, the debug entry trampoline the Debug Module plants for whichever hart it halts, and any library linked into more than one image. A wider `-march` on one of those lets the assembler emit an instruction a tile traps as illegal, so the build flags are pinned to this string and graded on the linked ELF by `//software/testtools:tile_isa_test`. Images that run on hart 0 alone keep the full ISA.

3 Package and pins

Note: The package model for this configuration (LQFP-176) has not been finalised for PentaWoundPt2mm; the bonding shown here is the planned assignment, not a confirmed bond-out. Pin assignments, package type, and pin count are subject to change.

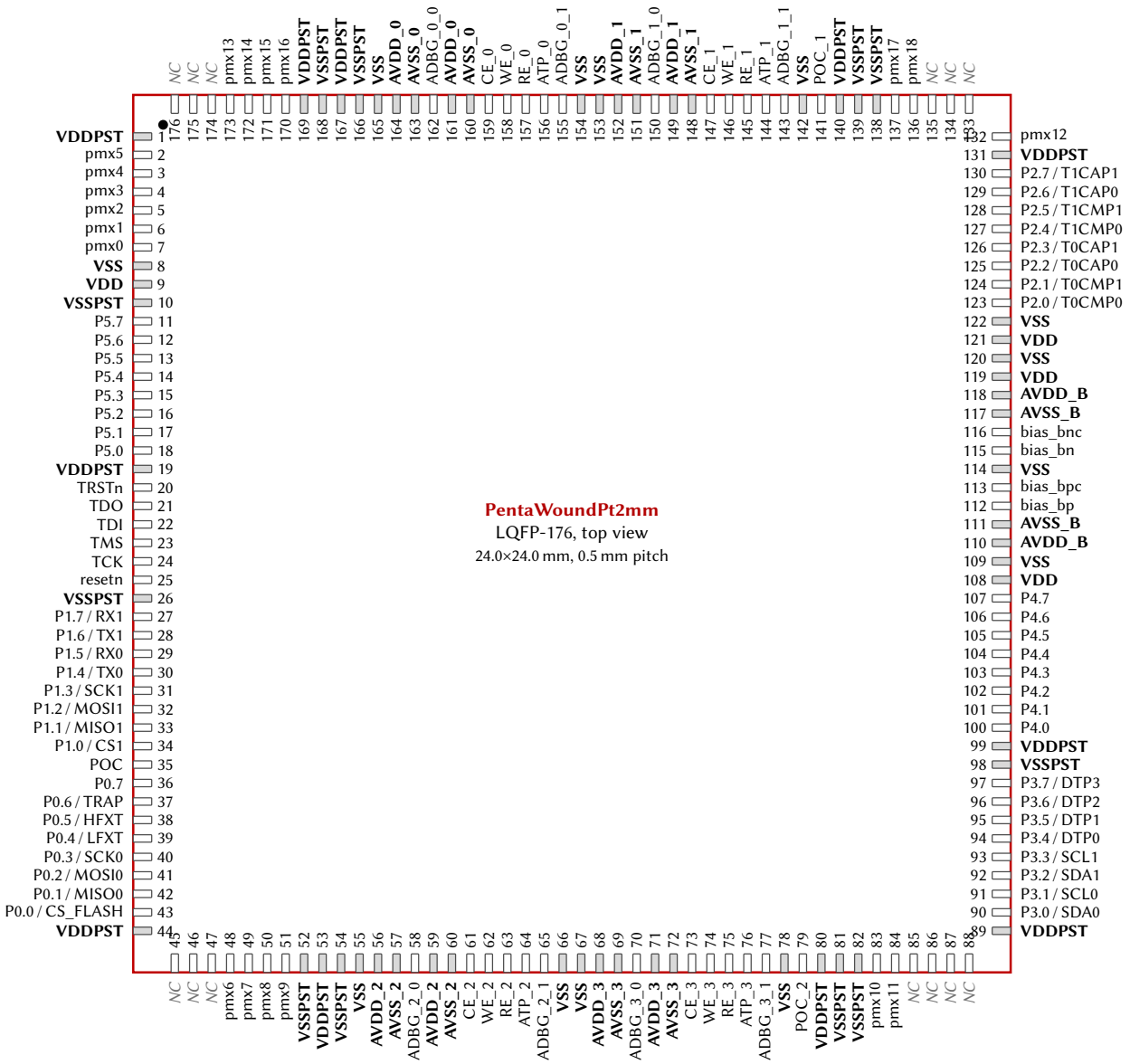


Figure 3.1: LQFP-176 pinout, top view (24.0 × 24.0 mm, 0.5 mm pitch); GPIO pins carry their AF0 function and power-rail pins are shaded.

The chip package is a LQFP-176 with dimensions 24.0 × 24.0 mm and pitch 0.5 mm. The package footprint, seen from the top, is shown in Figure 3.1, and the pins are listed in Table 3.1. The pin ordering, side assignment, and power domains in both the figure and the table are derived from the generator's

package model and are also exported machine-readably to `config/PadRing.json` at every `make chip` build.

Table 3.1: Package Pins

#	Pin	I/O	Power Domain	Power Pins
1	VDDPST	Power Input	Digital I/O (3.3 V)	VDDPST/VSSPST
2	pmx5	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
3	pmx4	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
4	pmx3	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
5	pmx2	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
6	pmx1	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
7	pmx0	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
8	VSS	Power Input	Digital Core (1.0 V)	VDD/VSS
9	VDD	Power Input	Digital Core (1.0 V)	VDD/VSS
10	VSSPST	Power Input	Digital I/O (3.3 V)	VDDPST/VSSPST
11	P5.7	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
12	P5.6	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
13	P5.5	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
14	P5.4	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
15	P5.3	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
16	P5.2	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
17	P5.1	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
18	P5.0	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
19	VDDPST	Power Input	Digital I/O (3.3 V)	VDDPST/VSSPST
20	TRSTn	Input	Digital I/O (3.3 V)	VDDPST/VSSPST
21	TDO	Output	Digital I/O (3.3 V)	VDDPST/VSSPST
22	TDI	Input	Digital I/O (3.3 V)	VDDPST/VSSPST
23	TMS	Input	Digital I/O (3.3 V)	VDDPST/VSSPST
24	TCK	Input	Digital I/O (3.3 V)	VDDPST/VSSPST
25	resetn	Input	Digital I/O (3.3 V)	VDDPST/VSSPST
26	VSSPST	Power Input	Digital I/O (3.3 V)	VDDPST/VSSPST
27	P1.7	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
28	P1.6	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
29	P1.5	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
30	P1.4	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
31	P1.3	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
32	P1.2	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
33	P1.1	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
34	P1.0	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
35	POC	Input	Digital I/O (3.3 V)	VDDPST/VSSPST
36	P0.7	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
37	P0.6	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
38	P0.5	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
39	P0.4	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
40	P0.3	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
41	P0.2	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST

Table 3.1 continued on next page

Table 3.1 continued from previous page

#	Pin	I/O	Power Domain	Power Pins
42	P0.1	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
43	P0.0	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
44	VDDPST	Power Input	Digital I/O (3.3 V)	VDDPST/VSSPST
45	*NC*	-	-	-
46	*NC*	-	-	-
47	*NC*	-	-	-
48	pmx6	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
49	pmx7	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
50	pmx8	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
51	pmx9	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
52	VSSPST	Power Input	Digital I/O (3.3 V)	VDDPST/VSSPST
53	VDDPST	Power Input	Digital I/O (3.3 V)	VDDPST/VSSPST
54	VSSPST	Power Input	Digital I/O (3.3 V)	VDDPST/VSSPST
55	VSS	Power Input	Digital Core (1.0 V)	VDD/VSS
56	AVDD_2	Power Input	Analog2 (2.5 V)	AVDD_2/AVSS_2
57	AVSS_2	Power Input	Analog2 (2.5 V)	AVDD_2/AVSS_2
58	ADBG_2_0	Input/Output	Analog2 (2.5 V)	AVDD_2/AVSS_2
59	AVDD_2	Power Input	Analog2 (2.5 V)	AVDD_2/AVSS_2
60	AVSS_2	Power Input	Analog2 (2.5 V)	AVDD_2/AVSS_2
61	CE_2	Input/Output	Analog2 (2.5 V)	AVDD_2/AVSS_2
62	WE_2	Input/Output	Analog2 (2.5 V)	AVDD_2/AVSS_2
63	RE_2	Input/Output	Analog2 (2.5 V)	AVDD_2/AVSS_2
64	ATP_2	Input/Output	Analog2 (2.5 V)	AVDD_2/AVSS_2
65	ADBG_2_1	Input/Output	Analog2 (2.5 V)	AVDD_2/AVSS_2
66	VSS	Power Input	Digital Core (1.0 V)	VDD/VSS
67	VSS	Power Input	Digital Core (1.0 V)	VDD/VSS
68	AVDD_3	Power Input	Analog3 (2.5 V)	AVDD_3/AVSS_3
69	AVSS_3	Power Input	Analog3 (2.5 V)	AVDD_3/AVSS_3
70	ADBG_3_0	Input/Output	Analog3 (2.5 V)	AVDD_3/AVSS_3
71	AVDD_3	Power Input	Analog3 (2.5 V)	AVDD_3/AVSS_3
72	AVSS_3	Power Input	Analog3 (2.5 V)	AVDD_3/AVSS_3
73	CE_3	Input/Output	Analog3 (2.5 V)	AVDD_3/AVSS_3
74	WE_3	Input/Output	Analog3 (2.5 V)	AVDD_3/AVSS_3
75	RE_3	Input/Output	Analog3 (2.5 V)	AVDD_3/AVSS_3
76	ATP_3	Input/Output	Analog3 (2.5 V)	AVDD_3/AVSS_3
77	ADBG_3_1	Input/Output	Analog3 (2.5 V)	AVDD_3/AVSS_3
78	VSS	Power Input	Digital Core (1.0 V)	VDD/VSS
79	POC_2	Input	Digital I/O (3.3 V)	VDDPST/VSSPST
80	VDDPST	Power Input	Digital I/O (3.3 V)	VDDPST/VSSPST
81	VSSPST	Power Input	Digital I/O (3.3 V)	VDDPST/VSSPST
82	VSSPST	Power Input	Digital I/O (3.3 V)	VDDPST/VSSPST
83	pmx10	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
84	pmx11	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
85	*NC*	-	-	-
86	*NC*	-	-	-

Table 3.1 continued on next page

Table 3.1 continued from previous page

#	Pin	I/O	Power Domain	Power Pins
87	*NC*	-	-	-
88	*NC*	-	-	-
89	VDDPST	Power Input	Digital I/O (3.3 V)	VDDPST/VSSPST
90	P3.0	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
91	P3.1	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
92	P3.2	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
93	P3.3	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
94	P3.4	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
95	P3.5	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
96	P3.6	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
97	P3.7	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
98	VSSPST	Power Input	Digital I/O (3.3 V)	VDDPST/VSSPST
99	VDDPST	Power Input	Digital I/O (3.3 V)	VDDPST/VSSPST
100	P4.0	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
101	P4.1	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
102	P4.2	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
103	P4.3	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
104	P4.4	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
105	P4.5	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
106	P4.6	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
107	P4.7	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
108	VDD	Power Input	Digital Core (1.0 V)	VDD/VSS
109	VSS	Power Input	Digital Core (1.0 V)	VDD/VSS
110	AVDD_B	Power Input	AnalogBias (2.5 V)	AVDD_B/AVSS_B
111	AVSS_B	Power Input	AnalogBias (2.5 V)	AVDD_B/AVSS_B
112	bias_bp	Input/Output	AnalogBias (2.5 V)	AVDD_B/AVSS_B
113	bias_bpc	Input/Output	AnalogBias (2.5 V)	AVDD_B/AVSS_B
114	VSS	Power Input	Digital Core (1.0 V)	VDD/VSS
115	bias_bn	Input/Output	AnalogBias (2.5 V)	AVDD_B/AVSS_B
116	bias_bnc	Input/Output	AnalogBias (2.5 V)	AVDD_B/AVSS_B
117	AVSS_B	Power Input	AnalogBias (2.5 V)	AVDD_B/AVSS_B
118	AVDD_B	Power Input	AnalogBias (2.5 V)	AVDD_B/AVSS_B
119	VDD	Power Input	Digital Core (1.0 V)	VDD/VSS
120	VSS	Power Input	Digital Core (1.0 V)	VDD/VSS
121	VDD	Power Input	Digital Core (1.0 V)	VDD/VSS
122	VSS	Power Input	Digital Core (1.0 V)	VDD/VSS
123	P2.0	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
124	P2.1	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
125	P2.2	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
126	P2.3	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
127	P2.4	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
128	P2.5	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
129	P2.6	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
130	P2.7	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
131	VDDPST	Power Input	Digital I/O (3.3 V)	VDDPST/VSSPST

Table 3.1 continued on next page

Table 3.1 continued from previous page

#	Pin	I/O	Power Domain	Power Pins
132	pmx12	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
133	*NC*	-	-	-
134	*NC*	-	-	-
135	*NC*	-	-	-
136	pmx18	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
137	pmx17	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
138	VSSPST	Power Input	Digital I/O (3.3 V)	VDDPST/VSSPST
139	VSSPST	Power Input	Digital I/O (3.3 V)	VDDPST/VSSPST
140	VDDPST	Power Input	Digital I/O (3.3 V)	VDDPST/VSSPST
141	POC_1	Input	Digital I/O (3.3 V)	VDDPST/VSSPST
142	VSS	Power Input	Digital Core (1.0 V)	VDD/VSS
143	ADBG_1_1	Input/Output	Analog1 (2.5 V)	AVDD_1/AVSS_1
144	ATP_1	Input/Output	Analog1 (2.5 V)	AVDD_1/AVSS_1
145	RE_1	Input/Output	Analog1 (2.5 V)	AVDD_1/AVSS_1
146	WE_1	Input/Output	Analog1 (2.5 V)	AVDD_1/AVSS_1
147	CE_1	Input/Output	Analog1 (2.5 V)	AVDD_1/AVSS_1
148	AVSS_1	Power Input	Analog1 (2.5 V)	AVDD_1/AVSS_1
149	AVDD_1	Power Input	Analog1 (2.5 V)	AVDD_1/AVSS_1
150	ADBG_1_0	Input/Output	Analog1 (2.5 V)	AVDD_1/AVSS_1
151	AVSS_1	Power Input	Analog1 (2.5 V)	AVDD_1/AVSS_1
152	AVDD_1	Power Input	Analog1 (2.5 V)	AVDD_1/AVSS_1
153	VSS	Power Input	Digital Core (1.0 V)	VDD/VSS
154	VSS	Power Input	Digital Core (1.0 V)	VDD/VSS
155	ADBG_0_1	Input/Output	Analog0 (2.5 V)	AVDD_0/AVSS_0
156	ATP_0	Input/Output	Analog0 (2.5 V)	AVDD_0/AVSS_0
157	RE_0	Input/Output	Analog0 (2.5 V)	AVDD_0/AVSS_0
158	WE_0	Input/Output	Analog0 (2.5 V)	AVDD_0/AVSS_0
159	CE_0	Input/Output	Analog0 (2.5 V)	AVDD_0/AVSS_0
160	AVSS_0	Power Input	Analog0 (2.5 V)	AVDD_0/AVSS_0
161	AVDD_0	Power Input	Analog0 (2.5 V)	AVDD_0/AVSS_0
162	ADBG_0_0	Input/Output	Analog0 (2.5 V)	AVDD_0/AVSS_0
163	AVSS_0	Power Input	Analog0 (2.5 V)	AVDD_0/AVSS_0
164	AVDD_0	Power Input	Analog0 (2.5 V)	AVDD_0/AVSS_0
165	VSS	Power Input	Digital Core (1.0 V)	VDD/VSS
166	VSSPST	Power Input	Digital I/O (3.3 V)	VDDPST/VSSPST
167	VDDPST	Power Input	Digital I/O (3.3 V)	VDDPST/VSSPST
168	VSSPST	Power Input	Digital I/O (3.3 V)	VDDPST/VSSPST
169	VDDPST	Power Input	Digital I/O (3.3 V)	VDDPST/VSSPST
170	pmx16	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
171	pmx15	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
172	pmx14	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
173	pmx13	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
174	*NC*	-	-	-
175	*NC*	-	-	-

Table 3.1 continued on next page

Table 3.1 continued from previous page

#	Pin	I/O	Power Domain	Power Pins
176	*NC*	-	-	-

3.1 GPIO pins and alternate functions

Most of the digital I/O pins are GPIO pins. A GPIO pin is in either primary (GPIO) mode or alternate function (peripheral) mode; see the GPIOx chapter (Chapter 12). Table 3.2 lists every GPIO pin with its primary function, its reset-selected alternate function (AF0), and the reset state of its PxSEL, PxOUT, PxDIR, and PxREN bits. The remaining alternate functions each pin can select (AF1 to AF7) are in the alternate-function matrix, Table 3.3.

Table 3.2: GPIO Pin Functions and Reset Values

	Primary	Alternate	*Reset Values*			
Pin	Function	Function (AF0)	SEL	OUT	DIR	REN
P0.0	GPIO0	CS_FLASH	0 *(Primary)*	1 *(High)*	1 *(Output)*	0 *(Disabled)*
P0.1	GPIO1	MISO0	1 *(Alternate)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P0.2	GPIO2	MOSI0	1 *(Alternate)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P0.3	GPIO3	SCK0	1 *(Alternate)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P0.4	GPIO4	LFXT	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P0.5	GPIO5	HFXT	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P0.6	GPIO6	TRAP	1 *(Alternate)*	0 *(Low)*	1 *(Output)*	0 *(Disabled)*
P0.7	BOOT	-	0 *(Primary)*	0 *(Low)*	0 *(Input)*	1 *(Enabled)*
P1.0	GPIO8	CS1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P1.1	GPIO9	MISO1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P1.2	GPIO10	MOSI1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P1.3	GPIO11	SCK1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P1.4	GPIO12	TX0	1 *(Alternate)*	0 *(Low)*	1 *(Output)*	0 *(Disabled)*
P1.5	GPIO13	RX0	1 *(Alternate)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P1.6	GPIO14	TX1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P1.7	GPIO15	RX1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P2.0	GPIO16	T0CMP0	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P2.1	GPIO17	T0CMP1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P2.2	GPIO18	T0CAP0	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P2.3	GPIO19	T0CAP1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P2.4	GPIO20	T1CMP0	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P2.5	GPIO21	T1CMP1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P2.6	GPIO22	T1CAP0	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P2.7	GPIO23	T1CAP1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P3.0	GPIO24	SDA0	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P3.1	GPIO25	SCL0	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P3.2	GPIO26	SDA1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P3.3	GPIO27	SCL1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P3.4	GPIO28	DTP0	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P3.5	GPIO29	DTP1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P3.6	GPIO30	DTP2	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*

Table 3.2 continued on next page

Table 3.2 continued from previous page

Pin	Primary Function	Alternate Function (AF0)	*Reset Values* SEL	OUT	DIR	REN
P3.7	GPIO31	DTP3	0 *(Primary)*	0 *(Low)*	0 *(Input)*	1 *(Enabled)*
P4.0	GPIO32	-	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P4.1	GPIO33	-	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P4.2	GPIO34	-	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P4.3	GPIO35	-	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P4.4	GPIO36	-	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P4.5	GPIO37	-	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P4.6	GPIO38	-	0 *(Primary)*	0 *(Low)*	0 *(Input)*	1 *(Enabled)*
P4.7	GPIO39	-	0 *(Primary)*	0 *(Low)*	0 *(Input)*	1 *(Enabled)*
P5.0	GPIO40	-	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P5.1	GPIO41	-	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P5.2	GPIO42	-	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P5.3	GPIO43	-	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P5.4	GPIO44	-	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P5.5	GPIO45	-	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P5.6	GPIO46	-	0 *(Primary)*	0 *(Low)*	0 *(Input)*	1 *(Enabled)*
P5.7	GPIO47	-	0 *(Primary)*	0 *(Low)*	0 *(Input)*	1 *(Enabled)*

When a pin is in alternate function mode (its PxSEL bit is 1), its 3-bit PxAFS field selects which of up to eight alternate functions (AF0 to AF7) drives the pad. Table 3.3 lists, for every GPIO pin, the function selected by each PxAFS value. Plane 0 is the pin's AF0 function and is the reset selection; planes marked **Hi-Z** have no function assigned for that pin and configure the pad as a high-impedance input. The superscript on each function denotes its direction (^{in} input, ^{out} output, ^{io} bidirectional), and a † marks each pin's reset plane (its PxAFS reset value). While a pin is in primary mode the selected plane has no effect on the pad, but it still routes relocatable peripheral inputs (Chapter 12).

Table 3.3: GPIO Alternate Function Selection (PxAFS)

Pin	PxAFS *field value (selected AF plane)* 0 (AF0)	1	2	3	4	5	6	7
P0.0	CS_FLASH$^{\text{out}\dagger}$	TX0^{out}	TX1^{out}	T0CMP0^{out}	T0CMP1^{out}	T1CMP0^{out}	T1CMP1^{out}	SCK1^{out}
P0.1	MISO0$^{\text{io}\dagger}$	TX1^{out}	T0CMP0^{out}	T0CMP1^{out}	T1CMP0^{out}	T1CMP1^{out}	SCK1^{out}	MOSI1^{out}
P0.2	MOSI0$^{\text{out}\dagger}$	T0CMP0^{out}	T0CMP1^{out}	T1CMP0^{out}	T1CMP1^{out}	SCK1^{out}	MOSI1^{out}	TX0^{out}
P0.3	SCK0$^{\text{out}\dagger}$	T0CMP1^{out}	T1CMP0^{out}	T1CMP1^{out}	SCK1^{out}	MOSI1^{out}	TX0^{out}	TX1^{out}
P0.4	LFXT$^{\text{io}\dagger}$	T1CMP0^{out}	T1CMP1^{out}	SCK1^{out}	MOSI1^{out}	TX0^{out}	TX1^{out}	T0CMP0^{out}
P0.5	HFXT$^{\text{io}\dagger}$	T1CMP1^{out}	SCK1^{out}	MOSI1^{out}	TX0^{out}	TX1^{out}	T0CMP0^{out}	T0CMP1^{out}
P0.6	TRAP$^{\text{out}\dagger}$	SCK1^{out}	MOSI1^{out}	TX0^{out}	TX1^{out}	T0CMP0^{out}	T0CMP1^{out}	T1CMP0^{out}
P0.7	*Hi-Z*†	MOSI1^{out}	TX0^{out}	TX1^{out}	T0CMP0^{out}	T0CMP1^{out}	T1CMP0^{out}	T1CMP1^{out}
P1.0	CS1$^{\text{in}\dagger}$	T0CMP0^{out}	TX1^{out}	T0CMP1^{out}	T1CMP0^{out}	T1CMP1^{out}	SCK1^{out}	MOSI1^{out}
P1.1	MISO1$^{\text{io}\dagger}$	T0CMP1^{out}	T0CMP0^{out}	T1CMP0^{out}	T1CMP1^{out}	SCK1^{out}	MOSI1^{out}	TX0^{out}
P1.2	MOSI1$^{\text{io}\dagger}$	T1CMP0^{out}	T1CMP1^{out}	SCK1^{out}	TX0^{out}	TX1^{out}	T0CMP0^{out}	T0CMP1^{out}
P1.3	SCK1$^{\text{io}\dagger}$	T1CMP1^{out}	MOSI1^{out}	TX0^{out}	TX1^{out}	T0CMP0^{out}	T0CMP1^{out}	T1CMP0^{out}
P1.4	TX0$^{\text{out}\dagger}$	SDA1^{io}	SCK1^{out}	MOSI1^{out}	TX1^{out}	T0CMP0^{out}	T0CMP1^{out}	T1CMP0^{out}
P1.5	RX0$^{\text{io}\dagger}$	SCL1^{io}	T1CMP1^{out}	SCK1^{out}	MOSI1^{out}	TX0^{out}	TX1^{out}	T0CMP0^{out}
P1.6	TX1$^{\text{out}\dagger}$	SDA0^{io}	TX0^{out}	T0CMP0^{out}	T0CMP1^{out}	T1CMP0^{out}	T1CMP1^{out}	SCK1^{out}
P1.7	RX1$^{\text{io}\dagger}$	SCL0^{io}	MOSI1^{out}	TX0^{out}	TX1^{out}	T0CMP0^{out}	T0CMP1^{out}	T1CMP0^{out}
P2.0	T0CMP0$^{\text{out}\dagger}$	TX1^{out}	SCK1^{out}	MOSI1^{out}	TX0^{out}	T0CMP1^{out}	T1CMP0^{out}	T1CMP1^{out}

Table 3.3 continued on next page

Table 3.3 continued from previous page

	PxAFS *field value (selected AF plane)*							
Pin	**0 (AF0)**	**1**	**2**	**3**	**4**	**5**	**6**	**7**
P2.1	T0CMP1^{out†}	RX1^{io}	T1CMP0^{out}	T1CMP1^{out}	SCK1^{out}	MOSI1^{out}	TX0^{out}	TX1^{out}
P2.2	T0CAP0^{in†}	SDA1^{io}	PWM0^{out}	T0CMP1^{out}	T1CMP0^{out}	T1CMP1^{out}	SCK1^{out}	MOSI1^{out}
P2.3	T0CAP1^{in†}	SCL1^{io}	PWM1^{out}	T1CMP0^{out}	T1CMP1^{out}	SCK1^{out}	MOSI1^{out}	TX0^{out}
P2.4	T1CMP0^{out†}	TX0^{out}	T0CMP1^{out}	T1CMP1^{out}	SCK1^{out}	MOSI1^{out}	TX1^{out}	T0CMP0^{out}
P2.5	T1CMP1^{out†}	RX0^{io}	TX0^{out}	TX1^{out}	T0CMP0^{out}	T0CMP1^{out}	T1CMP0^{out}	SCK1^{out}
P2.6	T1CAP0^{in†}	SDA0^{io}	SCK1^{out}	MOSI1^{out}	TX0^{out}	TX1^{out}	T0CMP0^{out}	T0CMP1^{out}
P2.7	T1CAP1^{in†}	SCL0^{io}	MOSI1^{out}	TX0^{out}	TX1^{out}	T0CMP0^{out}	T0CMP1^{out}	T1CMP0^{out}
P3.0	SDA0^{io†}	T0CAP0^{in}	TX0^{out}	TX1^{out}	T0CMP0^{out}	T0CMP1^{out}	T1CMP0^{out}	T1CMP1^{out}
P3.1	SCL0^{io†}	T0CAP1^{in}	TX1^{out}	T0CMP0^{out}	T0CMP1^{out}	T1CMP0^{out}	T1CMP1^{out}	SCK1^{out}
P3.2	SDA1^{io†}	T1CAP0^{in}	T0CMP0^{out}	T0CMP1^{out}	T1CMP0^{out}	T1CMP1^{out}	SCK1^{out}	MOSI1^{out}
P3.3	SCL1^{io†}	T1CAP1^{in}	T0CMP1^{out}	T1CMP0^{out}	T1CMP1^{out}	SCK1^{out}	MOSI1^{out}	TX0^{out}
P3.4	DTP0^{io†}	T0CMP0^{out}	TX0^{out}	TX1^{out}	T0CMP1^{out}	T1CMP0^{out}	T1CMP1^{out}	SCK1^{out}
P3.5	DTP1^{io†}	T0CMP1^{out}	RX0^{io}	T0CMP0^{out}	T1CMP0^{out}	T1CMP1^{out}	SCK1^{out}	MOSI1^{out}
P3.6	DTP2^{io†}	T1CMP0^{out}	T0CMP0^{out}	T0CMP1^{out}	T1CMP1^{out}	SCK1^{out}	MOSI1^{out}	MISO1^{io}
P3.7	DTP3^{io†}	T1CMP1^{out}	OW_DQ^{io}	T1CMP0^{out}	SCK1^{out}	MOSI1^{out}	TX0^{out}	TX1^{out}
P4.0	*Hi-Z*†	QSPI_SCK^{out}	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*
P4.1	*Hi-Z*†	QSPI_CS^{out}	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*
P4.2	*Hi-Z*†	QSPI_IO0^{io}	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*
P4.3	*Hi-Z*†	QSPI_IO1^{io}	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*
P4.4	*Hi-Z*†	QSPI_IO2^{io}	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*
P4.5	*Hi-Z*†	QSPI_IO3^{io}	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*
P4.6	*Hi-Z*†	I3C_SDA^{io}	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*
P4.7	*Hi-Z*†	I3C_SCL^{io}	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*
P5.0	*Hi-Z*	NFC_RF_CLK^{in†}	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*
P5.1	*Hi-Z*†	NFC_RF_RX^{in}	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*
P5.2	*Hi-Z*†	NFC_FIELD_DETECT^{in}	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*
P5.3	*Hi-Z*†	NFC_RF_TXMOD^{out}	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*
P5.4	*Hi-Z*†	NFC_RF_TX_EN^{out}	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*
P5.5	*Hi-Z*†	NFC_AFE_EN^{out}	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*
P5.6	*Hi-Z*†	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*
P5.7	*Hi-Z*†	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*	*Hi-Z*

4 Memory map

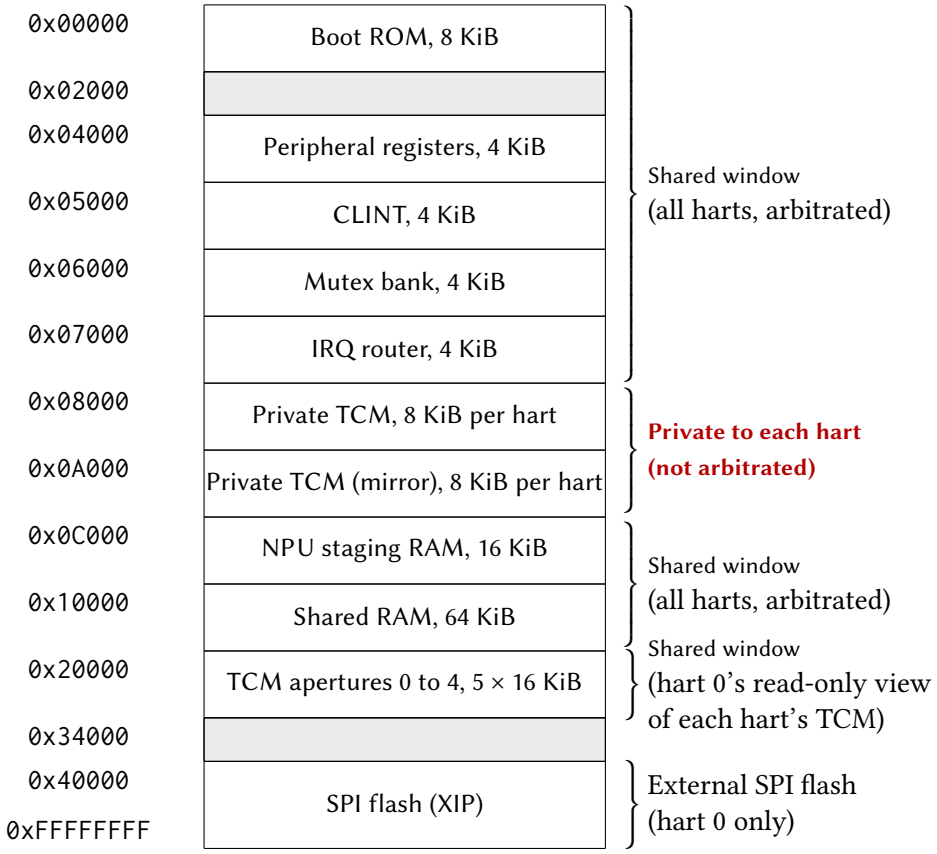


Figure 4.1: PentaWoundPt2mm address space: one gapless 32-bit column, drawn once for all five harts; grey rows are unmapped and read zero.

The 32-bit memory bus is a single von Neumann address space: the ROM, the RAMs and the memory-mapped peripheral registers are all reached through one bus, and a pointer can address any of them. An access to an unmapped address reads zero, and a write to one is ignored; no bus error is raised.

The ROM holds the boot loader and the ROM monitor and is read-only. Each RAM can be read and written and may hold both code and data. The chip carries no on-die non-volatile memory: it boots from an external SPI flash device, and hart 0, and only hart 0, can additionally read that flash directly, and execute in place from it, through the extended-flash window at `0x40000` and above (the last region in Figure 4.1; see Section 13.4.6). At reset every hart fetches from the boot ROM at `0x00000` and dispatches on its hart ID: hart 0 copies the application image out of the flash into memory and enters it, and the remaining harts park until hart 0 loads and launches them (Section 5.3). Each RAM and ROM block can be individually powered off and on through the `SYSTEM` peripheral; a block that is powered off draws less static power, does not respond, and a RAM loses its contents.

Figure 4.1 is a single address column drawn for all five harts, and its braces name who reaches each region. Everything below `0x40000` other than the private band is the shared window: every hart reaches it through the arbiter, and all five harts see the same memory there (Chapter 5). The private band is the exception: at those addresses a hart reaches its own TCM instead of the shared bus, so the same address

names a different 8 KiB of RAM in each hart, and no hart can reach another's directly.

Table 4.1 is the same column as a table, one row per decoded region with its access attributes; the register-level index of every peripheral is Appendix B.

Table 4.1: Address map (R = readable, W = writable, X = executable; unmapped rows read zero)

Start	End	Size	Region	Attr.
0x00000000	0x00001FFF	8 KiB	Boot ROM	R/X
0x00002000	0x00003FFF	8 KiB	Unmapped (reads zero)	-
0x00004000	0x00004FFF	4 KiB	Peripheral registers	R/W
0x00005000	0x00005FFF	4 KiB	CLINT	R/W
0x00006000	0x00006FFF	4 KiB	Mutex bank	R/W
0x00007000	0x00007FFF	4 KiB	IRQ router	R/W
0x00008000	0x00009FFF	8 KiB	Private TCM (private to each hart)	R/W/X
0x0000A000	0x0000BFFF	8 KiB	Private TCM (mirror) (private to each hart; Alias: the band decodes 16 KiB and the array answers 8 KiB, so it repeats 2 times)	R/W/X
0x0000C000	0x0000FFFF	16 KiB	NPU staging RAM	R/W/X
0x00010000	0x0001FFFF	64 KiB	Shared RAM	R/W/X
0x00020000	0x00023FFF	16 KiB	TCM aperture 0 (hart 0 read-only view of a hart's TCM)	R
0x00024000	0x00027FFF	16 KiB	TCM aperture 1 (hart 0 read-only view of a hart's TCM)	R
0x00028000	0x0002BFFF	16 KiB	TCM aperture 2 (hart 0 read-only view of a hart's TCM)	R
0x0002C000	0x0002FFFF	16 KiB	TCM aperture 3 (hart 0 read-only view of a hart's TCM)	R
0x00030000	0x00033FFF	16 KiB	TCM aperture 4 (hart 0 read-only view of a hart's TCM)	R
0x00034000	0x0003FFFF	48 KiB	Unmapped (reads zero)	-
0x00040000	0xFFFFFFFF	4194048 KiB	SPI flash (XIP) (hart 0 only; ; (addr + SPI0FOS) mod 2{24)	R/X

Every register owns one word-aligned 4-byte slot, and registers are 32 bits wide except those of GPIOx (PxDIR, PxIE, PxIES, PxIF, PxIN, PxOUT, PxOUTC, PxOUTS, PxOUTT, PxREN, PxSEL, PxTASK); I2Cx (I2CxAMR, I2CxAR, I2CxFCR, I2CxMRX, I2CxMTX, I2CxSR, I2CxSRX, I2CxSTX); SPIx (SPIxSR); SYSTEM (BLOCKPWR, CLKDIVCR, CRCDATA, CRCSTATE, DCO0BIAS, DCO1BIAS, SYSCLKCR, WDTCR, WDTSR); TIMERx (TIMxSR); UARTx (UARTxBR, UARTxCR, UARTxRX, UARTxSR, UARTxTX), which are narrower and occupy the low-order bits of their slot. The TCM apertures are the one sanctioned way across that line: hart 0, the orchestrator, reads hart h's private TCM through the aperture at 0x20000 + h × 0x4000. The aperture stride is 16 KiB whatever the TCM size, so a 8 KiB TCM is mirrored inside its aperture. The apertures are read-only, and any other hart reading them gets zeros (Chapter 5). The extended-flash window is likewise hart 0's alone; the other harts' accesses in that range read zeros without stalling.

The interrupt vector table (IVT) begins at 0x8000, the base of each hart's TCM, and holds one 4-byte jump instruction per vector (Chapter 9).

Peripheral registers occupy one 4-byte slot each, at 4-byte aligned offsets from the instance base address, whatever their width. The GPIOx port registers are 8 bits wide, one bit per pin, and occupy byte lane 0 of their slot; a 32-bit register uses the whole slot. The register index in Appendix B gives the width of

every register.

A read of a register that the peripheral's own clock domain writes, a status word or a receive buffer, returns a value up to two bus-clock edges older than the access, because the peripheral carries such a word into the bus clock through a synchroniser rather than sampling it on the select; the value is always a coherent snapshot of one register taken at one edge, a transfer launched by a write is already visible in that peripheral's busy bit on the next access, and a write-1-to-clear flag reads back cleared on the next access.

5 Multi-core architecture

PentaWoundPt2mm is a five-hart (five-core) multiprocessor built from five identical VestaRV cores. Each hart is a complete, single-issue, in-order RV32IMAC processor with its own private *tightly-coupled memory* (TCM), so the common case (instruction fetch and private data access) never contends with the other harts and runs at full core clock. Everything else in the low address space is *shared*: a single boot ROM, all peripherals, the inter-processor interrupt hardware (`CLINT`), a hardware mutex bank, an interrupt router, the NPU staging RAM, and 64 KiB of bulk RAM, all reached through one serializing round-robin arbiter. The main features of the multi-core architecture are listed below:

- 5 identical VestaRV harts, each identified by the read-only `mhartid` CSR (address `0xF14`)
- One shared boot ROM at `0x0`: all five harts reset to PC `0x0` and dispatch on `mhartid` (see Section 5.3)
- 8 KiB of private TCM per hart at `0x8000` to `0x9FFF` (the same local address in every hart), holding each hart's vector table, code, data, and stack
- All peripherals shared by all five harts at their legacy `0x4000`-page addresses, with the arbiter guaranteeing register-access atomicity
- 64 KiB of shared bulk RAM at `0x10000` to `0x1FFFF` for locks, mailboxes, staged tile programs, and inter-hart data
- `CLINT` with per-hart software interrupts (inter-processor interrupts) and a shared 64-bit `mtime` with per-hart compare registers
- 16 single-instruction hardware mutexes (`MUTEX` bank)
- Claim/complete peripheral interrupt delivery (`IRQROUTER`): any peripheral interrupt can be routed to any hart over one external-interrupt wire per hart, with exactly-once claiming
- `LR`/`SC` and `AMO` atomic instructions supported to shared RAM, including sub-word shared stores (byte lanes)
- Instruction fetch from shared memory is supported (the boot ROM is fetched this way by construction); parked harts sleep via the custom `sleep` instruction and are woken by a software interrupt; no busy-wait power burn

5.1 Harts and Roles

All five harts are instances of the same *hart tile*: an unmodified VestaRV core plus its own private TCM, replicating the proven single-core memory path, with every connection to the rest of the chip registered at the tile boundary. Since every hart sees the same address map (shared boot ROM, shared peripherals, private TCM at the same local address), the five harts are architecturally identical and code is hart-portable by construction.

Hart 0 is additionally the *management hart*. It is the only hart attached to the SPI-flash boot path and the extended flash region ($\geq$ `0x40000`), and by convention it owns system management: the `SYSTEM` controller's clock configuration registers reprogram the master clock that all five harts run on, so only the

management hart may touch them, and only after quiescing the other harts (see the SYSTEM chapter). Peripheral interrupts are uniform across harts: every hart, hart 0 included, routes and receives them through its IRQROUTER row and the claim/complete mechanism (see the IRQROUTER chapter).

Hart 0 (mhartid zero, the management and boot hart) is additionally the *orchestrator*, and it is the one hart that is not a corner tile. The channel harts, mhartid 1 through 4, are hardened *hart tile* macros at the corners of the die; the orchestrator is soft logic placed in the centre band alongside the shared fabric, with its own 8 KiB TCM at the same private 0x8000 to 0x9FFF window. It is architecturally identical to the tiles (same ISA, same private memory map, same mhartid discovery) and differs in exactly three ways that software can observe:

- **It is always on, and every other hart is not.** The orchestrator sits outside the switched-rail power fabric entirely: there are no header switches for the centre band, so PWRCR's bit 0 reads 0 and ignores writes, as it always has. What changes in this configuration is the other side of that sentence: *every* channel hart 1 to 4 now has a gate bit, including the corner tile that a four-hart chip would have called hart 0 and kept always-on. A blanket PWRCR write therefore gates all 4 channel tiles and leaves the orchestrator running (see the PWRCTRL chapter).
- **It boots the chip and starts the others.** Boot mastership is the orchestrator's, because the orchestrator is hart 0: it is the hart attached to the SPI-flash boot path and the extended flash region, it stages each channel hart's image into that hart's loader mailbox row and ignites it with a CLINT software interrupt (Section 5.3), and it holds the management privilege from reset rather than receiving it in a hand-over.
- **It can read every hart's private TCM.** Each hart's TCM is also visible read-only in the shared window, hart h at 0x20000+0x4000×h (the orchestrator's own included, at 0x20000), so the indexing is uniform. Each aperture is 16 KiB wide and the 8 KiB TCM repeats through it, because the aperture decode keeps the full window and the tile drops the top word-index bit. Only the management hart may read them; any other hart reads zero and has its write dropped, and writes are dropped for everyone (the windows are read-only by design: a write path into a running core's memory is a coherence hazard). A window belonging to a power-gated tile completes normally and returns zeros, so software checks the PWRCTRL sequencer state before trusting what it reads; zero is a legal TCM value.

The intended firmware model follows: the orchestrator boots the chip, starts and coordinates the channel harts, owns the front-end and its interrupts, inspects any channel hart's working memory through its TCM window, and stays up across any power-gating of the tiles.

Figure 5.1 follows one aperture read through the hardware, because the window is not a second port onto a shared memory: it is a request that leaves the shared fabric, borrows the target tile's own memory for four cycles, and comes back.

Software discovers which hart it is running on by reading the mhartid CSR:

```
csrr    t0, mhartid      # t0 = 0 .. (number of harts - 1)
```

5.2 The Shared Window

Everything below the extended flash region except each hart's private TCM is behind the arbiter. Every hart sees the same physical devices at the same addresses:

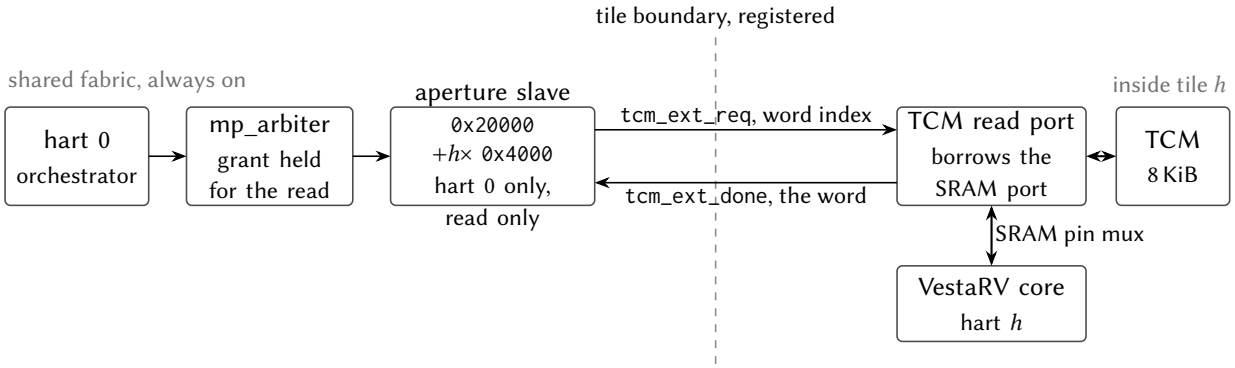


Figure 5.1: One read through a TCM aperture, from hart 0 to the target tile's TCM and back.

Address range	Device	Notes
0x00000 to 0x01FFF	Boot ROM (8 KiB)	Shared, read-only; all harts reset here
0x02000 to 0x03FFF	Unmapped	The ROM is smaller than its page; reads return zero
0x04000 to 0x04FFF	Peripheral slots	16 slots of 256 bytes, legacy numbering
0x05000 to 0x05FFF	CLINT	IPIs (MSIPx), MTIME/MTIMECMPx
0x06000 to 0x06FFF	MUTEX bank	16 hardware mutexes
0x07000 to 0x07FFF	IRQROUTER	Per-hart routing rows + CLAIM/COMPLETE
0x0C000 to 0x0FFFF	NPU staging RAM (16 KiB)	Vector storage; NPU-owned during a run
0x10000 to 0x1FFFF	Shared bulk RAM (64 KiB)	Locks, mailboxes, inter-hart data

(0x8000 to 0x9FFF is each hart's **private** TCM and never reaches the arbiter.) The sixteen peripheral slots at 0x4000 keep the original single-core VestaRV slot numbering (GPIO0 at 0x4000, UART0 at 0x4400, SYSTEM at 0x4900, and so on), so single-core driver code runs unchanged; the difference is that every hart can now reach every peripheral.

Instruction fetch from the shared window is supported: the boot flow depends on it, and programs may execute directly from shared RAM. One caveat: compressed (RVC) instructions in shared-window code are not currently validated; code intended to execute from shared memory should be built without the C extension. Code in the private TCM has no such restriction.

5.2.1 Arbitration and Stalling

A round-robin arbiter serializes all shared-window transactions. The arbiter grants one hart at a time and holds the grant until that hart's whole transaction completes, then advances the round-robin pointer, so no hart can be starved. While a hart waits for its grant, its clock is stalled transparently; software never sees a partial transaction and needs no special code for shared accesses. Because a stalled hart contributes nothing until its turn comes, frequently-polled data structures (spin locks in particular) should use a backoff strategy (see Section 5.4).

An AMO (atomic read-modify-write) instruction holds the arbiter grant across both halves of its read-modify-write pair, so AMOs to shared RAM are atomic with respect to all other harts. A shared-window access costs a few master-clock cycles uncontended (versus a single cycle to the private TCM), which is why per-hart code and stacks belong in the TCM. Figure 5.2 shows one uncontended read at the arbiter's own pins.

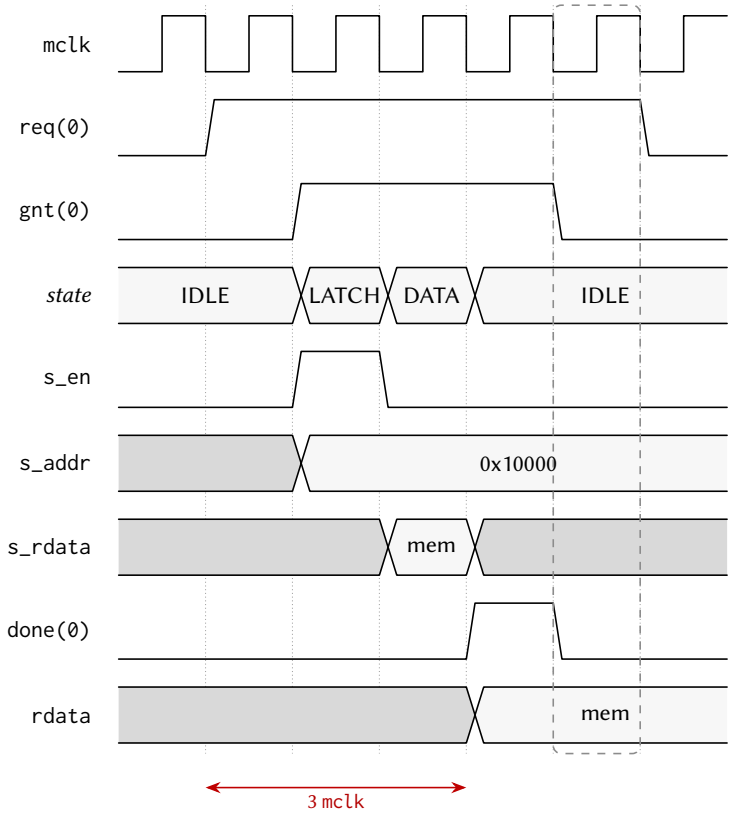


Figure 5.2: Timing diagram of one uncontended shared-window read at the `mp_arbiter` pins; in the dashed ghost window `req` is stale-high for one cycle after `done` and is masked by `need_release`.

One slave does not fit that walk. A read through a TCM aperture has to leave the shared fabric, borrow the target tile's own memory and come back, which takes six `mclk`, so the aperture slave holds the arbiter in *LATCH* with `s_stall` until the word is actually in its hand. It is the only slave in the design that does this, and the only one that can: nothing else could have been granted meanwhile, because the arbiter had already pinned the bus to this transaction. Figure 5.3 is the same set of pins as Figure 5.2, on that read.

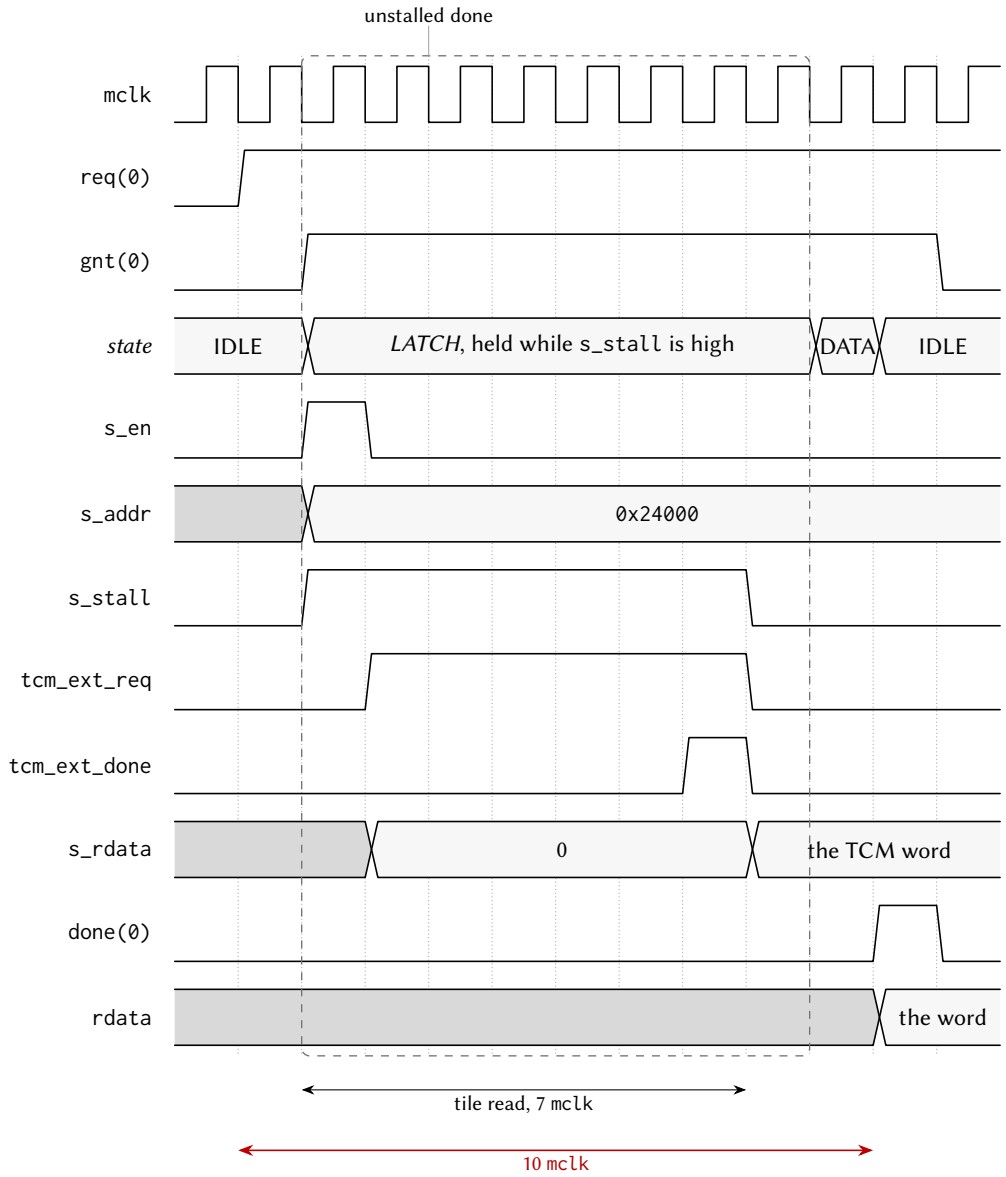


Figure 5.3: Timing diagram of one stalled shared-window read through a TCM aperture; the grey tick marks where an unstalled read would have completed, and the tile read (*SETTLE*, *READ*, *LATCH*, then one `mclk` of `tcm_ext_done`) runs while `s_stall` is high.

5.3 Boot and Tile Loading

All five harts come out of reset at PC `0x0` and fetch the shared boot ROM through the arbiter. The ROM immediately dispatches on `mhartid`; the whole flow is summarized in Figure 5.4:

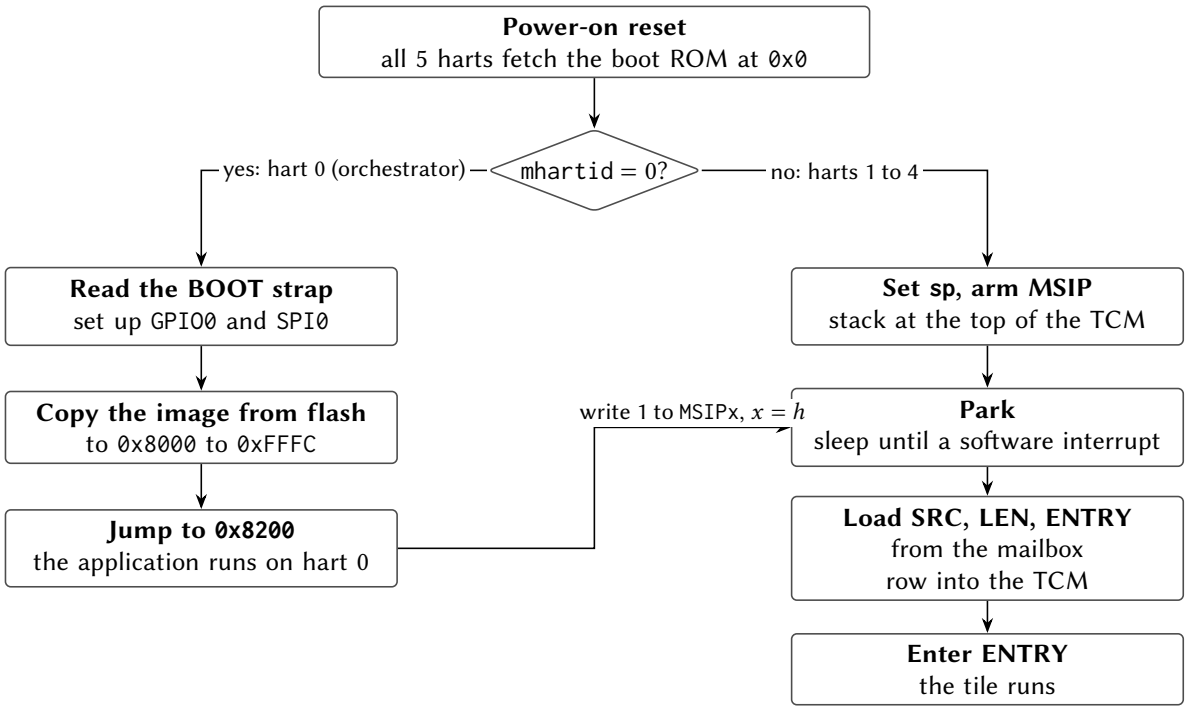


Figure 5.4: Flow chart of boot and tile loading across the harts.

Hart 0 runs the full boot sequence, exactly as on the single-core chip: it configures GPIO0/SPI0, copies the program from SPI flash into the load window 0x8000 to 0xFFFC (its TCM plus the NPU staging RAM), and jumps to the program at 0x8200. Before any tile can be released, the boot ROM zeroes the shared-RAM mailbox region 0x10000 to 0x107FF: shared RAM powers up with *unknown* contents on silicon, and this write-before-read guarantee is what makes "mailbox reads as zero" checks safe.

Harts 1 to 4 set up a minimal interrupt context (a stack at the top of their TCM and a software-interrupt vector) and park in low-power sleep. A parked tile wakes only when hart 0 (or any running program) sends it a CLINT software interrupt. On wake, the ROM-resident loader reads the tile's *loader mailbox row* in shared RAM:

Mailbox	Address	Meaning
SRC[h]	0x10500 + 16h + 0	Word-aligned source address in shared space
LEN[h]	0x10500 + 16h + 4	Words to copy to the tile's TCM at 0x8000 (0 = none)
ENTRY[h]	0x10500 + 16h + 8	PC the tile enters with

The loader clears the tile's own MSIP level, copies LEN[h] words from SRC[h] into the tile's private TCM at 0x8000 (a LEN of 2048 loads the full 8 KiB image: vector table, code, and interrupt handlers included), and enters ENTRY[h] with the stack pointer set to the top of the tile's TCM and all other registers undefined.

The canonical release sequence for hart h is therefore: stage the tile's program somewhere in shared RAM, write SRC[h] and LEN[h] and ENTRY[h], then write 1 to the hart's MSIP register (0x5000 + 4h). Because the mailbox row is consumed by the ROM only at wake time, the row must be complete *before* the MSIP write. Software interrupts sent to a tile *after* it has been loaded are handled by the loaded program's own vector table, so the same CLINT mechanism serves both boot-time loading and run-time inter-processor interrupts.

Stacks: each hart needs its own stack in its own private TCM. When an interrupt is taken, the hardware pushes the return program counter at sp − 4 (see Section 9), so every hart, including a freshly loaded tile

hart, must have a valid stack pointer before enabling interrupts or sleeping. The boot ROM guarantees this for parked and freshly-entered tiles; application code keeps the convention of placing each hart's stack at the top of its TCM, growing downward.

5.3.1 Harvested (field-powered) boot

In configurations wired for field power, hart 0's boot path has a second entry, selected by a hardware strap sampled at reset and reported by the PWRCTRL controller. Before any SPI or flash activity, hart 0 polls the strap-valid flag and, if the harvested-boot strap is set, branches to a ROM-resident service path instead of the SPI-flash download described above. This path is built for a chip running on harvested field energy, where the multi-kilobyte flash copy is the single most power-hungry cold-start step and is therefore skipped entirely.

The harvested path keeps both MCLK and SMCLK on HFXT (the SPI path's mid-boot SMCLK-to-LFXT switch does not run, so the clock configuration is written explicitly rather than inherited), then power-gates every tile hart through PWRCTRL (the tiles are already ROM-parked in WFI, the sanctioned quiesced state, so gating them is safe at any hart count). It configures the port-6 GPIO alternate-function plane for the six off-die NFC analog-front-end wires, provisions the NFC tag identity (a ROM-default UID, ATQA and SAK) and a short status payload, and arms the NFC block in LISTEN mode with auto-read enabled, so the hardware answers Type-2 read commands with no firmware in the loop. Finally it arms its own software-interrupt vector with a harmless clear-and-return handler (a stray MSIP must not disturb the parked hart), divides MCLK down (the parked service loop needs no speed, and dynamic power scales with the divide), and sleeps.

On a configuration built without the NFC block, the NFC register writes land in an unused alias of the MUTEX page and are harmless; the path is written to never *read* an NFC register, because a read of that alias would claim a mutex.

The exact sequence, in the order the ROM executes it (register addresses in the peripheral chapters):

1. **Common entry (shared with the SPI path):** interrupts off, all memory blocks powered, stack pointer set, and the shared-RAM mailbox region 0x10000 to 0x107FC zeroed (the write-before-read contract). At a 24 MHz MCLK this zeroing, 512 word writes through the shared-bus arbiter, dominates the harvested cold-start (about 625 µs of the roughly 690 µs from gate release to sleep).

2. **Strap branch:** poll PWRSTS until PWSTRAPVLD, then branch on PWSTRAP. (The sample completed a handful of cycles after reset release, long before this code runs; the poll is cheap insurance.)

3. **Clock story, explicitly:** write SYSCLKCR = 0; both MCLK and SMCLK stay on HFXT. The SPI path's mid-boot switch of SMCLK to LFXT never runs on this path, so the choice is written rather than inherited.

4. **Gate the tiles:** write all-ones to PWRCTRL PWRCR. The register masks the write to the tile bits that exist, so the same ROM gates every tile hart at any hart count; the tiles are parked in their boot-ROM WFI, the sanctioned quiesced state.

5. **Pin mux:** select the alternate-function planes for the six off-die NFC AFE wires on port 6 pins 0 to 5 (PxSEL, then the AF-select fields). Pins 6 and 7 (the strap and PGOOD) are untouched and stay plain inputs.

6. **Provision the tag:** write the ROM-default identity (UID 0xC0FFEE01, ATQA 0x0044, SAK 0x00), point the payload index register at byte 0 with auto-increment, and write a three-byte status payload: 'H', 'V', then the live PWRSTS snapshot, the status record of the base chip. A fielded product replaces all of this by relaunching with real firmware.

7. **Arm LISTEN:** a *byte* store of `NFCEN|LISTEN` to the NFC control register's low lane. A full-word store would also write the second byte lane and clear the auto-read enable, whose reset default is set; auto-read is what lets the hardware answer Type-2 `READ`s with the processor asleep.

8. **Stray-interrupt guard:** arm interrupt-vector slot 83 (the software interrupt) in hart 0's own TCM with a return stub, so an unexpected `msip` is cleared and returns the hart to sleep instead of vectoring into uninitialized memory.

9. **Slow down and sleep:** divide `MCLK` by eight (`CLKDIVCR`), then sleep. The clock reconfiguration is legal here because every other bus master is gated. From this point the NFC hardware serves readers autonomously; a field arrival can be taken as an interrupt only if firmware later opts in.

5.4 Synchronization

PentaWoundPt2mm offers three synchronization mechanisms, from cheapest to most flexible. Figure 5.5 summarizes how to choose between them:

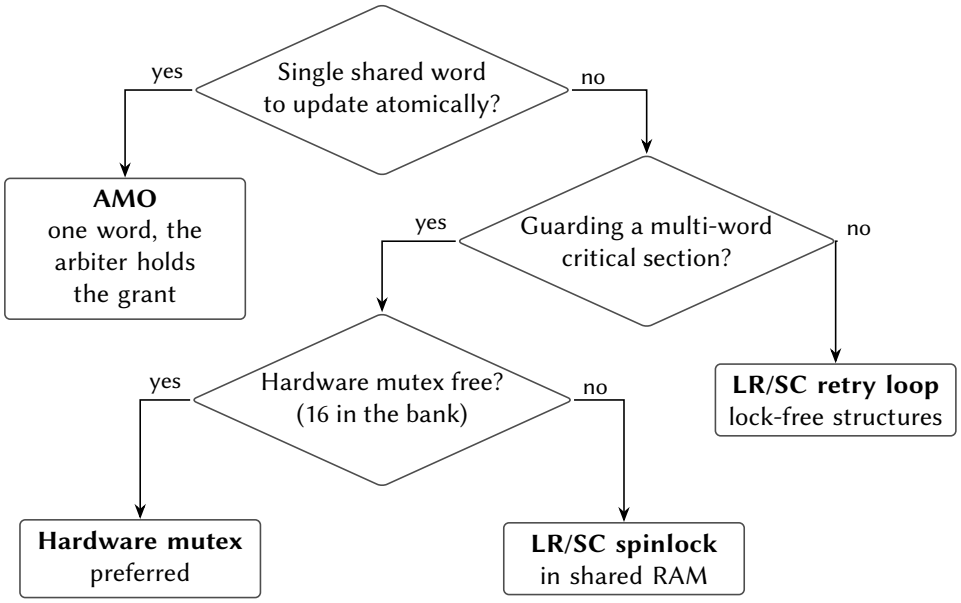


Figure 5.5: Decision chart for choosing a synchronization primitive.

1. **Hardware mutexes** (`MUTEX` bank): a single load claims a free mutex; a single store releases it. No retry loop hardware state, no reservation. This is the preferred lock for most uses. See the `MUTEX` chapter.

2. **`LR/SC` spinlocks in shared RAM**: standard RISC-V reservation-based locks. A failed `SC` has its write suppressed by the reservation unit, so it is safe under contention.

3. **`AMO` instructions to shared RAM**: atomic add, swap, and friends; the arbiter locks the grant across the read-modify-write pair.

Two rules apply to all of them:

- **Never** use LR/SC or AMO instructions on MUTEX bank addresses; hardware mutexes are claimed with plain loads and released with plain stores only.
- Under contention, spin with a *hart-scaled backoff*: delay between retries by an amount proportional to mhartid (for example delay $= k \cdot$ (mhartid + 1)) and bound the retry count. Multiple harts hammering the arbiter in lockstep can otherwise livelock a naive retry loop.
- Additionally, do not access the NPU staging RAM (0xC000 to 0xFFFF) while an NPU run may be active; poll the NPU's busy flag first (see the NPU chapter).

There are no data caches, so there is no cache-coherence protocol to consider: a shared-RAM write is globally visible as soon as the arbiter completes the transaction.

6 Reset, clocks and power

6.1 Reset

The chip is held in reset while the RESETN pin is low and leaves reset when it is released high. A power-on reset circuit asserts the internal reset while the core supply rises from zero and releases it once the supply has reached its operating level; the chip is in reset whenever either RESETN or the power-on reset is asserted. In reset, every memory-mapped register and every core state register is at its reset value. The RESETN input has an internal pull-up and may be left unconnected.

Every hart leaves reset fetching from the boot ROM at 0x00000. Hart 0 runs the boot loader; harts 1 to 4 park in the ROM until hart 0 releases them through the boot mailboxes in shared RAM (Section 5.3). The PWRCTRL boot gate can hold every hart in reset until a supply supervisor reports power good; see Section 6.4 and Chapter 18.

6.2 Boot mode selection

Hart 0's boot loader samples one strap after reset: GPIO pin P0.7, marked BOOT in Table 3.2. The pin is read once, through P0IN, after the boot loader has configured the SPI0 pins; it is an ordinary GPIO pin at every other time. Table 6.1 gives the two modes.

Table 6.1: Boot mode selected by the P0.7 strap.

P0.7 at reset	Mode	Behaviour
Low	ROM monitor	Hart 0 runs the interactive ROM monitor over UART0 at 115,200 baud (Chapter 35).
High	SPI flash	Hart 0 reads the boot image from the SPI flash on SPI0, copies its segments into RAM and enters it (Chapter 36).

On a field-powered board the harvested-boot strap sampled by PWRCTRL (PWRSTS) takes precedence: when it reads 1, hart 0 takes the harvested-boot path described in Chapter 18 before the P0.7 strap is consulted.

Note: The boot ROM reprograms the clock system during boot, so the SMCLK source at application entry is a boot-ROM implementation detail and may be the slow LFXT source. An application that uses an SMCLK-domain peripheral (a UART, SPI, I^2C or a timer clock-source switch) selects its SMCLK source in the SYSTEM peripheral first (Section 16.3.1).

6.3 Clock tree

6.3.1 Clock sources

The chip has four clock sources. HFXT and LFXT are external clocks supplied on the HFXT and LFXT pin functions (Section 3); DCO0 and DCO1 are on-chip digitally controlled oscillators whose frequency is set by the bias values in DCO0BIAS and DCO1BIAS, a higher bias giving a higher frequency. The design is characterised for a maximum clock frequency of 24 MHz, so HFXT must not exceed 24 MHz; LFXT is

intended to be 32.768 kHz so that time can be measured accurately. Operation above 24 MHz, from HFXT or from a DCO, is outside the characterised range and is not guaranteed.

HFXT and LFXT are enabled at reset and can be stopped with `HFXTOFF` and `LFXTOFF`; DCO0 and DCO1 are off at reset and are started with `DCO0ON` and `DCO1ON`. A source that is currently selected by either clock tree root stays enabled whatever its own bit says, so a mistaken write cannot stop the clock the chip is running on.

6.3.2 Roots and dividers

Two clocks form the roots of the tree in Figure 6.1: the main clock MCLK and the sub-main clock SMCLK. `SMCLKSEL` picks SMCLK's source from the four sources; `MCLKSEL` picks MCLK's from three of them and, on code `01`, from SMCLK itself. That tap is taken after the SMCLK divider but before the `SMCLKOFF` gate, so a chip running MCLK out of SMCLK keeps its main clock when SMCLK is stopped for the peripherals. Each root may then be divided again by `SYSMCLKDIV` and `SYSSMCLKDIV` in `CLKDIVCR`. Both roots use glitch-free multiplexers, so a source can be changed while the chip runs.

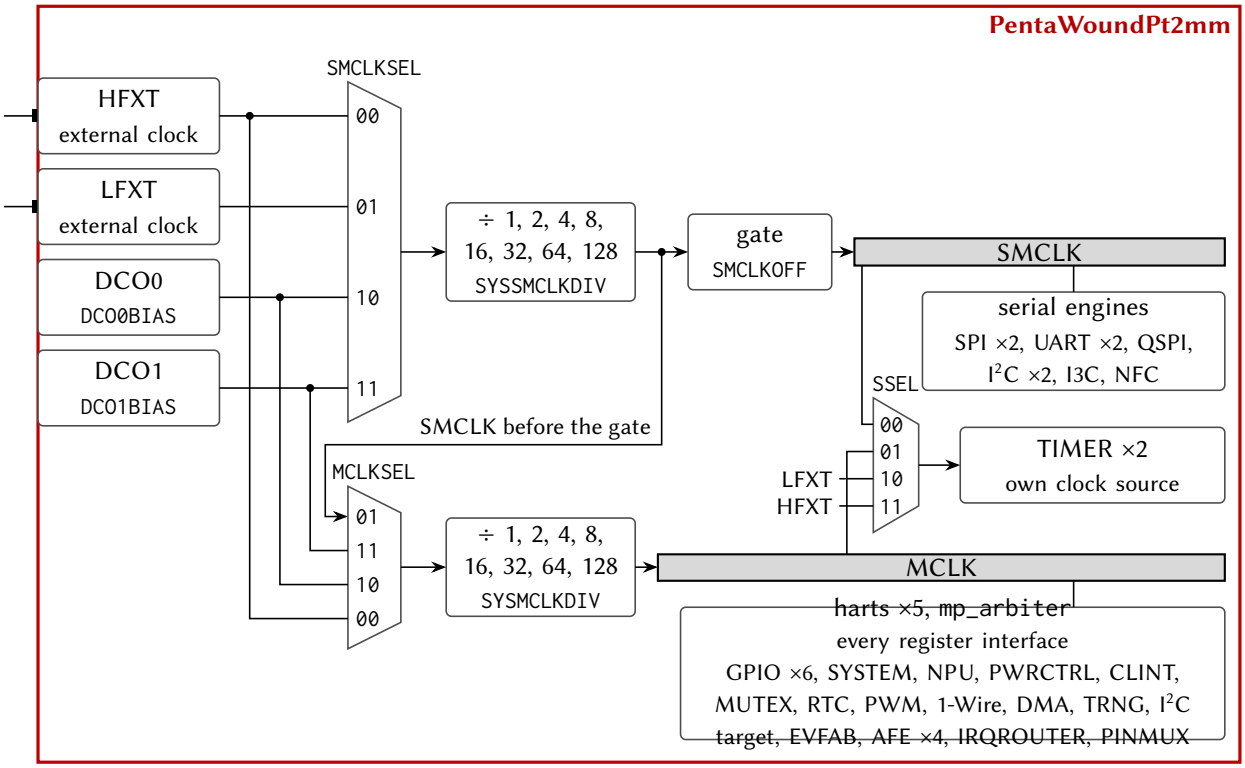


Figure 6.1: The clock tree: the four sources, the two selects and dividers, the SMCLK gate, and the blocks each root clocks.

MCLK drives all five harts, the multi-core arbiter and every always-on block, and it cannot be turned off. SMCLK drives the serial engines of the `SPIx`, `UARTx`, `I2Cx` and `NFCx` peripherals and is one of the `TIMERx` sources; it can be stopped with `SMCLKOFF`. The register interface of every peripheral is clocked from MCLK even where its engine runs on SMCLK, so a peripheral stays readable with SMCLK stopped. Two clocks come from outside this tree and are not reached by `SYSCLKCR`: the NFC protocol core runs on the carrier recovered off die, and the debug TAP runs on the probe's `TCK`.

6.3.3 Clock state at reset and after boot

Both roots select HFXT at reset, so a valid clock on HFXT is required while the chip boots; the board must keep the oscillator driving HFXT running across reset rather than gating it from a pin the chip drives only after boot. After boot, software may move MCLK to another source and then stop HFXT.

The boot ROM reprograms the clock system during the SPI-flash boot and may leave SMCLK on LFXT when it enters the application. The SMCLK source at application entry is therefore a boot-ROM implementation detail, and every application that uses an SMCLK-domain peripheral, or a TIMERx clock source switch, selects its own SMCLK source in SYSCLKCR first.

Because MCLK clocks every hart, a change to SYSCLKCR or CLKDIVCR changes the clock of all five harts at once. By convention only hart 0 reconfigures the clocks, after the other harts have been quiesced.

6.4 Power domains

The chip has one always-on power domain and one switchable domain per channel tile. Everything the rest of the chip needs (the arbiter, the boot ROM, the shared RAM, the peripherals, the pad ring and hart 0) sits on the unswitched VDD rail. Each channel tile, harts 1 to 4, has a rail of its own behind PMOS header switches, with isolation cells clamping every signal that leaves the tile while it is unpowered, and a per-tile hardware sequencer in PWRCTRL that applies the controls in the legal order in both directions. Figure 6.2 draws the two domains and the four controls that gate one tile.

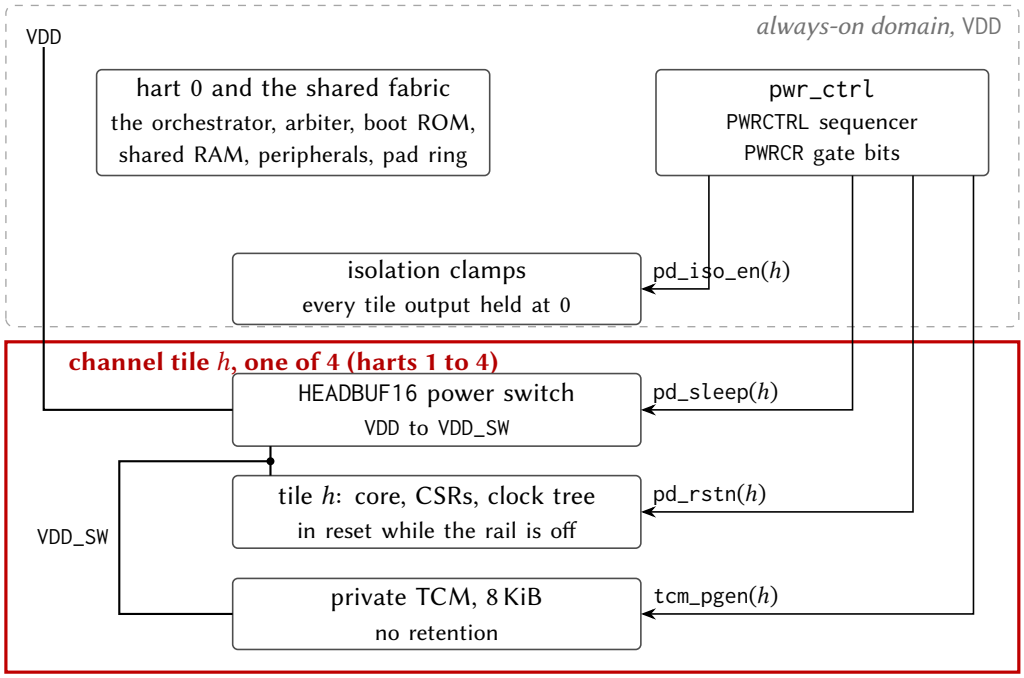


Figure 6.2: The always-on domain and one switched channel tile, with the four per-tile controls from pwr_ctrl.

Hart 0, the orchestrator, is soft logic in the always-on centre band with no header switches of its own; it is permanently powered and cannot be gated. All domains power up on at reset, so chip boot does not depend on the controller, and PWRCTRL is inert until software gates a tile.

6.4.1 Sequencing

Four per-tile controls leave `pwr_ctrl`, and Figure 6.2 draws each at the gate it drives. `pd_iso_en`(h) clamps: it closes AND gates on the always-on side of the boundary, so every outbound signal of tile h reads 0 while it is asserted. `pd_sleep`(h) opens the switches: a `HEADBUF16` chain carries `VDD` to the tile's `VDD_SW`, and the chain is opened on the gate and closed again on the wake. `pd_rstn`(h) holds the tile in reset: `tile_rstn`(h) is `resetn` AND `pd_rstn`(h) AND the field-power boot gate, asserted before the rail goes and released after it is back. `tcm_pgen` gates the TCM macro off the same `pd_sleep`(h), so the TCM dies with the rail and a wake is a cold boot; its aperture then reads zeros, so software checks `PWRSR` first.

The sequencer applies them in the legal order in both directions. On the gate it clamps, then resets, then opens the switches; on the wake it closes the switches, waits for the rail to settle, unclamps, and then releases the reset. The `PWRSR` nibble of tile h reports the state the sequencer is in: 0 ON, 1 ISO, 2 RSTOFF, 3 OFF, 4 RAIL and 5 UNISO.

6.4.2 Cold gating

Gating is cold: a gated tile consumes no dynamic or switched-leakage power and retains nothing, including its register file, CSRs and private TCM. The isolation clamps drive every outbound tile signal to its inactive level, which is identical to the tile's reset state, so the rest of the chip observes a gated tile as a silent one; a read of its TCM aperture completes normally and returns zeros. When the tile is ungated it cold-boots through the shared boot ROM, parks, and is relaunched with the loader-row and software-interrupt sequence of Section 5.3. Any data a tile must keep across a power cycle is staged to shared RAM before gating.

6.4.3 Memory block power

The ROM, the two per-hart memory blocks and the first four shared RAM banks each have a power gate in `BLOCKPWR` (Section 16.3.2). A gated block halves its leakage and loses its contents; an access to a gated block returns undefined data. Every block powers up on at reset.

6.4.4 Hart sleep

Each hart can stop its own clock with the `sleep` instruction and is woken by an interrupt (Section 16.3.3). Sleep is a clock gate, not a power gate: the hart keeps its state and resumes where it stopped, so it is the right tool for a hart that waits for an event, while cold gating is the right tool for a tile that has no work for a long time.

6.4.5 Boot gate and field power

In configurations wired for field power (the harvested-energy NFC operating mode), `PWRCTRL` also decides when the chip may start executing, so that the harts are held quiescent until the harvested supply can carry a transaction. Two package pins take part: a power-good input (`PGOOD`, pin 69, P6.7) driven by an external supply supervisor, and a boot-mode strap (pin 68, P6.6) that selects harvested boot. Both pads reset to inputs with pull-downs, so an unconnected pin reads as power-not-good and normal boot, and a board that does not use the feature behaves as if it did not exist.

The boot gate is a hold-in-reset: an internal release signal is combined into every hart's outer reset, hart 0 included, so a held hart fetches nothing and issues no bus request, and the rest of the chip cannot distinguish it from a hart still in power-on reset. The strap is sampled once, eight MCLK cycles after reset release, and sets the hardware defaults of the gate: a strapped board arms the gate, makes `PGOOD` a release

condition and re-holds on brownout with no firmware at all, which resolves the problem that the gate holds the very harts that would otherwise program it. Software may add overrides through PWRWAKE: arm the gate, add PGOOD or the NFC field-detect level as release conditions, force a release, and choose between a one-shot release and re-holding whenever a release condition later drops. Field detect used this way is a wake path that consumes no interrupt vector; the same level remains available as an ordinary interrupt to firmware that is already running.

With re-hold enabled, a PGOOD drop re-asserts the hold on the next cycle and its return releases the harts into a fresh boot; since the chip keeps no retention, a full cold boot is the accurate model of losing and regaining the harvested rail. Two schemes are supported on the same silicon. In the batteryless scheme the gate is the mechanism: the chip holds until PGOOD, cold-boots the harvested-boot ROM path (Section 5.3.1), serves the reader, and re-holds when the field collapses. In the supercap duty-cycled scheme an external buffer stays charged, the gate is left disarmed after the first boot, and the service loop sleeps between transactions and resumes on the field-detect interrupt without re-entering reset. The register-level mechanism and its timing are in the PWRCTRL chapter.

Part II Processor

7 VestaRV core

Every hart of the PentaWoundPt2mm MCU is a VestaRV core, a 32-bit RISC-V processor with a single unified instruction and data bus. This chapter states what the core implements: its instruction set, its register conventions, its hart identities and its control and status registers. The RISC-V specifications named in Section 1.3 are normative for the instructions and registers themselves and are not restated here.

7.1 Instruction set

The ISA string of this build, the `-march` value the toolchain is given, is listed under *ISA string* in Table 2.2 and is `rv32imac_zba_zbb_zbs_zbc`. The same information is available at run time in the read-only `misa` register (Section 7.4). The extensions the core family implements are listed below with the configuration knob that selects each; which ones are present in this build is exactly what the ISA string says.

- **RV32I**, the base integer instruction set: 32 registers of 32 bits, integer arithmetic, logic, shifts, comparisons, loads and stores of bytes, half-words and words, branches and jumps. Always present.
- **M** (`isa.mul`, `isa.div`): hardware multiply, divide and remainder, signed and unsigned. Multiply is single-cycle; divide is iterative (Chapter 10).
- **A** (`isa.atomics`): load-reserved and store-conditional, and the atomic read-modify-write operations on 32-bit words. An atomic operation holds the shared-window arbiter grant across its whole read-modify-write pair, so it is atomic with respect to every hart (Section 5.4).
- **C** (`isa.compressed`): 16-bit encodings of the most common instructions. Code built with it is substantially smaller; its fetch-alignment cost is measured in Section 10.2.
- **Zba, Zbb, Zbc, Zbs** (`isa.bitmanip`): address generation (shift-and-add), basic bit manipulation (count, min and max, rotate, sign and zero extension, byte reversal), carry-less multiplication, and single-bit set, clear, invert and extract.
- **Zicsr**: the CSR access instructions. Always present.
- **Zicntr**: the `cycle` and `instret` counters, readable from the user views. Always present; `time` is not implemented (Section 7.4).

An instruction of an extension that is not configured into the build raises an illegal-instruction exception; what happens next depends on the trap delivery mode (Section 7.4.1).

Three instructions are specific to this core and are used by the runtime: `retirq` returns from a legacy-mode interrupt (Chapter 9), `sleep` gates the hart's clock until an interrupt arrives (Section 16.3.3), and `wake` is its counterpart. Their encodings are in the generated `periph.S`.

7.2 Registers and calling convention

The 32 general-purpose registers and their roles under the standard RISC-V calling convention (the `ilp32` ABI) are listed in Table 7.1. The compiler follows this convention, so hand-written assembly that follows it is drop-in compatible with compiled code. A register whose saver is the caller may be overwritten by any function call; a register whose saver is the callee is preserved across calls.

Table 7.1: General-purpose registers and their ABI roles.

Register	ABI name	Role	Saver
x0	zero	Hard-wired zero (writes ignored)	none
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	none
x4	tp	Thread pointer	none
x5	t0	Temporary, alternate link register	Caller
x6, x7	t1, t2	Temporaries	Caller
x8	s0/fp	Saved register, frame pointer	Callee
x9	s1	Saved register	Callee
x10, x11	a0, a1	Function arguments and return values	Caller
x12 to x17	a2 to a7	Function arguments	Caller
x18 to x27	s2 to s11	Saved registers	Callee
x28 to x31	t3 to t6	Temporaries	Caller

7.3 Hart identities

Each hart reads its own identity in mhartid: 0 to 4. The boot ROM dispatches on it (Section 5.3), and it selects the hart's own CLINT and IRQROUTER rows. Hart 0 is the orchestrator: it is always powered, it runs the boot loader and the ROM monitor, and it alone reaches the TCM apertures and the extended-flash window (Chapter 5).

7.4 Control and status registers

Tables 7.2 to 7.3 list the CSRs the core implements, by the numbering of the RISC-V Privileged Architecture, version 1.12. The tables are partial in the sense the specification allows: a CSR not listed is not implemented, and Section 7.4.1 says what an access to it does. The trap CSRs of Table 7.3 are described field by field in Chapter 8, and the debug CSRs in Chapter 11.

Table 7.2: CSRs present in every configuration (partial list, Privileged Architecture 1.12 numbering).

Number	Name	Access	Implementation
0x301	misa	read-only	MXL = 1 (RV32) and one letter bit per configured extension. Writes are ignored.
0xF11	mvendorid	read-only	Reads zero (not a commercial vendor).
0xF12	marchid	read-only	Reads zero.
0xF13	mimpid	read-only	Reads zero.
0xF14	mhartid	read-only	The hart's identity, 0 to 4.
0xF15	mconfigptr	read-only	Reads zero (no configuration structure).
0xB00, 0xB80	mcycle, mcycleh	read/write	64-bit MCLK cycle counter, free-running; counts while the hart is stalled.
0xB02, 0xB82	minstret, minstreth	read/write	64-bit count of retired instructions.
0xC00, 0xC80	cycle, cycleh	read-only	User view of mcycle.
0xC02, 0xC82	instret, instreth	read-only	User view of minstret.

Number	Name	Access	Implementation
0x306	mcounteren	WARL	Hard-wired zero unless user mode is configured.
0x320	mcountinhibit	WARL	Hard-wired zero: the counters cannot be inhibited. Writes are ignored.
0xB03 to 0xB1F, 0xB83 to 0xB9F	mhpmcounter3 to mhpmcounter31 and their high halves	WARL	Hard-wired zero. Writes are ignored.
0x323 to 0x33F	mhpmevent3 to mhpmevent31	WARL	Hard-wired zero. Writes are ignored.
0xC03 to 0xC1F, 0xC83 to 0xC9F	hpmcounter3 to hpmcounter31 and their high halves	read-only	Read zero.

Table 7.3: CSRs present only in some configurations (partial list, Privileged Architecture 1.12 and Debug 1.0 numbering).

Number	Name	Configured by	Implementation
0x300	mstatus	priv.trapCsr (present here)	MIE, MPIE, MPP; MPRV and TW only with user mode. Chapter 8.
0x310	mstatush	priv.trapCsr	Reads zero; writes are ignored.
0x305	mtvec	priv.trapCsr	BASE read/write; MODE pinned to direct.
0x304, 0x344	mie, mip	priv.trapCsr	Bits MSIE/MSIP (3), MTIE/MTIP (7), MEIE/MEIP (11). mip is read-only.
0x340	mscratch	priv.trapCsr	Read/write scratch.
0x341, 0x342, 0x343	mepc, mcause, mtval	priv.trapCsr	Written by hardware on a standard-mode trap entry (Table 8.2); mcause keeps bit 31 and bits 3:0.
0x7C0	mtrapctl	priv.trapCsr	Custom: bit 0 LEGACY selects the trap delivery mode; resets to 1 (legacy).
0x3A0 to 0x3A3, 0x3B0 to 0x3BF	pmpcfg0 to pmpcfg3, pmpaddr0 to pmpaddr15	priv.pmp (absent here)	16 entries; entries above the count read WARL zero. Chapter 8.
0x7B0 to 0x7B3	dcsr, dpc, dscratch0, dscratch1	debug.enable (present here)	Accessible in debug mode only; an access outside debug mode is an illegal instruction. Chapter 11.

7.4.1 Unimplemented CSRs and illegal accesses

A CSR instruction naming a register that is not in Tables 7.2 and 7.3 for this build is an illegal instruction (exception code 2). In particular time and timeh (0xC01, 0xC81) are not implemented, because no real-time source is wired to the core; use cycle and the known MCLK frequency instead. Two further rules apply in every configuration: a CSR instruction in a write form (any form other than a pure read with x0 as the source) aimed at the read-only quadrant (0xC00 to 0xFFF) is illegal even when the register exists, and an access to the debug CSRs outside debug mode is illegal. A write to a WARL register listed as hard-wired (misa, mip, mstatush, mcountinhibit, the mhpm* ranges) is not illegal; it is accepted and ignored. An illegal CSR instruction commits nothing: no register, memory or CSR is changed.

What an illegal instruction does next depends on the trap delivery mode of Chapter 8. In the legacy mode, which every configuration selects at reset, there is no recoverable exception: the hart enters a terminal trap state and retires no further instruction until reset. In the standard mode, available where `priv.trapCsr` is configured and selected by clearing `mtrapctl.LEGACY`, the hart traps to `mtvec` with `mcause` = 2 and `mtval` holding the faulting encoding.

Caution: The `unimp` pseudo-instruction assembles to a CSR write to `cycle` (`0xC0001073`), which is exactly the read-only-quadrant write described above. Reaching an `unimp` in legacy delivery mode halts the hart until reset.

8 Privileged Architecture

Every VestaRV hart executes at the RISC-V *machine* (M) privilege level, the level that owns the whole address space and every control register. What differs between configurations is how a hart *takes a trap* (an interrupt or an exception) and whether it can drop to the *user* (U) privilege level and be fenced off from memory it does not own.

Two trap architectures exist in this family. The **legacy** architecture is the one described in Section 9: a table of jump instructions at `0x8000`, a hardware push of the return address, and the `retirq` instruction to return. It is inherited from the single-core VestaRV chip, it is what the boot ROM, the interrupt dispatcher and every shipped example use, and it is active out of reset in *every* configuration. The **standard** architecture is the one written in the RISC-V privileged specification: a single trap entry point in `mtvec`, the cause and value registers `mcause`/`mtval`, a saved program counter in `mepc`, an interrupt-enable stack in `mstatus`, and the `MRET` instruction to return. It is a configuration option (`priv.trapCsr` in Table 2.1), and when it is built the two architectures *coexist* in the same hart, selected at run time by one bit.

This configuration includes the standard trap architecture; the sections below document it.

8.1 Machine Information Registers

Five read-only registers identify the hart and the implementation. They are present in *every* configuration (they are not part of the trap architecture and are not affected by `priv.trapCsr`), and reading any of them is always legal, at machine level, in either delivery mode.

Address	Register	Value
`0xF11`	`mvendorid`	`0x00000000`: no commercially registered JEDEC vendor.
`0xF12`	`marchid`	`0x00000000`: no architecture ID assigned by the RISC-V registry.
`0xF13`	`mimpid`	`0x00000000`: no implementation revision encoded.
`0xF14`	`mhartid`	The hart's own index, `0` to 4 (Section 5). The only one of the five that differs between harts, and the only one with a non-zero value.
`0xF15`	`mconfigptr`	`0x00000000`: no configuration data structure in memory.

Zero is a defined answer, not a missing one. The privileged specification requires all five registers to exist and gives zero a specific meaning in each: it says *"not implemented"* or *"not assigned"*, which is the truthful answer for a non-commercial teaching chip. Discovery software must therefore read them and interpret zero, rather than treat a zero as evidence that the register is absent; there is no way to tell the two apart by reading, which is why the specification requires the registers rather than the values.

Writing one is an illegal instruction. All five sit in the read-only quadrant of the register address space (`csr[11:10] = 11`), and any CSR instruction in a *write* form aimed at that quadrant traps, in every configuration, including the read-modify-write forms that would write back an unchanged value. Reading with `csrr` (or any form whose source operand is `x0`, which the hardware recognizes as a pure read) is the only legal access.

8.2 The Legacy Trap Mechanism

The legacy mechanism is a pure *interrupt* architecture: it delivers the hardware interrupt sources of Section 9 and nothing else.

- **Delivery** is vectored in hardware. Each of the 125 interrupt sources owns one 4-byte slot of the interrupt vector table at `0x8000`, and the hardware jumps directly to the slot for the source it is taking. The slot holds a jump instruction to the service routine.
- **Entry** pushes the return program counter, and only that, to the stack at `sp` − 4 and decrements `sp` by four. The stack pointer must therefore be valid before any interrupt can arrive.
- **Return** is the custom `retirq` instruction, which pops the saved program counter, increments `sp` by four and resumes. The custom `sleep` and `wake` instructions park and un-park the hart (the generated header spells the three of them `irq_return()`, `cpu_sleep()` and `cpu_wake()`; the bare-metal assembly environment spells them `iret`, `extinguish` and `ignite`).
- **Exceptions do not exist.** There is no recoverable exception in the legacy architecture. An illegal instruction, including an access to an unimplemented control register, puts the hart into a terminal trap state: the program counter stops advancing and the hart never retires another instruction. This is a deliberate, deterministic failure mode for a teaching chip, not an error state software can catch.
- **Handling is non-recursive** and the enable state of the three per-hart interrupt lines is fixed in hardware; there is no interrupt-enable control register in the legacy path.

Because the legacy mechanism pushes to the stack on entry, it assumes the stack pointer belongs to trusted code. That assumption is exactly what the standard architecture removes, which is why a chip that wants user mode needs the standard architecture underneath it.

8.3 Selecting a Delivery Mode: `mtrapctl`

The two architectures are selected by bit 0 (`LEGACY`) of the custom machine register `mtrapctl` at address `0x7C0`, which lives in the architecturally reserved custom machine read/write range. All other bits read zero and ignore writes.

`mtrapctl.LEGACY`	Mode	Behavior
1 (reset)	Legacy	Exactly the mechanism of Section 8.2: vector table delivery, the stack push on entry, `retirq` to return, and a terminal halt on any illegal instruction.
`0`	Standard	Interrupts and exceptions vector through `mtvec` with the full `mepc`/`mcause`/`mtval`/`mstatus` sequence, and `MRET` returns. The vector table is inert, and **nothing is ever pushed to the stack** by trap entry.

`LEGACY` resets to `1`, so a chip built with the standard architecture still *boots* on the legacy path: the boot ROM, the tile loader, the interrupt dispatcher and existing application code run unmodified, and a program that never writes `mtrapctl` cannot tell the difference. Selecting the standard mode is an explicit software act.

Software contract: change `mtrapctl` only while no trap is in flight; that is, from ordinary program flow with interrupts quiesced, never from inside an interrupt service routine and never between a trap

entry and its matching return. The rule is the same class as the clock-reconfiguration contract in the SYSTEM chapter: the state of a trap that was taken under one delivery mode cannot be undone by the other. Switching back to legacy is legal and is the recovery path.

The two custom instruction groups keep their encodings in both modes. `retirq` is only *defined* in legacy mode, where an interrupt sequence is in progress for it to acknowledge; executing it in standard mode is a software error whose only consequence is the stack-pointer damage the instruction itself does.

8.4 The Machine Trap Registers

Table 8.1 lists every control register the standard trap architecture adds, with the behavior of each field. Fields not listed read as zero and ignore writes; every field is WARL (write any value, read a legal value), so software must read back what it wrote rather than assume a value took.

Table 8.1: Machine trap control and status registers

Address	Register	Fields and behavior
0x300	mstatus	MIE (bit 3) global machine interrupt enable, and MPIE (bit 7) its saved copy. MPP (bits 12:11) is the saved previous privilege and is pinned to 11 (machine). MPRV (bit 17) makes *data* accesses use the privilege in MPP. TW (bit 21) traps WFI (no effect without user mode).
0x310	mstatush	Implemented as a legal register that reads zero and ignores writes (the high half of mstatus holds no field this chip implements).
0x305	mtvec	BASE (bits 31:2) is the trap entry address, read/write. MODE (bits 1:0) is pinned to 00, *direct* mode: every trap enters at BASE, and the handler dispatches on mcause.
0x304	mie	Per-source interrupt enables: MSIE (bit 3) software, MTIE (bit 7) timer, MEIE (bit 11) external. **Resets to zero**: standard mode wakes up fully masked, unlike the legacy path whose three lines are hard-wired enabled.
0x344	mip	Read-only mirror of the three live interrupt lines: MSIP (bit 3), MTIP (bit 7), MEIP (bit 11). Writes are ignored: every source is level-driven and is cleared at the source (a CLINT MSIPh write, an MTIMECMPx write, or the IRQROUTER claim/complete sequence).
0x340	mscratch	32 bits of read/write scratch storage reserved for the trap handler. In standard mode the hardware saves nothing to memory, so this is where a handler keeps the pointer it needs before it owns a stack.
0x341	mepc	The trapped program counter. Bit 0 always reads zero; bit 1 is writable on a chip built with the compressed instruction set and reads zero otherwise.
0x342	mcause	The trap cause: bit 31 (the interrupt flag) distinguishes an interrupt from an exception, bits 3:0 carry the code. Bits 30:4 read zero. Hardware writes it on every trap entry per Table 8.2; software may write it.
0x343	mtval	The trap value, written by hardware per Table 8.2 and freely writable by software.

Address	Register	Fields and behavior
0x306	mcounteren	A legal register pinned to zero (its enables only have meaning with user mode built).
0x7C0	mtrapctl	Custom. LEGACY (bit 0) selects the delivery mode and resets to 1; bits 31:1 are pinned to zero. See Section 8.3.

8.5 Trap Entry, Trap Return, and Causes

In standard mode a trap entry performs **no memory transaction at all**. In one atomic step the hardware writes mepc, mcause and mtval per Table 8.2, copies MIE into MPIE and clears MIE (so the handler starts with interrupts disabled), records the interrupted privilege level in MPP, enters machine mode, and sets the program counter to mtvec's BASE. The stack pointer is not read, not written and not adjusted, which is the entire point, because the interrupted stack pointer may belong to untrusted code.

MRET reverses it: the program counter is loaded from mepc, MIE is restored from MPIE, MPIE is set to 1, and the privilege level returns to MPP. ECALL and EBREAK are decoded as the environment call and breakpoint exceptions; both are illegal instructions on a chip without the standard trap architecture.

A minimal handler installation looks like this (the assembler needs the Zicsr extension enabled for the control-register instructions):

```
.option push
.option arch, +zicsr
la      t0, trap_handler
csrw    mtvec, t0        # direct-mode trap entry point
csrw    mie, zero        # start fully masked
csrwi   mtrapctl, 0      # leave legacy delivery: standard mode from here
.option pop
```

Table 8.2: Trap causes, values, and saved program counters

Condition	mcause	mtval	mepc
Instruction address misaligned	0	The misaligned address	The faulting instruction
Instruction access fault (memory protection)	1	The faulting fetch address	The faulting instruction
Illegal instruction (including an access to an unimplemented or privileged control register, or a *write* to a read-only one)	2	The faulting instruction encoding	The faulting instruction
Breakpoint (EBREAK)	3	0	The EBREAK
Load access fault (memory protection)	5	The faulting data address	The faulting instruction
Store or atomic access fault (memory protection)	7	The faulting data address	The faulting instruction
Environment call from machine mode	11	0	The ECALL
Machine software interrupt (CLINT MSIPh)	0x80000003	0	The resume address

Condition	mcause	mtval	mepc
Machine timer interrupt (CLINT MTIMECMP0L)	0x80000007	0	The resume address
Machine external interrupt (IRQROUTER)	0x8000000B	0	The resume address

Three details of Table 8.2 need stating explicitly.

- **The illegal-instruction value for a compressed instruction is the expanded encoding, not the raw halfword.** When a 16-bit compressed instruction traps, `mtval` carries the 32-bit instruction the decompressor produced from it, which is what the core actually rejected. The specification permits either the faulting encoding or zero here; reporting the expansion is more informative than zero and costs nothing, but a debugger that expects the raw halfword will not find it. This is a deliberate, documented deviation.
- **Interrupt priority is fixed**: external, then software, then timer. An interrupt is taken only when `MIE` is set and the same bit is set in both `mip` and `mie`.
- **Interrupts are sampled at instruction boundaries only**, and the multi-cycle instruction sequences (the compressed push/pop sequences, the cache-block zero instruction, and the table-jump sequence) run to completion uninterrupted, exactly as in the legacy mechanism.

8.6 Waiting for an Interrupt

Two instructions park a hart, and they are not interchangeable.

- The standard `WFI` instruction is available whenever the standard trap registers are built, in either delivery mode. It stops the hart until $(\texttt{mip} \wedge \texttt{mie}) \neq 0$, *regardless* of `MIE`. If `MIE` is set the hart then takes the interrupt; if it is clear the hart simply resumes at the following instruction, which is what makes the standard "sleep until an event, handle it inline" idiom work.
- The custom `sleep` instruction keeps its legacy meaning: it parks the hart until an interrupt is actually *taken*, and the service routine must execute `wake` before returning or the hart goes straight back to sleep. This is the instruction the boot ROM parks tile harts with.

Because `WFI` waits on $\texttt{mip} \wedge \texttt{mie}$, a hart that executes it with `mie` still zero waits forever. That is the specified behavior, not a defect; `TW` exists so that machine-mode software can stop less privileged code from doing it.

9 Interrupts and exceptions

The VestaRV CPU implements an interrupt system with 125 interrupt vectors, numbered 0 to 124, whose sources are enumerated in Table 9.1. The numbering is frozen: a vector belongs to the block that owns it whether or not this configuration instantiates that block, so vectors whose block is absent are reserved and counted with the rest rather than shifting the ones above them. Two of the 125 are the `CLINT` pair (83, the software or inter-processor interrupt; 84, the timer interrupt) added for multi-core operation; the other 123 sit on the `IRQROUTER` side of the fabric: the peripheral and system sources, the reserved slots frozen among them, and `meip`'s own slot. Each hart takes three interrupt lines: its own `CLINT` software and timer interrupts (vectors 83 and 84, hardwired enabled, delivered on dedicated per-hart wires) and the external interrupt `meip` (vector 85, a slot of its own that no source pends on), which delivers all peripheral and system sources through the `IRQROUTER`'s claim and complete mechanism: the vector-85 handler reads the router's `CLAIM` register to discover and claim the pending source, dispatches on the returned vector number, and writes it back to complete (Chapter 33). Which sources reach which hart is programmed per hart in the router's routing rows, identical for every hart, hart 0 included. All peripheral sources are masked at reset and must be routed in software before use; source priority is fixed (the lowest pending vector number is claimed first). A peripheral must additionally be configured to generate its interrupt at the peripheral level.

Table 9.1: Interrupt vectors

Vector	Source	Description	Clear method	Chapter
0	SYSTEM	Watchdog Timer Interrupt IRQB_SYS_WDT	Write 1 to WDTSR.SYSWDTIF	16
1	GPIO0	GPIO0 Bit 0 Interrupt IRQB_GPIO0_B0	Write 1 to P0IF bit 0	12
2	GPIO0	GPIO0 Bit 1 Interrupt IRQB_GPIO0_B1	Write 1 to P0IF bit 1	12
3	GPIO0	GPIO0 Bit 2 Interrupt IRQB_GPIO0_B2	Write 1 to P0IF bit 2	12
4	GPIO0	GPIO0 Bit 3 Interrupt IRQB_GPIO0_B3	Write 1 to P0IF bit 3	12
5	GPIO0	GPIO0 Bit 4 Interrupt IRQB_GPIO0_B4	Write 1 to P0IF bit 4	12
6	GPIO0	GPIO0 Bit 5 Interrupt IRQB_GPIO0_B5	Write 1 to P0IF bit 5	12
7	GPIO0	GPIO0 Bit 6 Interrupt IRQB_GPIO0_B6	Write 1 to P0IF bit 6	12
8	GPIO0	GPIO0 Bit 7 Interrupt IRQB_GPIO0_B7	Write 1 to P0IF bit 7	12
9	SPI0	SPI0 Transmission Complete Interrupt IRQB_SPI0_TC	Write 1 to SPI0SR.SPITCIF, or read SPI0RX	13
10	SPI0	SPI0 Transmission Buffer Empty Interrupt IRQB_SPI0_TE	Write 1 to SPI0SR.SPITEIF	13
11	SPI1	SPI1 Transmission Complete Interrupt IRQB_SPI1_TC	Write 1 to SPI1SR.SPITCIF, or read SPI1RX	13

Vector	Source	Description	Clear method	Chapter
12	SPI1	SPI1 Transmission Buffer Empty Interrupt IRQB_SPI1_TE	Write 1 to SPI1SR.SPITEIF	13
13	UART0	UART0 Receive Complete Interrupt IRQB_UART0_RC	Write 1 to UART0SR.RCIF, or read UART0RX	14
14	UART0	UART0 Transmission Buffer Empty Interrupt IRQB_UART0_TE	Write 1 to UART0SR.TEIF	14
15	UART0	UART0 Transmission Complete Interrupt IRQB_UART0_TC	Write 1 to UART0SR.TCIF	14
16	TIMER0	TIMER0 Capture 0 Interrupt IRQB_TIM0_CAP0	Write 1 to TIM0SR.CAP0IF	15
17	TIMER0	TIMER0 Capture 1 Interrupt IRQB_TIM0_CAP1	Write 1 to TIM0SR.CAP1IF	15
18	TIMER0	TIMER0 Overflow Interrupt IRQB_TIM0_OVF	Write 1 to TIM0SR.OVIF	15
19	TIMER0	TIMER0 Compare 0 Interrupt IRQB_TIM0_CMP0	Write 1 to TIM0SR.CMP0IF	15
20	TIMER0	TIMER0 Compare 1 Interrupt IRQB_TIM0_CMP1	Write 1 to TIM0SR.CMP1IF	15
21	TIMER0	TIMER0 Compare 2 Interrupt IRQB_TIM0_CMP2	Write 1 to TIM0SR.CMP2IF	15
22	TIMER1	TIMER1 Capture 0 Interrupt IRQB_TIM1_CAP0	Write 1 to TIM1SR.CAP0IF	15
23	TIMER1	TIMER1 Capture 1 Interrupt IRQB_TIM1_CAP1	Write 1 to TIM1SR.CAP1IF	15
24	TIMER1	TIMER1 Overflow Interrupt IRQB_TIM1_OVF	Write 1 to TIM1SR.OVIF	15
25	TIMER1	TIMER1 Compare 0 Interrupt IRQB_TIM1_CMP0	Write 1 to TIM1SR.CMP0IF	15
26	TIMER1	TIMER1 Compare 1 Interrupt IRQB_TIM1_CMP1	Write 1 to TIM1SR.CMP1IF	15
27	TIMER1	TIMER1 Compare 2 Interrupt IRQB_TIM1_CMP2	Write 1 to TIM1SR.CMP2IF	15
28	GPIO1	GPIO1 Bit 0 Interrupt IRQB_GPIO1_B0	Write 1 to P1IF bit 0	12
29	GPIO1	GPIO1 Bit 1 Interrupt IRQB_GPIO1_B1	Write 1 to P1IF bit 1	12
30	GPIO1	GPIO1 Bit 2 Interrupt IRQB_GPIO1_B2	Write 1 to P1IF bit 2	12
31	GPIO1	GPIO1 Bit 3 Interrupt IRQB_GPIO1_B3	Write 1 to P1IF bit 3	12
32	GPIO1	GPIO1 Bit 4 Interrupt IRQB_GPIO1_B4	Write 1 to P1IF bit 4	12
33	GPIO1	GPIO1 Bit 5 Interrupt IRQB_GPIO1_B5	Write 1 to P1IF bit 5	12
34	GPIO1	GPIO1 Bit 6 Interrupt IRQB_GPIO1_B6	Write 1 to P1IF bit 6	12
35	GPIO1	GPIO1 Bit 7 Interrupt IRQB_GPIO1_B7	Write 1 to P1IF bit 7	12
36	GPIO2	GPIO2 Bit 0 Interrupt IRQB_GPIO2_B0	Write 1 to P2IF bit 0	12

Vector	Source	Description	Clear method	Chapter
37	GPIO2	GPIO2 Bit 1 Interrupt IRQB_GPIO2_B1	Write 1 to P2IF bit 1	12
38	GPIO2	GPIO2 Bit 2 Interrupt IRQB_GPIO2_B2	Write 1 to P2IF bit 2	12
39	GPIO2	GPIO2 Bit 3 Interrupt IRQB_GPIO2_B3	Write 1 to P2IF bit 3	12
40	GPIO2	GPIO2 Bit 4 Interrupt IRQB_GPIO2_B4	Write 1 to P2IF bit 4	12
41	GPIO2	GPIO2 Bit 5 Interrupt IRQB_GPIO2_B5	Write 1 to P2IF bit 5	12
42	GPIO2	GPIO2 Bit 6 Interrupt IRQB_GPIO2_B6	Write 1 to P2IF bit 6	12
43	GPIO2	GPIO2 Bit 7 Interrupt IRQB_GPIO2_B7	Write 1 to P2IF bit 7	12
44	GPIO3	GPIO3 Bit 0 Interrupt IRQB_GPIO3_B0	Write 1 to P3IF bit 0	12
45	GPIO3	GPIO3 Bit 1 Interrupt IRQB_GPIO3_B1	Write 1 to P3IF bit 1	12
46	GPIO3	GPIO3 Bit 2 Interrupt IRQB_GPIO3_B2	Write 1 to P3IF bit 2	12
47	GPIO3	GPIO3 Bit 3 Interrupt IRQB_GPIO3_B3	Write 1 to P3IF bit 3	12
48	GPIO3	GPIO3 Bit 4 Interrupt IRQB_GPIO3_B4	Write 1 to P3IF bit 4	12
49	GPIO3	GPIO3 Bit 5 Interrupt IRQB_GPIO3_B5	Write 1 to P3IF bit 5	12
50	GPIO3	GPIO3 Bit 6 Interrupt IRQB_GPIO3_B6	Write 1 to P3IF bit 6	12
51	GPIO3	GPIO3 Bit 7 Interrupt IRQB_GPIO3_B7	Write 1 to P3IF bit 7	12
52	UART1	UART1 Receive Complete Interrupt IRQB_UART1_RC	Write 1 to UART1SR.RCIF, or read UART1RX	14
53	UART1	UART1 Transmission Buffer Empty Interrupt IRQB_UART1_TE	Write 1 to UART1SR.TEIF	14
54	UART1	UART1 Transmission Complete Interrupt IRQB_UART1_TC	Write 1 to UART1SR.TCIF	14
55	QSPI0	QSPI0 Transfer Complete Interrupt IRQB_QSPI0_TC	See chapter	19
56	QSPI0	QSPI0 Receive-Register Full Interrupt IRQB_QSPI0_RXF	See chapter	19
57	I2C0	I2C0 start received Interrupt IRQB_I2C0_STR	Write 1 to I2C0SR.I2CSTR	20
58	I2C0	I2C0 stop received Interrupt IRQB_I2C0_spr	Write 1 to I2C0SR.I2CSPR	20
59	I2C0	I2C0 master mode start condition sent Interrupt IRQB_I2C0_msts	Write 1 to I2C0SR.I2CMSTS	20
60	I2C0	I2C0 master mode stop condition sent Interrupt IRQB_I2C0_msps	Write 1 to I2C0SR.I2CMSPS	20

Vector	Source	Description	Clear method	Chapter
61	I2C0	I2C0 master mode arbitration lost Interrupt IRQB_I2C0_marb	Write 1 to I2C0SR.I2CMARB	20
62	I2C0	I2C0 master mode transmit empty Interrupt IRQB_I2C0_mtxe	Write 1 to I2C0SR.I2CMTXE	20
63	I2C0	I2C0 master mode NACK received Interrupt IRQB_I2C0_mnr	Write 1 to I2C0SR.I2CMNR	20
64	I2C0	I2C0 master mode transfer complete Interrupt IRQB_I2C0_mxc	Write 1 to I2C0SR.I2CMXC	20
65	I2C0	I2C0 slave address Interrupt IRQB_I2C0_sa	Write 1 to I2C0SR.I2CSA	20
66	I2C0	I2C0 slave transmit empty Interrupt IRQB_I2C0_stxe	Write 1 to I2C0SR.I2CSTXE	20
67	I2C0	I2C0 slave overflow Interrupt IRQB_I2C0_sovf	Write 1 to I2C0SR.I2CSOVF	20
68	I2C0	I2C0 slave mode NACK received Interrupt IRQB_I2C0_snr	Write 1 to I2C0SR.I2CSNR	20
69	I2C0	I2C0 slave mode transfer complete Interrupt IRQB_I2C0_sxc	Write 1 to I2C0SR.I2CSXC	20
70	I2C1	I2C1 start received Interrupt IRQB_I2C1_STR	Write 1 to I2C1SR.I2CSTR	20
71	I2C1	I2C1 stop received Interrupt IRQB_I2C1_spr	Write 1 to I2C1SR.I2CSPR	20
72	I2C1	I2C1 master mode start condition sent Interrupt IRQB_I2C1_msts	Write 1 to I2C1SR.I2CMSTS	20
73	I2C1	I2C1 master mode stop condition sent Interrupt IRQB_I2C1_msps	Write 1 to I2C1SR.I2CMSPS	20
74	I2C1	I2C1 master mode arbitration lost Interrupt IRQB_I2C1_marb	Write 1 to I2C1SR.I2CMARB	20
75	I2C1	I2C1 master mode transmit empty Interrupt IRQB_I2C1_mtxe	Write 1 to I2C1SR.I2CMTXE	20
76	I2C1	I2C1 master mode NACK received Interrupt IRQB_I2C1_mnr	Write 1 to I2C1SR.I2CMNR	20
77	I2C1	I2C1 master mode transfer complete Interrupt IRQB_I2C1_mxc	Write 1 to I2C1SR.I2CMXC	20
78	I2C1	I2C1 slave address Interrupt IRQB_I2C1_sa	Write 1 to I2C1SR.I2CSA	20
79	I2C1	I2C1 slave transmit empty Interrupt IRQB_I2C1_stxe	Write 1 to I2C1SR.I2CSTXE	20
80	I2C1	I2C1 slave overflow Interrupt IRQB_I2C1_sovf	Write 1 to I2C1SR.I2CSOVF	20

Vector	Source	Description	Clear method	Chapter
81	I2C1	I2C1 slave mode NACK received Interrupt IRQB_I2C1_snr	Write 1 to I2C1SR.I2CSNR	20
82	I2C1	I2C1 slave mode transfer complete Interrupt IRQB_I2C1_sxc	Write 1 to I2C1SR.I2CSXC	20
83	CLINT	CLINT software interrupt (IPI) IRQB_CLINT_MSIP	Write 0 to the MSIPn register of the hart	21
84	CLINT	CLINT timer interrupt IRQB_CLINT_MTIP	Write MTIMECMPn of the hart to a value above mtime	21
85	Reserved	Reserved (vector 85; coincides with the meip external-interrupt IVT slot, never a pending source)	-	-
86	I3C0	I3C0 Transfer Complete Interrupt IRQB_I3C0_TC	See chapter	23
87	I3C0	I3C0 Receive-Register Full Interrupt IRQB_I3C0_RXF	See chapter	23
88	I3C0	I3C0 Transmit-Register Empty Interrupt IRQB_I3C0_TXE	See chapter	23
89	I3C0	I3C0 Address / Byte NACK Interrupt IRQB_I3C0_NACK	See chapter	23
90	I3C0	I3C0 Early End-of-Data Interrupt IRQB_I3C0_EOD	See chapter	23
91	I3C0	I3C0 Arbitration-Lost Interrupt IRQB_I3C0_ARB	See chapter	23
92	I3C0	I3C0 Dynamic-Address-Assignment Done Interrupt IRQB_I3C0_DAA	See chapter	23
93	I3C0	I3C0 In-Band Interrupt Pending Interrupt IRQB_I3C0_IBI	See chapter	23
94	NFC0	NFC0 RF Field-Detect Interrupt IRQB_NFC0_FIELD	Write 1 to NFC0SR.NFCFIELDF	24
95	NFC0	NFC0 Reader-Frame Received Interrupt IRQB_NFC0_RXF	Write 1 to NFC0SR.NFCRXFRAMEF	24
96	NFC0	NFC0 Tag-Response Transmit-Done Interrupt IRQB_NFC0_TXDONE	Write 1 to NFC0SR.NFCTXDONEF	24
97	NFC0	NFC0 RX CRC / Parity Error Interrupt IRQB_NFC0_CRCERR	Write 1 to NFC0SR.NFCCRCERRF and NFCPARERRF	24
98	GPIO4	GPIO4 Bit 0 Interrupt IRQB_GPIO4_B0	Write 1 to P4IF bit 0	12
99	GPIO4	GPIO4 Bit 1 Interrupt IRQB_GPIO4_B1	Write 1 to P4IF bit 1	12
100	GPIO4	GPIO4 Bit 2 Interrupt IRQB_GPIO4_B2	Write 1 to P4IF bit 2	12
101	GPIO4	GPIO4 Bit 3 Interrupt IRQB_GPIO4_B3	Write 1 to P4IF bit 3	12
102	GPIO4	GPIO4 Bit 4 Interrupt IRQB_GPIO4_B4	Write 1 to P4IF bit 4	12

Vector	Source	Description	Clear method	Chapter
103	GPIO4	GPIO4 Bit 5 Interrupt IRQB_GPIO4_B5	Write 1 to P4IF bit 5	12
104	GPIO4	GPIO4 Bit 6 Interrupt IRQB_GPIO4_B6	Write 1 to P4IF bit 6	12
105	GPIO4	GPIO4 Bit 7 Interrupt IRQB_GPIO4_B7	Write 1 to P4IF bit 7	12
106	GPIO5	GPIO5 Bit 0 Interrupt IRQB_GPIO5_B0	Write 1 to P5IF bit 0	12
107	GPIO5	GPIO5 Bit 1 Interrupt IRQB_GPIO5_B1	Write 1 to P5IF bit 1	12
108	GPIO5	GPIO5 Bit 2 Interrupt IRQB_GPIO5_B2	Write 1 to P5IF bit 2	12
109	GPIO5	GPIO5 Bit 3 Interrupt IRQB_GPIO5_B3	Write 1 to P5IF bit 3	12
110	GPIO5	GPIO5 Bit 4 Interrupt IRQB_GPIO5_B4	Write 1 to P5IF bit 4	12
111	GPIO5	GPIO5 Bit 5 Interrupt IRQB_GPIO5_B5	Write 1 to P5IF bit 5	12
112	GPIO5	GPIO5 Bit 6 Interrupt IRQB_GPIO5_B6	Write 1 to P5IF bit 6	12
113	GPIO5	GPIO5 Bit 7 Interrupt IRQB_GPIO5_B7	Write 1 to P5IF bit 7	12
114	RTC0	RTC0 combined alarm/periodic-tick Interrupt IRQB_RTC0	See chapter	25
115	PWM0	PWM0 fault-trip Interrupt IRQB_PWM0_FAULT	See chapter	26
116	PWM0	PWM0 period-event Interrupt IRQB_PWM0_EVT	See chapter	26
117	OW0	OW0 1-Wire combined transaction-complete/error Interrupt IRQB_OW0	See chapter	27
118	DMA0	DMA0 combined channels-done Interrupt IRQB_DMA0_DONE	Write 1 to the set DMA0SR.DMADONE channel bits	28
119	DMA0	DMA0 error (deny/LEN0/misalign/out-of-window) Interrupt IRQB_DMA0_ERR	Write 1 to the set DMA0SR.DMAERR channel bits	28
120	NPU	NPU0 think-done Interrupt IRQB_NPU0_TD	Write 1 to NPUSR.NPUTHINKDONE	17
121	TRNG0	TRNG0 combined data-ready/health-alarm Interrupt IRQB_TRNG0	See chapter	29
122	I2CT0	I2CT0 combined address-match/error Interrupt IRQB_I2CT0_AE	See chapter	30
123	I2CT0	I2CT0 combined tx-ready/rx-full Interrupt IRQB_I2CT0_DATA	See chapter	30
124	AFE0	AFE0-3 combined result-ready / error Interrupt (OR of the four sites) IRQB_AFE	Per site: write 1 to AFExSR.AFEDRDY (pop) until AFECNT = 0, and to AFEOVF / AFETO	32

Figure 33.1, in the IRQROUTER chapter (Chapter 33), draws the whole fabric in one view: the source population of Table 9.1 collapsed by type, the IRQROUTER as the spine every one of those sources terminates in, and the five harts with the three lines each of them takes. It shows the one asymmetry: meip arrives from the router, and msip and mtip do not go near it.

9.1 Exceptions

In the legacy trap delivery mode selected at reset there is no recoverable exception: an illegal instruction, including an access to an unimplemented CSR, puts the hart into a terminal trap state (Section 7.4.1). Where the standard delivery mode is configured and selected, exceptions vector through mtvec with the mcause codes of Table 8.2 in Chapter 8. Table 9.2 lists the causes that originate inside the hart rather than on a numbered vector.

Table 9.2: CPU-internal interrupt causes

Cause	Description
Timer	CPU internal timer
Instruction	EBREAK, ECALL, or illegal instruction
Bus error	Unaligned memory access

9.2 Interrupt vector table

The interrupt vector table (IVT) is located at the base of each hart's TCM, beginning at address 0x8000. Unlike a pointer-based vector table, the VestaRV IVT contains direct jump instructions to the interrupt service routines (ISRs): each entry is a 4-byte RISC-V jump instruction (for example j handler) that vectors execution immediately to the corresponding ISR when an interrupt occurs. This eliminates the indirection of loading a function pointer and gives a deterministic, low-latency interrupt response.

The hardware does not mandate a specific IVT content, so software populates the IVT with jump instructions during initialisation. The generated linker scripts and startup code place the IVT at the beginning of the TCM, and the generated header files provide macros for ISR placement and IVT entry configuration.

9.3 Interrupt entry, exit and recursion

When an interrupt is taken, the hardware saves a minimal context: it pushes the return program counter (one 32-bit word) onto the stack at sp−4, decrements sp by 4, and vectors execution through the IVT entry for the interrupt. No other state is saved by hardware. Software is responsible for saving and restoring every general-purpose register the ISR uses. The generated runtime provides an interrupt entry wrapper (in the startup code) that saves the general-purpose registers on the stack (about 120 bytes per interrupt level), calls the C-level handler, restores the registers, and returns with the retirq instruction, so C-coded ISRs need no special handling. Hand-written assembly ISRs must perform the equivalent save and restore themselves. Executing retirq pops the hardware-saved program counter, increments sp by 4, and resumes execution at the interrupted location.

Caution: Because the first thing interrupt entry does is a store at sp − 4, the stack pointer must point at valid, writable private memory before interrupts are enabled. This applies to every hart: a freshly released tile hart must load its stack pointer before unmasking interrupts or going to sleep (Chapter 5).

Interrupt handling is non-recursive: a running ISR is never preempted, and further pending interrupts (including a pending `CLINT` interrupt during a vector-85 service) are taken after the current ISR returns. Stack allocation must cover one interrupt level: the 4-byte hardware-saved return address plus the handler's register frame (about 120 bytes for the runtime wrapper) and the ISR's local variables. Source arbitration happens in the `IRQROUTER`; the core itself has no priority tiers.

9.4 Sleep mode and interrupt wake-up

If an interrupt occurs while the CPU is in sleep mode, the processor wakes from sleep and services the interrupt. On execution of the `retirq` instruction, the CPU returns to sleep mode unless the ISR or the woken code reconfigures the clock and power settings. During sleep mode the CPU clock is gated to conserve power, and all register contents and the program counter are retained. If the clock source for MCLK is disabled during sleep, an enabled interrupt powers up and enables the source clock for the duration of interrupt servicing.

Clock and power management, including sleep mode configuration, are controlled through the `SYSTEM` peripheral (Section 16.3.3). If SMCLK is disabled, peripherals in the SMCLK domain cannot assert their interrupts and cannot wake the CPU.

10 Core performance and instruction timing

The VestaRV core is a multi-cycle machine on a single unified instruction and data bus, but it issues the fetch of the next instruction during the execute cycle of the current one. Most instructions therefore retire at one per MCLK cycle, and the core's measured cycles-per-instruction (CPI) is much closer to 1 than the term "multi-cycle" on its own would suggest: over the benchmark suite of Section 10.3 it is 1.260. This chapter gives the cost of every instruction class and reports the CPI measured on that suite.

Every figure below is for one hart executing out of its own TCM with no shared-bus contention. An access that crosses the arbiter into the shared window, or that is made while another hart holds the grant, adds stall cycles on top of these numbers (Chapter 5). The core's clock is gated while it waits, so a stalled cycle is still an MCLK cycle in which nothing retires and it is charged to CPI like any other.

10.1 Instruction timing

Table 10.1 gives the cost of each instruction class. A row for an optional extension applies only when that extension is configured for this build; a disabled extension's instructions raise an illegal-instruction trap instead of executing. Atomic memory operations are the one class not re-measured here, because the bare-core bench has no second master; their 5-cycle cost is the design value.

Table 10.1: Measured instruction timing, in MCLK cycles.

Instruction class	MCLK cycles
Arithmetic and logical (`add`, `andi`, `lui`, `auipc`)	1
Shift (`sll`, `srl`, `sra` and immediate forms)	1
Comparison (`slt`, `sltu` and immediate forms)	1
Bit manipulation (Zba, Zbb, Zbc, Zbs)	1
Multiply (M): `mul`, `mulh`, `mulhu`, `mulhsu`	1
Divide and remainder (M): `div`, `divu`, `rem`, `remu`	36
Load and store (`lw`, `lh`, `lb`, `sw`, `sh`, `sb`)	2
Conditional branch, whether taken or not taken	1
Jump and link (`jal`, `jalr`)	1
CSR read (`csrr`)	1
Atomic memory operations (A)	5
Additional penalty: 32-bit instruction straddling a word boundary, reached sequentially	0
Additional penalty: the same, reached by a taken branch or jump	+1

Three entries are worth drawing out. Hardware multiply is single-cycle, so `mul` costs no more than `add` and there is no reason to hand-expand a multiplication into shifts and adds. Divide and remainder share one iterative divider and are, by a wide margin, the most expensive instructions the core executes; where the divisor is a constant, the compiler's multiply-and-shift expansion is far faster than `div` and is worth checking for in the disassembly of any hot loop. And a conditional branch costs the same whether it is taken or not, because the core resolves the condition and issues the redirected fetch in the same cycle: there is no branch predictor, and no misprediction penalty to design around.

The two straddling rows are one instruction class split by how the core arrives at it, and Section 10.2 is about that split.

10.2 Compressed instructions and fetch alignment

The core fetches one 32-bit word per bus cycle. With the C extension enabled instructions are only guaranteed 2-byte aligned, so a 32-bit instruction may straddle a word boundary, and the core must have both halves before it can decode. Compressed (2-byte) instructions and word-aligned 32-bit instructions are never affected.

The core covers most of that case in hardware, with a fetch-ahead. When it is about to step sequentially onto a straddling 32-bit instruction, it can already see that instruction's lower half in the word it is holding, and the fetch its ordinary sequencing would issue is for the word it is holding already, a bus cycle that would carry nothing new. The fetch-ahead spends that cycle on the next word instead and latches the lower half at the same edge, so the straddling instruction dispatches in one cycle and costs nothing at all. The whole mechanism is one flip-flop, it issues no bus cycle the core would not have issued one cycle later, and it changes cycle counts only, never an architectural result. It is the `core.fetchAhead` knob of Table 2.1 (the `ENABLE_IF_AHEAD` generic on the core), it is on in this build, and turning it off is a performance choice and nothing else.

What the fetch-ahead cannot cover is a taken control transfer whose target is a half-word-aligned 32-bit instruction. It arms only on a sequential advance, because that is the only case in which the core can prove, from a half-word it can already see, what it is about to fetch; after a taken branch or jump the core arrives at the target holding nothing, and genuinely needs two fetches before it can decode. That case still costs one extra cycle, and it is the whole of the residual reported below. Both rows of Table 10.1 are measured directly: a straight-line block in which every 32-bit instruction straddles runs at 1.000 cycles per instruction, while a block of a taken branch, a straddling 32-bit target and a compressed instruction (three instructions that cost one cycle each on their own) takes 4 cycles.

Table 10.2 isolates what the C extension still costs by rebuilding the whole benchmark suite for RV32IMA instead of RV32IMAC. The compiler emits the identical instruction sequence in both builds (the retired-instruction counts match exactly, benchmark for benchmark) and only the encoding differs, so the entire difference in cycle count is the straddling-fetch penalty that survives the fetch-ahead. Across the suite it is 0.031 CPI, or 2.2% of all cycles.

Table 10.2: Straddling fetches that survive the fetch-ahead, measured with and without the C extension.

Benchmark	CPI (RV32IMAC)	CPI (RV32IMA)	Penalty, CPI	Cycles lost
`dhrystone`	1.504	1.481	0.022	1.5%
`median`	1.398	1.397	0.000	0.0%
`memcpy`	1.726	1.726	0.000	0.0%
`multiply`	1.163	1.014	0.148	12.8%
`qsort`	1.398	1.330	0.068	4.8%
`rsort`	1.412	1.412	0.000	0.0%
`towers`	1.682	1.681	0.002	0.1%
`vvadd`	1.374	1.374	0.000	0.0%
Aggregate	**1.443**	**1.412**	**0.031**	**2.2%**

The aggregate row weights each benchmark by its retired-instruction count, and covers the eight benchmarks of Table 10.3 that were rebuilt both ways; `spmv` was not, which is why this aggregate differs from the one in that table.

Four of the eight benchmarks pay nothing measurable: their straddling instructions are all reached sequentially, and the hardware absorbs every one. What is left is concentrated in short, branch-dense loops, and `multiply` is the extreme case: its inner loop is seven instructions long and, in the measured

build, its head is a half-word-aligned 32-bit `and`, so the loop's own backward branch lands on a straddling instruction on every iteration, and one loop head accounts for the entire 12.8%.

That is the whole of the alignment advice. Aligning a hot loop's body buys nothing: the fetch-ahead already covers straight-line code, and a body of compressed instructions never straddled in the first place. What still pays is aligning branch targets: loop heads, and the entry of a hot function reached by a call. In C, `-falign-loops=4` does exactly that across a translation unit, and it complements hand alignment rather than replacing it: rebuilding `multiply` with it moves that one loop head onto a word boundary and recovers the entire residual, bringing the benchmark back to the RV32IMA figure in the table above, while an assembly hot spot or a computed jump target is aligned by hand with `.p2align 2` on the label. None of this is an argument against the C extension, which is configured on by default and earns its place by shrinking code substantially, decisive when a hart's private TCM is 8 KiB.

10.3 Measured CPI

Table 10.3 reports CPI over nine benchmarks derived from the riscv-tests benchmark suite, compiled `-O2` for RV32IMAC with the Zb* extensions enabled. The measured region is each benchmark's own timed kernel, so it excludes program startup, the cache-warming pass, and result verification.

Table 10.3: Measured CPI by benchmark (timed kernel, single hart, no bus contention).

Benchmark	Instructions retired	CPI
`spmv`	802,447	1.143
`multiply`	20,892	1.163
`vvadd`	2,408	1.374
`median`	4,015	1.398
`qsort`	123,205	1.398
`rsort`	146,548	1.412
`dhrystone`	200,523	1.504
`towers`	3,747	1.682
`memcpy`	11,017	1.726
Aggregate	**1,314,802**	**1.260**

The aggregate is the total cycle count over the total retired-instruction count, so each benchmark contributes in proportion to the work it does. Read against Table 10.1, the spread has a simple explanation: the further a workload sits above 1.0, the more of its instruction mix is loads and stores, which cost two cycles each on the unified bus. Fetch alignment barely enters into it: `multiply` is register-bound and runs at 1.163, of which 0.148 is the one unaligned loop head of Section 10.2 and the rest is its own work, and `towers` and `memcpy` sit at the other end because they are dominated by memory traffic. Branches move none of it, since they are free.

10.4 Measuring application code

The core implements the `cycle`/`cycleh` and `instret`/`instreth` counters, and their machine-mode aliases `mcycle` and `minstret`, in every configuration. `cycle` counts MCLK cycles, free-running and independent of whether the core is stalled; `minstret` counts instructions actually retired. Bracketing a region with `rdcycle` and `rdinstret` and taking the differences therefore gives that region's exact cycle count and CPI on hardware, with no external instrumentation:

```
unsigned c0, i0, c1, i1;
__asm__ volatile ("rdcycle %0; rdinstret %1" : "=r"(c0), "=r"(i0));
hot_loop();
__asm__ volatile ("rdcycle %0; rdinstret %1" : "=r"(c1), "=r"(i1));
/* cycles = c1 - c0, instructions = i1 - i0 */
```

`time` and `timeh` are not implemented in any configuration, because no real-time source is wired to the core, and reading either raises an illegal-instruction trap. Use `cycle` for elapsed time, converting through the known MCLK frequency, or a timer peripheral for wall-clock measurement.

10.5 Measurement methodology

The numbers in this chapter are measured on the RTL, not estimated. Each benchmark is run to completion in a VHDL simulation of the bare core with a one-cycle-latency memory and no bus back-pressure, which is the timing a single hart sees executing from its own TCM. Cycles are counted on the free-running core clock, so any cycle in which the core's clock is gated is still counted. Instructions are counted from the core's internal instruction-retire strobe, the same signal that increments `minstret`, so the instruction count is architectural rather than an estimate derived from the simulation. The core under measurement is configured as this build ships it, fetch-ahead included.

The per-class costs in Table 10.1 come from paired micro-kernels: a loop containing 64 copies of the instruction under test is run against an otherwise identical empty loop, and the difference in cycles divided by the difference in retired instructions gives the marginal cost of that instruction alone, with the loop's own overhead subtracted out. Every micro-kernel body is word-aligned, so no class is charged a fetch cost that is not its own, with two deliberate exceptions: the two kernels that measure the straddling rows are built to straddle. One is a straight-line block of four instructions, two of them straddling 32-bit ones and every one of them reached sequentially, which measures 1.000 cycles per instruction; the other is a taken branch onto a straddling 32-bit target followed by a compressed instruction, three instructions whose measured classes cost one cycle each, which measures 4 cycles for the block.

11 Debug

11.1 Overview

Hardware debug reaches into the chip over dedicated wires that depend on no software: it stops a hart between instructions, reads and writes its registers and memory, and starts it again. It works on a hart stuck in a loop, on a hart that has taken an illegal instruction and stopped retiring, and, with halt-on-reset armed, on a hart before its first instruction. The Forth monitor over `UART0` (Section 35) is the cooperative alternative and needs the chip to be executing correctly enough to answer.

- IEEE 1149.1 JTAG transport on five pins, with a 5-bit instruction register and a 41-bit DMI data register
- One Debug Module for the whole chip, reaching every hart, implementing RISC-V debug specification 1.0 (`version` = 3)
- Halt, resume, halt-on-reset and single-step per hart; halt groups across harts
- Abstract access-register commands with one data word and a two-word program buffer plus an implicit third word
- Debug mode in the hart with four CSRs (`dcsr`, `dpc`, `dscratch0`, `dscratch1`) and the `DRET` instruction
- Memory access through the halted hart's own load and store instructions; no system bus access, no hardware triggers

This configuration includes the debug system; the sections below document it.

11.2 Block diagram

Figure 11.1 shows the debug path as built in a configuration with the debug system: the probe and the transport on `TCK`, the two toggle signals that cross into the main clock domain, and the Debug Module as one more master on the shared bus with direct halt and resume wires to every hart.

11.3 Functional description

11.3.1 JTAG

JTAG is the IEEE 1149.1 serial protocol: four mandatory wires and an optional fifth, driving a sixteen-state Test Access Port (TAP) whose only input is `TMS`.

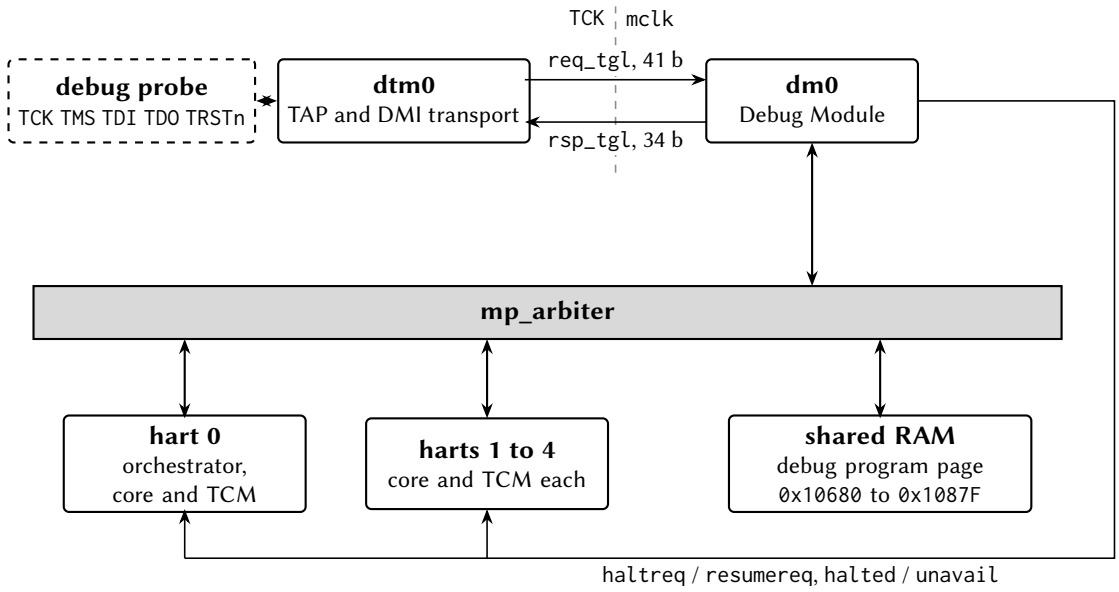


Figure 11.1: Block diagram of the debug path, from the probe to the harts.

Wire	Direction	Purpose
TCK	in	Test clock, supplied by the probe. Everything on the port is synchronous to it; it has no relationship to any clock inside the chip.
TMS	in	Test mode select, sampled on every rising TCK edge; this one wire drives the state machine.
TDI	in	Serial input to the selected shift register.
TDO	out	Serial output of that register.
TRSTn	in	Optional asynchronous reset of the port; present on this chip.

The states form two lobes, one for scanning an instruction and one for scanning data, joined at a common idle state. Test-Logic-Reset is the reset state, and holding TMS high for five TCK clocks reaches it from any state, which is the standard's guaranteed recovery for a probe that has lost track of the machine. The IR lobe (Select-IR, Capture-IR, Shift-IR, Exit1-IR, Pause-IR, Exit2-IR, Update-IR) loads the instruction register; the DR lobe loads whichever data register the current instruction selects. Figure 11.2 is the machine with every transition this chip implements.

A scan has three phases: Capture loads the selected register in parallel from what it shadows, Shift moves one bit in from TDI and one out on TDO per TCK (least-significant bit first), and Update commits the shifted-in value. One scan therefore reads the old contents and writes the new ones at once, so a debugger's traffic is a stream of fixed-length shifts rather than read-then-write pairs. TMS and TDI are sampled on the rising TCK edge and TDO changes on the falling edge, which is why its transitions sit half a cycle after the others. Figure 11.3 follows one four-bit scan.

The port is passive with respect to the chip's own clocks: it answers while the harts run, while they are halted, and while they are held in reset. Because the standard's BYPASS instruction selects a single flip-flop, several devices can be chained (TDO of one to TDI of the next) and a debugger addresses one while the others contribute one bit of padding each.

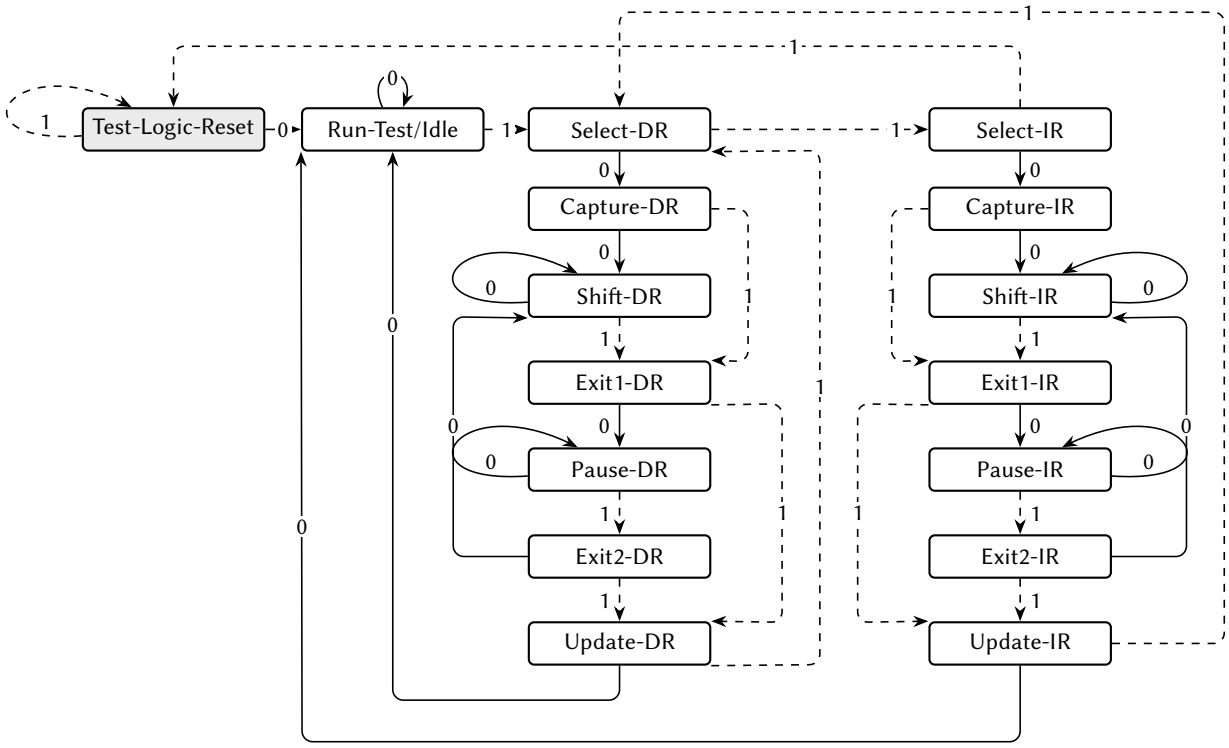


Figure 11.2: State diagram of the sixteen-state JTAG test access port.

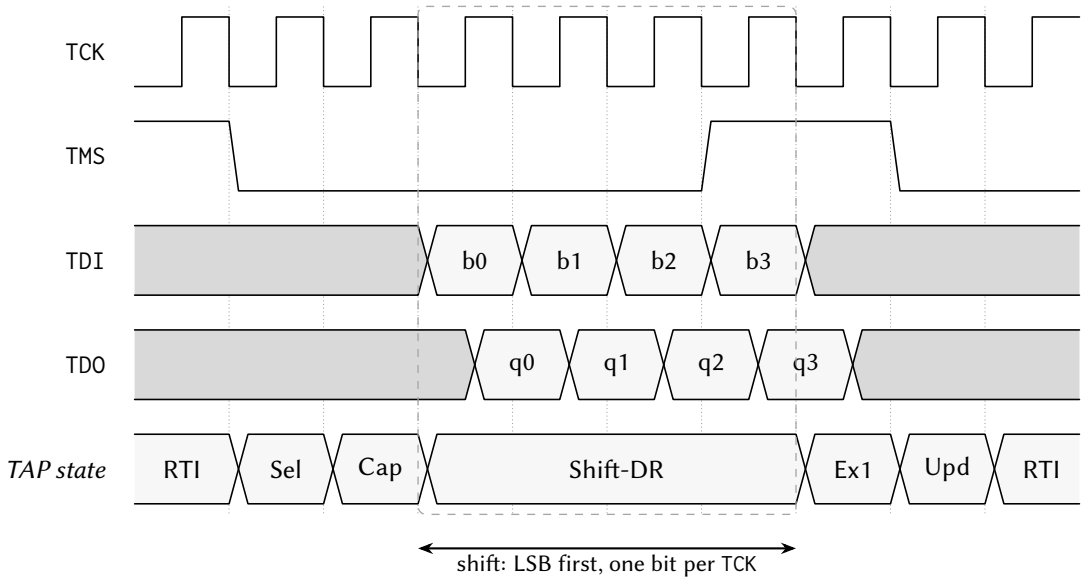


Figure 11.3: Timing diagram of one four-bit JTAG data-register scan.

11.3.2 The RISC-V debug stack

The RISC-V debug specification defines the meaning of the scanned bits in four layers, each independent of the internals of the one below it, so that one debugger drives different chips.

Layer	What it is
DTM	The Debug Transport Module: the TAP plus the logic that turns one DR scan into one transaction on the interface below. It has no knowledge of harts.
DMI	The Debug Module Interface: a 7-bit address, 32-bit data bus with three operations (no-op, read, write). Anything that drives a DMI can debug the chip; JTAG is the transport this chip provides.
DM	The Debug Module: registers on the DMI implementing run control (select a hart, halt it, query its state, run a short program on it, resume it). One DM serves every hart.
Debug mode	A state of the hart, effectively a privilege level above machine mode, in which the hart executes the debugger's code and has control registers that exist only there.

A halted hart is not a frozen hart. Halting does not stop the clock; it diverts the hart, between instructions, to a fixed address where the debugger's code waits, and the hart executes there until told what to do. To read a register the debugger writes a two- or three-instruction program into memory, has the hart run it, and reads back where the program put the answer. Every property that follows derives from this: the debugger needs memory for its code, a register read can report an exception, the hart's `s0` and `s1` must be saved before anything else, and memory accesses on the debugger's behalf go through the ordinary bus with ordinary arbitration.

11.3.3 Debug port

The debug port occupies five pins, present only on the LQFP-100 package; a smaller package has the debug system in silicon with nowhere to bond it. The pins are ordinary 3.3 V digital I/O on the `VDDPST/VSSPST` domain, may be left unconnected, and are placed on the south and east edges away from the analog band.

Pin	Signal	I/O	Idle state when unconnected
47	`TCK`	Input	Pulled low.
48	`TMS`	Input	Pulled high, as IEEE 1149.1 requires, so that five clocks from a disconnected probe still reach Test-Logic-Reset.
49	`TDI`	Input	Pulled high.
50	`TDO`	Output	Driven; held quiet between shift states so the pin is well behaved in a chain.
51	`TRSTn`	Input	Pulled low; see below.

The pull on `TRSTn` is a deliberate deviation from IEEE 1149.1, which wants the pin to idle high so that a port with a floating reset is available to a probe. This chip pulls it low, holding the TAP in reset, because the transport is optional: a board that does not use it leaves all five pins unconnected, and the desired behaviour is then a port that is inert rather than one that is attachable by anyone who touches the pins. A board that uses the port drives `TRSTn` high.

A DR scan of IDCODE returns `0x1CA57EEF`: version `1`, part number `0xCA57`, manufacturer `0x777`, mandatory LSB `1`. The manufacturer field is not a valid JEDEC JEP106 code and is not registered to anyone;

the value is deliberately invalid rather than borrowed, so discovery software identifies the part by its full IDCODE.

11.3.4 Debug Transport Module

The DTM implements the TAP and four data registers. The instruction register is five bits wide and resets to IDCODE (Table 11.1).

IR value	Instruction	DR width	Selects
`0x01`	IDCODE	32	The identification value. The reset instruction, so the first DR scan after a port reset returns the IDCODE.
`0x10`	DTMCS	32	The transport's control and status register.
`0x11`	DMI	41	One transaction on the Debug Module Interface.
`0x1F`	BYPASS	1	A single flip-flop, for a multi-device chain.
any other	BYPASS	1	Every unrecognised instruction selects the one-bit chain, so the selected length never depends on what was scanned before.

Table 11.1: Debug Transport Module instruction register values.

Test-Logic-Reset sets the instruction register to IDCODE and nothing else: it does not clear the sticky status below and does not discard a waiting result, so a debugger that has recovered a lost TAP still clears the status with `dmireset`. `TRSTn` resets the entire transport.

DTMCS reports the transport's capabilities and carries the two recovery strobes:

Bits	Field	Meaning
`[3:0]`	`version`	1: satisfies the 0.13 and 1.0 debug specifications.
`[9:4]`	`abits`	7: the DMI address is seven bits, so the DMI register is $7 + 32 + 2 = 41$ bits.
`[11:10]`	`dmistat`	Sticky transaction status.
`[14:12]`	`idle`	7: Run-Test/Idle clocks to insert between a DMI scan and the scan that collects its result.
`[16]`	`dmireset`	Write 1 to clear `dmistat` and abandon any outstanding transaction. Reads `0`.
`[17]`	`dmihardreset`	Write 1 to do the above and clear the stored result. Reads `0`.

The `idle` value is derived from the real latency of the slowest transaction class. A DMI access to one of the Debug Module's own registers completes in eight to ten cycles of the main clock; an access to `data0` or a program-buffer word is proxied into shared memory through the arbiter and takes tens of cycles. Against a `TCK` period of at least 140 ns (7.14 MHz or below) and a 24 MHz main clock, the worst class needs six idle cycles and 7 gives one cycle of margin. It is a recommendation, not a guarantee: at a faster `TCK`, or while the harts contend for the shared bus, a transaction can still be outstanding when the debugger returns, which is reported as busy and answered by clearing the status and reissuing.

The DMI data register is 41 bits (Figure 11.4): `op` `[1:0]`, `data` `[33:2]`, `address` `[40:34]`. On the way in, `op` is `00` for no operation, `01` for read and `10` for write; on the way out it is `00` for success, `10` for failed and `11` for busy. A no-operation scan starts nothing, which makes a capture-only scan (zeros shifted in to read the previous result) safe. The register is shifted least-significant bit first, so `op` goes in and comes out first; the field widths in the figure are drawn for legibility and the bit numbers are exact.

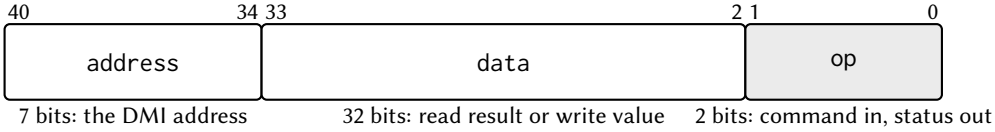


Figure 11.4: Field layout of the 41-bit DMI data register.

`dmistat` is sticky with two values: `3` (busy) if a scan is captured while a transaction is in flight or a new transaction is scanned in while one is outstanding, and `2` (failed) if a transaction fails, for instance on a DMI address the Debug Module does not implement. While `dmistat` is nonzero, further DMI scans are dropped, so that a debugger cannot issue a long sequence of writes after the first one failed. The status is cleared by `dmireset` or `dmihardreset` and by nothing else; neither Test-Logic-Reset nor a system reset clears it. Figure 11.5 follows one transaction across the clock boundary.

The Update-DR of a DMI scan writes the 41-bit request into a hold register and flips `req_tgl`; the main-clock side synchronises the toggle through three flops and presents the held payload as `dmi_req_valid` for exactly two cycles, retiring it on the registered `dmi_req_ready`. The Debug Module's re-accept lockout is a timer, not a handshake: it reopens 9 `mclk` after a capture. A master that held `valid` until its response came back would still be asserting it then and would earn a second, duplicate accept: two responses for one request, sliding every later pair by one. The symptom is not a wrong answer; it is the previous answer. Two cycles leaves seven inside the window. The response latches `op` and `data` into a 34-bit hold and flips `rsp_tgl`, which crosses back through three `TCK` flops before the shadow register updates.

The `idle` value of 7 follows from the same path. A debugger sees the result no earlier than 3 `TCK` (the response synchroniser) plus $\lceil t_{\mathrm{DM}}/T_{\mathrm{TCK}} \rceil$, where t_{DM} is 8 to 10 `mclk` for a Debug Module register and tens of cycles for `data0` or a program-buffer word, which are proxied into shared RAM through the arbiter. At $\mathtt{TCK} \leq 7.14$ MHz against a 24 MHz `mclk` that is 6 cycles for the worst class; 7 adds one of margin and is the largest value the 3-bit field can hold. A faster `TCK`, or a contended bus, can still report busy.

11.3.5 Debug Module

There is one Debug Module for the chip. It runs on the always-running main clock, is reached by the DTM through the DMI, and, because it must place instructions where a hart will fetch them, it is also a master on the shared-memory arbiter alongside the harts (Section 5). Table 11.2 is its DMI address map.

Table 11.2: Debug Module registers at their DMI addresses.

DMI	Register	Purpose
`0x04`	`data0`	The abstract-command data word, backed by shared memory.
`0x10`	`dmcontrol`	Run control: enable the module, select a hart, halt it, resume it.
`0x11`	`dmstatus`	The selected hart's state and the module's capabilities. Read-only.

Table 11.2 continued on next page

Table 11.2 continued from previous page

DMI	Register	Purpose
0x12	hartinfo	Reads zero: no scratch registers and no debugger-visible data0 address. Read-only.
0x13	haltsum1	Reads zero. Present because debuggers probe it during discovery.
0x16	abstractcs	Abstract-command status: busy, error code, sizes.
0x17	command	Write to start an abstract command. Reads zero.
0x18	abstractauto	Reads zero; automatic re-execution is not implemented.
0x20	progbuf0	Program-buffer word 0, backed by shared memory.
0x21	progbuf1	Program-buffer word 1, backed by shared memory.
0x32	dmcs2	Halt-group membership of the selected hart.
0x40	haltsum0	One bit per hart: halted and reachable. The only status register that ignores the hart selection.

Any other DMI address returns failed, which because dmistat is sticky also stalls the transport until cleared. haltsum1 and abstractauto answer with a successful zero because debuggers routinely read them during discovery. hartinfo reads zero on purpose: with dataaccess = 0 a conforming debugger does not consult dataaddr and reaches memory through the program buffer, which is the only path this chip supports, and the real data0 address does not fit the twelve-bit field.

dmcontrol is where every session starts. dmactive (bit 0) must be written 1 before the module does anything; writing 0 resets the module's own state but not the harts. hartsel (bits [25:16]) selects the hart the status and command registers refer to; all ten bits are implemented, and an index that does not exist is reported as nonexistent rather than clamped, which is what makes hart-count discovery work. haltreq (bit 31) is a level that asks the selected hart to halt while set. resumereq (bit 30) is a write-one request that is queued rather than refused and ignored for a hart that is not halted or not reachable. setresethaltreq and clrresethaltreq (bits 3 and 2) arm and disarm halt-on-reset for the selected hart. ndmreset, hartreset and hasel are not implemented; they read zero and ignore writes.

dmstatus reports version = 3 (specification 1.0), authenticated = 1 (no authentication), hasresethaltreq = 1 and impebreak = 1. Hart state is reported through six pairs of bits, and because one hart is selected at a time both bits of each pair carry the same value:

- allhalted, anyhalted
- allrunning, anyrunning
- allunavail, anyunavail
- allnonexistent, anynonexistent
- allresumeack, anyresumeack
- allhavereset, anyhavereset

abstractcs reports progbufsize = 2 and datacount = 1. busy (bit 12) is set while a command runs, and the module stays readable so the debugger can poll. cmderr (bits [10:8]) is write-one-to-clear:

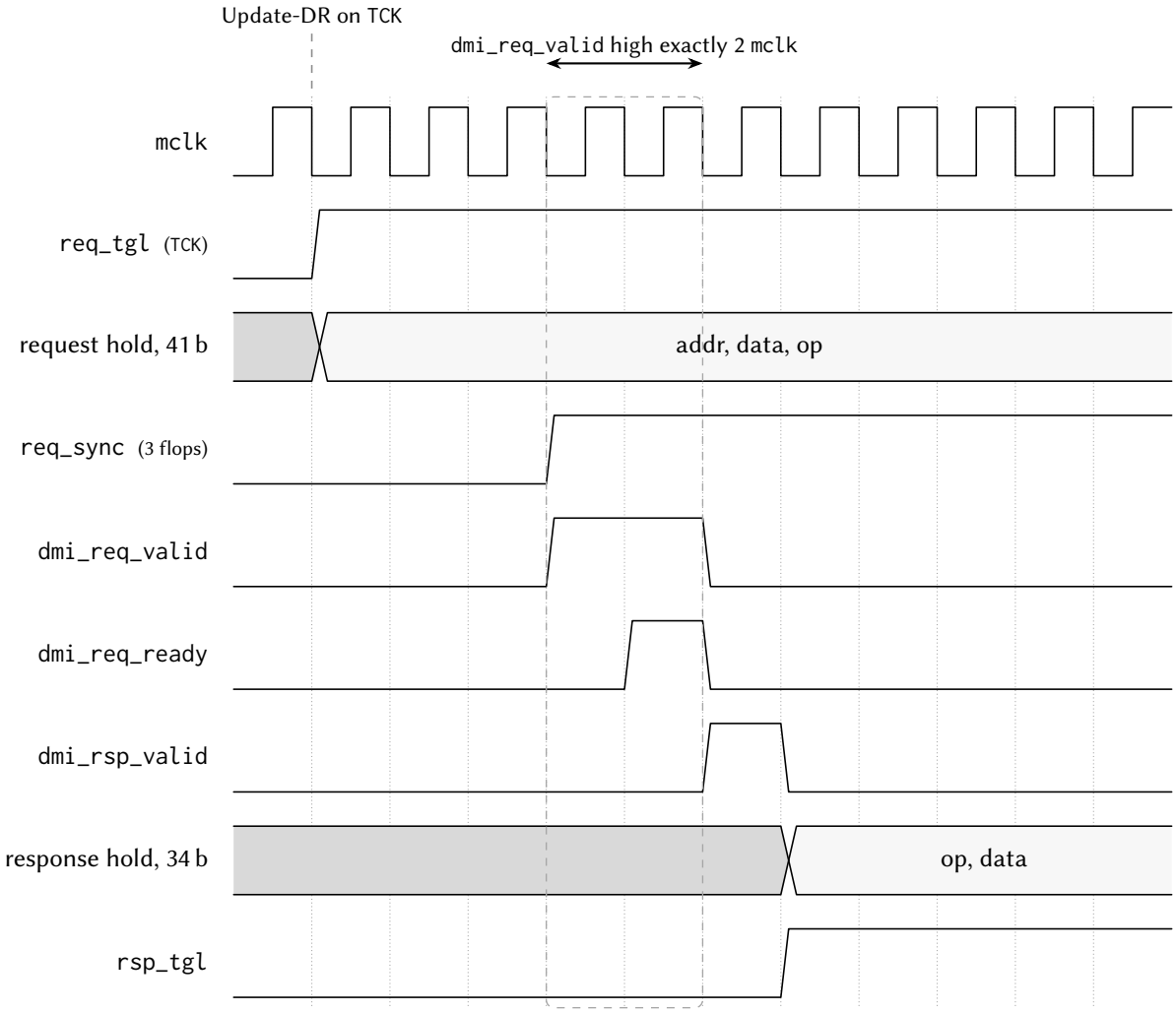


Figure 11.5: Timing diagram of one DMI transaction crossing between the two clock domains.

cmderr	Meaning	Raised when
0	none	No error.
1	busy	A command was written while one was running, or data0 or a program-buffer word was accessed while the bus engine was busy.
2	not supported	The command type is not access-register, or the transfer size is not 32-bit.
3	exception	The hart took an exception while executing the command.
4	halt/resume	The selected hart was not halted when the command was written.
7	other	The command did not complete within the module's internal timeout.

A command written while cmderr is nonzero is ignored entirely: it does not run and does not clear the error.

command implements one command type, access register: it transfers one 32-bit value between data0 and a register of the halted hart, optionally executing the program buffer afterwards. The fields are aarsize ([22:20], which must be 2), postexec (bit 18), transfer (bit 17), write (bit 16) and regno ([15:0]). Register numbers 0x1000 to 0x101F are x0 to x31; any other number is a CSR address. Access memory is not implemented and returns not supported; memory is reached by executing lw or sw from the program buffer.

data0 and the program-buffer words are shared memory, not flip-flops: a DMI access to 0x04, 0x20 or 0x21 becomes a read or write of a word in shared memory issued by the Debug Module through the arbiter. That is why those addresses are the slow class behind idle = 7, and why an access to them while the bus engine is busy answers with zero data and sets cmderr to busy. The program buffer is two DMI-visible words plus an implicit third word supplied by hardware, which is why impebreak reads 1.

11.3.6 Debug mode in the hart

A hart enters debug mode for one of four reasons, recorded in dcsr.cause:

cause	Reason	Notes
1	EBREAK	A software breakpoint, taken only when dcsr.ebreakm is set (or when the hart is already in debug mode).
3	Halt request	The Debug Module asserted haltreq.
4	Single step	The one instruction permitted by dcsr.step has retired.
5	Halt on reset	setresethaltreq was armed when this hart left reset.

On entry the hart saves its program counter to dpc, records the cause and previous privilege level in dcsr, sets debug mode, and jumps to the fixed address 0x00010780 in the shared window. Four CSRs exist only in debug mode:

Address	Register	Contents
0x7B0	dcsr	xdebugver [31:28] reads 4; ebreakm (15) makes EBREAK enter debug mode; ebreaku (12) does the same for user mode where it is configured; cause [8:6] is read-only; step (2) enables single-stepping; prv [1:0] is the privilege level to return to.
0x7B1	dpc	The saved program counter. Bit 0 is hardwired zero; bit 1 is writable only where the compressed extension is configured.
0x7B2	dscratch0	Scratch word for the debugger.
0x7B3	dscratch1	Second scratch word.

Every CSR instruction naming an address in 0x7B0 to 0x7BF is an illegal instruction outside debug mode, so a program cannot read the debugger's saved state or forge an entry. DRET (0x7B200073) leaves debug mode, restoring the program counter from dpc and the privilege level from dcsr.prv; outside debug mode it is an illegal instruction.

Five behaviours decide whether a session works:

- **The halt request is unmaskable.** It is recognised at the thirteen sequencer points at which an interrupt is recognised plus the terminal trap state, so a hart that has taken an illegal instruction and stopped retiring (Section 8.2) can be halted and inspected. Neither mstatus.MIE nor mtrapctl.LEGACY affects it.
- **An EBREAK records its own address in dpc**, not the address after it, so a debugger resuming past a software breakpoint adds the instruction's length itself. Every other cause is taken between instructions and dpc is the resume address.
- **No interrupt is taken in debug mode.** Sources stay pending as levels and are delivered after DRET, so a session neither loses interrupts nor runs the application's handlers underneath the debugger.
- **An exception in debug mode re-enters debug mode** at the same fixed address, leaving dpc, dcsr, mepc, mcause and mtval untouched, which is what makes a faulting abstract command report cmderr = 3 instead of ending the session.
- **Halt-on-reset is sampled once, at reset release.** Arming setresethaltreq affects the next time the hart leaves reset and does not halt a running hart.

Single-step diverts at the stepped instruction's own dispatch, so exactly one instruction retires per step; stepping is inactive inside debug mode. The hart therefore has exactly two states, running and debug mode; the four causes above enter debug mode at the debug entry address, and DRET returns to running at dpc.

11.3.7 Halt groups and unavailable harts

dmcs2 assigns the selected hart to one of eight halt groups; group 0, the reset value, is no group. When any member of a group halts for any reason, including its own EBREAK, the Debug Module requests a halt of every other member, so a breakpoint in one hart stops the cooperating set at very nearly the same instant. Halting is grouped; resuming is done per hart.

Unavailable is a third state, distinct from halted. A hart whose power domain is off cannot answer the Debug Module, and the signals that would report its state are clamped to zero at the domain boundary; a

clamped not-halted is indistinguishable from running, so the module takes a separate unavailable signal from the power controller and consults it before halted or running. The reported state resolves in the order nonexistent, unavailable, halted, running, and a gated hart reports `allunavail`/`anyunavail` with both halted and running clear. A halt request to a gated hart does nothing; the debugger powers the hart up through `PWRCTRL` first, and because a hart returning from power-down cold-boots the ROM and parks, halt-on-reset is armed before powering it up to catch it on its first instruction.

11.3.8 Debug program page

Because a halted hart executes code, the debug system owns one 512-byte region of the shared window:

Address	Contents
`0x10680`	`data0`, the abstract-command data word.
`0x10684`	`progbuf0`
`0x10688`	`progbuf1`
`0x1068C`	The implicit third program-buffer word, written by the Debug Module.
`0x10690` to `0x106EC`	Reserved for the Debug Module.
`0x106F0`, `0x106F4`	The entry stub's repair copies of `s0` and `s1`; not where a debugger reads those registers from.
`0x10700` + $4h$	The per-hart handshake word between the Debug Module and hart h.
`0x10780` to `0x1087F`	The debug entry page, 64 words: the entry stub, the area into which the Debug Module writes each abstract command, and the common exit sequence.

The boot ROM zeroes `0x10000` to `0x107FF` at every boot, so every Debug Module data word below `0x10800` starts from a known value; above `0x10800` nothing is zero-filled, and every word there (the last eight words of the entry stub, the command body and the exit sequence) is code the Debug Module writes before anything reads it. The Debug Module writes the entry page itself, once when `dmactive` is raised and again before acting on any hart it has just seen halt, so nothing is staged in advance and the page cannot be read-only memory. The page is assembled without compressed instructions, because execution of compressed or misaligned instructions from the shared window is outside the tested envelope (Section 5).

While a hart is halted, the single home of the interrupted program's `s0` and `s1` is the CSR pair `dscratch0` and `dscratch1`: a debugger that reads or writes `s0` reads or writes `dscratch0`, and a program-buffer change to `s0` survives to the next read because the exit sequence adopts the architectural pair into the CSRs first. The two words at `0x106F0` and `0x106F4` are the stub's own repair copies for a hart that re-enters debug mode from inside the page; the Debug Module writes them only when it authored a new value, and nothing else reads them. A plain halt and resume with no register writes leaves `s0` and `s1` as it found them.

11.3.9 DMI register map, field by field

Table 11.2 says what each Debug Module register is for; the field-level tables below carry the bits. They are generated from `hdl/common/regs/rdl/debug_module.rdl`, the SystemRDL description that is the single source for every register description in this manual. The Debug Module has no row in the register index (Chapter B) because its registers answer on the DMI, not on the memory bus: no load or store reaches them.

Two conventions differ from Table 11.2. SystemRDL addresses in bytes, so each **Offset** below is the DMI address times four: `dmcontrol`, DMI `0x10`, prints `0x40`. Names carry the `DM` prefix the RTL uses, so `dmcontrol` appears as `DMCONTROL`.

Table 11.3: DEBUG register summary

Offset	Name	Access	Reset	Description
0x10	DMDATA0	rw	0x00000000	Abstract data register 0, datacount = 1
0x14 to 0x3F	-	-	-	Reserved
0x40	DMCONTROL	rw/w	0x00000000	Debug Module control
0x44	DMSTATUS	r	0x00000000	Debug Module status, read only
0x48	DMHARTINFO	r0	0x00000000	Hart information, read only
0x4C	DMHALTSUM1	r0	0x00000000	Halt summary 1, read only
0x50 to 0x57	-	-	-	Reserved
0x58	DMABSTRACTCS	r/rw1	0x00000000	Abstract control and status
0x5C	DMCOMMAND	w	0x00000000	Abstract command
0x60	DMABSTRACTAUTO	rw	0x00000000	Abstract command autoexec
0x64 to 0x7F	-	-	-	Reserved
0x80	DMPROGBUF0	rw	0x00000000	Program buffer word 0
0x84	DMPROGBUF1	rw	0x00000000	Program buffer word 1
0x88 to 0xC7	-	-	-	Reserved
0xC8	DMCS2	rw/w	0x00000000	Halt-group assignment of the selected hart
0xCC to 0xDF	-	-	-	Reserved
0xE0	DMSBCS	r0	0x00000000	System Bus Access control and status
0xE4 to 0xFF	-	-	-	Reserved
0x100	DMHALTSUM0	r	0x00000000	Halt summary 0, read only

11.3.10 DMDATA0: Abstract data register 0, datacount = 1

Offset 0x10 **Reset** 0x00000000 **Width** 32

Abstract data register 0, datacount = 1. Not a flop in this Debug Module: every access is PROXIED to the backing word at DATA0_ADDR through the DM's own shared-bus master, so a read returns whatever the halted hart's abstract command left there. An access while the master engine is busy is refused with abstractcs.CMDERR = BUSY. When abstractauto.AUTOEXECDATA is set, both a read and a write re-issue the last command at the retire of this access.

Bits	Field	Type	Reset	Description
31:0	DMDATA0	rw	0x00000000	Abstract command operand / result word.

11.3.11 DMCONTROL: Debug Module control

Offset 0x40 **Reset** 0x00000000 **Width** 32

Debug Module control. DMACTIVE is the DM's own reset: writing 0 clears haltreq, the reset-halt requests, the halt-group pending set, the resume pending set, busy, cmderr, the plant-owed bits and abstractauto, and the RISING edge of DMACTIVE re-arms the entry-page plant. HAVERESET is deliberately NOT cleared by a dmactive toggle, because its value is a claim about the harts rather than about the DM. NDMRESET is not implemented and reads 0 (WARL).

Bits	Field	Type	Reset	Description
31	DMHALTREQ	rw	0	Halt request for the selected hart, a level held until cleared.
30	DMRESUMEREQ	w	0	Resume request for the selected hart. Queued, never refused and never dropped, but it means something only for a hart that is halted, exists and is available. Reads 0.
29	-	-	-	Reserved. Reads zero; write zero.
28	DMACKHAVERESET	w	0	Acknowledge the selected hart's HAVERESET report. A reset arriving in the same cycle as the acknowledge survives it. Reads 0.
27:26	-	-	-	Reserved. Reads zero; write zero.
25:16	DMHARTSEL	rw	0x000	Hart select. An index at or above the hart count selects a nonexistent hart, which dmstatus reports through ANYNONEXISTENT / ALLNONEXISTENT; the all-ones probe a debugger uses to size the array is handled without indexing the hart arrays.
15:4	-	-	-	Reserved. Reads zero; write zero.
3	DMSETRESETHALTREQ	rw	0	Set the selected hart's reset-halt request (halt it out of reset). Reads back the selected hart's current reset-halt state, so this one bit is a write-1 command in one direction and a status bit in the other.
2	DMCLRRESETHALTREQ	w	0	Clear the selected hart's reset-halt request. SET (bit 3) is evaluated first, so a write with both bits set leaves the request set. Reads 0.
1	-	-	-	Reserved. Reads zero; write zero.
0	DMACTIVE	rw	0	Debug Module active. 0 resets the DM's own state (not the harts); the 0-to-1 edge re-arms the entry-page plant. Every dmstatus poll writes this register with DMACTIVE set, which is why the plant is armed on the EDGE and not on the level.

11.3.12 DMSTATUS: Debug Module status, read only

Offset 0x44 **Reset** 0x00000000 **Width** 32

Debug Module status, read only. With hasel dropped, both halves of every any/all pair mirror the selected hart and BOTH are driven, because OpenOCD misreads them otherwise. VERSION = 3 (debug spec 1.0), IMPEBREAK = 1 (the implicit third program-buffer word is an ebreak), HASRESETHALTREQ = 1 and AUTHENTICATED = 1.

Bits	Field	Type	Reset	Description
31:23	-	-	-	Reserved. Reads zero; write zero.
22	DMIMPEBREAK	r	0	Implicit ebreak after the program buffer. Constant 1.
21:20	-	-	-	Reserved. Reads zero; write zero.
19	DMALLHAVERESET	r	0	Every selected hart has reset since the last acknowledge.
18	DMANYHAVERESET	r	0	Any selected hart has reset since the last acknowledge.

Bits	Field	Type	Reset	Description
17	DMALLRESUMEACK	r	0	Every selected hart has acknowledged its resume request.
16	DMANYRESUMEACK	r	0	Any selected hart has acknowledged its resume request.
15	DMALLNONEXISTENT	r	0	Every selected hart does not exist.
14	DMANYNONEXISTENT	r	0	Any selected hart does not exist.
13	DMALLUNAVAIL	r	0	Every selected hart is unavailable: it exists but the DM cannot reach it, because isolation or reset is holding it.
12	DMANYUNAVAIL	r	0	Any selected hart is unavailable.
11	DMALLRUNNING	r	0	Every selected hart is running.
10	DMANYRUNNING	r	0	Any selected hart is running.
9	DMALLHALTED	r	0	Every selected hart is halted.
8	DMANYHALTED	r	0	Any selected hart is halted.
7	DMAUTHENTICATED	r	0	Authenticated. Constant 1: this DM has no authentication.
6	-	-	-	Reserved. Reads zero; write zero.
5	DMHASRESETHALTREQ	r	0	Reset-halt request supported. Constant 1: the wire exists.
4	-	-	-	Reserved. Reads zero; write zero.
3:0	DMVERSION	r	0b0000	Debug Module version. Constant 3 = debug specification 1.0.

11.3.13 DMHARTINFO: Hart information, read only

Offset 0x48 **Reset** 0x00000000 **Width** 32

Hart information, read only. THE NULL CLAIM: all four fields (NSCRATCH, DATAACCESS, DATASIZE, DATAADDR) read 0, which says there are no scratch registers and no shadowed data region, so a debugger must use the program buffer. That is this design's only path. DATAADDR must NOT advertise DATA0_ADDR: the field is 12 bits and sign-extended, so 0x10680 would publish as 0x680, a resolvable wrong address inside the read-only boot ROM. Clearing DATAACCESS is what makes that unreachable; the DM's own master engine still targets DATA0_ADDR, it simply does not advertise it.

Bits	Field	Type	Reset	Description
31:0	-	-	-	Reserved. Reads zero; write zero.

11.3.14 DMHALTSUM1: Halt summary 1, read only

Offset 0x4C **Reset** 0x00000000 **Width** 32

Halt summary 1, read only. Reads 0 with SUCCESS: a debugger probes this address and must not see a failed transaction. One 32-bit haltsum0 covers every hart count this chip supports.

Bits	Field	Type	Reset	Description
31:0	-	-	-	Reserved. Reads zero; write zero.

11.3.15 DMABSTRACTCS: Abstract control and status

Offset 0x58 **Reset** 0x00000000 **Width** 32

Abstract control and status. PROGBUFSIZE = 2 and DATACOUNT = 1 are constants. CMDERR is sticky and write-1-to-clear (‘cmderr_r <= cmderr_r and not d(10 downto 8)‘), and NO ABSTRACT COMMAND STARTS WHILE IT IS SET: a command write arriving with a standing error is ignored outright, error and all, so the diagnosis is never overwritten with BUSY.

Bits	Field	Type	Reset	Description
31:29	-	-	-	Reserved. Reads zero; write zero.
28:24	DMPROGBUFSIZE	r	0x00	Program buffer size in words. Constant 2, with an implicit third word that is an ebreak.
23:13	-	-	-	Reserved. Reads zero; write zero.
12	DMBUSY	r	0	An abstract command is executing. The command WRITE itself completes immediately; this is what the debugger polls.
11	-	-	-	Reserved. Reads zero; write zero.
10:8	DMCMDERR	rw1	0b000	Sticky command error: 0 none, 1 busy, 2 not supported, 3 exception, 4 halt/resume state wrong. Write 1s to the bits to clear.
7:4	-	-	-	Reserved. Reads zero; write zero.
3:0	DMDATACOUNT	r	0b0000	Number of abstract data registers. Constant 1.

11.3.16 DMCOMMAND: Abstract command

Offset 0x5C **Reset** 0x00000000 **Width** 32

Abstract command. Write-only: the read mux returns 0. Only cmdtype 0 (access register) with aarsize 2 (32-bit) is accepted; cmdtype 2 (access memory) answers CMDERR = NOT_SUPPORTED so the debugger falls back to program-buffer lw/sw through the hart, which is this design’s memory path. A write is refused, with the reason in abstractcs.CMDERR, when a command error is already standing (ignored outright, no new error), when a command is already running (BUSY), when the selected hart is not halted (HALTRES), or when the DM’s master engine is busy (BUSY).

Bits	Field	Type	Reset	Description
31:0	DMCOMMAND	w	0x00000000	Abstract command word: [31:24] cmdtype, [22:20] aarsize, [17] postexec, [16] transfer, [15:0] regno. Reads 0.

11.3.17 DMABSTRACTAUTO: Abstract command autoexec

Offset 0x60 **Reset** 0x00000000 **Width** 32

Abstract command autoexec. AUTOEXECDATA is bit 0 and AUTOEXECPROGBUF is bits 17:16, because datacount is 1 and progbufsize is 2; every other bit is hardwired 0, so the register is WARL by construction and an all-ones write reads back 0x00030001. A write arriving while a command executes DOES NOT LAND and reports CMDERR = BUSY: re-arming the trigger under a running command is exactly the race that rule exists to refuse. Cleared by a DMCONTROL.DMACTIVE toggle.

Bits	Field	Type	Reset	Description
31:18	-	-	-	Reserved. Reads zero; write zero.
17:16	DMAUTOEXECPROGBUF	rw	0b00	Re-issue the last abstract command on any access to progbuf0 (bit 16) or progbuf1 (bit 17).
15:1	-	-	-	Reserved. Reads zero; write zero.

Bits	Field	Type	Reset	Description
0	DMAUTOEXECDATA	rw	0	Re-issue the last abstract command on any access to data0, read or write: a read burst pulls results out and a write burst pushes operands in, and both need the command run once per access.

11.3.18 DMPROGBUF0: Program buffer word 0

Offset 0x80 **Reset** 0x00000000 **Width** 32

Program buffer word 0. Proxied to the backing word like data0, and an access while the master engine is busy is refused with CMDERR = BUSY. A written word equal to the 32-bit ebreak encoding 0x00100073 is SUBSTITUTED with the position-correct jump to the epilogue, and a readback returns the substituted word because no shadow of the original is kept; that substitution does not touch the exception detector, so a program that genuinely traps still reports EXCEPTION.

Bits	Field	Type	Reset	Description
31:0	DMPROGBUF0	rw	0x00000000	Program buffer instruction word 0.

11.3.19 DMPROGBUF1: Program buffer word 1

Offset 0x84 **Reset** 0x00000000 **Width** 32

Program buffer word 1. Same proxy, same busy refusal and same ebreak substitution as progbuf0. The implicit third word is an ebreak (dmstatus.IMPEBREAK = 1), so a two-word program needs no explicit terminator.

Bits	Field	Type	Reset	Description
31:0	DMPROGBUF1	rw	0x00000000	Program buffer instruction word 1.

11.3.20 DMCS2: Halt-group assignment of the selected hart

Offset 0xC8 **Reset** 0x00000000 **Width** 32

Halt-group assignment of the selected hart. HGSELECT must be 0 (harts, not external triggers) and HGWRITE commits GROUP for the SELECTED hart; grouptype is 0. A read returns the selected hart's current group in bits 6:2.

Bits	Field	Type	Reset	Description
31:7	-	-	-	Reserved. Reads zero; write zero.
6:2	DMGROUP	rw	0x00	Halt group of the selected hart. Read returns the group; a write commits it only when HGWRITE is 1 and HGSELECT is 0. The stored group is 3 bits, so bits 6:5 read 0.
1	DMHGWRITE	w	0	Commit GROUP to the selected hart. Reads 0.
0	DMHGSELECT	rw	0	Halt-group selector: 0 = harts. Any other value makes the write a no-op.

11.3.21 DMSBCS: System Bus Access control and status

Offset 0xE0 **Reset** 0x00000000 **Width** 32

System Bus Access control and status. READ-ZERO-SUCCESS and WRITES IGNORED-SUCCESS, both halves load-bearing. A debugger reads this address once per hart during examination and a FAILED read

aborts examination outright, while it writes sbcs to clear sberror and a failed write is one more gratuitous sticky transport event. Zeros are the honest answer, not a placation: SBVERSION 0, SBASIZE 0 and every access-width bit clear is the debug specification's own encoding for no system bus access. DMI addresses 0x39-0x3F remain unimplemented and answer 'failed'.

Bits	Field	Type	Reset	Description
31:0	-	-	-	Reserved. Reads zero; write zero.

11.3.22 DMHALTSUM0: Halt summary 0, read only

Offset 0x100 **Reset** 0x00000000 **Width** 32
Halt summary 0, read only. One bit per hart, set when that hart is halted AND reachable (halted and not unavailable). One 32-bit word covers every hart count this chip supports, which is why haltsum1 reads 0.

Bits	Field	Type	Reset	Description
31:0	DMHALTSUM0	r	0x00000000	Halted-and-reachable bitmap, bit h = hart h.

11.4 Interrupts and events

Not applicable. The debug system raises no interrupt vector; a halted hart is observed through dmstatus, and interrupts pending on a hart in debug mode are delivered after DRET.

11.5 Programming

11.5.1 Halt, read a register, resume

The following session halts hart 2, reads its a1 register and resumes it. Each step is one or more JTAG scans; a DMI read is two scans (one to issue the read and one, after the idle clocks, to collect the result), because the transport returns the previous transaction's result on every capture. Figure 11.6 draws the same steps across the four agents that perform them.

1. Drive TRSTn high, then clock TMS high for five TCK cycles. The instruction register is now IDCODE; a 32-bit DR scan returns 0x1CA57EEF.
2. Scan 0x10 (DTMCS) into the instruction register and read it: version = 1, abits = 7, idle = 7. Insert seven Run-Test/Idle clocks after every DMI scan from here on.
3. Scan 0x11 (DMI) into the instruction register. Every scan below is a 41-bit DR scan of {address, data, op}.
4. Write 0x00000001 to dmcontrol (0x10): dmactive.
5. Write 0x80020001 to dmcontrol: haltreq, hartsel = 2, dmactive.
6. Read dmstatus (0x11) until allhalted (bit 9) is set. allunavail means hart 2 is powered down; allnonexistent means this configuration has fewer than three harts.
7. Write 0x00020001 to dmcontrol to drop the halt request. The hart stays halted, because halting is a state of the hart, not a level the module holds.

8. Write `0x00000700` to `abstractcs` (`0x16`) to clear any stale `cmderr`.
9. Write `0x0022100B` to `command` (`0x17`): `aarsize` = 2, `transfer`, `regno` = `0x100B` (x11).
10. Read `abstractcs` until `busy` (bit 12) is clear, then check `cmderr`.
11. Read `data0` (`0x04`); this access is proxied through shared memory and is the one the seven idle clocks are sized for.
12. Write `0x40020001` to `dmcontrol`: `resumereq`, hart 2 selected. Read `dmstatus` until `allresumeack` (bit 17) is set.

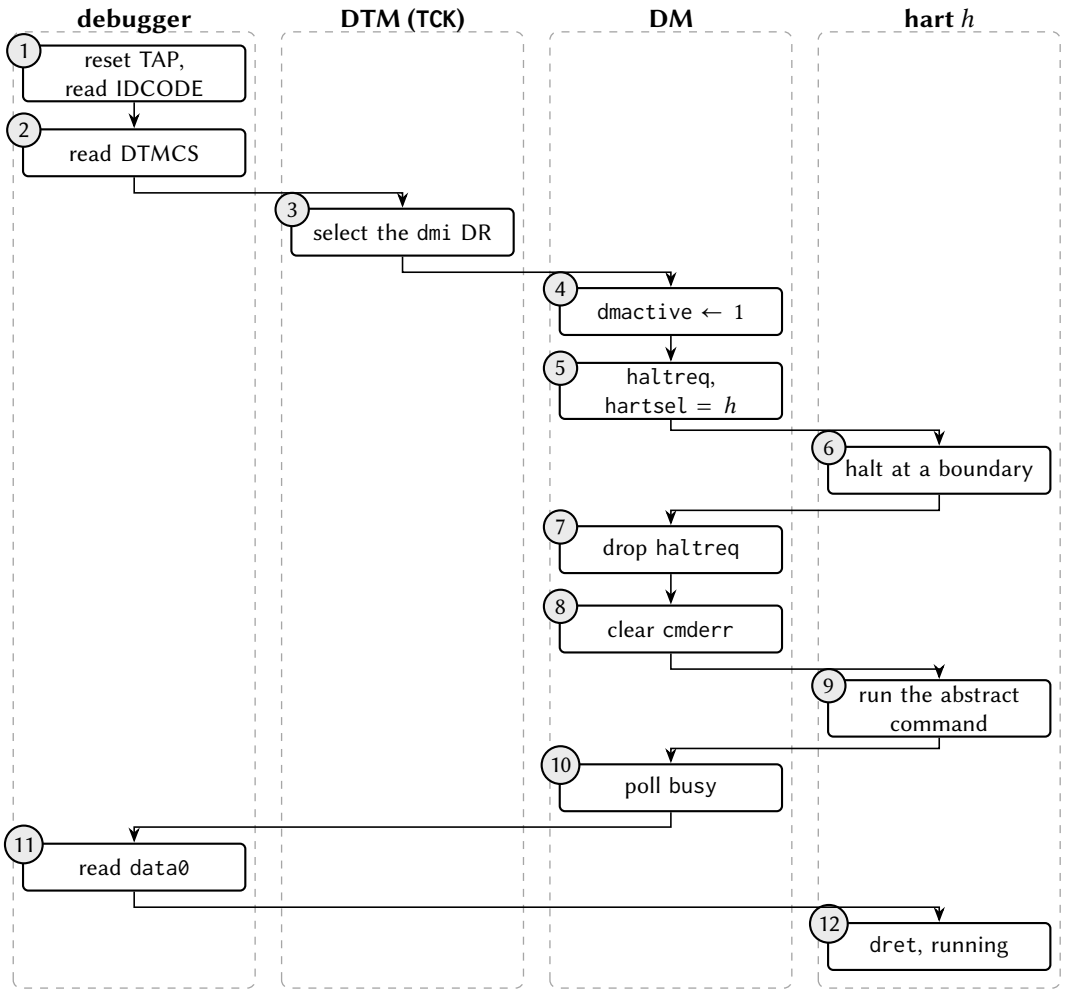


Figure 11.6: Swimlane diagram of the twelve steps across the four agents that perform them.

To write a register, put the value in `data0` first and set `write` (bit 16) in the command. To read or write memory, put an `lw` or `sw` in the program buffer, put the address in a register of the halted hart, and issue a command with `postexec` set; the hart executes the instruction with its own privilege and its own view of the address space. If a scan returns `op` = `10` or `11`, the transport has latched a sticky status and is dropping scans: read DTMCS, write `dmireset`, and reissue.

11.5.2 Connecting a debugger

The part has been brought up end to end against unmodified OpenOCD 0.12.0 and `riscv-none-elf-gdb` 13.2, which attach, enumerate every hart, halt and resume harts individually, read and write registers and memory, set software breakpoints and single-step. The adapter clock bound and the other constraints of that combination are in Section 11.6. The bound follows from the requirement that three `TCK` periods cover one DMI transaction in the module's clock domain, because stock OpenOCD does not honour `idle` and learns the delay from failures:

Traffic	**Minimum** T_{TCK}	**Maximum** f_{TCK}
Register class (`dmcontrol`, `dmstatus`, …)	139 ns	7.19 MHz
Program-buffer class (all memory access)	278 ns	3.60 MHz

11.6 Usage notes

Clock the adapter at 3 MHz or below. Every memory access and software breakpoint goes through the program buffer, whose bound is 3.60 MHz and shrinks under shared-bus contention; the 7.14 MHz port rating applies only to a master that honours `idle`.

Do not infer a reset from `reset halt`. The module has no system reset (`ndmreset` is unimplemented), so the command halts every hart, re-reads the IDCODE and reports success while resetting nothing. For a genuine halt-on-reset, arm `setresethaltreq` and power-cycle the hart's tile through `PWRCTRL`.

Experiment on harts 1 to 4, not hart 0. The per-tile power controller is the lever behind every recovery procedure here; hart 0 has no gate bit, and the only thing that resets it is the `resetn` pin, which also resets the Debug Module and ends the session.

Leave the program buffer through the exit sequence, not `EBREAK`. A DMI write of exactly the 32-bit `EBREAK` encoding into `progbuf0` or `progbuf1` stores a jump to the common exit sequence instead, and reads back as that jump; only that encoding in those two registers is substituted. A program that genuinely traps is still reported as an exception.

Write memory one word at a time. `abstractauto` is unimplemented, and a debugger that assumes it, as OpenOCD's burst-write path does, writes only the first word of a block and drops the rest; loading a program image over the debug port does not work.

Reach memory only through a halted, healthy hart. There is no system bus access: memory is read and written by `lw` and `sw` executed from the program buffer, with the hart's own permissions and view of the address space, so a chip with no debuggable hart cannot have its memory inspected.

Use software breakpoints only. There are no hardware triggers: no watchpoints and no fetch breakpoints. A breakpoint replaces the instruction with `EBREAK` under `dcsr.ebreakm`, so breakpoints cannot be set in the boot ROM or any read-only region; data watchpoints by stepping are impractical.

Keep application software out of `0x10680` to `0x1087F`. The region is ordinary shared RAM, so any master can overwrite it. Most damage repairs itself at the next halt, but a wrong first word at `0x10780` wedges the hart that halts into it: it takes an exception, re-enters at the same address, and is answered from the copy the tile already holds, so it never observes the repair. Recovery is a power cycle of that tile through PWRCTRL.

Build with `priv.trapCsr` when enabling debug. `debug.enable` requires the standard trap architecture (Section 8); the generator rejects the combination without it, because EBREAK is decoded on the standard privileged path.

Resume group members one at a time. Halt groups stop a set together; resume groups are not implemented.

Expect specification behaviour, not simulator behaviour. An unrecognised JTAG instruction selects BYPASS; `dmistat` is driven and reports sticky failure as well as sticky busy; `idle` reports the real requirement; an out-of-range `hartsel` reports nonexistent; `dmstatus.version` is 3. A debugger written against the specification finds the part conformant; one written against a particular simulator's quirks may not.

The debug port is not yet in a signed-off layout. The five pins exist in the chip-level netlist and the LQFP-100 package definition; the current signed-off layout predates them.

Part III Peripherals

12 General-purpose input and output (GPIOx)

12.1 Overview

Each GPIOx peripheral is one 8-pin general-purpose input/output port. A port is a parallel register set, so all eight pins are updated in one access, and every pin is also configurable on its own. Pin y of port x is written Px.y and is controlled by bit y of each port register. Every pin is either in GPIO mode, driven by the port registers, or in alternate-function mode, driven by one of up to eight peripheral functions selected per pin.

- 8 pins per port, individually configurable as input, output, or resistor-terminated input
- Atomic set, clear and toggle registers (PxOUTS, PxOUTC, PxOUTT), so no read-modify-write is needed
- Alternate-function mode per pin (PxSEL) with an 8-way function select per pin (PxAFS)
- Relocatable peripheral inputs, routed by PxAFS independently of PxSEL
- One interrupt per pin, 8 vectors per port, with programmable edge (PxIES)
- Event-fabric task select per pin (PxTASK) in configurations with the event fabric

12.2 Instances and signals

Table 12.1: GPIOx instances

Instance	Base address	Interrupt vector(s)	Pins (AF)
GPIO0	0x4000	1 to 8	P0.0 to P0.7
GPIO1	0x4100	28 to 35	P1.0 to P1.7
GPIO2	0x4800	36 to 43	P2.0 to P2.7
GPIO3	0x4D00	44 to 51	P3.0 to P3.7
GPIO4	0x6300	98 to 105	P4.0 to P4.7
GPIO5	0x6400	106 to 113	P5.0 to P5.7

Table 12.2: GPIOx signals

Signal	Direction	Description	Pin (AF)
P0.0	Bidirectional	General-purpose I/O GPIO0; AF0 CS_FLASH, AF1 TX0, AF2 TX1, AF3 T0CMP0, AF4 T0CMP1, AF5 T1CMP0, AF6 T1CMP1, AF7 SCK1	P0.0
P0.1	Bidirectional	General-purpose I/O GPIO1; AF0 MISO0, AF1 TX1, AF2 T0CMP0, AF3 T0CMP1, AF4 T1CMP0, AF5 T1CMP1, AF6 SCK1, AF7 MOSI1	P0.1

Signal	Direction	Description	Pin (AF)
P0.2	Bidirectional	General-purpose I/O GPIO2; AF0 MOSI0, AF1 T0CMP0, AF2 T0CMP1, AF3 T1CMP0, AF4 T1CMP1, AF5 SCK1, AF6 MOSI1, AF7 TX0	P0.2
P0.3	Bidirectional	General-purpose I/O GPIO3; AF0 SCK0, AF1 T0CMP1, AF2 T1CMP0, AF3 T1CMP1, AF4 SCK1, AF5 MOSI1, AF6 TX0, AF7 TX1	P0.3
P0.4	Bidirectional	General-purpose I/O GPIO4; AF0 LFXT, AF1 T1CMP0, AF2 T1CMP1, AF3 SCK1, AF4 MOSI1, AF5 TX0, AF6 TX1, AF7 T0CMP0	P0.4
P0.5	Bidirectional	General-purpose I/O GPIO5; AF0 HFXT, AF1 T1CMP1, AF2 SCK1, AF3 MOSI1, AF4 TX0, AF5 TX1, AF6 T0CMP0, AF7 T0CMP1	P0.5
P0.6	Bidirectional	General-purpose I/O GPIO6; AF0 TRAP, AF1 SCK1, AF2 MOSI1, AF3 TX0, AF4 TX1, AF5 T0CMP0, AF6 T0CMP1, AF7 T1CMP0	P0.6
P0.7	Bidirectional	General-purpose I/O BOOT; AF1 MOSI1, AF2 TX0, AF3 TX1, AF4 T0CMP0, AF5 T0CMP1, AF6 T1CMP0, AF7 T1CMP1	P0.7
P1.0	Bidirectional	General-purpose I/O GPIO8; AF0 CS1, AF1 T0CMP0, AF2 TX1, AF3 T0CMP1, AF4 T1CMP0, AF5 T1CMP1, AF6 SCK1, AF7 MOSI1	P1.0
P1.1	Bidirectional	General-purpose I/O GPIO9; AF0 MISO1, AF1 T0CMP1, AF2 T0CMP0, AF3 T1CMP0, AF4 T1CMP1, AF5 SCK1, AF6 MOSI1, AF7 TX0	P1.1
P1.2	Bidirectional	General-purpose I/O GPIO10; AF0 MOSI1, AF1 T1CMP0, AF2 T1CMP1, AF3 SCK1, AF4 TX0, AF5 TX1, AF6 T0CMP0, AF7 T0CMP1	P1.2
P1.3	Bidirectional	General-purpose I/O GPIO11; AF0 SCK1, AF1 T1CMP1, AF2 MOSI1, AF3 TX0, AF4 TX1, AF5 T0CMP0, AF6 T0CMP1, AF7 T1CMP0	P1.3
P1.4	Bidirectional	General-purpose I/O GPIO12; AF0 TX0, AF1 SDA1, AF2 SCK1, AF3 MOSI1, AF4 TX1, AF5 T0CMP0, AF6 T0CMP1, AF7 T1CMP0	P1.4

Signal	Direction	Description	Pin (AF)
P1.5	Bidirectional	General-purpose I/O GPIO13; AF0 RX0, AF1 SCL1, AF2 T1CMP1, AF3 SCK1, AF4 MOSI1, AF5 TX0, AF6 TX1, AF7 T0CMP0	P1.5
P1.6	Bidirectional	General-purpose I/O GPIO14; AF0 TX1, AF1 SDA0, AF2 TX0, AF3 T0CMP0, AF4 T0CMP1, AF5 T1CMP0, AF6 T1CMP1, AF7 SCK1	P1.6
P1.7	Bidirectional	General-purpose I/O GPIO15; AF0 RX1, AF1 SCL0, AF2 MOSI1, AF3 TX0, AF4 TX1, AF5 T0CMP0, AF6 T0CMP1, AF7 T1CMP0	P1.7
P2.0	Bidirectional	General-purpose I/O GPIO16; AF0 T0CMP0, AF1 TX1, AF2 SCK1, AF3 MOSI1, AF4 TX0, AF5 T0CMP1, AF6 T1CMP0, AF7 T1CMP1	P2.0
P2.1	Bidirectional	General-purpose I/O GPIO17; AF0 T0CMP1, AF1 RX1, AF2 T1CMP0, AF3 T1CMP1, AF4 SCK1, AF5 MOSI1, AF6 TX0, AF7 TX1	P2.1
P2.2	Bidirectional	General-purpose I/O GPIO18; AF0 T0CAP0, AF1 SDA1, AF2 PWM0, AF3 T0CMP1, AF4 T1CMP0, AF5 T1CMP1, AF6 SCK1, AF7 MOSI1	P2.2
P2.3	Bidirectional	General-purpose I/O GPIO19; AF0 T0CAP1, AF1 SCL1, AF2 PWM1, AF3 T1CMP0, AF4 T1CMP1, AF5 SCK1, AF6 MOSI1, AF7 TX0	P2.3
P2.4	Bidirectional	General-purpose I/O GPIO20; AF0 T1CMP0, AF1 TX0, AF2 T0CMP1, AF3 T1CMP1, AF4 SCK1, AF5 MOSI1, AF6 TX1, AF7 T0CMP0	P2.4
P2.5	Bidirectional	General-purpose I/O GPIO21; AF0 T1CMP1, AF1 RX0, AF2 TX0, AF3 TX1, AF4 T0CMP0, AF5 T0CMP1, AF6 T1CMP0, AF7 SCK1	P2.5
P2.6	Bidirectional	General-purpose I/O GPIO22; AF0 T1CAP0, AF1 SDA0, AF2 SCK1, AF3 MOSI1, AF4 TX0, AF5 TX1, AF6 T0CMP0, AF7 T0CMP1	P2.6
P2.7	Bidirectional	General-purpose I/O GPIO23; AF0 T1CAP1, AF1 SCL0, AF2 MOSI1, AF3 TX0, AF4 TX1, AF5 T0CMP0, AF6 T0CMP1, AF7 T1CMP0	P2.7

Signal	Direction	Description	Pin (AF)
P3.0	Bidirectional	General-purpose I/O GPIO24; AF0 SDA0, AF1 T0CAP0, AF2 TX0, AF3 TX1, AF4 T0CMP0, AF5 T0CMP1, AF6 T1CMP0, AF7 T1CMP1	P3.0
P3.1	Bidirectional	General-purpose I/O GPIO25; AF0 SCL0, AF1 T0CAP1, AF2 TX1, AF3 T0CMP0, AF4 T0CMP1, AF5 T1CMP0, AF6 T1CMP1, AF7 SCK1	P3.1
P3.2	Bidirectional	General-purpose I/O GPIO26; AF0 SDA1, AF1 T1CAP0, AF2 T0CMP0, AF3 T0CMP1, AF4 T1CMP0, AF5 T1CMP1, AF6 SCK1, AF7 MOSI1	P3.2
P3.3	Bidirectional	General-purpose I/O GPIO27; AF0 SCL1, AF1 T1CAP1, AF2 T0CMP1, AF3 T1CMP0, AF4 T1CMP1, AF5 SCK1, AF6 MOSI1, AF7 TX0	P3.3
P3.4	Bidirectional	General-purpose I/O GPIO28; AF0 DTP0, AF1 T0CMP0, AF2 TX0, AF3 TX1, AF4 T0CMP1, AF5 T1CMP0, AF6 T1CMP1, AF7 SCK1	P3.4
P3.5	Bidirectional	General-purpose I/O GPIO29; AF0 DTP1, AF1 T0CMP1, AF2 RX0, AF3 T0CMP0, AF4 T1CMP0, AF5 T1CMP1, AF6 SCK1, AF7 MOSI1	P3.5
P3.6	Bidirectional	General-purpose I/O GPIO30; AF0 DTP2, AF1 T1CMP0, AF2 T0CMP0, AF3 T0CMP1, AF4 T1CMP1, AF5 SCK1, AF6 MOSI1, AF7 MISO1	P3.6
P3.7	Bidirectional	General-purpose I/O GPIO31; AF0 DTP3, AF1 T1CMP1, AF2 OW_DQ, AF3 T1CMP0, AF4 SCK1, AF5 MOSI1, AF6 TX0, AF7 TX1	P3.7
P4.0	Bidirectional	General-purpose I/O GPIO32; AF1 QSPI_SCK	P4.0
P4.1	Bidirectional	General-purpose I/O GPIO33; AF1 QSPI_CS	P4.1
P4.2	Bidirectional	General-purpose I/O GPIO34; AF1 QSPI_IO0	P4.2
P4.3	Bidirectional	General-purpose I/O GPIO35; AF1 QSPI_IO1	P4.3
P4.4	Bidirectional	General-purpose I/O GPIO36; AF1 QSPI_IO2	P4.4
P4.5	Bidirectional	General-purpose I/O GPIO37; AF1 QSPI_IO3	P4.5
P4.6	Bidirectional	General-purpose I/O GPIO38; AF1 I3C_SDA	P4.6

Signal	Direction	Description	Pin (AF)
P4.7	Bidirectional	General-purpose I/O GPIO39; AF1 I3C_SCL	P4.7
P5.0	Bidirectional	General-purpose I/O GPIO40; AF1 NFC_RF_CLK	P5.0
P5.1	Bidirectional	General-purpose I/O GPIO41; AF1 NFC_RF_RX	P5.1
P5.2	Bidirectional	General-purpose I/O GPIO42; AF1 NFC_FIELD_DETECT	P5.2
P5.3	Bidirectional	General-purpose I/O GPIO43; AF1 NFC_RF_TXMOD	P5.3
P5.4	Bidirectional	General-purpose I/O GPIO44; AF1 NFC_RF_TX_EN	P5.4
P5.5	Bidirectional	General-purpose I/O GPIO45; AF1 NFC_AFE_EN	P5.5
P5.6	Bidirectional	General-purpose I/O GPIO46	P5.6
P5.7	Bidirectional	General-purpose I/O GPIO47	P5.7

Table 12.3: GPIOx interrupt vectors

Vector	Instance	Description	Clear method
1	GPIO0	GPIO0 Bit 0 InterruptIRQB_GPIO0_B0	Write 1 to P0IF bit 0
2	GPIO0	GPIO0 Bit 1 InterruptIRQB_GPIO0_B1	Write 1 to P0IF bit 1
3	GPIO0	GPIO0 Bit 2 InterruptIRQB_GPIO0_B2	Write 1 to P0IF bit 2
4	GPIO0	GPIO0 Bit 3 InterruptIRQB_GPIO0_B3	Write 1 to P0IF bit 3
5	GPIO0	GPIO0 Bit 4 InterruptIRQB_GPIO0_B4	Write 1 to P0IF bit 4
6	GPIO0	GPIO0 Bit 5 InterruptIRQB_GPIO0_B5	Write 1 to P0IF bit 5
7	GPIO0	GPIO0 Bit 6 InterruptIRQB_GPIO0_B6	Write 1 to P0IF bit 6
8	GPIO0	GPIO0 Bit 7 InterruptIRQB_GPIO0_B7	Write 1 to P0IF bit 7
28	GPIO1	GPIO1 Bit 0 InterruptIRQB_GPIO1_B0	Write 1 to P1IF bit 0
29	GPIO1	GPIO1 Bit 1 InterruptIRQB_GPIO1_B1	Write 1 to P1IF bit 1
30	GPIO1	GPIO1 Bit 2 InterruptIRQB_GPIO1_B2	Write 1 to P1IF bit 2
31	GPIO1	GPIO1 Bit 3 InterruptIRQB_GPIO1_B3	Write 1 to P1IF bit 3
32	GPIO1	GPIO1 Bit 4 InterruptIRQB_GPIO1_B4	Write 1 to P1IF bit 4
33	GPIO1	GPIO1 Bit 5 InterruptIRQB_GPIO1_B5	Write 1 to P1IF bit 5
34	GPIO1	GPIO1 Bit 6 InterruptIRQB_GPIO1_B6	Write 1 to P1IF bit 6
35	GPIO1	GPIO1 Bit 7 InterruptIRQB_GPIO1_B7	Write 1 to P1IF bit 7
36	GPIO2	GPIO2 Bit 0 InterruptIRQB_GPIO2_B0	Write 1 to P2IF bit 0
37	GPIO2	GPIO2 Bit 1 InterruptIRQB_GPIO2_B1	Write 1 to P2IF bit 1
38	GPIO2	GPIO2 Bit 2 InterruptIRQB_GPIO2_B2	Write 1 to P2IF bit 2
39	GPIO2	GPIO2 Bit 3 InterruptIRQB_GPIO2_B3	Write 1 to P2IF bit 3
40	GPIO2	GPIO2 Bit 4 InterruptIRQB_GPIO2_B4	Write 1 to P2IF bit 4
41	GPIO2	GPIO2 Bit 5 InterruptIRQB_GPIO2_B5	Write 1 to P2IF bit 5
42	GPIO2	GPIO2 Bit 6 InterruptIRQB_GPIO2_B6	Write 1 to P2IF bit 6
43	GPIO2	GPIO2 Bit 7 InterruptIRQB_GPIO2_B7	Write 1 to P2IF bit 7
44	GPIO3	GPIO3 Bit 0 InterruptIRQB_GPIO3_B0	Write 1 to P3IF bit 0
45	GPIO3	GPIO3 Bit 1 InterruptIRQB_GPIO3_B1	Write 1 to P3IF bit 1

Vector	Instance	Description	Clear method
46	GPIO3	GPIO3 Bit 2 InterruptIRQB_GPIO3_B2	Write 1 to P3IF bit 2
47	GPIO3	GPIO3 Bit 3 InterruptIRQB_GPIO3_B3	Write 1 to P3IF bit 3
48	GPIO3	GPIO3 Bit 4 InterruptIRQB_GPIO3_B4	Write 1 to P3IF bit 4
49	GPIO3	GPIO3 Bit 5 InterruptIRQB_GPIO3_B5	Write 1 to P3IF bit 5
50	GPIO3	GPIO3 Bit 6 InterruptIRQB_GPIO3_B6	Write 1 to P3IF bit 6
51	GPIO3	GPIO3 Bit 7 InterruptIRQB_GPIO3_B7	Write 1 to P3IF bit 7
98	GPIO4	GPIO4 Bit 0 InterruptIRQB_GPIO4_B0	Write 1 to P4IF bit 0
99	GPIO4	GPIO4 Bit 1 InterruptIRQB_GPIO4_B1	Write 1 to P4IF bit 1
100	GPIO4	GPIO4 Bit 2 InterruptIRQB_GPIO4_B2	Write 1 to P4IF bit 2
101	GPIO4	GPIO4 Bit 3 InterruptIRQB_GPIO4_B3	Write 1 to P4IF bit 3
102	GPIO4	GPIO4 Bit 4 InterruptIRQB_GPIO4_B4	Write 1 to P4IF bit 4
103	GPIO4	GPIO4 Bit 5 InterruptIRQB_GPIO4_B5	Write 1 to P4IF bit 5
104	GPIO4	GPIO4 Bit 6 InterruptIRQB_GPIO4_B6	Write 1 to P4IF bit 6
105	GPIO4	GPIO4 Bit 7 InterruptIRQB_GPIO4_B7	Write 1 to P4IF bit 7
106	GPIO5	GPIO5 Bit 0 InterruptIRQB_GPIO5_B0	Write 1 to P5IF bit 0
107	GPIO5	GPIO5 Bit 1 InterruptIRQB_GPIO5_B1	Write 1 to P5IF bit 1
108	GPIO5	GPIO5 Bit 2 InterruptIRQB_GPIO5_B2	Write 1 to P5IF bit 2
109	GPIO5	GPIO5 Bit 3 InterruptIRQB_GPIO5_B3	Write 1 to P5IF bit 3
110	GPIO5	GPIO5 Bit 4 InterruptIRQB_GPIO5_B4	Write 1 to P5IF bit 4
111	GPIO5	GPIO5 Bit 5 InterruptIRQB_GPIO5_B5	Write 1 to P5IF bit 5
112	GPIO5	GPIO5 Bit 6 InterruptIRQB_GPIO5_B6	Write 1 to P5IF bit 6
113	GPIO5	GPIO5 Bit 7 InterruptIRQB_GPIO5_B7	Write 1 to P5IF bit 7

12.3 Functional description

12.3.1 GPIO mode

In GPIO mode the pin is controlled by PxDIR, PxOUT and PxREN: PxDIR selects input or output, PxOUT selects the output level, and PxREN enables the internal pull resistor on an input. Table 12.4 lists the four states.

PxDIR[y]	PxOUT[y]	PxREN[y]	P$x.y$ state
0	-	0	High-impedance input, resistor disabled
0	-	1	Input with pull resistor enabled
1	0	-	Output driven low
1	1	-	Output driven high

Table 12.4: GPIO pin states.

PxIN always reads the digital level of the pad, in both directions and in both modes. The read value is latched on the falling edge of the memory enable signal, so a pin that changes during the access returns one stable sample.

12.3.2 Set, clear and toggle registers

Writing 1 to a bit of PxOUTS, PxOUTC or PxOUTT sets, clears or toggles the corresponding PxOUT bit; writing 0 has no effect. Each is a single write, so two harts, or a handler and the code it interrupted, can change

different pins of one port without a read-modify-write race. PxOUTS and PxOUTT read back PxOUT; PxOUTC reads back its complement.

12.3.3 Alternate-function mode

Two registers together decide what drives a pin. PxSEL[y] = 1 puts P$x.y$ in alternate-function mode, and the 4-bit field y of PxAFS (bits $4y + 3$ to $4y$, of which the low three are implemented) selects which function, AF0 to AF7, governs it. PxAFS resets to 0, so a pin in alternate-function mode drives its AF0 function, which is the pin's home peripheral function; software that only writes PxSEL therefore obtains the home function. Section 3 tabulates every pin's plane assignment.

In alternate-function mode the selected function owns the pin's direction, output and resistor controls, and PxDIR[y], PxOUT[y] and PxREN[y] have no effect until PxSEL[y] returns to 0. A field value that selects a plane on which the pin has no function leaves the pin as a high-impedance input. Some peripheral inputs, such as the timer capture pins, are inputs on every plane and are governed by PxDIR regardless of PxSEL, because the peripheral never drives them.

The planes above AF0 carry a shared pool of relocatable output functions (the UARTx transmitters, the TIMERx compare outputs, the SPI1 clock and master-out) fanned across the ports, so a peripheral output can be placed on whichever pin suits the board. Because there is one instance of each function, selecting it on several pins drives the same signal on all of them. A few plane slots carry bidirectional bus completions so that whole buses relocate as a set: both I2C pairs, the UART0 receiver alongside its transmitter, and the SPI1 master-in alongside its clock and master-out. A function that belongs to a peripheral this configuration omits leaves the pool, and its slots become high-impedance inputs.

12.3.4 Relocatable peripheral inputs

A peripheral input that appears at more than one pin (UART receivers, timer captures, I2C data and clock) is routed from the pin whose PxAFS field selects its plane, and from its home pin otherwise, regardless of PxSEL. Input routing ignores PxSEL so that the routing mirrors PxIN, which is always visible; a pin can therefore stay in GPIO output mode while its PxAFS field routes the pad into a peripheral input, which gives a software loopback for self-test. If the same input is selected on more than one pin, a fixed priority (newest alternate location first, then the older alternate, then home) picks one, so the outcome of that configuration error is deterministic.

12.3.5 Pin interrupts

Each pin has its own interrupt. With PxIE[y] set, the edge selected by PxIES[y] (0 = rising, 1 = falling) sets PxIF[y] and raises the pin's vector. The pin is sampled on the peripheral clock rather than clocking the flag directly, so the flag appears three clock cycles after the edge, a pulse shorter than one clock period is not captured, and no edge is captured while the clock is stopped. Pin interrupts stay available in alternate-function mode alongside any interrupt the governing peripheral raises. PxIF is latched on the falling edge of the memory enable signal, so a handler reads one stable sample.

12.3.6 Event-fabric tasks

PxTASK selects which pins respond to the event fabric's OUT-SET and OUT-CLR tasks for this port; in a configuration without the event fabric the register is writable and has no effect. A task pulse is applied after any coincident register write in the same cycle, and CLR after SET, so a same-cycle conflict resolves to the cleared, safe direction.

12.4 Interrupts and events

Port x owns eight consecutive vectors, one per pin, in the order listed in Table 12.5. The interrupt is a level derived from PxIF[y] and PxIE[y].

Port	Vectors	Condition	Set by	Cleared by
GPIO0	1 to 8	Selected edge on P0.y	PxIF[y]	Writing 1 to PxIF[y]
GPIO1	28 to 35	Selected edge on P1.y	PxIF[y]	Writing 1 to PxIF[y]
GPIO2	36 to 43	Selected edge on P2.y	PxIF[y]	Writing 1 to PxIF[y]
GPIO3	44 to 51	Selected edge on P3.y	PxIF[y]	Writing 1 to PxIF[y]
GPIO4	98 to 105	Selected edge on P4.y	PxIF[y]	Writing 1 to PxIF[y]
GPIO5	106 to 113	Selected edge on P5.y	PxIF[y]	Writing 1 to PxIF[y]

Table 12.5: GPIOx interrupt vectors; pin y uses the port's base vector plus y.

12.5 Programming

12.5.1 Configuring a pin as an output

1. Write the initial level to PxOUT[y].
2. Clear PxSEL[y].
3. Set PxDIR[y].

12.5.2 Routing a peripheral output to an alternate pin

1. Write the function's plane number to field y of PxAFS.
2. Set PxSEL[y].

12.5.3 Enabling a pin interrupt

1. Clear PxIE[y].
2. Write the edge to PxIES[y].
3. Write 1 to PxIF[y] to discard any stale flag.
4. Enable the pin's vector in the hart's IRQROUTER row.
5. Set PxIE[y].

12.6 Usage notes

Disable the pin interrupt before changing its edge. Writing PxIES[y] while PxIE[y] is set can raise a spurious interrupt, because the edge detector sees the selection change as an edge.

Clear PxIF[y] with a 1 before returning. The interrupt is a level; a handler that returns with the flag set is re-entered.

Configure capture pins as inputs in PxDIR. A timer capture input is governed by PxDIR on every plane, so setting PxSEL[y] alone does not make the pad an input.

Select an input function on one pin only. The priority among several selected pins is fixed and makes the error deterministic, not correct.

Use the set, clear and toggle registers from shared code. A read-modify-write of PxOUT from two harts loses one of the two updates.

12.7 Registers

Register offsets in this chapter are relative to the instance base address in Table 12.1 (GPIO0 0x4000, GPIO1 0x4100, GPIO2 0x4800, GPIO3 0x4D00, GPIO4 0x6300, GPIO5 0x6400); the generated header names each base GPIO0_BASE and so on.

Table 12.6: GPIOx register summary

Offset	Name	Access	Reset	Description
0x00	PxIN	r	0x00	GPIO read pin register
0x04	PxOUT	rw	0x00	GPIO output drive register
0x08	PxOUTS	rw1	0x00	GPIO output drive set register
0x0C	PxOUTC	rw1	0x00	GPIO output drive clear register
0x10	PxOUTT	rw1	0x00	GPIO output drive toggle register
0x14	PxDIR	rw	0x00	GPIO pin direction register
0x18	PxIF	rw1	0x00	GPIO interrupt flag register
0x1C	PxIES	rw	0x00	GPIO interrupt edge select register
0x20	PxIE	rw	0x00	GPIO interrupt enable register
0x24	PxSEL	rw	0x00	GPIO peripheral select register
0x28	PxREN	rw	0x00	GPIO resistor enable register
0x2C	PxAFS	rw	0x00000000	GPIO alternate function select register
0x30	PxTASK	rw	0x00	GPIO event-fabric task pin-select register

12.7.1 PxIN: GPIO read pin register

Offset 0x00 **Reset** 0x00 **Width** 8 **Address** see Table 12.1
GPIO read pin register. Each bit corresponds to the input logic state of the GPIO pin of the same number. The register is latched on the falling edge of the memory enable signal. Reading a 0 in a bit indicates a logic low pin state; reading a 1 indicates a logic high state. This register always reflects the pin state regardless of pin direction or peripheral select settings.

Bits	Field	Type	Reset	Description
7:0	IN	r	0x00	Pin input levels. Bit y reads the logic level of pin y as sampled at the start of the bus access, regardless of the pin direction or PxSEL setting. Bits 31:8 read 0.

12.7.2 PxOUT: GPIO output drive register

Offset 0x04 **Reset** 0x00 **Width** 8 **Address** see Table 12.1
GPIO output drive register. Each bit corresponds to the output logic state of the GPIO pin of the same number. Only has an effect if the pin is configured as an output in PxDIR and is set to GPIO (primary) mode in PxSEL. Write a 0 to the desired bit to make the corresponding pin output a logic low value; write a 1 to output a logic high value.

Bits	Field	Type	Reset	Description
7:0	OUT	rw	0x00	Output drive levels. Bit y is the level driven on pin y while the pin is an output (PxDIR bit y = 1) in GPIO mode (PxSEL bit y = 0). Resets to 0; bits 31:8 read 0 and ignore writes.

12.7.3 PxOUTS: GPIO output drive set register

Offset 0x08 **Reset** 0x00 **Width** 8 **Address** see Table 12.1
GPIO output drive set register. Each bit corresponds to the output logic state of the GPIO pin of the same number. Only has an effect if the pin is configured as an output in PxDIR and is in GPIO (primary) mode in PxSEL. Write a 1 to the desired bit to set the corresponding pin (make the pin output a logic high value). Writing a 0 has no effect. Reading this register is equivalent to reading the output drive register PxOUT.

Bits	Field	Type	Reset	Description
7:0	OUTS	rw1	0x00	Output set. Writing 1 to bit y sets PxOUT bit y; writing 0 has no effect. Reads return the current PxOUT value.

12.7.4 PxOUTC: GPIO output drive clear register

Offset 0x0C **Reset** 0x00 **Width** 8 **Address** see Table 12.1
GPIO output drive clear register. Each bit corresponds to the output logic state of the GPIO pin of the same number. Only has an effect if the pin is configured as an output in PxDIR and is in GPIO (primary) mode in PxSEL. Write a 1 to the desired bit to clear the corresponding pin (make the pin output a logic low value). Writing a 0 has no effect. Reading this register yields the inversion of the output drive register PxOUT.

Bits	Field	Type	Reset	Description
7:0	OUTC	rw1	0x00	Output clear. Writing 1 to bit y clears PxOUT bit y; writing 0 has no effect. Reads return the inverse of the current PxOUT value.

12.7.5 PxOUTT: GPIO output drive toggle register

Offset 0x10 **Reset** 0x00 **Width** 8 **Address** see Table 12.1
GPIO output drive toggle register. Each bit corresponds to the output logic state of the GPIO pin of the same number. Only has an effect if the pin is configured as an output in PxDIR and is in GPIO (primary)

mode in PxSEL. Write a 1 to the desired bit to toggle the corresponding pin state. Writing a 0 has no effect. Reading this register is equivalent to reading the output drive register PxOUT.

Bits	Field	Type	Reset	Description
7:0	OUTT	rw1	0x00	Output toggle. Writing 1 to bit y inverts PxOUT bit y; writing 0 has no effect. Reads return the current PxOUT value.

12.7.6 PxDIR: GPIO pin direction register

Offset 0x14 **Reset** 0x00 **Width** 8 **Address** see Table 12.1
GPIO pin direction register. Each bit corresponds to the GPIO pin of the same number. Only has an effect if the pin is configured in GPIO (primary) mode in PxSEL. Write a 0 to the desired bit to set the corresponding pin to input mode; write a 1 to set to output mode.

Bits	Field	Type	Reset	Description
7:0	DIR	rw	0x00	Pin direction. Bit y = 0 makes pin y an input and 1 makes it an output, effective only while the pin is in GPIO mode (PxSEL bit y = 0). Resets to 0 (all inputs); bits 31:8 read 0 and ignore writes.

12.7.7 PxIF: GPIO interrupt flag register

Offset 0x18 **Reset** 0x00 **Width** 8 **Address** see Table 12.1
GPIO interrupt flag register. Each bit corresponds to the GPIO pin of the same number. The register is latched on the falling edge of the memory enable signal. Reading a 0 in a bit indicates there is no pending interrupt for the corresponding pin; reading a 1 indicates there is a new interrupt pending for the corresponding pin. Write a 1 to each bit for which you wish to clear the interrupt flag. Writing 0 has no effect.

Bits	Field	Type	Reset	Description
7:0	IF	rw1	0x00	Interrupt flags. Hardware sets bit y when the edge selected by PxIES occurs on pin y while PxIE bit y is 1, and holds it until software writes 1 to the bit; writing 0 has no effect. The pin is sampled on the peripheral clock, so the flag appears three clock cycles after the edge, a pulse shorter than one clock period is not captured, and no edge is captured while the clock is stopped. Each flag drives the interrupt vector of its pin directly, and bits 31:8 read 0.

12.7.8 PxIES: GPIO interrupt edge select register

Offset 0x1C **Reset** 0x00 **Width** 8 **Address** see Table 12.1
GPIO interrupt edge select register. Each bit corresponds to the GPIO pin of the same number. Write a 0 to the desired bit to set the corresponding pin interrupt to trigger on low-to-high (rising) edge; write a 1 to set to high-to-low (falling) edge triggering.

Bits	Field	Type	Reset	Description
7:0	IES	rw	0x00	Interrupt edge select. Bit y = 0 makes a rising edge on pin y set PxIF bit y, 1 selects a falling edge. Resets to 0.

12.7.9 PxIE: GPIO interrupt enable register

Offset 0x20 **Reset** 0x00 **Width** 8 **Address** see Table 12.1
GPIO interrupt enable register. Each bit corresponds to the GPIO pin of the same number. Write a 0 to the desired bit to disable the pin interrupt; write a 1 to enable the pin interrupt. Each pin has an individual interrupt output that connects to the system interrupt vector table. Interrupts function in both GPIO (primary) and secondary function (peripheral) modes.

Bits	Field	Type	Reset	Description
7:0	IE	rw	0x00	Interrupt enable. Bit y = 1 allows the selected edge on pin y to set PxIF bit y; with the bit at 0 no flag is captured for the pin. Resets to 0.

12.7.10 PxSEL: GPIO peripheral select register

Offset 0x24 **Reset** 0x00 **Width** 8 **Address** see Table 12.1
GPIO peripheral select register. Each bit corresponds to the GPIO pin of the same number. Write a 0 to the desired bit to set the corresponding pin to GPIO (primary) mode; write a 1 to set the pin to alternate function (peripheral) mode. When a pin is in alternate function (peripheral) mode, the governing peripheral takes control of the pin output, direction, and resistor enable states, and the PxOUT, PxDIR, and PxREN registers have no effect on the pin. WHICH alternate function governs the pin is selected by the pin's field in the PxAFS register: at reset all PxAFS fields are 0, selecting alternate function 0 (AF0). Pin interrupts remain available when in alternate function (peripheral) mode in addition to any interrupts the governing peripheral may generate. If a pin has no alternate function defined at the selected PxAFS plane, setting PxSEL to 1 will configure the pin as a high-impedance input.

Bits	Field	Type	Reset	Description
7:0	SEL	rw	0x00	Pin function select. Bit y = 0 gives pin y to the GPIO registers, 1 hands its output level, direction and resistor enable to the alternate function chosen by the PxAFS field of the pin. Resets to 0; bits 31:8 read 0 and ignore writes.

12.7.11 PxREN: GPIO resistor enable register

Offset 0x28 **Reset** 0x00 **Width** 8 **Address** see Table 12.1
GPIO resistor enable register. Each bit corresponds to the GPIO pin of the same number. Only has an effect if the pin is configured in GPIO (primary) mode in PxSEL. Write a 0 to the desired bit to disable the pin pullup/pulldown resistor; write a 1 to enable the pin pullup/pulldown resistor.

Bits	Field	Type	Reset	Description
7:0	REN	rw	0x00	Pull resistor enable. Bit y = 1 enables the pull resistor of pin y while the pin is in GPIO mode (PxSEL bit y = 0); in alternate function mode the peripheral controls the resistor. Resets to 0; bits 31:8 read 0 and ignore writes.

12.7.12 PxAFS: GPIO alternate function select register

Offset 0x2C **Reset** 0x00000000 **Width** 32 **Address** see Table 12.1
GPIO alternate function select register. One 4-bit field per pin (pin y occupies bits 4y+3 downto 4y; only the low 3 bits of each field are implemented, the top bit is reserved and reads 0). When a pin is in alternate

function (peripheral) mode (PxSEL bit = 1), the value of its PxAFS field selects WHICH alternate function (AF0-AF7) controls the pin. Each pin's available alternate functions are listed in the pin configuration table in Section 3. The register resets to 0, so every pin comes out of reset selecting its AF0 (legacy) function. Selecting an AF plane with no function defined for the pin configures the pin as a high-impedance input. While a pin is in GPIO (primary) mode its PxAFS field has no effect on the pad, but it still routes relocatable peripheral INPUTS: a peripheral input function relocated to this pin (e.g. a UART receiver or timer capture) observes the pin whenever the PxAFS field selects it, regardless of PxSEL.

Bits	Field	Type	Reset	Description
31	-	-	-	Reserved. Reads zero; write zero.
30:28	AFS7	rw	0b000	Alternate function select for pin 7 (0 = AF0 ... 7 = AF7)
27	-	-	-	Reserved. Reads zero; write zero.
26:24	AFS6	rw	0b000	Alternate function select for pin 6 (0 = AF0 ... 7 = AF7)
23	-	-	-	Reserved. Reads zero; write zero.
22:20	AFS5	rw	0b000	Alternate function select for pin 5 (0 = AF0 ... 7 = AF7)
19	-	-	-	Reserved. Reads zero; write zero.
18:16	AFS4	rw	0b000	Alternate function select for pin 4 (0 = AF0 ... 7 = AF7)
15	-	-	-	Reserved. Reads zero; write zero.
14:12	AFS3	rw	0b000	Alternate function select for pin 3 (0 = AF0 ... 7 = AF7)
11	-	-	-	Reserved. Reads zero; write zero.
10:8	AFS2	rw	0b000	Alternate function select for pin 2 (0 = AF0 ... 7 = AF7)
7	-	-	-	Reserved. Reads zero; write zero.
6:4	AFS1	rw	0b000	Alternate function select for pin 1 (0 = AF0 ... 7 = AF7)
3	-	-	-	Reserved. Reads zero; write zero.
2:0	AFS0	rw	0b000	Alternate function select for pin 0 (0 = AF0 ... 7 = AF7)

12.7.13 PxTASK: GPIO event-fabric task pin-select register

Offset 0x30 **Reset** 0x00 **Width** 8 **Address** see Table 12.1

GPIO event-fabric task pin-select register. Each bit corresponds to the GPIO pin of the same number and selects whether that pin participates in the EVFAB0 event-fabric output tasks: when the fabric fires the port's OUT-SET task, every pin whose PxTASK bit is 1 has its PxOUT bit set; when it fires the OUT-CLR task, every selected pin has its PxOUT bit cleared. The two task pulses are applied AFTER the CPU register write in the same cycle (a task wins its own pins against a coincident PxOUT write) and CLR is applied after SET (a same-cycle set+clear on an overlapping pin resolves to CLEAR, the safe direction). A toggle task is deliberately not offered. Resets to 0, so no pin is fabric-driven out of reset. In a configuration WITHOUT the event fabric (peripherals.eventFabric false) the register is still readable and writable but has no effect, because both task inputs are tied inactive. Only the port's implemented pins have bits; the rest read 0.

Bits	Field	Type	Reset	Description
7:0	TASK	rw	0x00	Event fabric task pin select. Bit y = 1 lets the EVFAB0 output set and output clear tasks of this port set or clear PxOUT bit y; with the bit at 0 the tasks leave pin y untouched. Resets to 0, so no pin responds to a task until software selects it.

13 Serial peripheral interface (SPIx)

13.1 Overview

The Serial Peripheral Interface (SPIx) is a full-duplex synchronous serial interface. A bus has three shared wires (clock, master out, master in) and one chip select per slave. SPI0 is the boot-flash master and carries the memory-mapped flash window; SPI1, in configurations that include it, is a general-purpose port with both master and slave modes.

- Master mode on both ports; slave mode on SPI1
- 8-, 16- or 32-bit frames (SPIDL), LSB- or MSB-first (SPIMSB)
- SPI modes 0 to 3 as master (SPICPOL, SPICPHA); modes 1 and 3 as slave
- Optional byte swap on transmit and receive (SPITXSB, SPIRXSB)
- Baud divider from SMCLK (SPIBR)
- One-deep transmit queue for back-to-back frames without dead time
- Two interrupts per port: transfer complete, transmit register empty
- Flash extended memory on SPI0: memory-mapped reads and execute-in-place from the boot flash for hart 0 (SPIFEN, SPI0FOS)

13.2 Instances and signals

Table 13.1: SPIx instances

Instance	Base address	Interrupt vector(s)	Pins (AF)
SPI0	0x4200	9 to 10	MISO0 P0.1 (AF0), MOSI0 P0.2 (AF0), SCK0 P0.3 (AF0)
SPI1	0x4300	11 to 12	SCK1 P1.3 (AF0), MOSI1 P1.2 (AF0), CS1 P1.0 (AF0), MISO1 P1.1 (AF0)

Table 13.2: SPIx signals

Signal	Direction	Description	Pin (AF)
MISO0	Bidirectional	SPI0 Master In Slave Out (connected to SPI flash memory)	P0.1 (AF0)
MOSI0	Output	SPI0 Master Out Slave In (connected to SPI flash memory)	P0.2 (AF0)
SCK0	Output	SPI0 serial clock (connected to SPI flash memory)	P0.3 (AF0)

Signal	Direction	Description	Pin (AF)
SCK1	Output	SPI1 serial clock	P0.0 (AF7), P0.1 (AF6), P0.2 (AF5), P0.3 (AF4), P0.4 (AF3), P0.5 (AF2), P0.6 (AF1), P1.0 (AF6), P1.1 (AF5), P1.2 (AF3), P1.3 (AF0), P1.4 (AF2), P1.5 (AF3), P1.6 (AF7), P2.0 (AF2), P2.1 (AF4), P2.2 (AF6), P2.3 (AF5), P2.4 (AF4), P2.5 (AF7), P2.6 (AF2), P3.1 (AF7), P3.2 (AF6), P3.3 (AF5), P3.4 (AF7), P3.5 (AF6), P3.6 (AF5), P3.7 (AF4)
MOSI1	Output	SPI1 Master Out Slave In	P0.1 (AF7), P0.2 (AF6), P0.3 (AF5), P0.4 (AF4), P0.5 (AF3), P0.6 (AF2), P0.7 (AF1), P1.0 (AF7), P1.1 (AF6), P1.2 (AF0), P1.3 (AF2), P1.4 (AF3), P1.5 (AF4), P1.7 (AF2), P2.0 (AF3), P2.1 (AF5), P2.2 (AF7), P2.3 (AF6), P2.4 (AF5), P2.6 (AF3), P2.7 (AF2), P3.2 (AF7), P3.3 (AF6), P3.5 (AF7), P3.6 (AF6), P3.7 (AF5)
CS1	Input	SPI1 chip select	P1.0 (AF0)
MISO1	Bidirectional	SPI1 Master In Slave Out	P1.1 (AF0), P3.6 (AF7)

Table 13.3: SPIx interrupt vectors

Vector	Instance	Description	Clear method
9	SPI0	SPI0 Transmission Complete Interrupt IRQB_SPI0_TC	Write 1 to SPI0SR.SPITCIF, or read SPI0RX
10	SPI0	SPI0 Transmission Buffer Empty Interrupt IRQB_SPI0_TE	Write 1 to SPI0SR.SPITEIF
11	SPI1	SPI1 Transmission Complete Interrupt IRQB_SPI1_TC	Write 1 to SPI1SR.SPITCIF, or read SPI1RX
12	SPI1	SPI1 Transmission Buffer Empty Interrupt IRQB_SPI1_TE	Write 1 to SPI1SR.SPITEIF

13.3 Block diagram

Figure 13.1 shows how the two ports are wired in this configuration: SPI0 to the external boot flash on fixed pads, and SPI1 to the pool of relocatable pins. The flash chip select is a GPIO pad at reset, driven by the boot ROM while it loads the image, and bit 0 of P0SEL hands the pad driver to SPI0. Only hart 0 reaches the flash window, and it stalls while SPI0 reads on its behalf (Section 13.4.6).

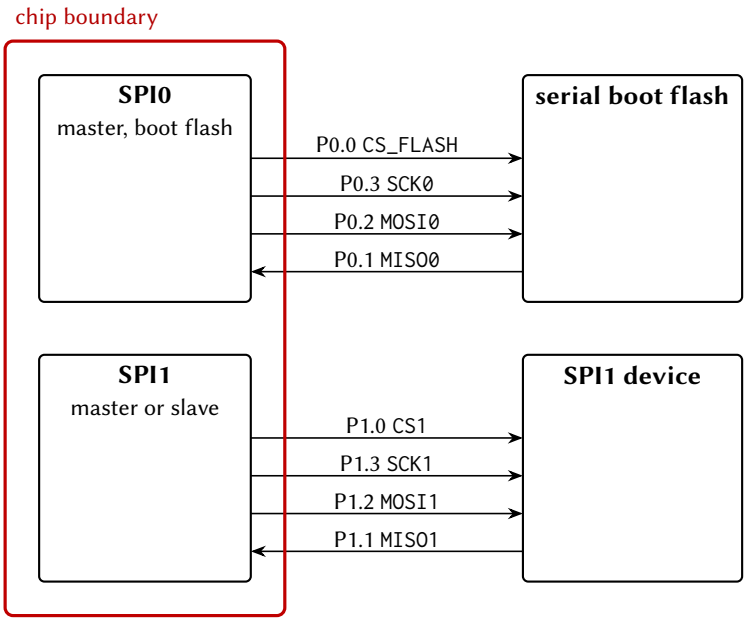


Figure 13.1: SPI connection diagram: SPI0 to the boot flash on fixed pads and SPI1, drawn as master, to an external device.

13.4 Functional description

13.4.1 Transfer

The bus is idle until the master asserts (drives low) the chip select of one slave. The master then drives the clock for one frame of 8, 16 or 32 bits; on each clock transition the master and the slave alternately update and latch the two data wires, so each frame moves one word in each direction at once. After a frame the master either sends another frame with the chip select still low, which makes the frames one packet, or deasserts the chip select. The slave must have a word ready for the first frame of a packet, because the master clocks it out whether or not the slave has data to send.

The order of update and latch, and the idle level of the clock, are set by clock polarity (CPOL) and clock phase (CPHA), together called the SPI mode (Table 13.4). Figure 13.2 draws all four modes for one 8-bit frame. Slave mode supports modes 1 and 3 only (CPHA = 1), because with CPHA = 0 the slave would have to present its first bit before the first clock edge, before it has seen the chip select settle.

Mode	CPOL	CPHA	Clock idle	Data update edge	Data latch edge
0	0	0	Low	Trailing (falling)	Leading (rising)
1	0	1	Low	Leading (rising)	Trailing (falling)
2	1	0	High	Trailing (rising)	Leading (falling)
3	1	1	High	Leading (falling)	Trailing (rising)

Table 13.4: SPI mode key.

13.4.2 Bit and byte order

SPIMSB selects MSB-first (1) or LSB-first (0) within a frame. For 16- and 32-bit frames, SPITXSB and SPIRXSB swap the byte order of the transmitted and received words, so a word exchanged with a device of the other

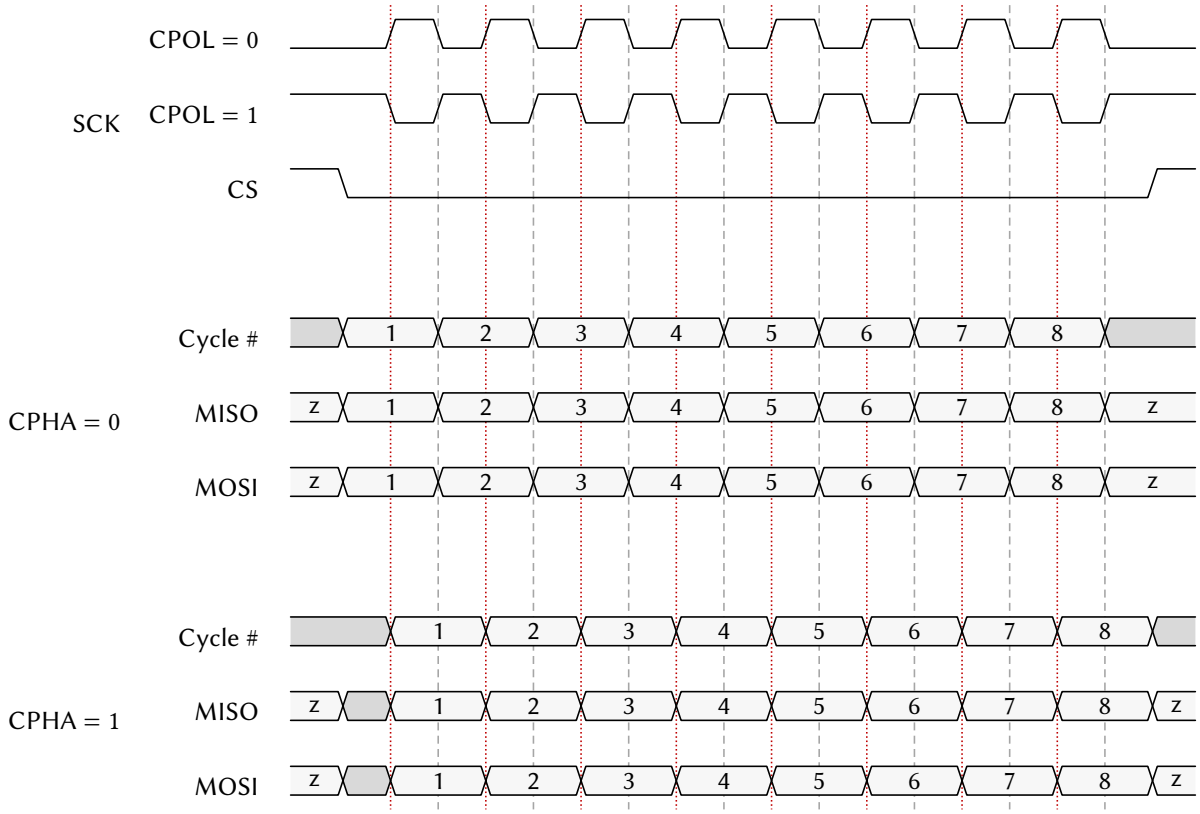


Figure 13.2: SPI timing for the four modes, one 8-bit frame.

endianness needs no software swap. The bit order is applied before the byte swap: a 16-bit MSB-first frame with SPITXSB set sends bits 7 to 0 and then bits 15 to 8. Figures 13.3 and 13.4 show both.

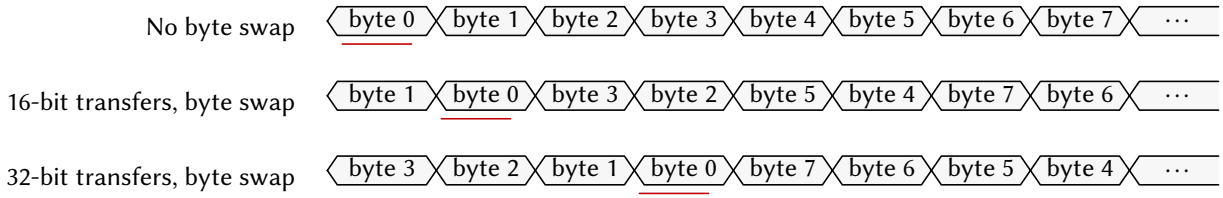


Figure 13.3: Byte order of a multi-byte frame with and without the swap.

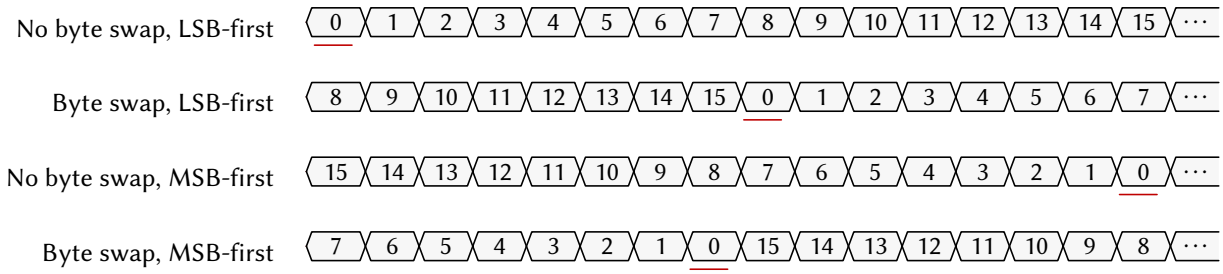


Figure 13.4: Bit order of a 16-bit frame.

13.4.3 Baud clock

In master mode the clock frequency is

$$f_{\mathrm{SCK}} = \frac{f_{\mathrm{SMCLK}}}{2 \times (1 + \mathrm{SPIBR})}$$

so the fastest clock is half SMCLK. The port's serial engine runs on SMCLK and its register interface on MCLK, so the registers stay readable with SMCLK stopped.

In slave mode the master supplies SCKx, and the port oversamples it on SMCLK rather than clocking flip-flops from the pad. SMCLK must be at least eight times the slave SCKx rate, which at 24 MHz allows a slave SCKx up to 3 MHz. The binding half of that ratio is MISOx: the port launches each bit from the leading SCKx edge and the master latches it at the trailing edge, so the master's half period has to cover the sampling latency. Receive alone would run at a quarter of SMCLK.

13.4.4 Transmit queue

A write to SPIxTX queues one word. In master mode with no frame in progress the frame starts at once and SPITEIF is set immediately; with a frame in progress the queued word starts when that frame ends, and SPITEIF is set then. In slave mode the last word written is loaded when the master's first clock edge arrives, and SPITEIF is set at that edge. The queue holds one word, so a second write during one frame overwrites the first; software waits for SPITEIF = 1 or SPIBUSY = 0 before each write.

13.4.5 Status flags

SPITCIF is set when a frame completes and the received word is in SPIxRX; it is cleared by writing 1 or by reading SPIxRX. SPITEIF is set as described above and cleared by writing 1. SPIBUSY is 1 while the master

is clocking a frame, and in slave mode while the chip select is asserted; it can read 0 between frames of one packet when no word was queued. Flash extended memory reads set none of the flags, because they are completed in hardware without software involvement.

13.4.6 Flash extended memory

With SPIFEN set, hart 0's loads and instruction fetches at or above 0x40000 become flash reads: SPI0 drives CS_FLASH, sends the read command and a 24-bit address, and stalls hart 0 until the word returns. The window is decoded on hart 0's private bus, not through the arbiter, so the other harts have no path to the flash and read zeros there without stalling. The flash address is

$$\text{Flash address} = (\text{CPU address} + \texttt{SPI0FOS}) \bmod 2^{24}$$

so SPI0FOS relocates the flash contents within the window without moving data; to place flash address 0 at 0x40000 itself, SPI0FOS holds the two's complement of the window base modulo 2^{24}. Each 32-bit read costs a command, an address and four data bytes on the serial clock, so the window suits read-only data, lookup tables and code that is not performance-critical.

13.4.7 Boot use of SPI0

The boot ROM uses SPI0 to load the application from the flash (Section 6). During that sequence the ROM drives CS_FLASH as a GPIO output and the flash drives MISO0; at the end of the sequence the ROM puts the flash into deep power-down. After boot the port is free for ordinary transfers and for the extended memory window.

13.5 Interrupts and events

Each flag has an enable in SPIxCR (SPITCIE, SPITEIE); the interrupt is a level asserted while both are set. Table 13.5 lists the vectors.

Vector	Port	Condition	Set by	Cleared by
9	SPI0	Frame complete	SPITCIF	Writing 1, or reading SPIxRX
10	SPI0	Transmit register empty	SPITEIF	Writing 1
11	SPI1	Frame complete	SPITCIF	Writing 1, or reading SPIxRX
12	SPI1	Transmit register empty	SPITEIF	Writing 1

Table 13.5: SPIx interrupt vectors.

13.6 Programming

13.6.1 Initialisation as a master

1. Select the SMCLK source in SYSCLKCR (Section 16.3.1).
2. Set the PxSEL bits of the SCKx, MOSIx and MISOx pins, and their PxAFS fields if the port is relocated.
3. Configure each slave's chip-select pin as a GPIO output driven high.

4. Write SPIxCR.SPIBR for the clock rate.
5. Write SPIxCR.SPICPOL and SPIxCR.SPICPHA for the mode.
6. Write SPIxCR.SPIDL and SPIxCR.SPIMSB for the frame format.
7. Clear SPIxCR.SPISM.
8. Set SPIxCR.SPIEN.

13.6.2 Master transfer of one packet

1. Write 1 to every flag in SPIxSR.
2. Drive the slave's chip-select pin low.
3. Wait for SPITEIF = 1 or SPIBUSY = 0, then write the next word to SPIxTX.
4. Wait for SPITCIF = 1, then read SPIxRX; the read clears the flag.
5. Repeat from step 3 for each further frame.
6. Drive the chip-select pin high.

13.6.3 Initialisation as a slave (SPI1)

1. Select the SMCLK source in SYSCLKCR.
2. Set the PxSEL bits of the SCK1, MOSI1 and MISO1 pins.
3. Configure the CS1 pin as a high-impedance GPIO input; the port never drives it.
4. Write SPI1CR.SPICPHA = 1 and SPI1CR.SPICPOL for the mode.
5. Write SPI1CR.SPIDL and SPI1CR.SPIMSB.
6. Set SPI1CR.SPISM.
7. Set SPI1CR.SPIEN.
8. Write the first word to SPI1TX.

In slave mode SPITEIF at the start of each frame asks for the next word, and SPITCIF at the end delivers the received one in SPI1RX; the received word must be read before the next frame ends, because the next frame overwrites it. The external master's SCK1 must not exceed an eighth of SMCLK, 3 MHz at 24 MHz.

13.6.4 Enabling the flash window

1. Initialise SPI0 as a master in mode 3 with 8-bit frames at a rate within the flash's specification.
2. Configure CS_FLASH as a GPIO output driven high.
3. Send the flash's release-from-deep-power-down command (often 0xAB), because the boot ROM leaves the flash in deep power-down.
4. Send any vendor-specific configuration commands.

5. Write SPI0FOS.
6. Set the PxSEL bit of CS_FLASH, which hands the pin to SPI0.
7. Set SPI0CR.SPIFEN.

13.7 Usage notes

Select the SMCLK source before the first transfer. The boot ROM may leave SMCLK on LFXT (32.768 kHz), where transfers appear to hang.

Keep SMCLK at least eight times the slave clock. The slave port samples SCK1, MOSI1 and CS1 on SMCLK, so a divided SMCLK caps the rate an external master may use; below the ratio the port misses bits rather than slowing down.

Never read SPIxRX speculatively from a hart that does not own the transfer. The read clears SPITCIF on any hart's access and consumes the owner's transfer-complete event; when ownership is unclear, clear flags by writing 1s to SPIxSR.

Serialize whole transfers between harts. The arbiter makes each register access atomic; frames from two harts interleave on the wire and race for SPIxRX (Section 5.4).

Wait for the queue before each write. A second write to SPIxTX within one frame overwrites the queued word.

Keep other SPI0 slaves deselected across reset. The boot ROM clocks the flash while every GPIO is a high-impedance input, so a slave whose chip select can float low drives MISO0 against the flash; pull the chip-select lines high externally or keep those slaves on SPI1.

Use the flash window from hart 0 only. Other harts read zeros there; a program that must run on a channel hart is staged into shared RAM or the hart's TCM first.

Wake the flash before enabling the window. The window issues read commands only; a flash left in deep power-down returns undefined data.

13.8 Registers

Register offsets in this chapter are relative to the instance base address in Table 13.1 (SPI0 0x4200, SPI1 0x4300); the generated header names each base SPI0_BASE and so on.

Table 13.6: SPIx register summary

Offset	Name	Access	Reset	Description
0x00	SPIxCR	rw	0x00000000	SPI control register
0x04	SPIxSR	r/rw1	0x00	SPI status register
0x08	SPIxTX	rw	0x00000000	SPI transmit buffer register
0x0C	SPIxRX	r	0x00000000	SPI receive buffer register

Offset	Name	Access	Reset	Description
0x10	SPIxFOS	rw	0x00000000	SPI Flash memory address offset

13.8.1 SPIxCR: SPI control register

Offset 0x00 **Reset** 0x00000000 **Width** 32 **Address** 0x4200 (SPI0), 0x4300 (SPI1)

Bits	Field	Type	Reset	Description
31:20	-	-	-	Reserved. Reads zero; write zero.
19	FEN	rw	0	Flash extended memory enable. When set, reads of the extended flash address range are served by the SPI peripheral, which issues the flash read command and returns the data over the system bus. Implemented on SPI0 only; reads 0 on SPI1. 0: Flash extended memory disabled 1: Flash extended memory enabled
18	SM	rw	0	Slave mode select. Effective on SPI1 only: SPI0 is master only and ignores this bit. 0: Master mode 1: Slave mode
17	TXSB	rw	0	Transmit byte swap. When set, 32-bit transfers swap bytes 3 with 0 and 2 with 1, and 16-bit transfers swap bytes 1 and 0; 8-bit transfers are unaffected. 0: Bytes not swapped 1: Bytes swapped
16	RXSB	rw	0	Receive byte swap. When set, 32-bit receptions swap bytes 3 with 0 and 2 with 1, and 16-bit receptions swap bytes 1 and 0; 8-bit receptions are unaffected. 0: Bytes not swapped 1: Bytes swapped
15:8	BR	rw	0x00	SPI clock (SCK) baud rate control for master mode. Baud rate = SMCLK / (2 * (1 + SPIBR)). For example, with SMCLK at 24 MHz: SPIBR=0 gives 12 MHz, SPIBR=1 gives 8 MHz, SPIBR=2 gives 6 MHz, SPIBR=5 gives 4 MHz, SPIBR=11 gives 2 MHz, SPIBR=23 gives 1 MHz.
7	EN	rw	0	SPI enable. When disabled, all SPI operations cease and the peripheral is held in reset state. 0: Disabled 1: Enabled
6	MSB	rw	0	Bit endianness select. Determines whether data is transmitted and received MSB-first or LSB-first. 0: LSB-first 1: MSB-first
5	TCIE	rw	0	SPI transmit complete interrupt enable. Interrupt triggers when a full SPI transfer (transmit and receive) completes. 0: Interrupt disabled 1: Interrupt enabled

Bits	Field	Type	Reset	Description
4	TEIE	rw	0	SPI transmit register empty interrupt enable. Interrupt triggers when SPIxTX register is empty and ready to accept new data for the next transfer. 0: Interrupt disabled 1: Interrupt enabled
3:2	DL	rw	0b00	SPI transmission data length select. Determines the number of bits transferred per SPI transaction. 0b00 SPIDL_8: 8-bit transfers 0b01 SPIDL_16: 16-bit transfers 0b10 SPIDL_32: 32-bit transfers 0b11 SPIDL_RES: Reserved (do not use)
1	CPOL	rw	0	SPI clock (SCK) polarity. Determines the idle state of the SCK line. 0: SCK idles low (SPIMODE0 or SPIMODE1) 1: SCK idles high (SPIMODE2 or SPIMODE3)
0	CPHA	rw	0	SPI clock (SCK) phase. Determines when data is sampled relative to the SCK edge. In slave mode, only SPICPHA=1 is supported. 0: Data sampled on leading edge, shifted on trailing edge (SPIMODE0 or SPIMODE2) 1: Data shifted on leading edge, sampled on trailing edge (SPIMODE1 or SPIMODE3)

13.8.2 SPIxSR: SPI status register

Offset 0x04 **Reset** 0x00 **Width** 8 **Address** 0x4204 (SPI0), 0x4304 (SPI1)
SPI status register. Provides real-time status of SPI transfer operations and interrupt flags.

Bits	Field	Type	Reset	Description
7:3	-	-	-	Reserved. Reads zero; write zero.
2	BUSY	r	0	Indicates whether a SPI transfer is currently in progress. In master mode, set when a transfer starts and cleared when complete. In slave mode, set when chip select is asserted (driven low) and cleared when deasserted. 0: SPI is idle 1: SPI transfer in progress
1	TCIF	rw1	0	SPI transfer complete interrupt flag. Set when a SPI transfer completes. Must be cleared by writing 1 to this bit or by reading SPIxRX register. 0: No transfer completed 1: Transfer completed
0	TEIF	rw1	0	Transmit register empty interrupt flag, set when the transmitter has taken the SPIxTX contents so the register can accept the next word; a word written while a transfer is in progress is sent after it. Write 1 to clear. 0: SPIxTX not empty 1: SPIxTX empty and ready

13.8.3 SPIxTX: SPI transmit buffer register

Offset 0x08 **Reset** 0x00000000 **Width** 32 **Address** 0x4208 (SPI0), 0x4308 (SPI1)
SPI transmit buffer register. In master mode, writing to this register initiates a new SPI transfer. In slave mode, writing to this register queues data to be transmitted during the next master-initiated transfer. Actual number of bits transmitted is determined by SPIDL setting.

Bits	Field	Type	Reset	Description
31:0	TX	rw	0x00000000	Transmit data. In master mode a write starts a transfer of the low 8, 16 or 32 bits as selected by SPIDL; in slave mode the write loads the data shifted out during the next master-driven transfer. Reads return the last value written; resets to 0.

13.8.4 SPIxRX: SPI receive buffer register

Offset 0x0C **Reset** 0x00000000 **Width** 32 **Address** 0x420C (SPI0), 0x430C (SPI1)
SPI receive buffer register. Contains the data received during the most recent SPI transfer. Reading this register also clears the SPITCIF flag. Valid data width depends on SPIDL setting; unused upper bits read as 0.

Bits	Field	Type	Reset	Description
31:0	RX	r	0x00000000	Received data from the most recent transfer, right aligned in the width selected by SPIDL with the unused upper bits reading 0. Reading this register clears SPITCIF.

13.8.5 SPIxFOS: SPI Flash memory address offset

Offset 0x10 **Reset** 0x00000000 **Width** 32 **Address** 0x4210 (SPI0), 0x4310 (SPI1)
SPI Flash memory address offset. This 24-bit value is added to memory access addresses when flash extended memory mode is enabled (SPIFEN=1). Allows remapping of flash memory to different virtual addresses. The addition wraps around at 0x00FFFFFF. Available only on SPI0; reads as 0 on SPI1.

Bits	Field	Type	Reset	Description
31:24	-	-	-	Reserved. Reads zero; write zero.
23:0	FOS	rw	0x000000	Flash address offset, added to the bus address of every flash extended memory read while SPIFEN = 1; the 24-bit sum wraps. Resets to 0. Implemented on SPI0 only; on SPI1 the field reads 0.

14 Universal asynchronous receiver and transmitter (UARTx)

14.1 Overview

The Universal Asynchronous Receiver/Transmitter (UARTx) is a two-wire, full-duplex asynchronous serial interface with hardware parity. UART0 is the console port used by the boot ROM's Forth interpreter; UART1, in configurations that include it, is identical.

- Independent transmitter and receiver, 8-bit frames, one stop bit
- Baud generator from SMCLK with 16 times oversampling (UARTxBR)
- Even or odd parity, generated and checked in hardware (UPEN, PSEL)
- Framing, parity and receive-overflow error flags (FEF, PEF, OVF)
- One-deep transmit queue for back-to-back frames
- Three interrupts per port: receive complete, transmit register empty, transmit complete
- Latched register reads for stable status during reception

14.2 Instances and signals

Table 14.1: UARTx instances

Instance	Base address	Interrupt vector(s)	Pins (AF)
UART0	0x4400	13 to 15	TX0 P1.4 (AF0), RX0 P1.5 (AF0)
UART1	0x4500	52 to 54	TX1 P1.6 (AF0), RX1 P1.7 (AF0)

Table 14.2: UARTx signals

Signal	Direction	Description	Pin (AF)
TX0	Output	UART0 transmitter	P0.0 (AF1), P0.2 (AF7), P0.3 (AF6), P0.4 (AF5), P0.5 (AF4), P0.6 (AF3), P0.7 (AF2), P1.1 (AF7), P1.2 (AF4), P1.3 (AF3), P1.4 (AF0), P1.5 (AF5), P1.6 (AF2), P1.7 (AF3), P2.0 (AF4), P2.1 (AF6), P2.3 (AF7), P2.4 (AF1), P2.5 (AF2), P2.6 (AF4), P2.7 (AF3), P3.0 (AF2), P3.3 (AF7), P3.4 (AF2), P3.7 (AF6)
RX0	Bidirectional	UART0 receiver	P1.5 (AF0), P2.5 (AF1), P3.5 (AF2)

Signal	Direction	Description	Pin (AF)
TX1	Output	UART1 transmitter	P0.0 (AF2), P0.1 (AF1), P0.3 (AF7), P0.4 (AF6), P0.5 (AF5), P0.6 (AF4), P0.7 (AF3), P1.0 (AF2), P1.2 (AF5), P1.3 (AF4), P1.4 (AF4), P1.5 (AF6), P1.6 (AF0), P1.7 (AF4), P2.0 (AF1), P2.1 (AF7), P2.4 (AF6), P2.5 (AF3), P2.6 (AF5), P2.7 (AF4), P3.0 (AF3), P3.1 (AF2), P3.4 (AF3), P3.7 (AF7)
RX1	Bidirectional	UART1 receiver	P1.7 (AF0), P2.1 (AF1)

Table 14.3: UARTx interrupt vectors

Vector	Instance	Description	Clear method
13	UART0	UART0 Receive Complete Interrupt IRQB_UART0_RC	Write 1 to UART0SR.RCIF, or read UART0RX
14	UART0	UART0 Transmission Buffer Empty Interrupt IRQB_UART0_TE	Write 1 to UART0SR.TEIF
15	UART0	UART0 Transmission Complete Interrupt IRQB_UART0_TC	Write 1 to UART0SR.TCIF
52	UART1	UART1 Receive Complete Interrupt IRQB_UART1_RC	Write 1 to UART1SR.RCIF, or read UART1RX
53	UART1	UART1 Transmission Buffer Empty Interrupt IRQB_UART1_TE	Write 1 to UART1SR.TEIF
54	UART1	UART1 Transmission Complete Interrupt IRQB_UART1_TC	Write 1 to UART1SR.TCIF

14.3 Block diagram

Figure 14.1 shows the two-wire connection to an external device.

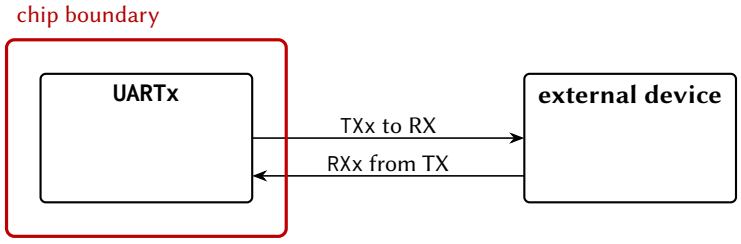


Figure 14.1: UART connection to an external device.

14.4 Functional description

14.4.1 Frame

Each wire is unidirectional and idles high. A frame is a start bit (low for one bit time $T_B = 1/\text{baud}$), eight data bits LSB first, an optional parity bit, and a stop bit (high for T_B). The transmitter may start the next frame immediately after the stop bit or leave the wire idle. Figure 14.2 shows one frame.

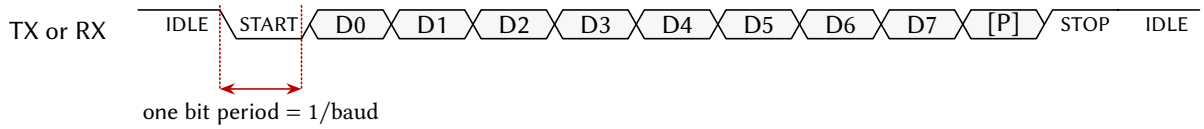


Figure 14.2: One UART frame with the optional parity bit.

14.4.2 Baud generation

Both ends must agree on the baud rate, because the receiver has no clock wire and samples each bit at the centre of its expected period. The receiver oversamples at 16 times the baud rate for noise immunity, and the rate is set by the 12-bit UARTxBR:

$$\text{baud} = \frac{f_{\text{SMCLK}}}{16 \times (\text{BR} + 1)}$$

With SMCLK at 24 MHz, BR = 12 gives 115,385 baud, BR = 25 gives 57,692 baud and BR = 51 gives 28,846 baud, each within 0.2 % of the standard rate. The serial engine runs on SMCLK and the register interface on MCLK, so the registers stay readable with SMCLK stopped.

14.4.3 Parity

With UPEN set the transmitter appends $p = b_7 \oplus \cdots \oplus b_0 \oplus \text{PSEL}$, which is even parity for PSEL = 0 and odd for 1, and the receiver checks the same function and sets PEF on a mismatch. Parity detects any odd number of bit errors in a frame and misses any even number.

14.4.4 Transmit queue

A write to UARTxTX queues one byte. If no frame is in progress the frame starts at once and TEIF is set immediately; otherwise the byte starts when the current frame ends, and TEIF is set then. The queue holds one byte, so software waits for TEIF = 1 or TXBF = 0 before each write. TCIF is set when the last stop bit has been sent and the transmitter is idle.

14.4.5 Reception and status

RXBF is 1 while a frame is being received. When the stop bit has been sampled, the byte is placed in UARTxRX, RCIF is set, and FEF, PEF and OVF are updated: FEF when the stop bit was not high, which usually means the two ends disagree on rate or format; PEF on a parity mismatch; OVF when the previous byte had not been read, in which case it is lost. A byte received with FEF or PEF set is corrupt. Reading UARTxRX clears RCIF, FEF, PEF and OVF; UARTxSR and UARTxRX are latched on the falling edge of the memory enable signal, so a read during reception returns one stable sample.

14.5 Interrupts and events

The three interrupt flags in UARTxSR have enables in UARTxCR (CIE, TEIE, TCIE); each interrupt is a level asserted while both are set. Table 14.4 lists the vectors.

Vector	Port	Condition	Set by	Cleared by
13	UART0	Byte received	RCIF	Reading UARTxRX, or writing 1
14	UART0	Transmit register empty	TEIF	Writing 1
15	UART0	Transmission complete	TCIF	Writing 1
52	UART1	Byte received	RCIF	Reading UARTxRX, or writing 1
53	UART1	Transmit register empty	TEIF	Writing 1
54	UART1	Transmission complete	TCIF	Writing 1

Table 14.4: UARTx interrupt vectors.

14.6 Programming

14.6.1 Initialisation

1. Select the SMCLK source in SYSCLKCR (Section 16.3.1).
2. Set the PxSEL bits of the TXx and RXx pins, and their PxAFS fields if the port is relocated.
3. Write UARTxBR.BR for the baud rate.
4. Write UARTxCR.UPEN and UARTxCR.PSEL for parity.
5. Set the interrupt enables required in UARTxCR.
6. Set UARTxCR.UEN; while it is 0 the port is held in reset.

14.6.2 Transmitting a byte

1. Wait for TXBF = 0 or TEIF = 1.
2. Write the byte to UARTxTX.

14.6.3 Receiving a byte

1. Wait for RCIF = 1.
2. Read UARTxSR and reject the byte if FEF or PEF is set.
3. Read UARTxRX; the read clears the flags.

14.7 Usage notes

Select the SMCLK source before the first frame. With SMCLK on LFXT (32.768 kHz), as the boot ROM may leave it, each frame takes about 10 ms.

Read the status before the data. Reading UARTxRX clears the error flags, so a handler that reads the byte first cannot tell whether it was corrupt.

Serialize whole messages between harts. The arbiter makes each register access atomic; bytes from two harts interleave on the wire (Section 5.4).

Clear TEIF and TCIF with a 1 before returning. Both are levels into the interrupt controller.

Leave UART0 to the console during boot. The boot ROM's Forth interpreter owns UART0 at 115,200 baud until the application starts (Section 6).

14.8 Registers

Register offsets in this chapter are relative to the instance base address in Table 14.1 (UART0 0x4400, UART1 0x4500); the generated header names each base UART0_BASE and so on.

Table 14.5: UARTx register summary

Offset	Name	Access	Reset	Description
0x00	UARTxCR	rw	0x00	UART control register
0x04	UARTxSR	r/rw1	0x00	UART status register
0x08	UARTxBR	rw	0x0000	UART baud rate register
0x0C	UARTxRX	r	0x00	UART receive buffer register
0x10	UARTxTX	rw	0x00	UART transmit buffer register

14.8.1 UARTxCR: UART control register

Offset 0x00 **Reset** 0x00 **Width** 8 **Address** 0x4400 (UART0), 0x4500 (UART1)
UART control register. Controls UART enable, parity configuration, and interrupt enable bits. When UCR.EN is disabled, the UART peripheral is held in reset state.

Bits	Field	Type	Reset	Description
7:6	-	-	-	Reserved. Reads zero; write zero.
5	EN	rw	0	UART enable. When disabled, the UART peripheral is in reset state and the transmitter is idle. 0: UART disabled 1: UART enabled
4	PEN	rw	0	Parity enable. Enables parity generation on transmit and parity checking on receive. 0: Parity disabled 1: Parity enabled
3	PSEL	rw	0	Parity select. Selects even or odd parity when parity is enabled. 0: Even parity 1: Odd parity

Bits	Field	Type	Reset	Description
2	CIE	rw	0	Receive complete interrupt enable. Enables interrupt generation when a byte is successfully received. 0: RX complete interrupt disabled 1: RX complete interrupt enabled
1	TEIE	rw	0	Transmit empty interrupt enable. Enables interrupt generation when the transmit buffer becomes empty and ready for new data. 0: TX empty interrupt disabled 1: TX empty interrupt enabled
0	TCIE	rw	0	Transmit complete interrupt enable. Enables interrupt generation when transmission is complete and the transmitter is idle. 0: TX complete interrupt disabled 1: TX complete interrupt enabled

14.8.2 UARTxSR: UART status register

Offset 0x04 **Reset** 0x00 **Width** 8 **Address** 0x4404 (UART0), 0x4504 (UART1)
UART status register. Contains receiver and transmitter status flags and interrupt flags. Error flags (FEF, PEF, OVF, RCIF) are cleared by reading UARTxRX. Interrupt flags (RCIF, TEIF, TCIF) can also be cleared by writing a 1 to the respective bit.

Bits	Field	Type	Reset	Description
7	RXBF	r	0	Receiver busy flag. Set while the UART receiver is actively receiving a byte. 0: Receiver idle 1: Reception in progress
6	TXBF	r	0	Transmitter busy flag. Set while the UART transmitter is actively transmitting a byte or has a pending transmission. 0: Transmitter idle 1: Transmission in progress
5	FEF	r	0	Framing error flag. Set when the stop bit is not detected (RX line not high at expected stop bit time). Cleared by reading UARTxRX. 0: No framing error 1: Framing error detected on last reception
4	PEF	r	0	Parity error flag. Set when received parity does not match expected parity (when parity is enabled). Cleared by reading UARTxRX. 0: No parity error 1: Parity error detected on last reception
3	OVF	r	0	Receive overflow flag. Set when a new byte is received before the previous byte in UARTxRX was read by the processor. Cleared by reading UARTxRX. 0: No receive data overrun 1: Receive data overflow detected
2	RCIF	rw1	0	Receive complete interrupt flag. Set when a byte is successfully received and placed in UARTxRX. Cleared by reading UARTxRX or writing a 1 to this bit. 0: No pending RX complete interrupt 1: RX complete interrupt pending

Bits	Field	Type	Reset	Description
1	TEIF	rw1	0	Transmit empty interrupt flag. Set when the transmit buffer is empty and ready to accept new data. Write a 1 to this bit to clear it. 0: No pending TX empty interrupt 1: TX empty interrupt pending
0	TCIF	rw1	0	Transmit complete interrupt flag. Set when transmission is complete (all bits sent) and the transmitter is idle. Write a 1 to this bit to clear it. 0: No pending TX complete interrupt 1: TX complete interrupt pending

14.8.3 UARTxBR: UART baud rate register

Offset 0x08 **Reset** 0x0000 **Width** 16 **Address** 0x4408 (UART0), 0x4508 (UART1)
UART baud rate register. Configures the baud rate divisor for the UART. The UART uses 16× oversampling.

Bits	Field	Type	Reset	Description
15:12	-	-	-	Reserved. Reads zero; write zero.
11:0	BR	rw	0x000	Baud rate divisor. UART baud rate = SMCLK ÷ (16 × (BR + 1)). For example, with SMCLK = 48 MHz: BR = 25 gives 115,200 baud; BR = 51 gives 57,600 baud; BR = 103 gives 28,800 baud.

14.8.4 UARTxRX: UART receive buffer register

Offset 0x0C **Reset** 0x00 **Width** 8 **Address** 0x440C (UART0), 0x450C (UART1)
UART receive buffer register. Contains the last received byte. Reading this register clears the FEF, PEF, OVF, and RCIF flags in UARTxSR. The value is latched on the falling edge of the memory enable signal.

Bits	Field	Type	Reset	Description
7:0	RX	r	0x00	Received data byte

14.8.5 UARTxTX: UART transmit buffer register

Offset 0x10 **Reset** 0x00 **Width** 8 **Address** 0x4410 (UART0), 0x4510 (UART1)
UART transmit buffer register. Writing to this register loads the byte to transmit and initiates transmission. Do not write to this register while TXBF is set in UARTxSR.

Bits	Field	Type	Reset	Description
7:0	TX	rw	0x00	Transmit data byte

15 Timer (TIMERx)

15.1 Overview

The TIMERx peripheral is a 32-bit general-purpose timer/counter with a glitch-free clock source multiplexer, three output compare units and two input capture units.

- 32-bit counter (TIMxVAL), readable and writable at any time
- Three output compare units (TIMxCMP0 to TIMxCMP2); units 0 and 1 drive PWM outputs, unit 2 sets the period
- Two input capture units (TIMxCAP0, TIMxCAP1) with selectable edge
- Four clock sources (SMCLK, MCLK, LFXT, HFXT) and a divider from 1 to 32,768
- Glitch-free clock source switching while the timer runs
- Six interrupts: two capture, three compare, one overflow
- Latched register reads for stable values while the counter runs

15.2 Instances and signals

Table 15.1: TIMERx instances

Instance	Base address	Interrupt vector(s)	Pins (AF)
TIMER0	0x4600	16 to 21	T0CMP0 P2.0 (AF0), T0CMP1 P2.1 (AF0), T0CAP0 P2.2 (AF0), T0CAP1 P2.3 (AF0)
TIMER1	0x4700	22 to 27	T1CMP0 P2.4 (AF0), T1CMP1 P2.5 (AF0), T1CAP0 P2.6 (AF0), T1CAP1 P2.7 (AF0)

Table 15.2: TIMERx signals

Signal	Direction	Description	Pin (AF)
T0CMP0	Output	TIMER0 Compare 0	P0.0 (AF3), P0.1 (AF2), P0.2 (AF1), P0.4 (AF7), P0.5 (AF6), P0.6 (AF5), P0.7 (AF4), P1.0 (AF1), P1.1 (AF2), P1.2 (AF6), P1.3 (AF5), P1.4 (AF5), P1.5 (AF7), P1.6 (AF3), P1.7 (AF5), P2.0 (AF0), P2.4 (AF7), P2.5 (AF4), P2.6 (AF6), P2.7 (AF5), P3.0 (AF4), P3.1 (AF3), P3.2 (AF2), P3.4 (AF1), P3.5 (AF3), P3.6 (AF2)

Signal	Direction	Description	Pin (AF)
T0CMP1	Output	TIMER0 Compare 1	P0.0 (AF4), P0.1 (AF3), P0.2 (AF2), P0.3 (AF1), P0.5 (AF7), P0.6 (AF6), P0.7 (AF5), P1.0 (AF3), P1.1 (AF1), P1.2 (AF7), P1.3 (AF6), P1.4 (AF6), P1.6 (AF4), P1.7 (AF6), P2.0 (AF5), P2.1 (AF0), P2.2 (AF3), P2.4 (AF2), P2.5 (AF5), P2.6 (AF7), P2.7 (AF6), P3.0 (AF5), P3.1 (AF4), P3.2 (AF3), P3.3 (AF2), P3.4 (AF4), P3.5 (AF1), P3.6 (AF3)
T0CAP0	Input	TIMER0 Capture 0	P2.2 (AF0), P3.0 (AF1)
T0CAP1	Input	TIMER0 Capture 1	P2.3 (AF0), P3.1 (AF1)
T1CMP0	Output	TIMER1 Compare 0	P0.0 (AF5), P0.1 (AF4), P0.2 (AF3), P0.3 (AF2), P0.4 (AF1), P0.6 (AF7), P0.7 (AF6), P1.0 (AF4), P1.1 (AF3), P1.2 (AF1), P1.3 (AF7), P1.4 (AF7), P1.6 (AF5), P1.7 (AF7), P2.0 (AF6), P2.1 (AF2), P2.2 (AF4), P2.3 (AF3), P2.4 (AF0), P2.5 (AF6), P2.7 (AF7), P3.0 (AF6), P3.1 (AF5), P3.2 (AF4), P3.3 (AF3), P3.4 (AF5), P3.5 (AF4), P3.6 (AF1), P3.7 (AF3)
T1CMP1	Output	TIMER1 Compare 1	P0.0 (AF6), P0.1 (AF5), P0.2 (AF4), P0.3 (AF3), P0.4 (AF2), P0.5 (AF1), P0.7 (AF7), P1.0 (AF5), P1.1 (AF4), P1.2 (AF2), P1.3 (AF1), P1.5 (AF2), P1.6 (AF6), P2.0 (AF7), P2.1 (AF3), P2.2 (AF5), P2.3 (AF4), P2.4 (AF3), P2.5 (AF0), P3.0 (AF7), P3.1 (AF6), P3.2 (AF5), P3.3 (AF4), P3.4 (AF6), P3.5 (AF5), P3.6 (AF4), P3.7 (AF1)
T1CAP0	Input	TIMER1 Capture 0	P2.6 (AF0), P3.2 (AF1)
T1CAP1	Input	TIMER1 Capture 1	P2.7 (AF0), P3.3 (AF1)

Table 15.3: TIMERx interrupt vectors

Vector	Instance	Description	Clear method
16	TIMER0	TIMER0 Capture 0 InterruptIRQB_TIM0_CAP0	Write 1 to TIM0SR.CAP0IF
17	TIMER0	TIMER0 Capture 1 InterruptIRQB_TIM0_CAP1	Write 1 to TIM0SR.CAP1IF
18	TIMER0	TIMER0 Overflow InterruptIRQB_TIM0_OVF	Write 1 to TIM0SR.OVIF
19	TIMER0	TIMER0 Compare 0 InterruptIRQB_TIM0_CMP0	Write 1 to TIM0SR.CMP0IF
20	TIMER0	TIMER0 Compare 1 InterruptIRQB_TIM0_CMP1	Write 1 to TIM0SR.CMP1IF
21	TIMER0	TIMER0 Compare 2 InterruptIRQB_TIM0_CMP2	Write 1 to TIM0SR.CMP2IF
22	TIMER1	TIMER1 Capture 0 InterruptIRQB_TIM1_CAP0	Write 1 to TIM1SR.CAP0IF
23	TIMER1	TIMER1 Capture 1 InterruptIRQB_TIM1_CAP1	Write 1 to TIM1SR.CAP1IF
24	TIMER1	TIMER1 Overflow InterruptIRQB_TIM1_OVF	Write 1 to TIM1SR.OVIF

Vector	Instance	Description	Clear method
25	TIMER1	TIMER1 Compare 0 InterruptIRQB_TIM1_CMP0	Write 1 to TIM1SR.CMP0IF
26	TIMER1	TIMER1 Compare 1 InterruptIRQB_TIM1_CMP1	Write 1 to TIM1SR.CMP1IF
27	TIMER1	TIMER1 Compare 2 InterruptIRQB_TIM1_CMP2	Write 1 to TIM1SR.CMP2IF

15.3 Block diagram

Figure 15.1 shows the clock path: the source multiplexer, the divider and the counter.

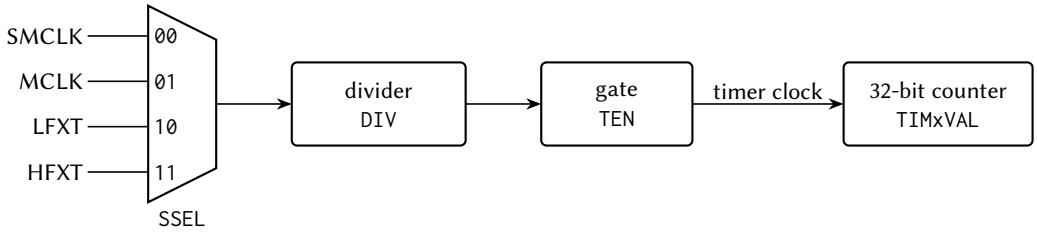


Figure 15.1: Clock path of the TIMERx peripheral.

15.4 Functional description

15.4.1 Clock source and divider

SSEL selects the timer clock from SMCLK, MCLK, LFXT or HFXT. The multiplexer is glitch-free, so the source can be changed while the timer is enabled without a runt pulse reaching the counter; the price is that the timer clock stops for a few cycles of the slower of the two clocks involved while the old source is released and the new one engages. DIV then divides the selected clock, and TIMxVAL increments on every rising edge of the divided clock while TEN is set.

The multiplexer powers up parked on the SMCLK slice and needs three edges of the clock being released before another source can engage. The first source selection after reset therefore waits three SMCLK edges at whatever rate SMCLK has at that moment, because the boot ROM may leave SMCLK on LFXT (32.768 kHz), and in that state the wait is about 92 µs even when the selected source is MCLK or HFXT.

15.4.2 Counting

Setting TEN starts the counter from its current value; clearing it holds the value, which is kept until the timer is enabled again, written, or reset. TIMxVAL can be written at any time, enabled or not, and the new value takes effect immediately. Reads of TIMxVAL are latched on the falling edge of the memory enable signal, so a read during counting returns one stable sample.

15.4.3 Overflow and rollover

When the counter reaches $2^{32} - 1$ it rolls over to zero on the next clock and sets OVIF. With CMP2RST set, the counter instead rolls over to zero on the clock after it equals TIMxCMP2, which sets the period of the PWM outputs, and CMP2IF is set at each such rollover while OVIF is not. The comparison is equality, not greater-or-equal: a counter that is above TIMxCMP2, because software wrote TIMxVAL or lowered TIMxCMP2 below the current value, counts on to $2^{32} - 1$ before it wraps. Figure 15.2 shows the rollover.

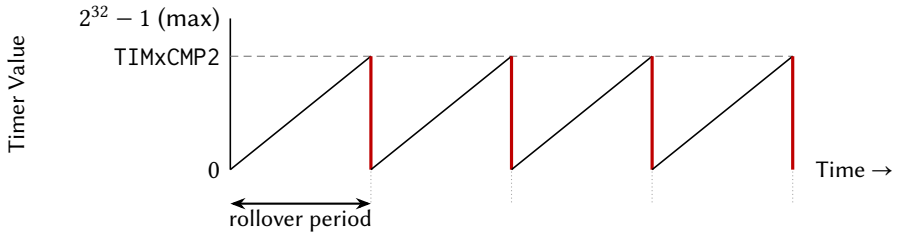


Figure 15.2: Rollover at TIMxCMP2 with CMP2RST set.

15.4.4 Output compare and PWM

Compare units 0 and 1 each hold a compare value (TIMxCMP0, TIMxCMP1) that is compared with TIMxVAL continuously. The output TxCMPy toggles when the counter equals the compare value and again when the counter rolls over, so with CMP2RST set the pair TIMxCMP2 and TIMxCMPy sets the period and the duty cycle. CMP0IH and CMP1IH select the output level while the counter is below the compare value. CMP0OUT and CMP1OUT in TIMxSR report the current output levels whether or not the pins are in alternate-function mode. Figure 15.3 shows one PWM cycle.

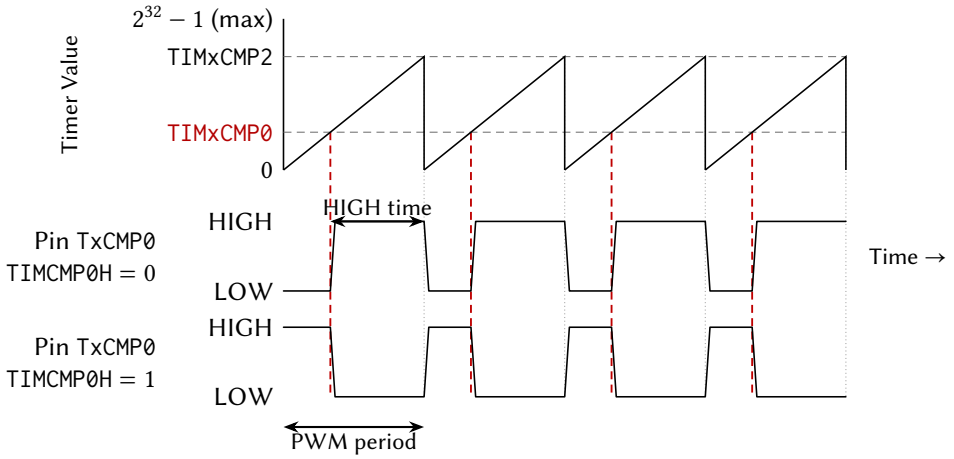


Figure 15.3: Output compare and PWM generation on TxCMP0.

15.4.5 Input capture

Capture unit y, enabled by CAPyEN, waits for the edge selected by CAPyFE (0 = rising, 1 = falling) on TxCAPy, then copies TIMxVAL into TIMxCAPy and sets CAPyIF. A capture that arrives while CAPyIF is still set overwrites TIMxCAPy, so the flag is cleared, by writing 1 to it in TIMxSR, as soon as the value has been read. Figure 15.4 shows one capture: the selected edge on TxCAP0 latches TIMxVAL into TIMxCAP0 and sets CAP0IF on the same timer clock.

15.4.6 Status register

TIMxSR holds the two output indicators and the six interrupt flags. It is latched on the falling edge of the memory enable signal, and every flag is write-1-to-clear.

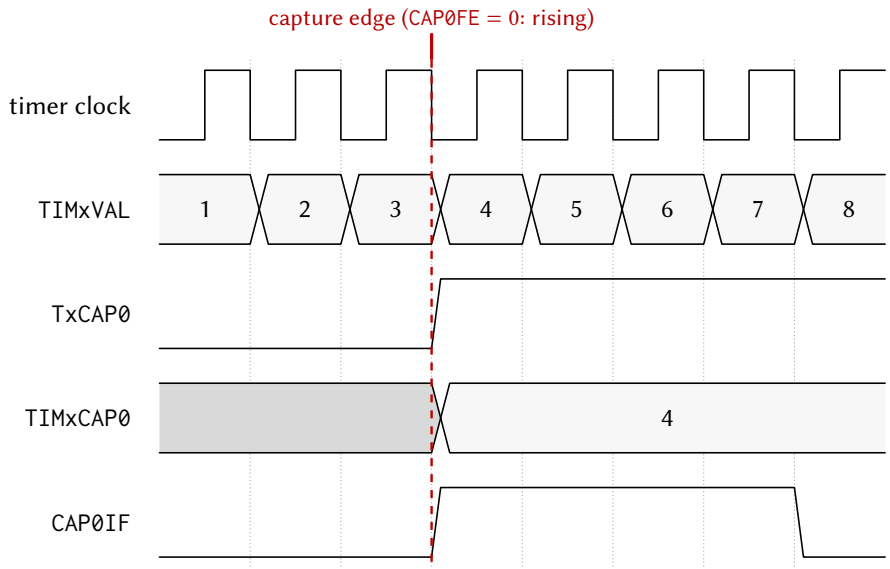


Figure 15.4: Input capture: an edge on TxCAP0 latches TIMxVAL into TIMxCAP0.

15.5 Interrupts and events

Each flag has an enable bit in TIMxCR; the interrupt is a level asserted while both are set. Table 15.4 lists the vectors of TIMER0; TIMER1, in configurations that include it, uses the same order from vector 22 (22 to 27).

Vector	Condition	Set by	Cleared by
16	Capture on T0CAP0	CAP0IF	Writing 1 to CAP0IF
17	Capture on T0CAP1	CAP1IF	Writing 1 to CAP1IF
18	Counter overflow from $2^{32} - 1$	OVIF	Writing 1 to OVIF
19	Counter equals TIMxCMP0	CMP0IF	Writing 1 to CMP0IF
20	Counter equals TIMxCMP1	CMP1IF	Writing 1 to CMP1IF
21	Rollover at TIMxCMP2 (CMP2RST set)	CMP2IF	Writing 1 to CMP2IF

Table 15.4: TIMER0 interrupt vectors; TIMER1 follows the same order from vector 22.

15.6 Programming

15.6.1 Initialisation as a PWM generator

1. Select a fast SMCLK source in SYSCLKCR (Section 16.3.1).
2. Write the period minus one to TIMxCMP2.
3. Write the duty value to TIMxCMP0.
4. Write 0 to TIMxVAL.
5. Set the PxSEL bit of the TxCMP0 pin, and its PxAFS field if the output is relocated.

6. Write TIMxCR.SSEL and TIMxCR.DIV.
7. Set TIMxCR.CMP2RST and TIMxCR.CMP0IH as required.
8. Set TIMxCR.TEN.
9. Poll TIMxVAL until it has changed, because the clock handoff may hold the counter for a few cycles.

15.6.2 Initialisation for input capture

1. Clear the PxDIR bit of the TxCAPy pin, and write its PxAFS field if the input is relocated.
2. Write TIMxCR.CAPyFE for the edge.
3. Set TIMxCR.CAPyIE if an interrupt is wanted, and enable the vector in the hart's IRQROUTER row.
4. Set TIMxCR.CAPyEN.
5. Set TIMxCR.TEN.

15.7 Usage notes

Select the SMCLK source before the first timer clock selection. The multiplexer releases its reset slice only after three SMCLK edges, so with SMCLK on LFXT the first selection stalls the timer for about 92 µs.

Poll TIMxVAL after enabling. Two reads immediately after setting TEN may return the same value while the clock handoff completes; the timer is running once the value has changed.

Do not change CAPyFE while CAPyEN is set. The edge detector can see the change as a capture event.

Keep TIMxVAL below TIMxCMP2 when CMP2RST is set. The rollover fires on equality only; a counter above the compare value runs to $2^{32} - 1$.

Clear each flag with a 1 before returning. Every flag in TIMxSR is a level into the interrupt controller.

Own a timer from one hart at a time. The arbiter serializes register accesses but not sequences; a second hart writing TIMxCR changes the configuration under the owner.

15.8 Registers

Register offsets in this chapter are relative to the instance base address in Table 15.1 (TIMER0 0x4600, TIMER1 0x4700); the generated header names each base TIMER0_BASE and so on.

Table 15.5: TIMERx register summary

Offset	Name	Access	Reset	Description
0x00	TIMxCR	rw	0x00000000	Timer/Counter control register
0x04	TIMxSR	r/rw1	0x00	Timer/Counter status register

Offset	Name	Access	Reset	Description
0x08	TIMxVAL	rw	0x00000000	Timer/Counter value register
0x0C	TIMxCMP0	rw	0x00000000	Timer/Counter Compare 0 threshold register
0x10	TIMxCMP1	rw	0x00000000	Timer/Counter Compare 1 threshold register
0x14	TIMxCMP2	rw	0x00000000	Timer/Counter Compare 2 threshold register
0x18 + 4n	TIMxCAPn (n = 0..1)	r	0x00000000	Timer/Counter Capture n value register

15.8.1 TIMxCR: Timer/Counter control register

Offset 0x00 **Reset** 0x00000000 **Width** 32 **Address** 0x4600 (TIMER0), 0x4700 (TIMER1)
Timer/Counter control register. Configures clock source, clock divider, capture/compare settings, and interrupt enables.

Bits	Field	Type	Reset	Description
31:20	-	-	-	Reserved. Reads zero; write zero.
19:16	DIV	rw	0b0000	Timer/Counter clock divider. Divides the clock source selected by SSEL to generate the timer clock for TIMxVAL increments. 0b0000 DIV_1: /1 (no division) 0b0001 DIV_2: /2 0b0010 DIV_4: /4 0b0011 DIV_8: /8 0b0100 DIV_16: /16 0b0101 DIV_32: /32 0b0110 DIV_64: /64 0b0111 DIV_128: /128 0b1000 DIV_256: /256 0b1001 DIV_512: /512 0b1010 DIV_1024: /1,024 0b1011 DIV_2048: /2,048 0b1100 DIV_4096: /4,096 0b1101 DIV_8192: /8,192 0b1110 DIV_16384: /16,384 0b1111 DIV_32768: /32,768
15	CMP1IH	rw	0	Compare 1 initial PWM output level. Sets the TxCMP1 pin level when TIMxVAL < TIMxCMP1. 0: PWM output starts LOW 1: PWM output starts HIGH
14	CMP0IH	rw	0	Compare 0 initial PWM output level. Sets the TxCMP0 pin level when TIMxVAL < TIMxCMP0. 0: PWM output starts LOW 1: PWM output starts HIGH

Bits	Field	Type	Reset	Description
13	CAP1FE	rw	0	Capture 1 edge select. Selects which edge on TxCAP1 pin triggers a capture event. 0: Capture on rising edge 1: Capture on falling edge
12	CAP0FE	rw	0	Capture 0 edge select. Selects which edge on TxCAP0 pin triggers a capture event. 0: Capture on rising edge 1: Capture on falling edge
11	CAP1EN	rw	0	Capture 1 enable. Enables input capture on TxCAP1 pin. 0: Capture 1 disabled 1: Capture 1 enabled
10	CAP0EN	rw	0	Capture 0 enable. Enables input capture on TxCAP0 pin. 0: Capture 0 disabled 1: Capture 0 enabled
9:8	SSEL	rw	0b00	Timer/Counter clock source select. Glitch-free multiplexer prevents spurious transitions when switching sources. 0b00 SSEL_SMCLK: SMCLK 0b01 SSEL_MCLK: MCLK 0b10 SSEL_LFXT: LFXT (low frequency crystal) 0b11 SSEL_HFXT: HFXT (high frequency crystal)
7	CMP2RST	rw	0	Timer/Counter reset on Compare 2 enable. Resets TIMxVAL to 0 when TIMxVAL equals TIMxCMP2. 0: Free-running mode (resets at 2^{32}-1) 1: Resets on TIMxVAL = TIMxCMP2
6	EN	rw	0	Timer/Counter enable. When disabled, timer clock is gated off and TIMxVAL holds its value. 0: Timer disabled 1: Timer enabled
5	CAP1IE	rw	0	Capture 1 interrupt enable. Enables interrupt when CAP1IF flag is set. 0: Interrupt disabled 1: Interrupt enabled
4	CAP0IE	rw	0	Capture 0 interrupt enable. Enables interrupt when CAP0IF flag is set. 0: Interrupt disabled 1: Interrupt enabled
3	OVIE	rw	0	Timer/Counter overflow interrupt enable. Enables interrupt when OVIF flag is set. 0: Interrupt disabled 1: Interrupt enabled
2	CMP2IE	rw	0	Compare 2 interrupt enable. Enables interrupt when CMP2IF flag is set. 0: Interrupt disabled 1: Interrupt enabled

Bits	Field	Type	Reset	Description
1	CMP1IE	rw	0	Compare 1 interrupt enable. Enables interrupt when CMP1IF flag is set. 0: Interrupt disabled 1: Interrupt enabled
0	CMP0IE	rw	0	Compare 0 interrupt enable. Enables interrupt when CMP0IF flag is set. 0: Interrupt disabled 1: Interrupt enabled

15.8.2 TIMxSR: Timer/Counter status register

Offset 0x04 **Reset** 0x00 **Width** 8 **Address** 0x4604 (TIMER0), 0x4704 (TIMER1)
Timer/Counter status register. Contains current compare output levels and interrupt flags. The register is latched on the falling edge of the memory enable signal for stable reads.

Bits	Field	Type	Reset	Description
7	CMP1OUT	r	0	Current value of the Compare 1 output pin TxCMP1. Reflects the actual PWM output level. 0: TxCMP1 output LOW 1: TxCMP1 output HIGH
6	CMP0OUT	r	0	Current value of the Compare 0 output pin TxCMP0. Reflects the actual PWM output level. 0: TxCMP0 output LOW 1: TxCMP0 output HIGH
5	CAP1IF	rw1	0	Capture 1 interrupt flag. Set when TxCAP1 pin triggers a capture event. Write 1 to clear. 0: No pending capture 1 interrupt 1: Capture 1 interrupt pending
4	CAP0IF	rw1	0	Capture 0 interrupt flag. Set when TxCAP0 pin triggers a capture event. Write 1 to clear. 0: No pending capture 0 interrupt 1: Capture 0 interrupt pending
3	OVIF	rw1	0	Timer/Counter overflow interrupt flag. Set when timer overflows from 2^{32}-1 to 0. Write 1 to clear. 0: No pending overflow interrupt 1: Overflow interrupt pending
2	CMP2IF	rw1	0	Compare 2 interrupt flag. Set when TIMxVAL equals TIMxCMP2. Write 1 to clear. 0: No pending compare 2 interrupt 1: Compare 2 interrupt pending
1	CMP1IF	rw1	0	Compare 1 interrupt flag. Set when TIMxVAL equals TIMxCMP1. Write 1 to clear. 0: No pending compare 1 interrupt 1: Compare 1 interrupt pending

Bits	Field	Type	Reset	Description
0	CMP0IF	rw1	0	Compare 0 interrupt flag. Set when TIMxVAL equals TIMxCMP0. Write 1 to clear. 0: No pending compare 0 interrupt 1: Compare 0 interrupt pending

15.8.3 TIMxVAL: Timer/Counter value register

Offset 0x08 **Reset** 0x00000000 **Width** 32 **Address** 0x4608 (TIMER0), 0x4708 (TIMER1)
Timer/Counter value register. Reads the current timer count. Writes immediately update the timer value regardless of enable state. The value is latched on the falling edge of the memory enable signal for stable reads.

Bits	Field	Type	Reset	Description
31:0	VAL	rw	0x00000000	Current timer count value

15.8.4 TIMxCMP0: Timer/Counter Compare 0 threshold register

Offset 0x0C **Reset** 0x00000000 **Width** 32 **Address** 0x460C (TIMER0), 0x470C (TIMER1)
Timer/Counter Compare 0 threshold register. When TIMxVAL equals this value, CMP0IF is set and the TxCMP0 PWM output toggles.

Bits	Field	Type	Reset	Description
31:0	CMP0	rw	0x00000000	Compare 0 threshold value

15.8.5 TIMxCMP1: Timer/Counter Compare 1 threshold register

Offset 0x10 **Reset** 0x00000000 **Width** 32 **Address** 0x4610 (TIMER0), 0x4710 (TIMER1)
Timer/Counter Compare 1 threshold register. When TIMxVAL equals this value, CMP1IF is set and the TxCMP1 PWM output toggles.

Bits	Field	Type	Reset	Description
31:0	CMP1	rw	0x00000000	Compare 1 threshold value

15.8.6 TIMxCMP2: Timer/Counter Compare 2 threshold register

Offset 0x14 **Reset** 0x00000000 **Width** 32 **Address** 0x4614 (TIMER0), 0x4714 (TIMER1)
Timer/Counter Compare 2 threshold register. When TIMxVAL equals this value, CMP2IF is set. If CMP2RST is enabled, timer resets to 0 (PWM period control).

Bits	Field	Type	Reset	Description
31:0	CMP2	rw	0x00000000	Compare 2 threshold value (PWM period)

15.8.7 TIMxCAPn (n = 0..1): Timer/Counter Capture n value register

Offset 0x18 + 4n **Reset** 0x00000000 **Width** 32
Timer/Counter Capture n value register. Automatically latches TIMxVAL when TxCAPn pin edge triggers a capture event. The captured value is latched on the falling edge of the memory enable signal for stable reads.

Bits	Field	Type	Reset	Description
31:0	CAPn	r	0x00000000	Captured timer value from TxCAPn event

16 System control (SYSTEM)

16.1 Overview

The SYSTEM peripheral groups the chip-level control blocks: the clock tree roots and dividers, the memory block power gates, a CRC16 accumulator and the watchdog timer. Its effects are global (MCLK clocks all harts and SMCLK clocks the peripherals), so by convention only hart 0 reconfigures the clocks, after quiescing the other harts.

- Source selection and enables for MCLK and SMCLK (SYSCLKCR), dividers for both (CLKDIVCR), and bias control of the two on-chip oscillators (DCO0BIAS, DCO1BIAS)
- Power gates in BLOCKPWR for the ROM, the per-hart memory blocks and the first four shared RAM banks
- CRC16-CDMA2000 accumulator (CRCDATA, CRCSTATE)
- 24-bit watchdog with sixteen timeouts, password-protected control, optional interrupt and hardware reset (WDTCR, WDTPASS, WDTSR, WDTVAL)
- One interrupt: watchdog timeout, vector 0

Interrupt routing is not managed here: every hart enables peripheral interrupts through its IRQROUTER row, and the CLINT vectors are hardwired into every hart. The register slots of the single-core chip's vectored interrupt controller (IRQENx, IRQPRIx, IRQCR) are reserved and read as zero.

16.2 Instances and signals

Table 16.1: SYSTEM instances

Instance	Base address	Interrupt vector(s)	Pins (AF)
SYSTEM	0x4900	0	none

Table 16.2: SYSTEM interrupt vectors

Vector	Instance	Description	Clear method
0	SYSTEM	Watchdog Timer InterruptIRQB_SYS_WDT	Write 1 to WDTSR.SYSWDTIF

16.3 Functional description

16.3.1 Clock control

The clock tree, its four sources and the clock state at reset and after boot are described in the Reset, clocks and power chapter (Section 6.3). SYSCLKCR holds the root selects and the source enables: MCLKSEL and SMCLKSEL pick the source of each root, SMCLKOFF stops SMCLK, HFXTOFF and LFXTOFF stop the external

sources, and DCO0ON and DCO1ON start the on-chip oscillators. A source selected by either root stays enabled regardless of its enable bit, so the running clock cannot be stopped by mistake. CLKDIVCR divides each root after its multiplexer, and DCO0BIAS and DCO1BIAS set the oscillator frequencies. Both root multiplexers are glitch-free, so a source change while running produces no runt pulse.

Because MCLK clocks every hart and SMCLK clocks the peripheral engines, writing SYSCLKCR or CLKDIVCR changes the timing of all five harts and every serial peripheral at once. Reducing the MCLK frequency, through the divider, a slower source or a lower DCO bias, is the main lever on dynamic power.

16.3.2 Memory block power

BLOCKPWR gates the supply of the ROM (SYSROMOFF), the two per-hart memory blocks (SYSRAM0OFF, SYSRAM1OFF) and the first four shared RAM banks individually (SYSSHB0OFF to SYSSHB3OFF); banks beyond the fourth are permanently powered. A gated block reduces leakage and loses its contents, because the gate disconnects the array's supply rail and there is no retention; an access to a gated block returns undefined data. All blocks power up on at reset, and the contents of a block that has been gated are undefined until written.

16.3.3 Hart sleep

Each hart can stop its own clock with the `sleep` instruction, which reduces that hart's dynamic power to zero while the peripherals and the other harts continue. A sleeping hart is woken only by an interrupt (a CLINT software or timer interrupt, or a peripheral interrupt routed to it) or by reset. On an interrupt the clock is re-enabled for the handler; when every pending interrupt has been serviced the hart returns to sleep, unless a handler executed the `wake` instruction, in which case the hart stays awake and continues after the `sleep`. The average power of a duty-cycled hart is its running power scaled by $t_{\text{on}}/(t_{\text{on}} + t_{\text{off}})$.

16.3.4 CRC accumulator

The CRC module computes CRC16-CDMA2000 over a byte sequence: width 16, polynomial 0xC857, initial value 0xFFFF, no input or output reflection, no output XOR. CRCSTATE holds the running value and CRCDATA accepts one byte per write. CRCSTATE is a true seed register: the value written to it is what the next CRCDATA byte folds against, so a partial state read out mid-message can be written back later and the chain resumes, and a protocol whose initial value is not 0xFFFF is computed by seeding that value instead. Reset behaves as a 0xFFFF seed write, so the first byte after reset needs no seed at all.

16.3.5 Watchdog

The watchdog is a 24-bit counter that increments on every MCLK cycle while SYSWDTEN is set. SYSWDTCDIV selects which counter bit, 16 to 31, raises the event on its 0-to-1 transition, so the timeout is $2^{\text{SYSWDTCDIV}+16}$ MCLK cycles: at 24 MHz from about 2.73 ms to about 89.5 s. On the event, SYSWDTIF is set; if SYSWDTIE is set and vector 0 is routed to some hart, the interrupt is delivered through the IRQROUTER and, if SYSWDTHWRST is also set, the chip resets after that hart completes the claim. If the interrupt is disabled or unrouted, the chip resets immediately on the event when SYSWDTHWRST is set.

WDTCR is write-protected: a write of the unlock password 0x5F3759DF to WDTPASS opens a 64-MCLK-cycle window in which WDTCR accepts a write, so that a runaway program cannot disable the watchdog by chance. A write of the clear password 0xA0C8A620 to WDTPASS resets the counter to zero. SYSWDTRF in WDTSR records that the last reset was caused by the watchdog and survives reset. WDTVAL reads the counter, or zero while the watchdog is disabled.

16.4 Interrupts and events

Table 16.3 lists the one vector.

Vector	Condition	Set by	Cleared by
0	Watchdog timeout	SYSWDTIF	Writing 1 to SYSWDTIF

Table 16.3: SYSTEM interrupt vector.

16.5 Programming

16.5.1 Selecting the clock sources

1. Quiesce the other harts, because the change reaches every hart.
2. Set SYSCLKCR.DCO0ON or SYSCLKCR.DCO1ON if a DCO is to be used, and write its bias register.
3. Write CLKDIVCR.SYSMCLKDIV and CLKDIVCR.SYSSMCLKDIV.
4. Write SYSCLKCR.SMCLKSEL.
5. Write SYSCLKCR.MCLKSEL.
6. Set SYSCLKCR.HFXTOFF or SYSCLKCR.LFXTOFF for any source no longer needed.

16.5.2 Computing a CRC

1. Write 0xFFFF to CRCSTATE, or the protocol's initial value, or a partial state saved earlier.
2. Write each byte, in order, to CRCDATA.
3. Read the result from CRCSTATE.

16.5.3 Starting the watchdog

1. Route vector 0 to the servicing hart in its IRQROUTER row if the interrupt is wanted.
2. Write 0x5F3759DF to WDTPASS.
3. Write WDTCR with SYSWDTCDIV, SYSWDTIE, SYSWDTHWRST and SYSWDTEN set in the same write, within 64 MCLK cycles of the unlock.
4. Write 0xA0C8A620 to WDTPASS periodically, within the timeout.

16.5.4 Putting a hart to sleep

1. Set a valid stack pointer, because interrupt entry stores to it.
2. Enable the wake source's vector in the hart's interrupt controller.
3. Execute sleep.
4. In the handler, execute wake before returning if the hart is to stay awake.

16.6 Usage notes

Reconfigure the clocks from hart 0 only, with the other harts quiesced. SYSCLKCR and CLKDIVCR change the clock of every hart and every peripheral engine at once.

Keep HFXT running across reset. Both roots select HFXT at reset; a board that gates the oscillator from a chip-driven pin does not boot.

Select the SMCLK source before using an SMCLK-domain peripheral. The boot ROM may leave SMCLK on LFXT (32.768 kHz).

Keep the bank in use powered. A gated shared RAM bank or memory block loses its contents and returns undefined data; track which banks hold live stack or payload data before writing BLOCKPWR.

Write WDTCR in one access within the unlock window. The window closes 64 MCLK cycles after the unlock password; a read-modify-write that straddles it is discarded.

Clear SYSWDTIF with a 1 in the handler. The interrupt is a level; with SYSWDTHWRST set the chip resets after the claim is completed regardless.

16.7 Registers

Register offsets in this chapter are relative to the instance base address in Table 16.1 (SYSTEM 0x4900); the generated header names each base SYSTEM_BASE.

Table 16.4: SYSTEM register summary

Offset	Name	Access	Reset	Description
0x00	SYSCLKCR	rw	0x0000	System clock control register
0x04	CLKDIVCR	rw	0x00	MCLK and SMCLK clock divider control register
0x08	BLOCKPWR	rw	0x00	Block power control register
0x0C	CRCDATA	rw	0x00	CRC input data register
0x10	CRCSTATE	rw	0x0000	CRC state register
0x14 to 0x2F	-	-	-	Reserved
0x30	WDTPASS	w	0x00000000	Watchdog timer password register
0x34	WDTCR	rw	0x00	Watchdog timer control register
0x38	WDTSR	rw1	0x00	Watchdog timer status register
0x3C	WDTVAL	r	0x00000000	Watchdog timer value register
0x40 + 4n	DCOnBIAS (n = 0..1)	rw	0x0800	Digitally controlled oscillator n bias register

16.7.1 SYSCLKCR: System clock control register

Offset 0x00 **Reset** 0x0000 **Width** 16 **Address** 0x4900

Bits	Field	Type	Reset	Description
15:9	-	-	-	Reserved. Reads zero; write zero.
8	DCO1ON	rw	0	Digitally controlled oscillator 1 (DCO1) power enable. Set to power on DCO1. DCO1 is automatically kept on if it is currently selected as the source for MCLK or SMCLK, regardless of this bit. 0: DCO1 powered off 1: DCO1 powered on
7	DCO0ON	rw	0	Digitally controlled oscillator 0 (DCO0) power enable. Set to power on DCO0. DCO0 is automatically kept on if it is currently selected as the source for MCLK or SMCLK, regardless of this bit. 0: DCO0 powered off 1: DCO0 powered on
6	HFXTOFF	rw	0	High frequency external crystal clock disable. Cannot be disabled if it is currently selected as the source for MCLK or SMCLK. 0: HFXT enabled 1: HFXT disabled
5	LFXTOFF	rw	0	Low frequency external crystal clock disable. Cannot be disabled if it is currently selected as the source for SMCLK. 0: LFXT enabled 1: LFXT disabled
4	SMCLKOFF	rw	0	Submain clock disable. Globally and unconditionally disables SMCLK, gating all peripherals clocked from SMCLK. 0: SMCLK enabled 1: SMCLK disabled
3:2	SMCLKSEL	rw	0b00	Submain clock source select 0b00 SMCLKSEL_HFXT: High Frequency Crystal Clock 0b01 SMCLKSEL_LFXT: Low Frequency Crystal Clock 0b10 SMCLKSEL_DCO0: Digitally Controlled Oscillator 0 0b11 SMCLKSEL_DCO1: Digitally Controlled Oscillator 1
1:0	MCLKSEL	rw	0b00	Main clock source select (also CPU clock source select) 0b00 MCLKSEL_HFXT: High Frequency Crystal Clock 0b01 MCLKSEL_SMCLK: Submain Clock 0b10 MCLKSEL_DCO0: Digitally Controlled Oscillator 0 0b11 MCLKSEL_DCO1: Digitally Controlled Oscillator 1

16.7.2 CLKDIVCR: MCLK and SMCLK clock divider control register

Offset 0x04 **Reset** 0x00 **Width** 8 **Address** 0x4904

MCLK and SMCLK clock divider control register. Configures clock division for main and submain clocks using glitch-free multiplexers.

Bits	Field	Type	Reset	Description
7:6	-	-	-	Reserved. Reads zero; write zero.

Bits	Field	Type	Reset	Description
5:3	SMCLKDIV	rw	0b000	SMCLK clock division selection. Division is applied after clock source selection through glitch-free divider multiplexer. 0b000 SYSSMCLKDIV_1: /1 (no division) 0b001 SYSSMCLKDIV_2: /2 0b010 SYSSMCLKDIV_4: /4 0b011 SYSSMCLKDIV_8: /8 0b100 SYSSMCLKDIV_16: /16 0b101 SYSSMCLKDIV_32: /32 0b110 SYSSMCLKDIV_64: /64 0b111 SYSSMCLKDIV_128: /128
2:0	MCLKDIV	rw	0b000	MCLK clock division selection. Division is applied after clock source selection through glitch-free divider multiplexer. 0b000 SYSMCLKDIV_1: /1 (no division) 0b001 SYSMCLKDIV_2: /2 0b010 SYSMCLKDIV_4: /4 0b011 SYSMCLKDIV_8: /8 0b100 SYSMCLKDIV_16: /16 0b101 SYSMCLKDIV_32: /32 0b110 SYSMCLKDIV_64: /64 0b111 SYSMCLKDIV_128: /128

16.7.3 BLOCKPWR: Block power control register

Offset 0x08 **Reset** 0x00 **Width** 8 **Address** 0x4908

Block power control register. Controls power gating for the on-chip memory blocks; all bits reset to 0 (every block powered). Bits 6:3 gate the four low shared bulk-RAM banks individually: a gated bank loses its contents and stops responding, so software must keep the bank holding its stack, mailboxes or live data powered and must treat a re-powered bank as uninitialized. In configurations with more than four banks, banks 4 and up are always on.

Bits	Field	Type	Reset	Description
7	-	-	-	Reserved. Reads zero; write zero.
6	SHB3OFF	rw	0	Shared bulk-RAM bank 3 (0x1C000-0x1FFFF) power control. When set, the bank is powered off: contents are LOST and accesses no longer respond. Reduces static power (a gated sram1p16k halves its leakage). 0: Bank 3 powered on 1: Bank 3 powered off (contents lost)
5	SHB2OFF	rw	0	Shared bulk-RAM bank 2 (0x18000-0x1BFFF) power control. When set, the bank is powered off: contents are LOST and accesses no longer respond. Reduces static power (a gated sram1p16k halves its leakage). 0: Bank 2 powered on 1: Bank 2 powered off (contents lost)

Bits	Field	Type	Reset	Description
4	SHB1OFF	rw	0	Shared bulk-RAM bank 1 (0x14000-0x17FFF) power control. When set, the bank is powered off: contents are LOST and accesses no longer respond. Reduces static power (a gated sram1p16k halves its leakage). 0: Bank 1 powered on 1: Bank 1 powered off (contents lost)
3	SHB0OFF	rw	0	Shared bulk-RAM bank 0 (0x10000-0x13FFF) power control. When set, the bank is powered off: contents are LOST and accesses no longer respond. Reduces static power (a gated sram1p16k halves its leakage). 0: Bank 0 powered on 1: Bank 0 powered off (contents lost)
2	RAM1OFF	rw	0	RAM block 1 power control. When set, the block is powered off: its contents are lost and accesses to it no longer respond. 0: RAM block 1 powered on 1: RAM block 1 powered off
1	RAM0OFF	rw	0	RAM block 0 power control. When set, the block is powered off: its contents are lost and accesses to it no longer respond. 0: RAM block 0 powered on 1: RAM block 0 powered off
0	ROMOFF	rw	0	ROM power control. When set, the boot ROM is powered off and no longer responds to reads. 0: ROM powered on 1: ROM powered off

16.7.4 CRCDATA: CRC input data register

Offset 0x0C **Reset** 0x00 **Width** 8 **Address** 0x490C

CRC input data register. Write the next byte of data to this register to update the CRC calculation. Uses CRC16-CDMA2000 polynomial 0xC857.

Bits	Field	Type	Reset	Description
7:0	CRCDATA	rw	0x00	CRC data input byte. Writing to this register feeds the byte into the CRC calculation and updates CRCSTATE.

16.7.5 CRCSTATE: CRC state register

Offset 0x10 **Reset** 0x0000 **Width** 16 **Address** 0x4910

CRC state register. Contains the current CRC16 calculation result. Write to this register to initialize or restart the CRC calculation. Read to obtain the computed CRC16 checksum.

Bits	Field	Type	Reset	Description
15:0	CRCSTATE	rw	0x0000	CRC state value. Initialize to 0xFFFF before starting CRC calculation. Read after processing all data bytes to get final CRC16 checksum.

16.7.6 WDTPASS: Watchdog timer password register

Offset 0x30 **Reset** 0x00000000 **Width** 32 **Address** 0x4930
Watchdog timer password register. Write-only register for two security functions: (1) Write 0x5F3759DF to unlock WDTCR for 64 MCLK cycles, enabling writes to watchdog configuration. (2) Write 0xA0C8A620 to clear watchdog counter to 0, preventing timeout. Reading always returns 0.

Bits	Field	Type	Reset	Description
31:0	WDTPASS	w	0x00000000	Watchdog password. Write 0x5F3759DF (unlock password) to enable WDTCR writes for 64 MCLK cycles. Write 0xA0C8A620 (clear password) to reset watchdog counter to 0.

16.7.7 WDTCR: Watchdog timer control register

Offset 0x34 **Reset** 0x00 **Width** 8 **Address** 0x4934
Watchdog timer control register. This register is protected and requires password unlock via WDTPASS before writing. Configures watchdog operation mode and timeout period.

Bits	Field	Type	Reset	Description
7	WDTEN	rw	0	Watchdog enable. When set, the 24-bit watchdog counter counts MCLK cycles and a timeout occurs when the counter bit selected by SYSWDTCDIV rises. WDTCR is write protected: unlock it through WDTPASS first. 0: Watchdog disabled 1: Watchdog enabled
6	-	-	-	Reserved. Reads zero; write zero.

Bits	Field	Type	Reset	Description
5:2	WDTCDIV	rw	0b0000	Watchdog timeout select. The timeout event occurs when counter bit SYSWDTCDIV + 16 rises, so the period is 2^(SYSWDTCDIV + 16) MCLK cycles. 0b0000 SYSWDTCDIV_65536: Bit 16: 65,536 MCLK cycles 0b0001 SYSWDTCDIV_131072: Bit 17: 131,072 MCLK cycles 0b0010 SYSWDTCDIV_262144: Bit 18: 262,144 MCLK cycles 0b0011 SYSWDTCDIV_524288: Bit 19: 524,288 MCLK cycles 0b0100 SYSWDTCDIV_1048576: Bit 20: 1,048,576 MCLK cycles 0b0101 SYSWDTCDIV_2097152: Bit 21: 2,097,152 MCLK cycles 0b0110 SYSWDTCDIV_4194304: Bit 22: 4,194,304 MCLK cycles 0b0111 SYSWDTCDIV_8388608: Bit 23: 8,388,608 MCLK cycles 0b1000 SYSWDTCDIV_16777216: Bit 24: 16,777,216 MCLK cycles 0b1001 SYSWDTCDIV_33554432: Bit 25: 33,554,432 MCLK cycles 0b1010 SYSWDTCDIV_67108864: Bit 26: 67,108,864 MCLK cycles 0b1011 SYSWDTCDIV_134217728: Bit 27: 134,217,728 MCLK cycles 0b1100 SYSWDTCDIV_268435456: Bit 28: 268,435,456 MCLK cycles 0b1101 SYSWDTCDIV_536870912: Bit 29: 536,870,912 MCLK cycles 0b1110 SYSWDTCDIV_1073741824: Bit 30: 1,073,741,824 MCLK cycles 0b1111 SYSWDTCDIV_2147483648: Bit 31: 2,147,483,648 MCLK cycles
1	WDTIE	rw	0	Watchdog interrupt enable. When set, a timeout sets SYSWDTIF and raises the watchdog interrupt (vector 0) to the harts the IRQROUTER routes it to; with SYSWDTHWRST also set, the system reset follows the servicing hart completing the interrupt. If the vector is routed to no hart, the reset (when enabled) occurs immediately on timeout. 0: Watchdog interrupt disabled 1: Watchdog interrupt enabled
0	WDTHWRST	rw	0	Watchdog reset enable. When set, a timeout resets the system: immediately if the watchdog interrupt is disabled or routed to no hart, otherwise after the servicing hart completes the interrupt. 0: Watchdog reset disabled 1: Watchdog reset enabled

16.7.8 WDTSR: Watchdog timer status register

Offset 0x38 **Reset** 0x00 **Width** 8 **Address** 0x4938

Watchdog timer status register. Contains flags indicating watchdog reset and interrupt events. Flags are cleared by writing 1 to the respective bit.

Bits	Field	Type	Reset	Description
7:2	-	-	-	Reserved. Reads zero; write zero.

Bits	Field	Type	Reset	Description
1	WDTIF	rw1	0	Watchdog interrupt flag, set on a timeout while SYSWDTIE = 1 and held until software writes 1 to it. 0: No watchdog interrupt pending 1: Watchdog interrupt occurred
0	WDTRF	rw1	0	Watchdog reset flag, set when the last system reset was caused by the watchdog and held across resets until software writes 1 to it. 0: Reset not caused by watchdog 1: Reset caused by watchdog

16.7.9 WDTVAL: Watchdog timer value register

Offset 0x3C **Reset** 0x00000000 **Width** 32 **Address** 0x493C

Watchdog timer value register. Read-only register containing current watchdog counter value. Counter increments on MCLK when watchdog is enabled. Returns 0 when watchdog is disabled.

Bits	Field	Type	Reset	Description
31:24	-	-	-	Reserved. Reads zero; write zero.
23:0	WDTVAL	r	0x000000	Watchdog counter value. 24-bit up-counter that increments on MCLK cycles. Watchdog event occurs when bit selected by WDTCDIV transitions from 0 to 1.

16.7.10 DCOnBIAS (n = 0..1): Digitally controlled oscillator n bias register

Offset 0x40 + 4n **Reset** 0x0800 **Width** 16

Digitally controlled oscillator n bias register. Controls DCOn output frequency through bias voltage adjustment. Higher bias values generally produce higher frequencies.

Bits	Field	Type	Reset	Description
15:12	-	-	-	Reserved. Reads zero; write zero.
11:0	DCOnBIAS	rw	0x800	DCOn bias adjustment value. 12-bit bias control for DCOn frequency tuning. Default value loaded from constants on reset.

17 Neural processing unit (NPU)

17.1 Overview

The NPU is a fixed-point neural network processing unit built around one multiply-accumulate datapath and a configurable sequencer. One engine provides four datapath modes, selected per computation by NPUMODE: a fully connected layer, a one-dimensional convolution, a binary XNOR-popcount layer and a general matrix multiply. Operands and results live in the 16 KiB staging RAM at 0xC000 to 0xFFFF, which is shared between the harts and the engine's compute port.

- Four datapath modes (NPUMODE): dense layer, 1-D convolution, XNOR-popcount, GEMM
- Q0.24 inputs, Q7.24 weights and accumulator, per-step rounding and saturation
- Five activation functions on one hardware sigmoid approximator (NPUACTF, NPUAEN): sigmoid, ReLU, tanh, clamp, exponential
- Optional bias term per neuron (NPUBEN); packed input and weight formats (NPUXPK, NPUWPK)
- Three word-index pointers into the staging RAM (NPUIVSAR, NPUWVSAR, NPUOVSAR) and two per-mode configuration words (NPUCFG1, NPUCFG2)
- One interrupt, think-done, vector 120

17.2 Instances and signals

Table 17.1: NPU instances

Instance	Base address	Interrupt vector(s)	Pins (AF)
NPU	0x4A00	120	none

Table 17.2: NPU interrupt vectors

Vector	Instance	Description	Clear method
120	NPU	NPU0 think-done InterruptIRQB_NPU0_TD	Write 1 to NPUSR.NPUTHINKDONE

17.3 Block diagram

Figure 17.1 draws the whole arrangement: the register file and the staging RAM as the two paths a hart reaches the NPU on, the port multiplexer that decides which side drives the RAM's single port, and the engine behind it. The three pointer registers are word indices into the one staging array, so the input, weight and output regions in the figure are roles rather than fixed addresses. The mode and activation codes the engine implements are the value tables of NPUMODE and NPUACTF in NPUCR (Section 17.8.1); they are not repeated here.

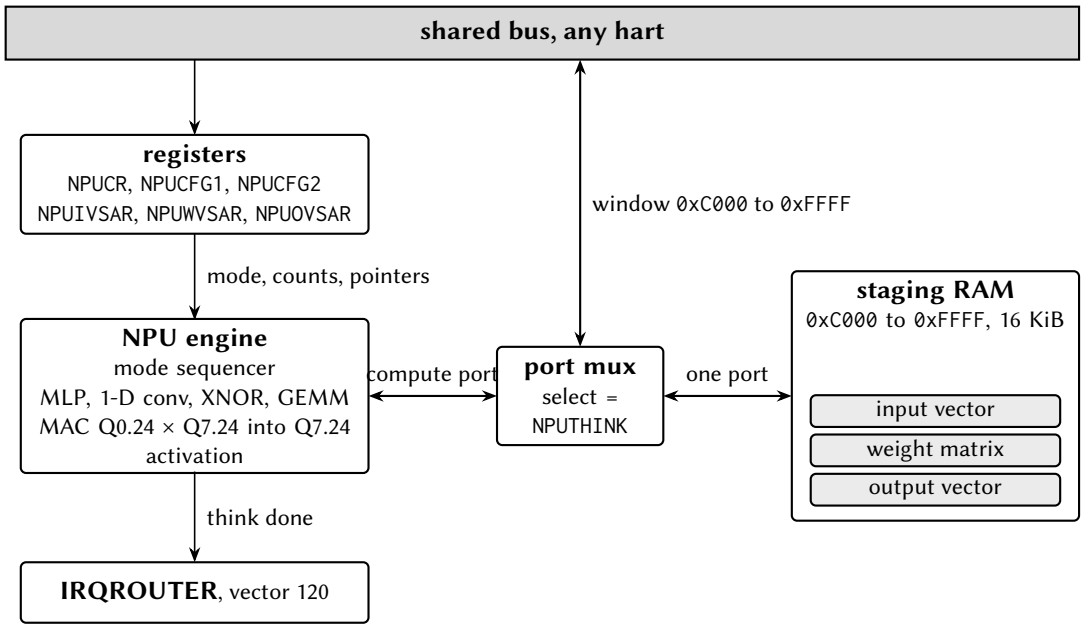


Figure 17.1: The NPU datapath and the staging RAM it borrows.

17.4 Functional description

17.4.1 Staging RAM ownership

The staging RAM is one single-port SRAM macro. While NPUTHINK is set the port belongs to the engine; the multiplexer select is NPUTHINK re-clocked on the free-running chip clock, so the port changes hands one cycle after a hart's start write lands and one cycle after the engine's completion pulse, and a hart's in-flight access is never cut in half. The hardware does not stop a hart from reaching into the window while the engine owns it: the access completes at the arbiter in the ordinary time and returns or overwrites whatever the RAM is holding for the engine. The rule that no hart touches 0xC000 to 0xFFFF while NPUTHINK reads 1 is therefore software's, and the poll on NPUTHINK is the whole of it.

The pointers NPUIVSAR, NPUWVSAR and NPUOVSAR are word indices into that array (byte offset from 0xC000 divided by 4), not addresses of separate memories. The sequencer trusts its configuration and bounds-checks nothing: a working set larger than the 4096 words, or count fields inconsistent with the staged data, wraps the 12-bit word index and corrupts neighbouring regions without the engine ever hanging.

17.4.2 Data formats

Inputs (dense and convolution inputs, GEMM A elements) are signed Q0.24 in bits 24:0 of a word, representing $[-1, 1)$; bits 31:25 are ignored. Weights (and GEMM B elements) are signed Q7.24, representing $[-128, 128)$. Outputs are Q7.24 unless an activation narrows them. The accumulator is Q7.24 with round-to-nearest and saturation at ± 128 applied at every multiply-accumulate step in the fixed sequencer order, so a long dot product with large coefficients can clip before its end. XNOR mode packs 1-bit operands 32 per word and emits ± 1.0 Q7.24 words.

17.4.3 Packed operand formats

Two bits in NPUCR halve the staging footprint of a layer at the cost of resolution. With NPUXPK set, the input vector holds two 16-bit Q0.15 elements per word, element $2i$ in bits 15:0 and element $2i + 1$ in bits 31:16, so element i is read from word NPUIVSAR + $i/2$; the range stays $[-1, 1)$ and firmware packs by arithmetic-shifting each Q0.24 input right by 9. The output vector is never packed, because outputs are full-width accumulator or activation words, so a layer whose outputs feed the next layer's inputs either leaves NPUXPK clear or repacks between computations.

With NPUWPK set, the weight matrix holds two 16-bit Q3.12 weights per word in the same element order as the unpacked layout; only the storage changes, and the element walk of every mode, including the convolution filter-block reload and the GEMM per-row reload, is unchanged. A maximal 256 by 256 layer then needs 8 re-stagings of the 4096-word staging RAM instead of 16. Q3.12 narrows the weight range from $[-128, 128)$ to $[-8, 8)$ and coarsens the resolution from 2^{-24} to 2^{-12}: firmware packs by arithmetic-shifting each Q7.24 weight right by 12 and must saturate first, because a weight outside $[-8, 8)$ has no packed representation and the hardware only ever sees the packed half. The bias weight rides the same stream under the same range. Both bits are ignored in XNOR mode, which already packs 32 one-bit operands per word, and both are latched when NPUTHINK is set, so a write during a run takes effect at the next run.

17.4.4 Datapath modes

NPUNI and NPUNN hold a count minus one and are reinterpreted by each mode, as are NPUCFG1 and NPUCFG2.

- **Mode 0, dense layer** (reset default): NPUNI + 1 inputs at NPUIVSAR, NPUNN + 1 neurons. The weight matrix at NPUWVSAR holds one contiguous row per neuron; with NPUBEN set each row begins with a bias weight multiplied by an implicit input of 1.0. NPUCFG1 and NPUCFG2 are unused.
- **Mode 1, 1-D convolution**: NPUNI = taps −1, NPUNN = filters −1. NPUCFG1 packs stride, dilation, input length and channel count; NPUCFG2 holds the output length, which the sequencer trusts, so the padding convention is a firmware decision. Inputs are channel-major, weights filter-major (bias first per filter with NPUBEN), outputs filter-major.
- **Mode 2, XNOR-popcount**: activations and weights packed 32 per word LSB first. NPUNI = packed words per neuron −1; the exact bit count $K \leq 4096$ is in NPUCFG2 bits 12:0 and the unused tail bits of the last word are masked. A neuron emits +1.0 when $2 \cdot \text{popcount}(\text{XNOR}(a, w)) - K$ is at least the signed threshold in NPUCFG1, else −1.0. NPUBEN and NPUAEN must be 0.
- **Mode 3, GEMM**: $C = A \times B$ with A ($M \times K$, row-major, Q0.24) at NPUIVSAR, B ($K \times N$, column-major, Q7.24) at NPUWVSAR, C ($M \times N$, row-major) at NPUOVSAR; $K - 1$ in NPUNI, $N - 1$ in NPUNN, $M - 1$ in NPUCFG1 bits 7:0. There is no accumulation across runs, so K must fit one run; larger M or N tile by re-pointing the registers. Because C is row-major, a result chains directly as the next layer's A. NPUBEN must be 0.

Mode codes 4 to 7 run as mode 0. The mode, counts and activation selection are latched into run shadows when NPUTHINK is set, so a write during a run takes effect at the next run.

17.4.5 Activation functions

With NPUAEN set, each output passes through the function selected by NPUACTF before it is written; with NPUAEN clear the raw Q7.24 accumulator is written regardless. All five are shift-and-add wrappers around one piecewise-quadratic sigmoid approximator: 0 = sigmoid (Q0.24 in $[0, 1)$, saturating for $|x| \geq 4$); 1 =

ReLU (Q7.24; an output at or above 1.0 is not a valid Q0.24 next-layer input); 2 = tanh as $2\sigma(2x) - 1$ (in $[-1, 1)$, saturating for $|x| \geq 2$, a valid next-layer input); 3 = clamp to $[-1, 1)$ (always chains); 4 = exponential as $2\sigma(x)$ (Q1.24 in $[0, 2)$, the per-element scores of a softmax whose sum and normalisation firmware performs; subtracting the maximum logit first maps it to exactly 1.0). Codes 5 to 7 read as sigmoid. XNOR mode ignores the activation path.

17.4.6 Run time

Run time scales with the mode's loop product: a GEMM costs on the order of $5 \cdot M \cdot N \cdot K$ cycles and a convolution $5 \cdot C_{out} \cdot L_{out} \cdot C_{in} \cdot K$, so a full-scope convolution can take hundreds of milliseconds at 24 MHz. XNOR mode is about an order of magnitude cheaper per output, because one fetched word pair carries 32 synapses.

17.5 Interrupts and events

The completion pulse that clears NPUTHINK also sets the sticky flag NPUTHINKDONE in NPUSR; the interrupt is the level of that flag gated by NPUTDIE, on the free-running clock. NPUTDIE resets to 0, so polling software is unaffected. Table 17.3 lists the vector.

Vector	Condition	Set by	Cleared by
120	Computation complete	NPUTHINKDONE	Writing 1 to NPUTHINKDONE

Table 17.3: NPU interrupt vector.

17.6 Programming

17.6.1 Running one computation

1. Poll NPUCR.NPUTHINK until it reads 0.
2. Stage the operands in the staging RAM in the selected mode's layout.
3. Write the input word index to NPUIVSAR.
4. Write the weight word index to NPUWVSAR.
5. Write the output word index to NPUOVSAR.
6. Write NPUCFG1 and NPUCFG2 if the mode uses them.
7. Write NPUCR with NPUMODE, NPUNI, NPUNN, NPUBEN, NPUAEN, NPUACTF and, for an interrupt, NPUTDIE.
8. Set NPUCR.NPUTHINK.
9. Poll NPUCR.NPUTHINK until it reads 0, or take vector 120 and write 1 to NPUSR.NPUTHINKDONE in the handler.
10. Read the outputs from the staging RAM at NPUOVSAR.

17.7 Usage notes

Do not touch the staging RAM while NPUTHINK reads 1. The hardware neither blocks nor stalls the access; it returns or corrupts the engine's data.

Check NPUTHINK from every hart, not only the one that took the interrupt. The interrupt tells one hart the run has finished; any other hart consults the flag before reading results.

Size the working set yourself. Nothing in the sequencer checks bounds; an oversized working set wraps the 12-bit index and corrupts its neighbours silently.

Saturate weights to $[-8, 8)$ before packing them. A Q7.24 weight outside that range has no Q3.12 representation; shifting without saturating wraps it.

Chain layers through Q0.24-valid activations. ReLU outputs at or above 1.0 are not valid inputs; use tanh, clamp or normalised weights when a result feeds the next layer.

Clear NPUTHINKDONE with a 1 before completing the interrupt. Reading NPUSR does not clear it, and the level re-enters the handler otherwise.

17.8 Registers

Register offsets in this chapter are relative to the instance base address in Table 17.1 (NPU 0x4A00); the generated header names each base NPU_BASE.

Table 17.4: NPU register summary

Offset	Name	Access	Reset	Description
0x00	NPUCR	rw	0x00000000	NPU control register
0x04	NPUIVSAR	rw	0x00000000	
0x08	NPUWVSAR	rw	0x00000000	
0x0C	NPUOVSAR	rw	0x00000000	
0x10	NPUSR	rw1	0x00000000	NPU status register
0x14	NPUCFG1	rw	0x00000000	NPU mode configuration word 1
0x18	NPUCFG2	rw	0x00000000	NPU mode configuration word 2

17.8.1 NPUCR: NPU control register

Offset 0x00 **Reset** 0x00000000 **Width** 32 **Address** 0x4A00

Bits	Field	Type	Reset	Description
31:28	-	-	-	Reserved. Reads zero; write zero.

Bits	Field	Type	Reset	Description
27	XPK	rw	0	Packed input format enable: when set, the input vector holds two 16-bit Q0.15 elements per staging word (element 2i in bits 15:0, element 2i+1 in bits 31:16), so element i is read from word NPUIVSAR + i/2; when clear each input occupies its own 32-bit word as Q0.24. The setting is latched when NPUTHINK is set and is ignored in XNOR-popcount mode. See the packed operand formats in the functional description. 0: One input element per 32-bit word (reset default) 1: Two 16-bit Q0.15 input elements per 32-bit word
26	WPK	rw	0	Packed weight format enable: when set, the weight matrix holds two 16-bit Q3.12 weights per staging word (element 2i in bits 15:0, element 2i+1 in bits 31:16) in the same element order as the unpacked layout; when clear each weight occupies its own 32-bit word as Q7.24. The setting is latched when NPUTHINK is set and is ignored in XNOR-popcount mode. See the packed operand formats in the functional description for the range and saturation rules. 0: One weight per 32-bit word (reset default) 1: Two 16-bit Q3.12 weights per 32-bit word
25:23	ACTF	rw	0b000	Activation function applied to the accumulator output while NPUAEN = 1; with NPUAEN = 0 the raw accumulator passes through regardless of this field. The selection is latched when NPUTHINK is set; see the activation functions in the functional description for the exact formulas and output formats. 0b000: logistic sigmoid approximation (reset default), output Q0.24 in [0, 1) 0b001: ReLU, output Q7.24 0b010: tanh approximation, output in [-1, 1) 0b011: clamp to [-1, 1) 0b100: exponential approximation 2*sigmoid(x), output Q1.24 in [0, 2) 0b101 to 0b111: reserved, and behave as 0
22:20	MODE	rw	0b000	Datapath mode select. The mode is latched when NPUTHINK is set; see the datapath modes in the functional description for the operand layout of each mode. 0b000: multilayer-perceptron mode (reset default) 0b001: one-dimensional convolution mode 0b010: XNOR-popcount binary mode (NPUBEN and NPUAEN must be 0) 0b011: general matrix-multiply (GEMM) mode (NPUBEN must be 0) 0b100 to 0b111: reserved, and behave as mode 0
19	TDIE	rw	0	Think-done interrupt enable. When set, NPUSR.NPUTHINKDONE drives the NPU think-done interrupt (vector 120); when clear the flag is available for polling only. 0: Interrupt disabled (polling only) 1: Interrupt enabled

Bits	Field	Type	Reset	Description
18	BEN	rw	0	Bias enable. When set, the first weight of each output neuron's row in the weight matrix is used as a bias term: it is multiplied by an implicit input of 1.0 and accumulated before the synaptic weights. 0: Disabled 1: Enabled
17	AEN	rw	0	Activation function enable. When set, the logistic sigmoid approximation activation function is applied to the accumulator output. When cleared, the raw accumulator output is used (linear/identity). 0: Disabled (linear output) 1: Enabled (logistic sigmoid approximation)
16	THINK	rw	0	Computation start and busy flag. Writing 1 starts the NPU; the bit reads 1 until the computation completes, when hardware clears it. While it reads 1 the staging RAM belongs to the NPU. 0: Idle (computation complete or not started) 1: Running (write 1 to start)
15:8	NI	rw	0x00	Number of inputs in the input vector minus 1. The actual number of inputs is NPUNI + 1.
7:0	NN	rw	0x00	Number of output neurons minus 1. The actual number of outputs is NPUNN + 1.

17.8.2 NPUIVSAR

Offset 0x04 **Reset** 0x00000000 **Width** 32 **Address** 0x4A04

Input vector start word index within the shared NPU staging RAM: the byte offset from the start of the staging RAM (0xC000) divided by 4. For example, an input vector at byte address 0xC100 has word index 0x40. Each input is a signed Q0.24 value stored in bits 24:0 of its 32-bit SRAM word; bits 31:25 are ignored. Bit 24 is the sign bit. The input at index 0 is at word index NPUIVSAR. The input at index 1 is at word index NPUIVSAR + 1. The rest of the inputs follow in consecutive words. This one-input-per-word layout is what NPUCR.NPUXPK = 0 (the reset default) selects; with NPUXPK = 1 the vector is packed two 16-bit Q0.15 inputs per word and input index i is read from word index NPUIVSAR + i/2, low half first.

Bits	Field	Type	Reset	Description
31:12	-	-	-	Reserved. Reads zero; write zero.
11:0	IVSAR	rw	0x000	Input vector start word index within the NPU staging RAM (byte offset from 0xC000 divided by 4)

17.8.3 NPUWVSAR

Offset 0x08 **Reset** 0x00000000 **Width** 32 **Address** 0x4A08

Synaptic weight matrix start word index within the shared NPU staging RAM: the byte offset from the start of the staging RAM (0xC000) divided by 4. Each weight is a signed Q7.24 value occupying all 32 bits of its SRAM word; bit 31 is the sign bit. Weights are stored row-major, one per 32-bit word, in the following order: for each output neuron (0 through NPUNN), if bias is enabled (NPUBEN = 1), the first word in the row is the bias weight (multiplied by an implicit input of 1.0), followed by NPUNI + 1 synaptic weights for inputs 0 through NPUNI. If bias is disabled, each row contains NPUNI + 1 synaptic weights only. This one-weight-per-word layout is what NPUCR.NPUWPK = 0 (the reset default) selects; with NPUWPK = 1

the same weight sequence is packed two 16-bit Q3.12 weights per word, low half first, halving the word count of the block.

Bits	Field	Type	Reset	Description
31:12	-	-	-	Reserved. Reads zero; write zero.
11:0	WVSAR	rw	0x000	Synaptic weight matrix start word index within the NPU staging RAM (byte offset from 0xC000 divided by 4)

17.8.4 NPUOVSAR

Offset 0x0C **Reset** 0x00000000 **Width** 32 **Address** 0x4A0C

Output vector start word index within the shared NPU staging RAM: the byte offset from the start of the staging RAM (0xC000) divided by 4. Each output is a signed Q7.24 value occupying all 32 bits of its SRAM word; bit 31 is the sign bit. The output at index 0 is written to word index NPUOVSAR. The output at index 1 is at word index NPUOVSAR + 1, and so on.

Bits	Field	Type	Reset	Description
31:12	-	-	-	Reserved. Reads zero; write zero.
11:0	OVSAR	rw	0x000	Output vector start word index within the NPU staging RAM (byte offset from 0xC000 divided by 4)

17.8.5 NPUSR: NPU status register

Offset 0x10 **Reset** 0x00000000 **Width** 32 **Address** 0x4A10

NPU status register. NPUTHINKDONE is write-1-to-clear and is never cleared by a read; the think-done interrupt (vector 120) is NPUTHINKDONE and NPUCR.NPUTDIE. Clearing NPUTHINKDONE does not affect NPUCR.NPUTHINK, and the staging RAM stays owned by the NPU until NPUCR.NPUTHINK reads 0.

Bits	Field	Type	Reset	Description
31:1	-	-	-	Reserved. Reads zero; write zero.
0	THINKDONE	rw1	0	Think-done flag, set by hardware when a computation finishes (the same event that clears NPUTHINK) and held until software writes 1 to it. It drives the think-done interrupt (vector 120) while NPUTDIE = 1. A completion that coincides with a write of 1 leaves the flag set. 0: No completed computation pending 1: A computation has completed

17.8.6 NPUCFG1: NPU mode configuration word 1

Offset 0x14 **Reset** 0x00000000 **Width** 32 **Address** 0x4A14

NPU mode configuration word 1. The interpretation depends on NPUCR.NPUMODE. Multilayer-perceptron mode (0): unused, no effect. One-dimensional convolution mode (1): bits 3:0 = stride S (1-15), bits 7:4 = dilation D (1-15), bits 23:8 = input length L per channel in samples, bits 31:24 = number of input channels minus 1 (the actual channel count Cin is this field + 1). XNOR-popcount mode (2): the full 32-bit signed firing threshold THRESH. A neuron fires (output +1.0) when 2*popcount(XNOR(activations, weights)) - K is greater than or equal to THRESH, else outputs -1.0. GEMM mode (3): bits 7:0 = M - 1, the number of A rows minus 1 (the actual row count M is this field + 1); bits 31:8 are ignored in this mode.

Bits	Field	Type	Reset	Description
31:0	CFG1	rw	0x00000000	Per-mode configuration word 1 (interpretation selected by NPUCR.NPUMODE)

17.8.7 NPUCFG2: NPU mode configuration word 2

Offset 0x18 **Reset** 0x00000000 **Width** 32 **Address** 0x4A18

NPU mode configuration word 2. The interpretation depends on NPUCR.NPUMODE. Multilayer-perceptron mode (0): unused, no effect. One-dimensional convolution mode (1): bits 15:0 = output length Lout per filter in samples. Lout is computed by the host (the sequencer trusts this value; the valid/same padding convention is a firmware decision) and every referenced input sample index j*S + k*D must be less than L. XNOR-popcount mode (2): bits 12:0 = K, the exact input bit count (1-4096); NPUNI must hold ceil(K/32) - 1, the packed words per neuron, and when K mod 32 is nonzero the unused high bits of each last packed word are masked out of the popcount by hardware. GEMM mode (3): unused, no effect. Bits 31:16 are reserved and read 0.

Bits	Field	Type	Reset	Description
31:16	-	-	-	Reserved. Reads zero; write zero.
15:0	CFG2	rw	0x0000	Per-mode configuration word 2 (interpretation selected by NPUCR.NPUMODE)

18 Power control (PWRCTRL)

18.1 Overview

The power controller (PWRCTRL) switches the power domains of the channel tiles, harts 1 to 4, and in configurations wired for field power it also owns the boot gate that holds every hart in reset until the harvested supply is good. The power domains themselves, and what cold gating means for software, are described in the Reset, clocks and power chapter (Section 6.4).

- One gate bit per channel tile in PWRCR (mask 0x1E); hart 0 has no gate bit
- A per-tile hardware sequencer that orders isolation, reset and the header switches in both directions
- Read-only sequencer state per tile in PWRSR, one nibble per hart
- Cold gating: a gated tile loses all state and cold-boots through the shared ROM on wake
- Boot gate configuration in PWRWAKE and live status in PWRSTS
- An event-fabric wake mask in TASKWKM: a task-wake pulse clears the PWRCR gate bits it selects
- All domains power up on at reset; the controller is inert until written

18.2 Instances and signals

Table 18.1: PWRCTRL instances

Instance	Base address	Interrupt vector(s)	Pins (AF)
PWRCTRL	0x4B00	none	none

18.3 Functional description

18.3.1 Gate sequencer

Setting bit h of PWRCR starts tile h's power-down sequence: the isolation clamps assert on the always-on side of the boundary, the tile is held in reset, and only then do the header switches open, so that no floating signal ever reaches the shared fabric. Clearing the bit runs the sequence in the other order, with a rail-settle delay sized for the tile's header-switch chain before the reset is released. The sequencer never aborts a transition: a gate-bit change made mid-sequence takes effect when the current sequence completes. Bit 0 (PWRH0) is reserved, reads 0 and ignores writes, because hart 0 is always on; a write of 0x1E therefore gates every channel tile and leaves hart 0 running. The write is qualified by byte lane 0 only, so full-word stores are used.

PWRSR reports the sequencer state of each tile as a 4-bit code in nibble h (PWRST0 to PWRST4); 0 is on and 3 is off, with the intermediate codes marking the sequence in progress. Software polls the nibble after a write to PWRCR, because the transition takes several cycles and the tile must not be relaunched before its nibble reads 0.

18.3.2 Task wake

TASKWKM selects which tiles the event fabric may wake. A EVFAB task-wake pulse clears the PWRCR.PWRGATE bits that PWRTASKWKM selects, so those tiles run the same rail-up sequence a software gate-clear runs, with no hart awake to write PWRCR. The pulse is applied per bit after the bus writes of the same cycle, so a coincident PWRCR write merges and the task wins its own bits. The mask is qualified by byte lane 0 like PWRCR, bit 0 has no storage, and it resets to 0, so task wake is inert until software selects a tile. The tap never reaches the boot-gate release logic: a held boot gate stays held.

18.3.3 Boot gate

The boot gate is a hold-in-reset combined into every hart's outer reset. All of its state lives on the always-on MCLK domain and evaluates once per cycle. The three pad-side inputs (PGOOD, the strap and the NFC field-detect level) are double-flop synchronized, so they reach the gate two MCLK cycles late.

The strap is sampled once: a counter starts at reset release and, eight MCLK cycles later, latches the synchronized strap level into PWSTRAP and sets PWSTRAPVLD. A strapped board thereby arms the gate, adds PGOOD as a release condition and enables re-hold with no software. The gate is released during that sampling window so that a normal board's boot is never delayed; a strapped board's harts may therefore begin a first fetch and then be re-held, which is harmless because the hold is the same outer reset that cleared them at power-on and the eventual release runs a clean cold boot.

The effective policy combines the strap defaults with the PWRWAKE overrides:

- armed = PWGATEEN or strapped
- release on PGOOD = PWRLSPGOOD or strapped
- release on field detect = PWRLSFIELD
- re-hold = PWREHOLD or strapped
- release = PWSWRLS, or (PGOOD live and release on PGOOD), or (field live and release on field detect)
- hold = armed and not (release, when re-hold is set; the sticky latch of every past release, when it is clear)

The gate output is registered, so releases and re-holds are glitch-free and take effect one cycle after their condition resolves. With re-hold set the hold tracks the release condition: a PGOOD drop re-asserts the hold on the next cycle and the harts re-enter reset at once, because without retention there is nothing a grace period could save. With re-hold clear the first release is final until the next chip reset, for systems in which supervising software owns brownout policy. An armed gate with no release source holds until PWSWRLS, which production test uses to prove the gate is load-bearing.

PWRSTS reports the synchronized live levels (PWPGOODLIV, PWFIELDLIV), the sampled strap (PWSTRAP, PWSTRAPVLD), the gate state (PWBOOTHOLD) and the release latch (PWRRLSLATCH). Software can only observe the gate released or arm it for the next hold, because while the hold is active nothing executes.

18.4 Interrupts and events

Not applicable. The controller raises no vector; the NFC field-detect level used as a release source is available separately as NFCx vector 94, and tile state is polled in PWRSR.

18.5 Programming

18.5.1 Gating tile *h*

1. Confirm the tile is parked in its boot-ROM sleep or has finished its work, and that any data it must keep is in shared RAM.
2. Set bit *h* of PWRCR.PWRGATE with a full-word store.
3. Poll nibble *h* of PWRSR until it reads 3.

18.5.2 Waking tile *h*

1. Clear bit *h* of PWRCR.PWRGATE with a full-word store.
2. Poll nibble *h* of PWRSR until it reads 0.
3. Write the tile's loader mailbox row and send it a software interrupt (Section 5.3).

18.5.3 Arming the boot gate from software

1. Set PWRWAKE.PWRLSPGOOD, and PWRWAKE.PWRLSFIELD if the RF field is also a release source.
2. Set PWRWAKE.PWREHOLD if the hold is to track PGOOD.
3. Set PWRWAKE.PWGATEEN.

A harvested board needs none of this: with the strap high and the supervisor's output on PGOOD, the hardware defaults arm the gate, wait on PGOOD and re-hold on brownout, so the first instruction any hart executes runs after the supply is declared good.

18.5.4 Supercap duty-cycled service loop

1. Gate the unused tiles through PWRCR.
2. Route NFCx vector 94 to the serving hart in its IRQROUTER row.
3. Provision and arm the NFC block in listen mode with auto-read.
4. Gate unused memories and banks in BLOCKPWR and divide MCLK in CLKDIVCR.
5. Execute `sleep`; a reader's field raises the interrupt and the hart resumes with its state intact.

18.6 Usage notes

Gate only parked or quiesced tiles. Gating is cold; a tile gated mid-transaction loses its work and any shared-bus transaction it had in flight is answered by the clamps.

Use full-word stores to PWRCR. The write is qualified by byte lane 0 only.

Poll PWRSR rather than adding delays. The sequencer includes the rail-settle delay; the nibble reads 0 when the tile can be relaunched.

Check PWRSR before trusting an aperture read. A gated tile's TCM aperture returns zeros, and zero is a legal TCM value.

Keep the boot gate disarmed on a normally powered board. An armed gate with no release source holds every hart, hart 0 included, until PWSWRLS is written, which nothing can do while the hold is active.

18.7 Registers

Register offsets in this chapter are relative to the instance base address in Table 18.1 (PWRCTRL 0x4B00); the generated header names each base PWRCTRL_BASE.

Table 18.2: PWRCTRL register summary

Offset	Name	Access	Reset	Description
0x00	PWRCR	rw/r	0x00000000	Power gate control
0x04	PWRSR	r	0x00000000	Power sequencer state, one read-only nibble per hart (bits 4h+3:4h)
0x08 to 0x13	-	-	-	Reserved
0x14	PWRWAKE	rw	0x00000000	Boot gate and wake source control
0x18	PWRSTS	r	0x00000000	
0x1C	TASKWKM	rw	0x00000000	Event-fabric task-wake mask, one bit per gateable tile hart

18.7.1 PWRCR: Power gate control

Offset 0x00 **Reset** 0x00000000 **Width** 32 **Address** 0x4B00
Power gate control. Setting PWRGATE bit h powers tile hart h down (isolation, reset, rail off); clearing it powers the tile back up and cold-boots it. A request made while the sequencer is mid-sequence is honored when the sequence completes. Bit 0 (hart 0) is always on: it reads 0 and ignores writes.

Bits	Field	Type	Reset	Description
31:5	-	-	-	Reserved. Reads zero; write zero.
4:1	GATE	rw	0b0000	Gate request mask, one bit per tile hart: bit h = 1 powers tile hart h down and 0 powers it up, with a cold boot on the way up. Poll the PWRSR nibble of the hart for sequencer completion. Only the all-powered value is enumerated; any combination of bits is valid. 0b0000 PWRGATE_NONE: All tile harts powered
0	H0	r	0	Hart 0 is always-on: reads 0, writes ignored.

18.7.2 PWRSR: Power sequencer state, one read-only nibble per hart (bits 4h+3:4h)

Offset 0x04 **Reset** 0x00000000 **Width** 32 **Address** 0x4B04
Power sequencer state, one read-only nibble per hart (bits 4h+3:4h). 0 = ON, 1 = ISO (clamps asserting), 2 = RSTOFF (reset held, rail dying), 3 = OFF (gated), 4 = RAIL (waking, rail settling), 5 = UNISO (clamps releasing). Hart 0's nibble always reads 0. A tile is safely gated when its nibble reads 3, and fully awake (booting or parked in the ROM) when it returns to 0.

Bits	Field	Type	Reset	Description
31:20	-	-	-	Reserved. Reads zero; write zero.
19:16	ST4	r	0b0000	Tile hart 4 sequencer state.
15:12	ST3	r	0b0000	Tile hart 3 sequencer state.
11:8	ST2	r	0b0000	Tile hart 2 sequencer state.
7:4	ST1	r	0b0000	Tile hart 1 sequencer state.
3:0	ST0	r	0b0000	Hart 0 state: always 0 (ON, always-on domain).

18.7.3 PWRWAKE: Boot gate and wake source control

Offset 0x14 **Reset** 0x00000000 **Width** 32 **Address** 0x4B14

Boot gate and wake source control. The harvested-boot strap drives the hardware defaults (a strapped board arms the gate, waits on PGOOD and re-holds on brownout with no software involvement); these bits OR software overrides on top. Resets to 0 (no override), so the gate is released on every normal boot. Write with full-word stores (byte lane 0 qualified, like PWRCR).

Bits	Field	Type	Reset	Description
31:5	-	-	-	Reserved. Reads zero; write zero.
4	EHOLD	rw	0	Re-hold policy: 1 re-asserts the boot hold whenever the release condition drops (a later PGOOD return then cold-boots the harts through the shared ROM); 0 makes the release one-shot, latched in PWRSTS.PWRRLSLATCH. The strap ORs this bit in on a harvested board.
3	PWSWRLS	rw	0	Software release: 1 releases an armed boot gate unconditionally. With no other release source enabled, an armed gate holds until this bit is set.
2	LSFIELD	rw	0	Field release enable: 1 makes the synchronized NFC field level (PWRSTS.PWFIELDLIV) a release condition, so a reader arriving releases the boot gate. The field level is tied to 0 when NFC is absent or field power is disabled.
1	LSPGOOD	rw	0	PGOOD release enable: 1 makes the synchronized PGOOD pad level (PWRSTS.PWPGOODLIV) a release condition. The strap ORs this bit in on a harvested board, which always waits on the supply supervisor.
0	PWGATEEN	rw	0	Boot gate arm: 1 holds every hart in reset until a release condition fires. The strap ORs this bit in, so a strapped board arms with no software.

18.7.4 PWRSTS

Offset 0x18 **Reset** 0x00000000 **Width** 32 **Address** 0x4B18

Boot gate and wake source status, read only: the synchronized live pad levels, the one-shot strap sample that selects the harvested-boot branch of the boot ROM, and the gate state.

Bits	Field	Type	Reset	Description
31:6	-	-	-	Reserved. Reads zero; write zero.
5	RLSLATCH	r	0	One-shot release latched: set when an armed gate has been released with PWREHOLD = 0 and held until reset.

Bits	Field	Type	Reset	Description
4	PWBOOTHOLD	r	0	Boot gate state: 1 while the gate holds every hart in reset.
3	PWSTRAPVLD	r	0	Strap sample valid: set a few MCLK cycles after reset release once the one-shot strap sample has been taken. Poll it before reading PWSTRAP.
2	PWSTRAP	r	0	Harvested-boot strap sample: 1 = harvested boot (the boot ROM skips the SPI flash copy and runs the ROM-resident service loop), 0 = normal SPI boot. Sampled once after reset; later strap changes are ignored.
1	PWFIELDLIV	r	0	Synchronized NFC field detect level; 0 when NFC is absent or field power is disabled.
0	PWPGOODLIV	r	0	Synchronized PGOOD pad level (P6.7); reads 0 (power not good) when the pin is unconnected, because of the reset pull-down.

18.7.5 TASKWKM: Event-fabric task-wake mask, one bit per gateable tile hart

Offset 0x1C **Reset** 0x00000000 **Width** 32 **Address** 0x4B1C

Event-fabric task-wake mask, one bit per gateable tile hart. A task_wake pulse from the event fabric clears the PWRCR gate bits selected here, waking those tiles through the same MTCMOS rail-up sequence a register-cleared gate uses; the tap never touches the boot-gate release logic. Byte-lane-0 qualified like PWRCR, and bit 0 (hart 0, always on) has no storage. Resets to 0, so task-wake is inert out of reset.

Bits	Field	Type	Reset	Description
31:5	-	-	-	Reserved. Reads zero; write zero.
4:1	TASKWKM	rw	0b0000	Task-wake mask, one bit per tile hart: bit h = 1 lets a fabric task_wake pulse clear PWRCR.PWRGATE[h] and power tile hart h back up.
0	-	-	-	Reserved. Reads zero; write zero.

19 Quad serial peripheral interface (QSPIx)

19.1 Overview

The Quad Serial Peripheral Interface (QSPIx) is a command-driven controller for external SPI-family memories (serial NOR flash and compatible devices). It issues a transaction as a sequence of up to four phases (command, address, dummy, and data) over a six-wire bus (a serial clock, an active-low chip select, and four bidirectional data lines). The lane width of each phase is independently configurable, so a single engine drives legacy single-lane (1-1-1) flash, dual-output (1-1-2), and quad-output / quad-I/O (1-1-4 / 1-4-4) devices. The main features of the QSPI controller are listed below:

- Command / address / dummy / data transaction structure with a single-write launch
- Independent single-, dual-, and quad-lane width per phase (QSPICMDW, QSPIADRW, QSPIDATW)
- Configurable address phase: none, 24-bit, or 32-bit (QSPIAWID)
- Programmable dummy-cycle count with the bus released, for dual/quad fast reads (QSPIDUMMY)
- Configurable clock polarity and phase (QSPICPOL, QSPICPHA)
- Baud generator off SMCLK: SCK = SMCLK $/(2 \times (1 + \text{QSPIBR}))$
- Data lengths of 8, 16, or 32 bits, right-justified in the transmit and receive registers
- Registered read path: register reads have *no* side effects (reading QSPIxRX clears nothing)
- Two interrupt sources: transfer complete and receive-register full

19.2 Instances and signals

Table 19.1: QSPIx instances

Instance	Base address	Interrupt vector(s)	Pins (AF)
QSPI0	0x4C00	55 to 56	QSPI_SCK P4.0 (AF1), QSPI_CS P4.1 (AF1), QSPI_IO0 P4.2 (AF1), QSPI_IO1 P4.3 (AF1), QSPI_IO2 P4.4 (AF1), QSPI_IO3 P4.5 (AF1)

Table 19.2: QSPIx signals

Signal	Direction	Description	Pin (AF)
QSPI_SCK	Output	QSPI0 serial clock	P4.0 (AF1)
QSPI_CS	Output	QSPI0 chip select	P4.1 (AF1)
QSPI_IO0	Bidirectional	QSPI0 quad data 0	P4.2 (AF1)
QSPI_IO1	Bidirectional	QSPI0 quad data 1	P4.3 (AF1)
QSPI_IO2	Bidirectional	QSPI0 quad data 2	P4.4 (AF1)

Signal	Direction	Description	Pin (AF)
QSPI_IO3	Bidirectional	QSPI0 quad data 3	P4.5 (AF1)

Table 19.3: QSPIx interrupt vectors

Vector	Instance	Description	Clear method
55	QSPI0	QSPI0 Transfer Complete InterruptIRQB_QSPI0_TC	See chapter
56	QSPI0	QSPI0 Receive-Register Full InterruptIRQB_QSPI0_RXF	See chapter

19.3 Functional description

19.3.1 QSPI Pins

The QSPIx peripheral drives six pins: the serial clock SCK, an active-low chip select CS, and four bidirectional data lines IO0–IO3. In single-lane mode IO0 carries the outbound data and IO1 the inbound data (classic MOSI/MISO); in dual and quad modes the same lines are driven as a two- or four-bit bus in the direction of the current phase. The controller releases every data line (high-impedance) during the dummy phase. Only chip select 0 is wired to a pin in this configuration; program QSPICSSEL to 0.

19.3.2 Transaction Structure

A transaction is fully described by the control, address, and command registers. The byte-lane-0 write of QSPIxCMD *launches* it; it is ignored while the controller is disabled or busy. Writing QSPIxTX never launches a transaction. For a read, the captured word appears in QSPIxRX and QSPIRXFULL is set; reading QSPIxRX has no side effects.

19.4 Interrupts and events

The three status flags QSPITXEIF, QSPIRXFULL, and QSPITCIF are write-1-to-clear: write a 1 to a bit to clear it.

The two interrupt sources are vector 55 (transfer complete) and vector 56 (receive-register full), delivered through the interrupt router like every other shared-peripheral source.

19.5 Programming

The launch sequence is:

1. Program QSPIxCR with the per-phase lane widths, SPI mode, address width, dummy count, baud divider, and interrupt enables, and set QSPIEN.
2. Write the transaction address to QSPIxADR (used only when QSPIAWID is non-zero).
3. For a write-direction transaction, place the payload (right-justified) in QSPIxTX. Writing QSPIxTX never launches a transaction.
4. Write QSPIxCMD last, with the opcode, data length, and direction. The byte-lane-0 write of QSPIxCMD *launches* the transaction; it is ignored while the controller is disabled or busy.

5. Poll QSPIBUSY (or wait for the transfer-complete interrupt). For a read, the captured word appears in QSPIxRX and QSPIRXFULL is set; reading QSPIxRX has no side effects.

19.6 Usage notes

Because the serial core runs in the SMCLK domain, software must set SYSCLKCR to 0 (SMCLK from HFXT) before using the controller, exactly as for the SPI and UART peripherals. Dual- and quad-lane read commands require at least one dummy cycle (QSPIDUMMY ≥ 1) so the device can turn its data lines around.

In configurations that include it, QSPI0 lives in the shared peripheral window behind the multi-core arbiter (see Section 5) at slot 0x4C00, so any hart can use it.

19.7 Registers

Register offsets in this chapter are relative to the instance base address in Table 19.1 (QSPI0 0x4C00); the generated header names each base QSPI0_BASE.

Table 19.4: QSPIx register summary

Offset	Name	Access	Reset	Description
0x00	QSPIxCR	rw	0x00000000	QSPI control register
0x04	QSPIxCMD	rw	0x00000000	QSPI command register
0x08	QSPIxADR	rw	0x00000000	QSPI transaction address
0x0C	QSPIxTX	rw	0x00000000	QSPI transmit data
0x10	QSPIxRX	r	0x00000000	QSPI receive data
0x14	QSPIxSR	r/rw1	0x00000000	QSPI status register

19.7.1 QSPIxCR: QSPI control register

Offset 0x00 **Reset** 0x00000000 **Width** 32 **Address** 0x4C00

QSPI control register. Configures the per-phase lane widths, SPI mode, address size, dummy cycles, chip select, baud rate, and interrupt enables. Take care to reconfigure this register only while the controller is idle (QSPIBUSY = 0).

Bits	Field	Type	Reset	Description
31:29	-	-	-	Reserved. Reads zero; write zero.
28	RXFIE	rw	0	Receive-full interrupt enable. When set, the QSPIRXFULL flag drives interrupt vector 56. 0: Interrupt disabled 1: Interrupt enabled
27	TCIE	rw	0	Transfer-complete interrupt enable. When set, the QSPITCIF flag drives interrupt vector 55. 0: Interrupt disabled 1: Interrupt enabled
26:19	BR	rw	0x00	Baud-rate divider. The serial clock is SCK = SMCLK / (2 * (1 + QSPIBR)), so 0 gives the fastest clock (SMCLK/2). SMCLK is the SYSTEM clock source (write SYS_CLK_CR = 0 to run SMCLK from HFXT before using the controller).

Bits	Field	Type	Reset	Description
18:16	CSSEL	rw	0b000	Chip-select index. Selects which chip-select output drives the transaction. In this MVP only CS0 is wired to a pin; write 0.
15:11	DUMMY	rw	0x00	Dummy SCK cycles inserted after the address phase and before the data phase, with the bus released (all IO lines tri-stated). Required for dual/quad fast-read commands (at least 1 dummy cycle); set to 0 for commands with no dummy phase.
10:9	AWID	rw	0b00	Address-phase width in bits. Selects whether the transaction emits an address phase and how wide it is (from QSPIxADR). 0b00: No address phase 0b01: 24-bit address (low 24 bits of QSPIxADR) 0b10: 32-bit address 0b11: Reserved
8	CPHA	rw	0	Clock phase. Selects the SCK edge on which data is sampled. 0: Sample on the leading edge 1: Sample on the trailing edge
7	CPOL	rw	0	Clock polarity. Sets the idle level of SCK. 0: SCK idles low 1: SCK idles high
6:5	DATW	rw	0b00	Data-phase lane width. Selects how many IO lines carry the payload. 0b00: 1-bit (single lane, IO0 only) 0b01: 2-bit (dual lane, IO0-1) 0b10: 4-bit (quad lane, IO0-3) 0b11: Reserved
4:3	ADRW	rw	0b00	Address-phase lane width. Selects how many IO lines carry the transaction address (when QSPIAWID is non-zero). 0b00: 1-bit (single lane, IO0 only) 0b01: 2-bit (dual lane, IO0-1) 0b10: 4-bit (quad lane, IO0-3) 0b11: Reserved
2:1	CMDW	rw	0b00	Command-phase lane width. Selects how many IO lines carry the 8-bit opcode. 0b00: 1-bit (single lane, IO0 only) 0b01: 2-bit (dual lane, IO0-1) 0b10: 4-bit (quad lane, IO0-3) 0b11: Reserved
0	EN	rw	0	QSPI enable. When 0 the controller is held idle: the serial pins are released, no transaction launches, and a write to QSPIxCMD is ignored. Set to 1 before launching a transaction. 0: Disabled 1: Enabled

19.7.2 QSPIxCMD: QSPI command register

Offset 0x04 **Reset** 0x00000000 **Width** 32 **Address** 0x4C04

QSPI command register. Holds the opcode, data length, and direction of the next transaction. Writing this register with byte lane 0 asserted LAUNCHES the transaction (the write is ignored when QSPIEN = 0 or QSPIBUSY = 1). Program QSPIxCR, QSPIxADR, and (for writes) QSPIxTX first, then write QSPIxCMD last.

Bits	Field	Type	Reset	Description
31:11	-	-	-	Reserved. Reads zero; write zero.
10	DIR	rw	0	Data-phase direction. 0: Write (drive QSPIxTX out on the IO lines) 1: Read (capture the IO lines into QSPIxRX)
9:8	DLEN	rw	0b00	Data-phase length. Selects how many payload bits the data phase transfers (right-justified in QSPIxTX / QSPIxRX). 0b00: No data phase 0b01: 8-bit 0b10: 16-bit 0b11: 32-bit
7:0	CMD	rw	0x00	Command opcode, shifted out most-significant bit first during the command phase at the QSPICMDW lane width.

19.7.3 QSPIxADR: QSPI transaction address

Offset 0x08 **Reset** 0x00000000 **Width** 32 **Address** 0x4C08

QSPI transaction address. Emitted during the address phase at the QSPIADRW lane width when QSPIAWID is non-zero. When QSPIAWID selects 24-bit, only the low 24 bits are used.

Bits	Field	Type	Reset	Description
31:0	ADR	rw	0x00000000	Transaction address value.

19.7.4 QSPIxTX: QSPI transmit data

Offset 0x0C **Reset** 0x00000000 **Width** 32 **Address** 0x4C0C

QSPI transmit data. Holds the write-direction payload, right-justified (the low QSPIDLEN bits are significant). Writing this register never triggers a transaction; only a QSPIxCMD write does.

Bits	Field	Type	Reset	Description
31:0	TX	rw	0x00000000	Write payload, right-justified.

19.7.5 QSPIxRX: QSPI receive data

Offset 0x10 **Reset** 0x00000000 **Width** 32 **Address** 0x4C10

QSPI receive data. A snapshot of the most recent read-direction data phase, right-justified (the low QSPIDLEN bits are significant). Reads have NO side effects: reading QSPIxRX does not clear any flag or advance any state (deliberately unlike the SPI RX-read-clears-TCIF behaviour).

Bits	Field	Type	Reset	Description
31:0	RX	r	0x00000000	Read payload snapshot, right-justified.

19.7.6 QSPIxSR: QSPI status register

Offset 0x14 **Reset** 0x00000000 **Width** 32 **Address** 0x4C14

QSPI status register. QSPIBUSY is read-only; the three event flags are write-1-to-clear (write a 1 to a bit to clear it; writing 0 leaves it unchanged).

Bits	Field	Type	Reset	Description
31:4	-	-	-	Reserved. Reads zero; write zero.
3	TCIF	rw1	0	Transfer-complete flag. Set when a transaction finishes; drives vector 55 when QSPITCIE is set. Write 1 to clear. 0: No event 1: Transfer complete
2	RXFULL	rw1	0	Receive-full flag. Set when a read-direction data phase has captured a fresh word into QSPIxRX; drives vector 56 when QSPIRXFIE is set. Write 1 to clear. (Reading QSPIxRX does NOT clear it.) 0: No event 1: Receive register full
1	TXEIF	rw1	0	Transmit-empty flag. Set when the transmit path has consumed QSPIxTX and can accept the next word. Write 1 to clear. 0: No event 1: Transmit register empty
0	BUSY	r	0	Busy. Reads 1 while a transaction is in progress; a QSPIxCMD launch is ignored while this bit is set. 0: Idle 1: Transaction in progress

20 Inter-integrated circuit interface (I2Cx)

20.1 Overview

The I2Cx peripherals are general-purpose I^2C interfaces, each able to act as a bus master or as an addressed slave. Master and slave are driven through separate sets of control and status bits that share one pin pair, so a port can be configured for either role, or for both on a multi-master bus.

- Master transmitter and master receiver modes with START, repeated START and STOP generation (I2CMST, I2CMSP) and a programmable bus clock divider (I2CMDIV)
- Multi-master arbitration: losing the bus releases it and sets I2CMARB
- Slave transmitter and slave receiver modes with a 7-bit address (I2CxAR), a per-bit address mask (I2CxAMR) and optional general-call response (I2CGCE)
- Optional slave clock stretching (I2CSCS) through the ACK bit
- Separate transmit and receive data registers per role (I2CxMTX/I2CxMRX, I2CxSTX/I2CxSRX)
- Thirteen individually maskable interrupt sources, one vector each

20.2 Instances and signals

Table 20.1: I2Cx instances

Instance	Base address	Interrupt vector(s)	Pins (AF)
I2C0	0x4E00	57 to 69	SDA0 P3.0 (AF0), SCL0 P3.1 (AF0)
I2C1	0x4F00	70 to 82	SDA1 P3.2 (AF0), SCL1 P3.3 (AF0)

Table 20.2: I2Cx signals

Signal	Direction	Description	Pin (AF)
SDA0	Bidirectional	I2C0 serial data	P1.6 (AF1), P2.6 (AF1), P3.0 (AF0)
SCL0	Bidirectional	I2C0 serial clock	P1.7 (AF1), P2.7 (AF1), P3.1 (AF0)
SDA1	Bidirectional	I2C1 serial data	P1.4 (AF1), P2.2 (AF1), P3.2 (AF0)
SCL1	Bidirectional	I2C1 serial clock	P1.5 (AF1), P2.3 (AF1), P3.3 (AF0)

Table 20.3: I2Cx interrupt vectors

Vector	Instance	Description	Clear method
57	I2C0	I2C0 start received Interrupt IRQB_I2C0_STR	Write 1 to I2C0SR.I2CSTR
58	I2C0	I2C0 stop received Interrupt IRQB_I2C0_spr	Write 1 to I2C0SR.I2CSPR

Vector	Instance	Description	Clear method
59	I2C0	I2C0 master mode start condition sent Interrupt IRQB_I2C0_msts	Write 1 to I2C0SR.I2CMSTS
60	I2C0	I2C0 master mode stop condition sent Interrupt IRQB_I2C0_msps	Write 1 to I2C0SR.I2CMSPS
61	I2C0	I2C0 master mode arbitration lost Interrupt IRQB_I2C0_marb	Write 1 to I2C0SR.I2CMARB
62	I2C0	I2C0 master mode transmit empty Interrupt IRQB_I2C0_mtxe	Write 1 to I2C0SR.I2CMTXE
63	I2C0	I2C0 master mode NACK received Interrupt IRQB_I2C0_mnr	Write 1 to I2C0SR.I2CMNR
64	I2C0	I2C0 master mode transfer complete Interrupt IRQB_I2C0_mxc	Write 1 to I2C0SR.I2CMXC
65	I2C0	I2C0 slave address Interrupt IRQB_I2C0_sa	Write 1 to I2C0SR.I2CSA
66	I2C0	I2C0 slave transmit empty Interrupt IRQB_I2C0_stxe	Write 1 to I2C0SR.I2CSTXE
67	I2C0	I2C0 slave overflow Interrupt IRQB_I2C0_sovf	Write 1 to I2C0SR.I2CSOVF
68	I2C0	I2C0 slave mode NACK received Interrupt IRQB_I2C0_snr	Write 1 to I2C0SR.I2CSNR
69	I2C0	I2C0 slave mode transfer complete Interrupt IRQB_I2C0_sxc	Write 1 to I2C0SR.I2CSXC
70	I2C1	I2C1 start received Interrupt IRQB_I2C1_STR	Write 1 to I2C1SR.I2CSTR
71	I2C1	I2C1 stop received Interrupt IRQB_I2C1_spr	Write 1 to I2C1SR.I2CSPR
72	I2C1	I2C1 master mode start condition sent Interrupt IRQB_I2C1_msts	Write 1 to I2C1SR.I2CMSTS
73	I2C1	I2C1 master mode stop condition sent Interrupt IRQB_I2C1_msps	Write 1 to I2C1SR.I2CMSPS
74	I2C1	I2C1 master mode arbitration lost Interrupt IRQB_I2C1_marb	Write 1 to I2C1SR.I2CMARB
75	I2C1	I2C1 master mode transmit empty Interrupt IRQB_I2C1_mtxe	Write 1 to I2C1SR.I2CMTXE

Vector	Instance	Description	Clear method
76	I2C1	I2C1 master mode NACK received Interrupt IRQB_I2C1_mnr	Write 1 to I2C1SR.I2CMNR
77	I2C1	I2C1 master mode transfer complete Interrupt IRQB_I2C1_mxc	Write 1 to I2C1SR.I2CMXC
78	I2C1	I2C1 slave address Interrupt IRQB_I2C1_sa	Write 1 to I2C1SR.I2CSA
79	I2C1	I2C1 slave transmit empty Interrupt IRQB_I2C1_stxe	Write 1 to I2C1SR.I2CSTXE
80	I2C1	I2C1 slave overflow Interrupt IRQB_I2C1_sovf	Write 1 to I2C1SR.I2CSOVF
81	I2C1	I2C1 slave mode NACK received Interrupt IRQB_I2C1_snr	Write 1 to I2C1SR.I2CSNR
82	I2C1	I2C1 slave mode transfer complete Interrupt IRQB_I2C1_sxc	Write 1 to I2C1SR.I2CSXC

20.3 Functional description

20.3.1 Wire protocol

Both roles use the same two-wire protocol on SDAx and SCLx. Both lines are open drain and require external pull-up resistors: a device drives a bit low by pulling the line down and releases it to let the pull-up present a one. Because every device can only pull down, two devices driving at once cannot damage each other, and that is what makes multi-master arbitration and slave clock stretching possible on the same pair of wires.

A transfer is framed by two conditions that are illegal during data: START is a falling edge on SDAx while SCLx is high, and STOP is a rising edge on SDAx while SCLx is high. Between them SDAx changes only while SCLx is low, so every data bit is stable across the whole SCLx high period and any receiver may sample it there. After the address byte and after each data byte, the transmitter releases SDAx for one clock and the receiver pulls it low for an ACK or leaves it high for a NACK. Figure 20.1 shows one byte write.

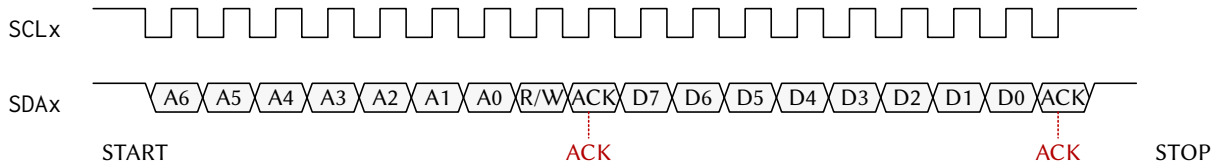


Figure 20.1: One I2C byte write on the wire: START, address, ACK, data byte, ACK, STOP.

The address byte carries the 7-bit slave address in its upper bits and the direction in its least significant bit: 0 selects a write (master transmitter), 1 a read (master receiver). A master that changes direction or address without giving up the bus issues a repeated START instead of a STOP.

20.3.2 Master operation

With I2CMEN set, the port drives SCLx at the rate set by I2CMDIV. Setting I2CMST in I2CxFCR sends a START (or a repeated START while the bus is held) followed by the byte in I2CxMTX; each subsequent write to I2CxMTX sends one byte, and each received byte lands in I2CxMRX. I2CMXC marks the end of each byte: in transmitter mode after the slave's ACK/NACK, in receiver mode before this master sends its own ACK/NACK, so software chooses the acknowledge before the next byte. I2CMNR reports a NACK from the slave. Setting I2CMSP sends a STOP. If another master wins the bus during arbitration, the port releases the lines and sets I2CMARB, because continuing to drive would corrupt the winner's transfer.

20.3.3 Slave operation

With I2CSEN set, the port compares every address byte against I2CxAR under the mask in I2CxAMR (a masked bit matches either value), and against the general-call address when I2CGCE is set. A match sets I2CSA, with I2CSTM reporting the direction. In slave receiver mode each byte lands in I2CxSRX and sets I2CSXC; a byte arriving before the previous one was read sets I2CSOVF. In slave transmitter mode each byte is taken from I2CxSTX, and I2CSTXE asks for the next one. With I2CSCS set, the port holds SCLx low through the ACK bit until firmware has serviced the flag, which trades bus latency for lossless flow control on a port whose hart may be busy elsewhere.

20.3.4 Clocking

The bus engine runs on SMCLK, divided by I2CMDIV to the bus rate; the register interface runs on MCLK, so the registers stay readable with SMCLK stopped. The boot ROM may leave SMCLK on LFXT (32.768 kHz), which would put the bus in the low kilohertz range.

The slave is oversampled: SDAx and SCLx are sampled on SMCLK, and START, STOP and both SCLx edges are decoded from the sampled values rather than clocking flip-flops directly. SMCLK must therefore be at least 32 times the SCLx rate an external master drives, which at 24 MHz allows SCLx up to 750 kHz and covers both standard mode (100 kHz) and fast mode (400 kHz). The same sampler suppresses spikes shorter than two SMCLK periods, 83 ns at 24 MHz, which meets the fast-mode requirement of 50 ns. Master mode is unaffected: it generates SCLx itself from I2CMDIV.

20.4 Interrupts and events

Each flag in I2CxSR has its own enable in I2CxCR and its own vector. All thirteen flags are write-1-to-clear. Table 20.4 lists them for I2C0; I2C1, in configurations that include it, uses the same order from vector 70 (70 to 82).

20.5 Programming

20.5.1 Initialisation as a master

1. Select the SMCLK source in SYSCLKCR (Section 16.3.1).
2. Set the PxSEL bits of the SDAx and SCLx pins, and their PxAFS fields if the bus is relocated.
3. Write I2CxCR.I2CMDIV for the bus rate.
4. Set the interrupt enables required in I2CxCR.

Vector	Condition	Set by	Cleared by
57	START or repeated START seen on the bus	I2CSTR	Writing 1 to I2CSTR
58	STOP seen on the bus	I2CSPR	Writing 1 to I2CSPR
59	Master START sent	I2CMSTS	Writing 1 to I2CMSTS
60	Master STOP sent	I2CMSPS	Writing 1 to I2CMSPS
61	Master arbitration lost	I2CMARB	Writing 1 to I2CMARB
62	Master transmit register empty	I2CMTXE	Writing 1 to I2CMTXE
63	Master received NACK	I2CMNR	Writing 1 to I2CMNR
64	Master byte transfer complete	I2CMXC	Writing 1 to I2CMXC
65	Slave addressed	I2CSA	Writing 1 to I2CSA
66	Slave transmit register empty	I2CSTXE	Writing 1 to I2CSTXE
67	Slave receive overflow	I2CSOVF	Writing 1 to I2CSOVF
68	Slave received NACK	I2CSNR	Writing 1 to I2CSNR
69	Slave byte transfer complete	I2CSXC	Writing 1 to I2CSXC

Table 20.4: I2C0 interrupt vectors; I2C1 follows the same order from vector 70.

5. Set I2CxCR.I2CMEN.

20.5.2 Initialisation as a slave

1. Select the SMCLK source in SYSCLKCR.
2. Set the PxSEL bits of the SDAx and SCLx pins.
3. Write the slave address to I2CxAR and the mask to I2CxAMR.
4. Set I2CxCR.I2CSCS if clock stretching is wanted, and I2CGCE for general-call response.
5. Set the interrupt enables required in I2CxCR.
6. Set I2CxCR.I2CSEN.

20.5.3 Master write of one byte

1. Write the address byte (address and direction 0) to I2CxMTX.
2. Set I2CxFCR.I2CMST.
3. Wait for I2CMXC; if I2CMNR is set, abort with a STOP.
4. Write the data byte to I2CxMTX and wait for I2CMXC.
5. Set I2CxFCR.I2CMSP.

The register descriptions in Section 20 give the remaining sequences (master read, slave transmit, slave receive) field by field.

20.6 Usage notes

Set the SMCLK source before using the bus. The bus clock is derived from SMCLK, and the boot-time SMCLK source is a boot-ROM implementation detail.

Keep SMCLK at least 32 times the slave bus rate. The slave samples SDAx and SCLx on SMCLK, so dividing SMCLK below that ratio, or moving it to LFXT, stops the port answering an external master even though the registers stay readable. At 24 MHz the ratio holds for any SCLx up to 750 kHz.

Choose the acknowledge before the next byte in master receiver mode. I2CMXC is raised before the master's ACK/NACK, because the last byte of a read must be answered with a NACK to end the transfer cleanly.

Handle arbitration loss by retrying the whole transfer. After I2CMARB the port has released the bus mid-transfer; the winner's transaction must complete before a new START.

Serialize whole transactions between harts. The arbiter makes each register access atomic but not a transaction; a hart that shares a port claims a mutex around each transfer (Section 5.4).

Clear every flag with a 1. Writing 0 to I2CxSR has no effect; a handler that returns with its flag set is re-entered.

20.7 Registers

Register offsets in this chapter are relative to the instance base address in Table 20.1 (I2C0 0x4E00, I2C1 0x4F00); the generated header names each base I2C0_BASE and so on.

Table 20.5: I2Cx register summary

Offset	Name	Access	Reset	Description
0x00	I2CxCR	rw	0x00000000	I2C control register
0x04	I2CxFCR	w1	0x00	I2C flow control register
0x08	I2CxSR	r/rw1	0x0000	I2C status register
0x0C	I2CxMTX	rw	0x00	I2C master transmit register
0x10	I2CxMRX	r	0x00	I2C master receive register
0x14	I2CxSTX	rw	0x00	I2C slave transmit register
0x18	I2CxSRX	r	0x00	I2C slave receive register
0x1C	I2CxAR	rw	0x00	I2C this slave address register
0x20	I2CxAMR	rw	0x00	I2C this slave address mask register

20.7.1 I2CxCR: I2C control register

Offset 0x00 **Reset** 0x00000000 **Width** 32 **Address** 0x4E00 (I2C0), 0x4F00 (I2C1)

Bits	Field	Type	Reset	Description
31:22	-	-	-	Reserved. Reads zero; write zero.

Bits	Field	Type	Reset	Description
21	MEN	rw	0	Master enable. When set, the device waits for I2CMST, acts as a master from that start condition until the stop condition commanded by I2CMSP, and then resumes slave operation if I2CSEN is also set. 0: Disabled 1: Enabled
20	SEN	rw	0	Slave enable. When set, the device listens for its address as an I2C slave. If master mode is also enabled, the device acts as a slave until I2CMST starts a master transfer and returns to slave operation when that transfer completes. 0: Disabled 1: Enabled
19	SN	rw	0	Slave NACK select. When set, the slave answers its address and every received byte with a NACK instead of an ACK. With clock stretching enabled, software can change this bit before releasing SCL to choose the reply for the current byte. 0: ACK 1: NACK
18	SCS	rw	0	I2C slave clock stretching enable. When enabled, this slave will hold the SCL line low during the ACK phase of the transmission to allow this slave more time. Note that the master will be left waiting for the ACK/NACK as long as this bit is set to 1. 0: Disabled 1: Enabled
17	GCE	rw	0	I2C slave general call enable. When enabled, this slave will be addressed if a global call is issued on the bus. 0: Disabled 1: Enabled
16:13	MDIV	rw	0b0000	I2C master mode clock divider. The master mode finite state machine clock source is SMCLK, which is divided by a factor of 4 * 2**I2CMDIV. 0b0000 I2CMDIV_1: /1 (no division) 0b0001 I2CMDIV_2: /2 0b0010 I2CMDIV_4: /4 0b0011 I2CMDIV_8: /8 0b0100 I2CMDIV_16: /16 0b0101 I2CMDIV_32: /32 0b0110 I2CMDIV_64: /64 0b0111 I2CMDIV_128: /128 0b1000 I2CMDIV_256: /256 0b1001 I2CMDIV_512: /512 0b1010 I2CMDIV_1024: /1,024 0b1011 I2CMDIV_2048: /2,048 0b1100 I2CMDIV_4096: /4,096 0b1101 I2CMDIV_8192: /8,192 0b1110 I2CMDIV_16384: /16,384 0b1111 I2CMDIV_32768: /32,768

Bits	Field	Type	Reset	Description
12	SAIE	rw	0	I2C slave mode addressed interrupt enable 0: Interrupt disabled 1: Interrupt enabled
11	STXEIE	rw	0	I2C slave transmit register empty interrupt enable 0: Interrupt disabled 1: Interrupt enabled
10	SOVFIE	rw	0	I2C slave receive register overflow interrupt enable 0: Interrupt disabled 1: Interrupt enabled
9	SNRIE	rw	0	I2C slave mode NACK received from master interrupt enable 0: Interrupt disabled 1: Interrupt enabled
8	SXCIE	rw	0	I2C slave mode transfer complete interrupt enable 0: Interrupt disabled 1: Interrupt enabled
7	MSTSIE	rw	0	I2C master mode start condition sent interrupt enable 0: Interrupt disabled 1: Interrupt enabled
6	MSPSIE	rw	0	I2C master mode stop condition sent interrupt enable 0: Interrupt disabled 1: Interrupt enabled
5	MARBIE	rw	0	I2C master mode arbitration error interrupt enable 0: Interrupt disabled 1: Interrupt enabled
4	MTXEIE	rw	0	I2C master transmit register empty interrupt enable 0: Interrupt disabled 1: Interrupt enabled
3	MNRIE	rw	0	I2C master mode NACK received interrupt enable 0: Interrupt disabled 1: Interrupt enabled
2	MXCIE	rw	0	I2C master transfer complete interrupt enable 0: Interrupt disabled 1: Interrupt enabled
1	STRIE	rw	0	I2C start received interrupt enable 0: Interrupt disabled 1: Interrupt enabled
0	SPRIE	rw	0	I2C stop received interrupt enable 0: Interrupt disabled 1: Interrupt enabled

20.7.2 I2CxFCR: I2C flow control register

Offset 0x04 **Reset** 0x00 **Width** 8 **Address** 0x4E04 (I2C0), 0x4F04 (I2C1)
I2C flow control register. Writing a 1 to a bit in this register initiates or queues the associated command. Writing a 0 to a bit does nothing. Reading this register always returns the value 0.

Bits	Field	Type	Reset	Description
7:4	-	-	-	Reserved. Reads zero; write zero.
3	SC	w1	0	I2C slave continue command. When clock stretching is enabled in slave mode, set this bit to tell the slave to continue with the ACK/NACK phase of the current byte by releasing SCL. This may only be set if clock stretching is enabled, slave mode is enabled, and the slave transfer complete flag has just been set. 0: No effect 1: Continue: release SCL for the ACK/NACK phase
2	MST	w1	0	Master send start command. Writing 1 sends a start condition, or a repeated start if this master already controls the bus, after which at least one address frame must be sent before the command is used again. If the bus is busy the start is sent once it becomes idle, and if this master is mid-transaction the repeated start follows the transaction. 0: No effect 1: Send a start condition
1	MSP	w1	0	Master send stop command. Writing 1 sends a stop condition once this master has sent a start condition and at least one address frame. If this master is mid-transaction the stop follows the transaction. 0: No effect 1: Send a stop condition
0	MRB	w1	0	I2C master read byte command. Set this bit to 1 while in master receiver mode to read one byte from the slave. This master is required to have already sent the slave address and received an ACK from the slave before initiating this command. 0: No effect 1: Read next byte

20.7.3 I2CxSR: I2C status register

Offset 0x08 **Reset** 0x0000 **Width** 16 **Address** 0x4E08 (I2C0), 0x4F08 (I2C1)

Bits	Field	Type	Reset	Description
15	BS	r	0	I2C bus state indicator. This bit cannot be cleared by writing to the status register. 0: The I2C bus is idle 1: The I2C bus is active
14	MCB	r	0	I2C master controls bus indicator. This bit cannot be cleared by writing to the status register. 0: This master does not control the bus 1: This master controls the bus
13	STM	r	0	Slave transmitter mode indicator: the mode this slave was addressed for, valid only while I2CSA = 1. Read only; a status register write does not clear it. 0: Slave receiver mode 1: Slave transmitter mode

Bits	Field	Type	Reset	Description
12	SA	rw1	0	I2C slave mode addressed interrupt flag. Indicates that this slave has been addressed by another master. Write a 1 to this bit to clear it. 0: No pending interrupt 1: Pending interrupt
11	STXE	rw1	0	Slave transmit register empty interrupt flag, set when the slave latches the byte held in I2CxSTX for transmission, so the register can take the next byte. Write 1 to clear. 0: No pending interrupt 1: Pending interrupt
10	SOVF	rw1	0	I2C slave receive register overflow interrupt flag. Indicates that this slave has failed to read one or more bytes from the I2CxSRX register before they were overwritten by another transmission. Write a 1 to this bit to clear it. 0: No pending interrupt 1: Pending interrupt
9	SNR	rw1	0	Slave NACK received interrupt flag, set in slave transmitter mode when the master answers a transmitted byte with a NACK; an ACK does not set it. Write 1 to clear. 0: No pending interrupt 1: Pending interrupt
8	SXC	rw1	0	I2C slave mode transfer complete interrupt flag. This bit is set after this slave receives a byte of data from a master, but before this slave sends an ACK/NACK. Write a 1 to this bit to clear it. 0: No pending interrupt 1: Pending interrupt
7	MSTS	rw1	0	I2C master mode start condition sent interrupt flag. This flag is set after this master sends a start condition or repeated start condition. Write a 1 to this bit to clear it. 0: No pending interrupt 1: Pending interrupt
6	MSPS	rw1	0	I2C master mode stop condition sent interrupt flag. This flag is set after this master sends a stop condition. Write a 1 to this bit to clear it. 0: No pending interrupt 1: Pending interrupt
5	MARB	rw1	0	Master arbitration loss interrupt flag, set when the value this master drives on SDA is overridden by another master; this master then releases the bus. Write 1 to clear. 0: No pending interrupt 1: Pending interrupt
4	MTXE	rw1	0	I2C master transmit register empty interrupt flag. This bit is set when this master latches the data stored in the master transmit register to indicate that the master transmit register is ready to accept another byte and queue it for transmission. Write a 1 to this bit to clear it. 0: No pending interrupt 1: Pending interrupt

Bits	Field	Type	Reset	Description
3	MNR	rw1	0	Master NACK received interrupt flag, set in master transmitter mode when the slave answers with a NACK; an ACK does not set it. Write 1 to clear. 0: No pending interrupt 1: Pending interrupt
2	MXC	rw1	0	Master transfer complete interrupt flag, set in master transmitter mode after the data byte and the slave ACK/NACK, and in master receiver mode after the data byte and before this master sends its ACK/NACK. Write 1 to clear. 0: No pending interrupt 1: Pending interrupt
1	STR	rw1	0	I2C start condition received interrupt flag. This flag is set whenever a start condition or repeated start condition is detected on the bus, regardless of if this master or another sent it. Write a 1 to this bit to clear it. 0: No pending interrupt 1: Pending interrupt
0	SPR	rw1	0	I2C stop condition received interrupt flag. This flag is set whenever a stop condition condition is detected on the bus, regardless of which device sent it. Write a 1 to this bit to clear it. 0: No pending interrupt 1: Pending interrupt

20.7.4 I2CxMTX: I2C master transmit register

Offset 0x0C **Reset** 0x00 **Width** 8 **Address** 0x4E0C (I2C0), 0x4F0C (I2C1)
I2C master transmit register. Write the desired slave address and read/write bit to this register after sending a start condition to begin a transmission with a slave. Note that the desired slave address must occupy the upper seven bits and the read/write bit must occupy the least significant bit. If the read bit is 0, the master enters master transmitter mode. If the read bit is 1, the master enters master receiver mode. Write a byte of data to this register after sending the address frame or a data frame to send that byte of data to the slave. If this master is busy with a transmission when this register is written, it will send the byte after it finishes the transmission.

Bits	Field	Type	Reset	Description
7:0	xMTX	rw	0x00	Master transmit data. While I2CMEN = 1 a write loads the byte and starts sending it: the first byte after a start condition is the 7-bit slave address in bits 7:1 with the read/write bit in bit 0, and later bytes are data. Reads return the last byte written; resets to 0.

20.7.5 I2CxMRX: I2C master receive register

Offset 0x10 **Reset** 0x00 **Width** 8 **Address** 0x4E10 (I2C0), 0x4F10 (I2C1)
I2C master receive register. When in master receiver mode, read this register after the master transfer complete interrupt flag (I2CMXC) is set to get the received data byte.

Bits	Field	Type	Reset	Description
7:0	xMRX	r	0x00	Master receive data. Hardware loads the byte received from the slave when I2CMXC is set in master receiver mode. Resets to 0.

20.7.6 I2CxSTX: I2C slave transmit register

Offset 0x14 **Reset** 0x00 **Width** 8 **Address** 0x4E14 (I2C0), 0x4F14 (I2C1)
I2C slave transmit register. When in slave transmitter mode, write to this register after the slave addressed flag (I2CSA) or the slave transaction complete flag (I2CSXC) has been set to queue the next byte to send to the master.

Bits	Field	Type	Reset	Description
7:0	xSTX	rw	0x00	Slave transmit data. The byte written here is sent to the master during the next slave transmitter data phase, after which I2CSTXE is set. Resets to 0.

20.7.7 I2CxSRX: I2C slave receive register

Offset 0x18 **Reset** 0x00 **Width** 8 **Address** 0x4E18 (I2C0), 0x4F18 (I2C1)
I2C slave receive register. When in slave receiver mode, read this register after the slave transaction complete flag (I2CSXC) has been set to get the data byte sent from the master. Note that if this slave fails to clear the status register before another byte is received, the slave receive overflow flag (I2CSOVF) will be set.

Bits	Field	Type	Reset	Description
7:0	xSRX	r	0x00	Slave receive data. Hardware loads the byte received from the master when I2CSXC is set in slave receiver mode; a new byte arriving before the previous one is read sets I2CSOVF. Resets to 0.

20.7.8 I2CxAR: I2C this slave address register

Offset 0x1C **Reset** 0x00 **Width** 8 **Address** 0x4E1C (I2C0), 0x4F1C (I2C1)
I2C this slave address register. When in slave mode, any master that sends an address frame containing this address will activate this slave.

Bits	Field	Type	Reset	Description
7	-	-	-	Reserved. Reads zero; write zero.
6:0	xAR	rw	0x00	Own 7-bit slave address. A received address frame that matches this value under the I2CxAMR mask addresses this slave. Resets to the default slave address of the instance (0x79 for I2C0, 0x23 for I2C1).

20.7.9 I2CxAMR: I2C this slave address mask register

Offset 0x20 **Reset** 0x00 **Width** 8 **Address** 0x4E20 (I2C0), 0x4F20 (I2C1)
I2C this slave address mask register. Any bit set to 1 in this register indicates that the corresponding bit in the slave address register is a wildcard. Only the slave address register bits that correspond to 0s in this register will be compared to the received slave address.

Bits	Field	Type	Reset	Description
7	-	-	-	Reserved. Reads zero; write zero.
6:0	xAMR	rw	0x00	Slave address mask. A 1 in bit i makes bit i of I2CxAR a wildcard for address matching; a 0 compares the bit. Resets to 0 (every bit compared).

21 Core-local interruptor (CLINT)

21.1 Overview

The Core-Local Interruptor (CLINT) provides the two interrupt sources on which multi-core software is built: a per-hart software interrupt, used as the inter-processor interrupt (IPI), and a per-hart timer interrupt driven by one shared free-running 64-bit counter. The block sits in the shared window behind the multi-core arbiter, so any hart can raise, clear or program any hart's interrupts.

- One software-interrupt register per hart (MSIP0 to MSIP4), writable by any hart
- One shared free-running 64-bit counter (MTIMEL/MTIMEH), incrementing once per MCLK cycle
- One 64-bit compare register per hart (MTIMECMP0L/MTIMECMP0H to MTIMECMP4L/MTIMECMP4H)
- Level-sensitive delivery on two hardwired vectors per hart: 83 (software) and 84 (timer)
- Wakes a hart parked by the sleep instruction, so parked harts consume no dynamic power
- Compare registers reset to all ones, so no timer interrupt fires before one is programmed

21.2 Instances and signals

Table 21.1: CLINT instances

Instance	Base address	Interrupt vector(s)	Pins (AF)
CLINT	0x5000	83 to 84	none

Table 21.2: CLINT interrupt vectors

Vector	Instance	Description	Clear method
83	CLINT	CLINT software interrupt (IPI) IRQB_CLINT_MSIP	Write 0 to the MSIPn register of the hart
84	CLINT	CLINT timer interrupt IRQB_CLINT_MTIP	Write MTIMECMPn of the hart to a value above mtime

21.3 Functional description

21.3.1 Software interrupts

Writing 1 to MSIPh asserts the software-interrupt level into hart *h*; writing 0 deasserts it. The interrupt is a level, so the handler on hart *h* must write 0 to its own MSIPh before returning, because the level would otherwise re-enter the handler immediately.

The software interrupt is the wake mechanism for a parked hart: a hart that has enabled vector 83 and executed sleep resumes when any hart writes 1 to its MSIPh. The woken hart's handler clears the level and executes wake before returning, because a handler that returns without wake puts the hart back to sleep (Section 16.3.3). The boot ROM uses this same path to launch a freshly loaded tile (Section 5.3).

21.3.2 Timer interrupts

The counter is one 64-bit value shared by all five harts and clocked by the free-running MCLK. Each hart owns one 64-bit compare value; the timer level into hart h is asserted while the counter is greater than or equal to hart h's compare value, and it is deasserted by moving the compare value into the future or back to all ones (the disarmed reset value).

The registers are 32 bits wide, so every 64-bit value is a low/high pair. The counter cannot be read atomically; Section 21.5 gives the read sequence that tolerates a carry between the two halves.

21.3.3 Clocking

The CLINT runs on MCLK and is never clock-gated, because a hart whose own clock is stopped by `sleep` can only be woken by logic that keeps running. The counter therefore advances through every sleep state, and its rate follows the MCLK divider in CLKDIVCR.

21.3.4 Address decoding

The block occupies shared-window page 1 from `0x5000`. MSIPh is at `0x5000` + $4h$, the counter pair starts at `0x5020`, and hart h's compare pair is at `0x5030` + $8h$. Only the low address bits are decoded, so the registers alias every 128 bytes up to `0x5FFF`; software uses the canonical addresses because the alias pattern is not part of the register interface and may change with the hart count.

21.4 Interrupts and events

Both vectors are hardwired into every hart's interrupt controller and do not pass through the IRQROUTER: each hart receives only its own MSIPh and its own compare match. Table 21.3 lists them.

Vector	Condition	Set by	Cleared by
83	Software interrupt to hart h	Any hart writing 1 to MSIPh	Writing 0 to MSIPh
84	Timer match for hart h	Counter reaching hart h's compare value	Writing a compare value above the counter

Table 21.3: CLINT interrupt vectors.

21.5 Programming

21.5.1 Reading the counter

1. Read MTIMEH.
2. Read MTIMEL.
3. Read MTIMEH again; if it differs from the first read, repeat from step 1, because the low half wrapped between the two reads.

21.5.2 Arming a timer interrupt for hart h

1. Write all ones to MTIMECMPhH, so that no intermediate compare value can be below the counter.
2. Write the low half of the target time to MTIMECMPhL.
3. Write the high half of the target time to MTIMECMPhH.
4. Enable vector 84 in hart h's interrupt controller.

21.5.3 Sending a software interrupt to hart h

1. Write 1 to MSIPh.

21.6 Usage notes

Clear the software-interrupt level before returning. The level stays asserted until MSIPh is written 0; a handler that returns first is re-entered at once.

Execute wake in the handler of a sleeping hart. A hart woken by an interrupt returns to sleep after the handler unless the handler executed wake (Section 16.3.3).

Program the compare pair high half first. Writing the low half first can momentarily produce a compare value below the counter and raise a spurious timer interrupt.

Do not send a software interrupt to a hart whose loader mailbox row is incomplete. The boot ROM consumes the row when the parked hart wakes, so the row must be written before MSIPh (Section 5.3).

21.7 Registers

Register offsets in this chapter are relative to the instance base address in Table 21.1 (CLINT 0x5000); the generated header names each base CLINT_BASE.

Table 21.4: CLINT register summary

Offset	Name	Access	Reset	Description
0x00 + 4n	MSIPn (n = 0..4)	rw	0x00000000	Hart n software interrupt (IPI) register
0x14 to 0x1F	-	-	-	Reserved
0x20	MTIMEL	rw	0x00000000	Machine time counter, lower 32 bits
0x24	MTIMEH	rw	0x00000000	Machine time counter, upper 32 bits
0x28 to 0x2F	-	-	-	Reserved
0x30 + 8n	MTIMECMPnL (n = 0..4)	rw	0xFFFFFFFF	Hart n timer compare register, lower 32 bits
0x34 + 8n	MTIMECMPnH (n = 0..4)	rw	0xFFFFFFFF	Hart n timer compare register, upper 32 bits

21.7.1 MSIPn (n = 0..4): Hart n software interrupt (IPI) register

Offset 0x00 + 4n **Reset** 0x00000000 **Width** 32

Hart n software interrupt (IPI) register. Any hart may write it through the shared window: writing 1 raises the software interrupt level into hart n (interrupt vector 83); writing 0 clears it. The interrupt is level-sensitive, so hart n's service routine must write 0 here before returning.

Bits	Field	Type	Reset	Description
31:1	-	-	-	Reserved. Reads zero; write zero.
0	MSIPHn	rw	0	Software interrupt (IPI) level for hart n. 0: No software interrupt pending 1: Software interrupt raised

21.7.2 MTIMEL: Machine time counter, lower 32 bits

Offset 0x20 **Reset** 0x00000000 **Width** 32 **Address** 0x5020

Machine time counter, lower 32 bits. mtime is a free-running 64-bit counter shared by all five harts that increments once per MCLK cycle. It is writable for initialization; a write merges only the addressed 32-bit half.

Bits	Field	Type	Reset	Description
31:0	MTIMEL	rw	0x00000000	mtime bits 31:0.

21.7.3 MTIMEH: Machine time counter, upper 32 bits

Offset 0x24 **Reset** 0x00000000 **Width** 32 **Address** 0x5024

Machine time counter, upper 32 bits. Read MTIMEH, then MTIMEL, then MTIMEH again (retry if the two MTIMEH reads differ) to obtain a coherent 64-bit time value.

Bits	Field	Type	Reset	Description
31:0	MTIMEH	rw	0x00000000	mtime bits 63:32.

21.7.4 MTIMECMPnL (n = 0..4): Hart n timer compare register, lower 32 bits

Offset 0x30 + 8n **Reset** 0xFFFFFFFF **Width** 32

Hart n timer compare register, lower 32 bits. The timer interrupt level for hart n (interrupt vector 84) is raised while mtime >= mtimecmpn. Resets to all-ones (no interrupt). To set a new compare value without a spurious interrupt, first write all-ones to MTIMECMPnH, then the new lower half, then the real upper half.

Bits	Field	Type	Reset	Description
31:0	MTIMECMPnL	rw	0xFFFFFFFF	mtimecmpn bits 31:0.

21.7.5 MTIMECMPnH (n = 0..4): Hart n timer compare register, upper 32 bits

Offset 0x34 + 8n **Reset** 0xFFFFFFFF **Width** 32

Bits	Field	Type	Reset	Description
31:0	MTIMECMPnH	rw	0xFFFFFFFF	mtimecmpn bits 63:32.

22 Hardware mutexes (MUTEX)

22.1 Overview

The hardware mutex bank (MUTEX) provides sixteen word-mapped advisory locks with single-instruction cross-hart mutual exclusion. A claim is one load and a release is one store; there is no reservation state and no retry loop in hardware. Atomicity comes from the multi-core arbiter, which serializes whole shared-window transactions, so two harts reading the same free mutex in the same cycle are still served one after the other.

- 16 independent mutexes, one word each (MUTEX0 to MUTEX15); the readback is a 4-bit owner marker in bits 3:0 and bits 31:4 read 0
- Claim with one load: reading a free mutex returns 0 and claims it in the same transaction
- Reading a held mutex returns the owner's marker (mhartid + 1) and leaves it held
- Release with one store of 0; any hart may release, so a supervisor can recover a dead hart's lock
- Nonzero writes are ignored, so ownership cannot be forged
- All mutexes reset to free

22.2 Instances and signals

Table 22.1: MUTEX instances

Instance	Base address	Interrupt vector(s)	Pins (AF)
MUTEX	0x6000	none	none

22.3 Functional description

22.3.1 Claim and release

Each mutex word holds either 0 (free) or the marker of the hart that holds it, mhartid + 1. The marker is four bits wide (the arbiter's granted-master width plus one), so it occupies bits 3:0 and bits 31:4 always read 0; the release comparison is against the full 32-bit written word, so only a store of exactly 0 frees a mutex. A read of a free mutex returns 0 and, in the same arbiter transaction, writes the reading hart's marker into the word; the arbiter attributes the read to the requesting hart, which is how the bank knows whose marker to store. A read of a held mutex returns the holder's marker and changes nothing, so a hart that reads its own mutex twice sees its own marker and is not disturbed. Figure 22.1 shows two harts racing for one mutex.

A store of 0 frees the mutex. The store is deliberately not qualified by owner, so that a supervising hart can release the mutex of a hart that has died while holding it; correct software releases only mutexes it holds. A store of any nonzero value is ignored, because the only legitimate nonzero content is a marker the hardware writes itself.

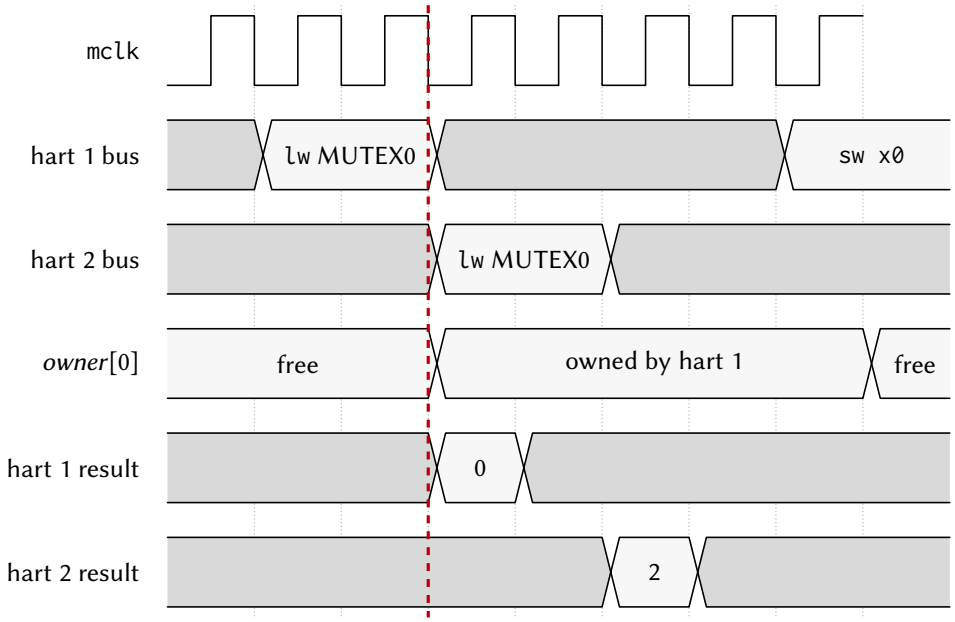


Figure 22.1: Two harts racing for the same mutex: the first read returns 0 and takes the word, the second returns the holder's marker (`mhartid` +1) and must retry, and the holder releases with `sw x0`.

22.3.2 Address decoding

The bank occupies shared-window page 2 from `0x6000`, with mutex n at `0x6000`$+4n$. Only the low address bits are decoded, so the words alias throughout the page; software uses the canonical addresses.

22.4 Interrupts and events

Not applicable. The bank raises no interrupt; a hart that fails a claim retries under software control.

22.5 Programming

22.5.1 Claiming mutex n

1. Load `MUTEXn` with a plain `lw`.
2. If the value is 0, the mutex is held by this hart; otherwise the value is the holder's marker and the claim failed.
3. On failure, delay by an amount proportional to mhartid + 1 and repeat from step 1, with a bounded retry count.

22.5.2 Releasing mutex n

1. Store 0 to `MUTEXn` with a plain `sw`.

22.6 Usage notes

Treat the locks as advisory. The hardware does not block a hart that accesses a protected resource without claiming its mutex; the protocol is enforced by software only.

Never use `lr.w`, `sc.w` or an AMO on a mutex address. A suppressed store-conditional reaches the bank as an ordinary read and claims the mutex, and an AMO performs a read and a write whose values the bank does not expect. Only `lw` and `sw` are legal (Section 5.4).

Back off between retries. five harts re-reading a contested mutex in lockstep waste arbiter bandwidth and can starve useful work; scale the delay with the hart index and bound the retry count.

Release only what this hart holds. The bank does not check the releasing hart; a stray store of 0 frees another hart's lock silently.

22.7 Registers

Register offsets in this chapter are relative to the instance base address in Table 22.1 (`MUTEX 0x6000`); the generated header names each base `MUTEX_BASE`.

Table 22.2: `MUTEX` register summary

Offset	Name	Access	Reset	Description
`0x00 + 4n`	`MUTEXn (n = 0..15)`	`rw`	`0x00000000`	Hardware mutex n

22.7.1 `MUTEXn (n = 0..15)`: Hardware mutex n

Offset `0x00 + 4n` **Reset** `0x00000000` **Width** 32

Hardware mutex n. Read to claim: a returned value of 0 means the mutex was free and the reading hart now holds it; a nonzero value is the current owner's marker (hartid+1). Write 0 to release.

Bits	Field	Type	Reset	Description
31:4	-	-	-	Reserved. Reads zero; write zero.
3:0	`OWNn`	`rw`	`0b0000`	Owner marker: 0 = free (and a read that returns 0 claims the mutex), h+1 = held by hart h. `0b0000` `MTXOWNn_FREE`: Mutex free (a read returning this value claims the mutex) `0b0001` `MTXOWNn_H0`: Held by hart 0 `0b0010` `MTXOWNn_H1`: Held by hart 1 `0b0011` `MTXOWNn_H2`: Held by hart 2 `0b0100` `MTXOWNn_H3`: Held by hart 3 `0b0101` `MTXOWNn_H4`: Held by hart 4

23 Improved inter-integrated circuit interface (I3Cx)

23.1 Overview

The I3C controller (I3Cx) is a single-controller MIPI I3C Basic interface for a two-wire (SDA, SCL) sensor bus. It acts as the active bus controller and interoperates with legacy I2C targets sharing the same wires. Transactions are described by the control and command registers and launched by a single register write; the registered read path returns snapshots with no read side effects. The serial core runs in the SMCLK domain (the SYSCLKCR = 0 rule applies), with independent open-drain and push-pull baud dividers. The main features of the I3C controller are listed below:

- Single-byte SDR private read and write transfers, with repeated-START chaining (I3CREPSTART, I3CSTOPEN)
- Legacy-I2C interoperability on the same bus (I3CBUSMODE), including a forced open-drain data mode (I3CSDAPP)
- Common Command Code (CCC) transactions, broadcast or direct (I3CCCC, I3CCCCDIR, I3CCCCOP)
- Hardware Dynamic Address Assignment: ENTDAA (I3CDAARUN) and SETDASA (I3CDASA), driven from a four-entry Device Address Table
- In-Band Interrupt (IBI) acceptance with optional mandatory-data-byte capture (I3CIBIEN, I3CxIBI)
- Independent open-drain and push-pull baud dividers off SMCLK: SCL = SMCLK $/(2\times(1+\texttt{I3CODBR}))$ (open drain) and $/(2\times(1+\texttt{I3CPPBR}))$ (push-pull)
- Registered read path: register reads have *no* side effects (reading I3CxRX clears nothing)
- Eight interrupt sources: transfer complete, receive full, transmit empty, address/byte NACK, early end-of-data, arbitration lost, DAA done, and IBI pending

23.2 Instances and signals

Table 23.1: I3Cx instances

Instance	Base address	Interrupt vector(s)	Pins (AF)
I3C0	0x6100	86 to 93	I3C_SDA P4.6 (AF1), I3C_SCL P4.7 (AF1)

Table 23.2: I3Cx signals

Signal	Direction	Description	Pin (AF)
I3C_SDA	Bidirectional	I3C0 serial data, open-drain	P4.6 (AF1)
I3C_SCL	Bidirectional	I3C0 serial clock, open-drain	P4.7 (AF1)

Table 23.3: I3Cx interrupt vectors

Vector	Instance	Description	Clear method
86	I3C0	I3C0 Transfer Complete Interrupt IRQB_I3C0_TC	See chapter
87	I3C0	I3C0 Receive-Register Full Interrupt IRQB_I3C0_RXF	See chapter
88	I3C0	I3C0 Transmit-Register Empty Interrupt IRQB_I3C0_TXE	See chapter
89	I3C0	I3C0 Address / Byte NACK Interrupt IRQB_I3C0_NACK	See chapter
90	I3C0	I3C0 Early End-of-Data Interrupt IRQB_I3C0_EOD	See chapter
91	I3C0	I3C0 Arbitration-Lost Interrupt IRQB_I3C0_ARB	See chapter
92	I3C0	I3C0 Dynamic-Address-Assignment Done Interrupt IRQB_I3C0_DAA	See chapter
93	I3C0	I3C0 In-Band Interrupt Pending Interrupt IRQB_I3C0_IBI	See chapter

23.3 Functional description

23.3.1 I3C Pins

The I3Cx peripheral drives two open-drain / push-pull bidirectional pins: the serial data line SDA and the serial clock SCL. Each is a tri-state pad (an output value with a direction control and an input); the controller releases the line (high-impedance, pulled high by the bus) when it is not actively driving. The open-drain phases (the arbitrated address header, legacy-I2C data, and IBI arbitration) run at the I3CODBR rate; SDR data runs push-pull at the I3CPPBR rate once the bus is in push-pull mode.

23.3.2 Dynamic Address Assignment and In-Band Interrupts

For Dynamic Address Assignment, program the desired dynamic addresses into the Device Address Table entries (select an entry with I3CIDX, then write I3CDYNADDR / I3CDYNVALID in I3CxDAT), then issue an ENTDAA transaction (I3CCCC = 1, I3CCCCOP = 0x07, I3CDAARUN = 1). The controller runs the arbitrated discovery loop, captures each discovered device's provisional ID and characteristic registers into a DAT entry (I3CxDATPID, I3CxDATINFO), assigns the programmed dynamic address, and sets I3CDAADONE. SETDASA (I3CDASA) assigns a dynamic address to a device already known by its static address.

When I3CIBIEN is set, the controller ACKs a target-initiated In-Band Interrupt, captures it into I3CxIBI (the requesting device's address, its optional mandatory data byte, and the ACK result), and sets I3CIBIP. The handler reads I3CxIBI, services the device, and clears I3CIBIP (write 1), which releases the snapshot for the next IBI.

23.4 Interrupts and events

The status flags I3CTCIF, I3CRXFULL, I3CTXEIF, I3CANACK, I3CEODF, I3CARBLOST, I3CDAADONE, I3CDAAFULL, and I3CIBIP are write-1-to-clear: write a 1 to a bit to clear it.

The eight interrupt sources occupy vectors 86 through 93 (transfer complete, receive full, transmit empty, address/byte NACK, early end-of-data, arbitration lost, DAA done, and IBI pending, in that order), delivered through the interrupt router like every other shared-peripheral source. These sources sit above the external-interrupt (meip) delivery vector, which stays fixed at vector 85.

23.5 Programming

A transaction is fully described by the control and command registers. The launch sequence is:

1. Program I3CxCR with the bus mode, drive style, open-drain and push-pull baud dividers, and the interrupt enables, and set I3CEN.
2. For a write-direction transfer, place the first byte in I3CxTX (this arms the per-byte transmit handshake; writing I3CxTX never launches a transaction).
3. Write I3CxCMD last, with the target address, direction, framing, and (for CCC / DAA) the operation. The byte-lane-0 write of I3CxCMD *launches* the transaction; the content is always captured, but the launch is suppressed while the controller is disabled (I3CEN = 0) or busy (I3CBUSY = 1).
4. Poll I3CBUSY (or wait for the transfer-complete interrupt). For a read, each captured byte appears in I3CxRX with I3CRXFULL set; reading I3CxRX has no side effects.

23.6 Usage notes

Because the serial core runs in the SMCLK domain, software must set SYSCLKCR to 0 (SMCLK from HFXT) before using the controller, exactly as for the SPI, UART, and I2C peripherals.

In configurations that include it, I3C0 lives in the shared peripheral window behind the multi-core arbiter (see Section 5) in the mutex-bank page, sub-slot 1 at 0x6100, so any hart can use it.

23.7 Registers

Register offsets in this chapter are relative to the instance base address in Table 23.1 (I3C0 0x6100); the generated header names each base I3C0_BASE.

Table 23.4: I3Cx register summary

Offset	Name	Access	Reset	Description
0x00	I3CxCR	rw	0x00000004	I3C control register
0x04	I3CxCMD	rw	0x00000000	I3C command register
0x08	I3CxTX	rw	0x00000000	I3C transmit data
0x0C	I3CxRX	r	0x00000000	I3C receive data
0x10	I3CxSR	r/rw1	0x00000000	I3C status register
0x14	I3CxDAT	rw	0x00000000	Device Address Table window

Offset	Name	Access	Reset	Description
0x18	I3CxDATPID	rw	0x00000000	Provisional ID (low 32 bits) of the DAT entry selected by I3CIDX
0x1C	I3CxDATINFO	rw	0x00000000	
0x20	I3CxIBI	r	0x00000000	In-band interrupt capture

23.7.1 I3CxCR: I3C control register

Offset 0x00 **Reset** 0x00000004 **Width** 32 **Address** 0x6100

I3C control register. Configures the bus mode, SDA drive style, open-drain and push-pull baud dividers, and the interrupt enables. Reconfigure only while the controller is idle (I3CBUSY = 0); the mode and drive bits are additionally latched at each transaction launch. Resets to 0 except I3CSDAPP, which resets to 1.

Bits	Field	Type	Reset	Description
31:30	-	-	-	Reserved. Reads zero; write zero.
29	TXEIE	rw	0	Transmit-empty interrupt enable (interrupt vector 88). 0: Interrupt disabled 1: Interrupt enabled
28	RXFIE	rw	0	Receive-full interrupt enable (interrupt vector 87). 0: Interrupt disabled 1: Interrupt enabled
27	IBIIE	rw	0	In-band interrupt pending interrupt enable (interrupt vector 93). 0: Interrupt disabled 1: Interrupt enabled
26	DAAIE	rw	0	Dynamic-address-assignment done interrupt enable (interrupt vector 92). 0: Interrupt disabled 1: Interrupt enabled
25	ERRIE	rw	0	Error interrupt enable: gates the address-NACK (vector 89), early-end-of-data (vector 90), and arbitration-lost (vector 91) sources. 0: Interrupt disabled 1: Interrupt enabled
24	TCIE	rw	0	Transfer-complete interrupt enable (interrupt vector 86). 0: Interrupt disabled 1: Interrupt enabled
23:16	PPBR	rw	0x00	Push-pull baud divider. The push-pull SCL rate is SMCLK / (2 * (1 + I3CPPBR)); used for SDR data once the bus is in push-pull. SMCLK is the SYSTEM clock source (write SYS_CLK_CR = 0 to run SMCLK from HFXT before using the controller).
15:8	ODBR	rw	0x00	Open-drain baud divider. The open-drain SCL rate is SMCLK / (2 * (1 + I3CODBR)); used for the arbitrated address header and legacy-I2C phases.
7:4	-	-	-	Reserved. Reads zero; write zero.
3	IBIEN	rw	0	In-band interrupt accept enable. When set, the controller ACKs a target-initiated IBI and captures it into I3CxIBI; when clear, IBIs are NACKed. 0: IBIs NACKed 1: IBIs accepted

Bits	Field	Type	Reset	Description
2	SDAPP	rw	1	SDA drive style for SDR data, latched at launch (resets to 1). When 0 the data phase is forced open-drain (required for legacy-I2C targets). 0: Force open-drain 1: Push-pull SDR data
1	BUSMODE	rw	0	Bus mode, latched at launch. Selects the framing for the next transaction. 0: I3C SDR 1: Legacy I2C
0	EN	rw	0	Controller enable. When 0 the serial core is held in reset (no transaction launches and the serial pins are released) but the RX register, DAT table, and status flags are preserved. Set to 1 before launching a transaction. 0: Disabled (serial core in reset) 1: Enabled

23.7.2 I3CxCMD: I3C command register

Offset 0x04 **Reset** 0x00000000 **Width** 32 **Address** 0x6104

I3C command register. Describes the next transaction: target address, direction, START/STOP framing, optional CCC and dynamic-address-assignment operations, and the data length. Writing this register with byte lane 0 asserted LAUNCHES the transaction; the content is always captured, but the launch is suppressed when I3CEN = 0 or I3CBUSY = 1. Program I3CxCR (and, for a write, I3CxTX) first, then write I3CxCMD last. All command and control fields are latched at launch.

Bits	Field	Type	Reset	Description
31:24	CCCOP	rw	0x00	CCC opcode (used when I3CCCC = 1; e.g. 0x07 = ENTDAA).
23:16	DLEN	rw	0x00	Data length, 0 to 255 bytes. The number of payload bytes the data phase transfers.
15:14	-	-	-	Reserved. Reads zero; write zero.
13	DASA	rw	0	Run SETDASA (set dynamic address from static address) for the addressed DAT entry. 0: No SETDASA 1: Run SETDASA
12	DAARUN	rw	0	Run ENTDAA dynamic address assignment. When set (with I3CCCC and I3CCCCOP = 0x07), the controller runs the ENTDAA loop, assigning the dynamic addresses programmed in the DAT entries to newly discovered targets. 0: No ENTDAA 1: Run ENTDAA
11	CCCDIR	rw	0	CCC direction. 0: Broadcast CCC 1: Direct CCC

Bits	Field	Type	Reset	Description
10	CCC	rw	0	Common Command Code transaction. When set, the transaction is a CCC (0x7E broadcast auto-prepended; the opcode is in I3CCCCOP). 0: Private transfer 1: CCC
9	STOPEN	rw	0	STOP enable. 0: Hold the bus at end (for a following repeated START) 1: Issue STOP at end
8	REPSTART	rw	0	Repeated-START (Sr) entry. When set, the transaction begins with a repeated START instead of a START, chaining onto a bus previously held (see I3CSTOPEN). 0: START 1: Repeated START
7	RNW	rw	0	Direction. 0: Write (controller drives I3CxTX out) 1: Read (controller captures into I3CxRX)
6:0	ADDR	rw	0x00	7-bit target address. The broadcast address 0x7E is auto-prepended by hardware only when I3CCCC = 1.

23.7.3 I3CxTX: I3C transmit data

Offset 0x08 **Reset** 0x00000000 **Width** 32 **Address** 0x6108

I3C transmit data. Writing the low byte arms the per-byte transmit handshake for the data phase; writing this register never launches a transaction (only an I3CxCMD write does). The upper bits are reserved for future word-packing.

Bits	Field	Type	Reset	Description
31:8	-	-	-	Reserved. Reads zero; write zero.
7:0	TX	rw	0x00	Next write byte.

23.7.4 I3CxRX: I3C receive data

Offset 0x0C **Reset** 0x00000000 **Width** 32 **Address** 0x610C

I3C receive data. The most recently received byte. Reads have NO side effects (reading I3CxRX clears nothing). Reset-cleared to 0.

Bits	Field	Type	Reset	Description
31:8	-	-	-	Reserved. Reads zero; write zero.
7:0	RX	r	0x00	Last received byte.

23.7.5 I3CxSR: I3C status register

Offset 0x10 **Reset** 0x00000000 **Width** 32 **Address** 0x6110

I3C status register. I3CBUSY and I3CIBIWON are read-only; the event flags are write-1-to-clear (write a 1 to a bit to clear it; writing 0 leaves it unchanged).

Bits	Field	Type	Reset	Description
31:11	-	-	-	Reserved. Reads zero; write zero.
10	IBIWON	r	0	IBI-won (live). Reads 1 while a target is currently winning IBI arbitration on the bus. 0: No live IBI 1: IBI in arbitration
9	IBIP	rw1	0	In-band interrupt pending flag. Set when an accepted IBI has been captured into I3CxIBI; drives vector 93 when I3CIBIIE is set. Write 1 to clear (which also releases the I3CxIBI snapshot). 0: No event 1: IBI captured
8	DAAFULL	rw1	0	DAA capture-full flag. Set when a DAA run captured a newly discovered device into a DAT entry. Write 1 to clear. 0: No event 1: DAT entry captured
7	DAADONE	rw1	0	Dynamic-address-assignment done flag. Set when an ENTDAA/SETDASA run completes; drives vector 92 when I3CDAAIE is set. Write 1 to clear. 0: No event 1: DAA complete
6	ARBLOST	rw1	0	Arbitration-lost flag. Set when the controller loses address-header arbitration; drives vector 91 when I3CERRIE is set. Write 1 to clear. 0: No event 1: Arbitration lost
5	EODF	rw1	0	Early end-of-data flag (read). Set when a target ends a read (T = 0) before I3CDLEN bytes; drives vector 90 when I3CERRIE is set. Write 1 to clear. 0: No event 1: Early end-of-data
4	ANACK	rw1	0	Address-NACK flag. Set when a target NACKs the address (or a legacy per-byte NACK occurs); drives vector 89 when I3CERRIE is set. Write 1 to clear. 0: No event 1: Address / byte NACK
3	TXEIF	rw1	0	Transmit-empty flag. Set when the transmit path has consumed I3CxTX and can accept the next byte; drives vector 88 when I3CTXEIE is set. Write 1 to clear. 0: No event 1: Transmit register empty
2	RXFULL	rw1	0	Receive-full flag. Set when the data phase captures a fresh byte into I3CxRX; drives vector 87 when I3CRXFIE is set. Write 1 to clear. (Reading I3CxRX does NOT clear it.) 0: No event 1: Receive register full

Bits	Field	Type	Reset	Description
1	TCIF	rw1	0	Transfer-complete flag. Set when a transaction finishes; drives vector 86 when I3CTCIE is set. Write 1 to clear. 0: No event 1: Transfer complete
0	BUSY	r	0	Busy. Reads 1 while a transaction is in progress; an I3CxCMD launch is ignored while this bit is set. 0: Idle 1: Transaction in progress

23.7.6 I3CxDAT: Device Address Table window

Offset 0x14 **Reset** 0x00000000 **Width** 32 **Address** 0x6114

Device Address Table window. I3CIDX selects which of the 4 DAT entries this window (and I3CxDATPID / I3CxDATINFO) addresses; the index persists across accesses. Each entry holds a target's dynamic address, static address, and validity bits used by DAA and by private transfers.

Bits	Field	Type	Reset	Description
31:20	-	-	-	Reserved. Reads zero; write zero.
19	STATVALID	rw	0	Static address valid. 0: No static address 1: Static address present
18:12	STATADDR	rw	0x00	Static (legacy-I2C) address (7-bit) of this device, used by SETDASA.
11	DYNVALID	rw	0	Dynamic address valid. 0: No dynamic address 1: Dynamic address assigned
10:4	DYNADDR	rw	0x00	Dynamic address (7-bit) assigned to (or to assign to) this device.
3	EVALID	rw	0	Entry valid. Marks the selected DAT entry as populated. 0: Empty 1: Valid
2:0	IDX	rw	0b000	DAT entry index (0 to 3; the upper values are ignored). Selects the entry for this register, I3CxDATPID, and I3CxDATINFO.

23.7.7 I3CxDATPID: Provisional ID (low 32 bits) of the DAT entry selected by I3CIDX

Offset 0x18 **Reset** 0x00000000 **Width** 32 **Address** 0x6118

Provisional ID (low 32 bits) of the DAT entry selected by I3CIDX. During ENTDAA the controller captures the discovered device's 48-bit PID; software reads it here (and the high 16 bits from I3CxDATINFO) to identify the device.

Bits	Field	Type	Reset	Description
31:0	PIDLO	rw	0x00000000	Provisional ID bits 31:0.

23.7.8 I3CxDATINFO

Offset 0x1C **Reset** 0x00000000 **Width** 32 **Address** 0x611C

High provisional-ID bits and the bus/device characteristic registers of the DAT entry selected by I3CIDX.

Bits	Field	Type	Reset	Description
31:24	DCR	rw	0x00	Device Characteristic Register of the device (device type code).
23:16	BCR	rw	0x00	Bus Characteristic Register of the device. BCR bit 2 = 1 indicates the device's IBIs carry a mandatory data byte (captured into I3CIBIMDB).
15:0	PIDHI	rw	0x0000	Provisional ID bits 47:32.

23.7.9 I3CxIBI: In-band interrupt capture

Offset 0x20 **Reset** 0x00000000 **Width** 32 **Address** 0x6120

In-band interrupt capture. A read-only snapshot of the most recently accepted IBI: which target raised it, its optional mandatory data byte, and whether it was ACKed. The snapshot is held until the I3CIBIP flag is cleared (write 1 to I3CIBIP in I3CxSR).

Bits	Field	Type	Reset	Description
31:18	-	-	-	Reserved. Reads zero; write zero.
17	IBIACKED	r	0	IBI ACK result. 0: NACKed 1: ACKed
16	IBIHASDATA	r	0	Mandatory-data-byte present. 0: No data byte 1: I3CIBIMDB valid
15:8	IBIMDB	r	0x00	Mandatory data byte, when the device's BCR bit 2 = 1 (see I3CIBIHASDATA).
7	IBIVALID	r	0	IBI snapshot valid. 0: Empty 1: Valid capture
6:0	IBIADDR	r	0x00	Dynamic address of the target that raised the IBI.

24 Near-field communication interface (NFCx)

24.1 Overview

The NFC controller (NFCx) is an ISO/IEC 14443 Type A tag emulation engine. It presents a passive contactless tag to an external reader: it decodes the reader-to-tag frames, runs the tag transaction state machine, and load-modulates the tag's response. The 13.56 MHz analog front end is off-die; the block is digital only and connects to an external front end through a small demodulated interface.

- ISO 14443A tag emulation: REQA/WUPA to ATQA, bit-frame anticollision with a 4-byte UID to SAK, then Type-2 READ answered from a firmware-filled payload window (NFCAUTOREAD)
- Miller decoding of reader frames; Manchester subcarrier (fc/16) load modulation of tag responses
- Byte framing with odd parity and CRC_A (reflected 0x8408, initial value 0x6363)
- Provisioned 4-byte UID (NFCxUID), programmable ATQA and SAK (NFCxCFG), hardware BCC
- Two 64-byte indexed windows behind one index/data pair (NFCxIDX, NFCxDATA): the payload window and the received-frame buffer
- Programmable protocol timing (NFCxTIM): bit period, subcarrier divisor, frame delay time
- Registered read path: no register read has a side effect
- Four interrupt sources, vectors 94 to 97

24.2 Instances and signals

Table 24.1: NFCx instances

Instance	Base address	Interrupt vector(s)	Pins (AF)
NFC0	0x6200	94 to 97	NFC_RF_CLK P5.0 (AF1), NFC_RF_RX P5.1 (AF1), NFC_FIELD_DETECT P5.2 (AF1), NFC_RF_TXMOD P5.3 (AF1), NFC_RF_TX_EN P5.4 (AF1), NFC_AFE_EN P5.5 (AF1)

Table 24.2: NFCx signals

Signal	Direction	Description	Pin (AF)
NFC_RF_CLK	Input	NFC0 off-die RF carrier clock	P5.0 (AF1)
NFC_RF_RX	Input	NFC0 off-die RX Miller envelope	P5.1 (AF1)
NFC_FIELD_DETECT	Input	NFC0 off-die RF field detect	P5.2 (AF1)
NFC_RF_TXMOD	Output	NFC0 off-die TX load modulation	P5.3 (AF1)
NFC_RF_TX_EN	Output	NFC0 off-die TX enable	P5.4 (AF1)
NFC_AFE_EN	Output	NFC0 off-die AFE enable	P5.5 (AF1)

Table 24.3: NFCx interrupt vectors

Vector	Instance	Description	Clear method
94	NFC0	NFC0 RF Field-Detect Interrupt IRQB_NFC0_FIELD	Write 1 to NFC0SR.NFCFIELDF
95	NFC0	NFC0 Reader-Frame Received Interrupt IRQB_NFC0_RXF	Write 1 to NFC0SR.NFCRXFRAMEF
96	NFC0	NFC0 Tag-Response Transmit-Done Interrupt IRQB_NFC0_TXDONE	Write 1 to NFC0SR.NFCTXDONEF
97	NFC0	NFC0 RX CRC / Parity Error Interrupt IRQB_NFC0_CRCERR	Write 1 to NFC0SR.NFCCRCERRF and NFCPARERRF

24.3 Functional description

24.3.1 Analog front-end boundary

The block never touches the carrier. Its interface to the external front end is a demodulated receive envelope (1 = field present, 0 = a reader pause), an asynchronous field-present level, the carrier-derived clock rf_clk that clocks the protocol core, a load-modulation drive output, a transmit-active window, and a listen-power enable. These wires are alternate functions on port 6 (Section 3).

The block spans three clock domains: the bus clock for the register interface, a free-running SMCLK reference that hosts the synchronizers and the write-1-to-clear retirement, and rf_clk. Quasi-static configuration (UID, ATQA, SAK, timing) is latched per transaction so that a multi-byte value is never sampled while software is updating it, and received-frame bytes cross by a flag-gated handshake: the buffer is written before NFCRXFRAMEF is raised, so a handler that sees the flag can read the frame.

24.3.2 Transaction engine

With NFCEN set the protocol core runs; with NFCLISTEN also set it answers readers. The engine walks the 14443A sequence on its own: REQA or WUPA is answered with the ATQA from NFCxCFG, the anticollision cascade with the UID from NFCxUID and its hardware-derived BCC, and SELECT with the SAK. With NFCAUTOREAD set (its reset value), a Type-2 READ is answered directly from the payload window without firmware. Any other command, or every command when NFCAUTOREAD is clear, is surfaced to firmware: NFCRXFRAMEF is set, NFCxRXST reports the command byte, length and CRC/parity result, and the raw frame is readable through the frame buffer. Firmware answers with NFCxTXCTL: it fills the payload window, writes the length to NFCTXLEN, selects CRC appending with NFCTXAPPCRC, and sets NFCTXGO.

24.3.3 Indexed windows

Both 64-byte windows are reached through one index/data pair: NFCIDXSEL in NFCxIDX selects the payload window (0) or the received-frame buffer (1), NFCIDX selects the byte, and each access to NFCxDATA reads or writes it. With NFCIDXAINC set the index advances after every access, so a record is streamed with consecutive data accesses.

24.3.4 Protocol timing

NFCxTIM carries the bit period (NFCETU), the subcarrier half period (NFCSUBCDIV) and the frame delay time (NFCFDT), all counted in rf_clk ticks. The reset values follow the 13.56 MHz grid; a simulation bench

writes them small to compress protocol time.

24.4 Interrupts and events

The four flags in NFCxSR are write-1-to-clear and each has an enable in NFCxCR. Table 24.4 lists them. The field-detect level is also available to the boot gate in PWRCTRL as a wake source, which does not use a vector.

Vector	Condition	Set by	Cleared by
94	RF field detected	NFCFIELDF	Writing 1 to NFCFIELDF
95	Reader frame received for firmware	NFCRXFRAMEF	Writing 1 to NFCRXFRAMEF
96	Tag response transmit done	NFCTXDONEF	Writing 1 to NFCTXDONEF
97	Receive CRC or parity error	NFCCRCERRF, NFCPARERRF	Writing 1 to the flag

Table 24.4: NFC0 interrupt vectors.

24.5 Programming

24.5.1 Provisioning and arming the tag

1. Write SYSCLKCR so that SMCLK runs from HFXT (Section 16.3.1).
2. Set the PxSEL bits and PxAFS fields of the six front-end pins on port 6.
3. Write the UID to NFCxUID.
4. Write NFCxCFG.NFCATQA and NFCxCFG.NFCSAK.
5. Write NFCxIDX with NFCIDXSEL = 0, NFCIDX = 0 and NFCIDXAINC = 1.
6. Write each payload byte to NFCxDATA.
7. Set the interrupt enables required in NFCxCR.
8. Set NFCxCR.NFCEN and NFCxCR.NFCLISTEN.

24.5.2 Servicing a frame delivered to firmware

1. Read NFCxRXST for the command, length and CRC/parity result.
2. Write NFCxIDX with NFCIDXSEL = 1 and NFCIDXAINC = 1, then read the frame bytes from NFCxDATA.
3. Fill the payload window with the response.
4. Write NFCxTXCTL with NFCTXLEN, NFCTXAPPCRC and NFCTXGO.
5. Write 1 to NFCxSR.NFCRXFRAMEF.

24.6 Usage notes

Keep SMCLK on HFXT while the tag is armed. The synchronizers and flag retirement run on SMCLK; a stopped or slow SMCLK stalls them.

Use a byte store to arm LISTEN when the payload is auto-read. A full-word write of NFCxCR also writes the byte lane holding NFCAUTOREAD; writing 0 there disables the hardware answer to READ.

Provision before enabling. The UID, ATQA, SAK and timing are latched per transaction, so a value written while a reader is mid-transaction takes effect at the next one.

Do not read the block through an alias when it is absent. In a configuration without the NFC block its address range aliases the MUTEX page, and a read there claims a mutex.

24.7 Registers

Register offsets in this chapter are relative to the instance base address in Table 24.1 (NFC0 0x6200); the generated header names each base NFC0_BASE.

Table 24.5: NFCx register summary

Offset	Name	Access	Reset	Description
0x00	NFCxCR	rw/w1	0x00001000	NFC control register
0x04	NFCxSR	r/rw1	0x00000000	NFC status register
0x08	NFCxUID	rw	0x00000000	Provisioned single-size 4-byte tag UID, used by bit-frame anticollision
0x0C	NFCxCFG	rw	0x00000044	
0x10	NFCxTIM	rw	0x088004D4	
0x14	NFCxRXST	r	0x00000000	
0x18	NFCxIDX	rw	0x00000000	
0x1C	NFCxDATA	rw	0x00000000	The byte at NFCIDX in the window selected by NFCIDXSEL
0x20	NFCxTXCTL	rw	0x00000000	
0x24	NFCxDBG	r	0x00000000	Debug telemetry / bench cross-checks: frame counters

24.7.1 NFCxCR: NFC control register

Offset 0x00 **Reset** 0x00001000 **Width** 32 **Address** 0x6200
NFC control register. Enables the protocol core, arms the tag to respond, selects the auto-answer path, and holds the interrupt enables and the carrier-division note. Resets to 0 except NFCAUTOREAD, which resets to 1. Program the identity and timing registers before setting NFCEN.

Bits	Field	Type	Reset	Description
31:20	-	-	-	Reserved. Reads zero; write zero.
19:16	RFDIV	rw	0b0000	Carrier-division note. Documents the assumed division from the 13.56 MHz carrier to rf_clk (informational; the off-die AFE supplies rf_clk).
15:13	-	-	-	Reserved. Reads zero; write zero.

Bits	Field	Type	Reset	Description
12	AUTOREAD	rw	1	Auto-answer READ path (resets to 1). When set, hardware answers a Type-2 READ command directly from the payload window without firmware intervention. When clear, standard frames are surfaced to firmware through NFCxRXST and the RX buffer for a firmware-composed response. 0: Firmware-handled reads 1: Hardware auto-answer
11	CRCIE	rw	0	CRC / parity-error interrupt enable. When set, an RX CRC or parity error (NFCCRCERRF or NFCPARERRF) drives interrupt vector 97. 0: Interrupt disabled 1: Interrupt enabled
10	TXIE	rw	0	Transmit-done interrupt enable. When set, the NFCTXDONEF flag drives interrupt vector 96. 0: Interrupt disabled 1: Interrupt enabled
9	RXFIE	rw	0	Frame-received interrupt enable. When set, the NFCRXFRAMEF flag drives interrupt vector 95. 0: Interrupt disabled 1: Interrupt enabled
8	FIELDIE	rw	0	Field-detect interrupt enable. When set, the NFCFIELDF flag drives interrupt vector 94. 0: Interrupt disabled 1: Interrupt enabled
7:3	-	-	-	Reserved. Reads zero; write zero.
2	HALTCLR	w1	0	Halt-clear pulse. Writing 1 forces the transaction FSM from the HALT state back to IDLE (a self-clearing, write-1-style event pulse); it reads 0. 0: No action 1: Force HALT to IDLE
1	LISTEN	rw	0	Listen arm. When set, the tag responds to reader commands once a field is present; when clear, the tag stays deaf even in a field. 0: Deaf (no response) 1: Armed to respond
0	EN	rw	0	NFC enable. When 0 the rf_clk protocol core is held in reset; it does NOT wipe the UID, CFG, payload window, or status flags (disable-preserves-data rule). Set to 1 to run the tag engine. 0: Disabled (protocol core in reset) 1: Enabled

24.7.2 NFCxSR: NFC status register

Offset 0x04 **Reset** 0x00000000 **Width** 32 **Address** 0x6204

NFC status register. NFCBUSY, NFCFIELDLIVE, NFCHALTED, and NFCSTATE are read-only live status; the five event flags (bits 1-5) are write-1-to-clear (write a 1 to a bit to clear it; writing 0 leaves it unchanged). The volatile bits are captured by the registered pre-latch read.

Bits	Field	Type	Reset	Description
31:12	-	-	-	Reserved. Reads zero; write zero.
11:8	STATE	r	0b0000	Transaction FSM state: 0 = POWER_OFF, 1 = IDLE, 2 = READY, 3 = ACTIVE, 4 = HALT.
7	HALTED	r	0	Halted. Reads 1 while the transaction FSM is in the HALT state (the tag has been HLTA / halted by the reader). 0: Not halted 1: FSM halted
6	FIELDLIVE	r	0	Field-live level. The synchronized RF field-present level (1 while a reader field is on the tag). 0: No field 1: Field present
5	PARERRF	rw1	0	RX parity-error flag. Set when a received frame fails an odd-parity check; drives vector 97 (folded with NFCCRCERRF) when NFCCRCIE is set. Write 1 to clear. 0: No event 1: RX parity error
4	CRCERRF	rw1	0	RX CRC-error flag. Set when a received frame fails the CRC_A check; drives vector 97 when NFCCRCIE is set. Write 1 to clear. 0: No event 1: RX CRC error
3	TXDONEF	rw1	0	Transmit-done flag. Set when the tag response end-of-frame has been sent; drives vector 96 when NFCTXIE is set. Write 1 to clear. 0: No event 1: Tag response sent
2	RXFRAMEF	rw1	0	Reader-frame-received flag. Set when a reader frame has landed for firmware (its CRC / parity result is summarized in NFCxRXST); drives vector 95 when NFCRXFIE is set. Write 1 to clear. 0: No event 1: Reader frame received
1	FIELDF	rw1	0	Field-detect flag. Set on a synchronized rising edge of the RF field-present input; drives vector 94 when NFCFIELDIE is set. Write 1 to clear. 0: No event 1: RF field detected
0	BUSY	r	0	Busy. Reads 1 while a reader frame is being received or a tag response is in flight. 0: Idle 1: RX or TX in progress

24.7.3 NFCxUID: Provisioned single-size 4-byte tag UID, used by bit-frame anticollision

Offset 0x08 **Reset** 0x00000000 **Width** 32 **Address** 0x6208

Provisioned single-size 4-byte tag UID, used by bit-frame anticollision. Byte order on air is UID byte 0 first: UID[7:0] is the first byte, UID[31:24] the last. Hardware derives the BCC check byte. Latched transaction-

locally at the start of each transaction.

Bits	Field	Type	Reset	Description
31:0	UID	rw	0x00000000	4-byte UID (UID0 in bits 7:0, first on air).

24.7.4 NFCxCFG

Offset 0x0C **Reset** 0x00000044 **Width** 32 **Address** 0x620C
Tag identity response configuration: the ATQA answer-to-request word and the SAK select-acknowledge byte. Latched transaction-locally.

Bits	Field	Type	Reset	Description
31:24	-	-	-	Reserved. Reads zero; write zero.
23:16	SAK	rw	0x00	Select Acknowledge byte (resets to 0x00) returned after the final anticollision level.
15:0	ATQA	rw	0x0044	Answer To Request, Type A (resets to 0x0044). Bits 7:0 are the first byte on air.

24.7.5 NFCxTIM

Offset 0x10 **Reset** 0x088004D4 **Width** 32 **Address** 0x6210
Protocol timing divisors, all in rf_clk ticks, latched transaction-locally so a mid-count reload never glitches. The reset values are the real 13.56 MHz grid; a bench compresses all three for fast simulation.

Bits	Field	Type	Reset	Description
31:24	SUBCDIV	rw	0x08	Subcarrier half-period, in rf_clk ticks (resets to 8, giving the fc/16 load-modulation subcarrier).
23:16	ETU	rw	0x80	Elementary Time Unit (bit period), in rf_clk ticks (resets to 128).
15:0	FDT	rw	0x04D4	Frame Delay Time, in rf_clk ticks (resets to approximately 1236, the ISO 14443A tag response grid). The composed response is released when the FDT down-counter reaches zero.

24.7.6 NFCxRXST

Offset 0x14 **Reset** 0x00000000 **Width** 32 **Address** 0x6214
Received-frame inspection, for firmware-handled frames (NFCAUTOREAD = 0 or an unrecognized command). Read side-effect-free; the frame byte payload is read separately through the indexed RX buffer (NFCxIDX / NFCxDATA with NFCIDXSEL = 1). Pre-latched.

Bits	Field	Type	Reset	Description
31:18	-	-	-	Reserved. Reads zero; write zero.
17	RXPAROK	r	0	Parity OK for the last received frame.0: Parity bad1: Parity good
16	RXCRCOK	r	0	CRC OK for the last received frame.0: CRC bad1: CRC good
15:8	RXLEN	r	0x00	Received length in bytes, including the CRC.
7:0	CMD	r	0x00	Command byte: the first byte of the last received standard frame.

24.7.7 NFCxIDX

Offset 0x18 **Reset** 0x00000000 **Width** 32 **Address** 0x6218
Byte-index pointer into one of the two 64-byte windows accessed through NFCxDATA. The index persists across accesses; NFCIDXSEL selects which window. Mirrors the indexed-window idiom used by I3C's Device Address Table.

Bits	Field	Type	Reset	Description
31:9	-	-	-	Reserved. Reads zero; write zero.
8	IDXSEL	rw	0	Window select for NFCxDATA. 0: Payload TX window (64 B, firmware-filled, HW-read) 1: RX frame buffer (64 B, HW-filled, firmware-read)
7	-	-	-	Reserved. Reads zero; write zero.
6	IDXAINC	rw	0	Auto-increment. When set, NFCIDX increments after each NFCxDATA access (streaming). 0: Index held 1: Auto-increment after each access
5:0	IDX	rw	0x00	Byte index (0 to 63) into the selected window. Hardware advances it by one modulo 64 after every NFCxDATA access while NFCIDXAINC = 1.

24.7.8 NFCxDATA: The byte at NFCIDX in the window selected by NFCIDXSEL

Offset 0x1C **Reset** 0x00000000 **Width** 32 **Address** 0x621C
The byte at NFCIDX in the window selected by NFCIDXSEL. The payload TX window is read/write (firmware fills the record the reader collects; hardware reads it for the auto-answer READ); the RX frame buffer is read-only (hardware fills it from the reader frame; writes are ignored). Auto-increments per NFCIDXAINC. Pre-latched on read (no side effects).

Bits	Field	Type	Reset	Description
31:8	-	-	-	Reserved. Reads zero; write zero.
7:0	DATA	rw	0x00	Window data byte at the current index.

24.7.9 NFCxTXCTL

Offset 0x20 **Reset** 0x00000000 **Width** 32 **Address** 0x6220
Firmware-composed response control (used when NFCAUTOREAD = 0 or for a vendor command). INERT in this MVP (auto-answer is the default path); present in the frozen register map so the firmware-WRITE stage bolts on without a map change.

Bits	Field	Type	Reset	Description
31:10	-	-	-	Reserved. Reads zero; write zero.
9	TXAPPCRC	rw	0	Append CRC_A to the firmware-composed response. 0: No CRC appended 1: Append CRC_A

Bits	Field	Type	Reset	Description
8	TXGO	rw	0	Launch a firmware-composed response (a lane-0 write is a held-level launch, suppressed unless the FSM is in ACTIVE and awaiting a firmware reply). 0: No launch 1: Send response
7:0	TXLEN	rw	0x00	Number of payload bytes to send from the payload TX window.

24.7.10 NFCxDBG: Debug telemetry / bench cross-checks: frame counters

Offset 0x24 **Reset** 0x00000000 **Width** 32 **Address** 0x6224
Debug telemetry / bench cross-checks: frame counters. Read-only; resets to 0.

Bits	Field	Type	Reset	Description
31:16	TXFRAMECNT	r	0x0000	Transmitted-frame count.
15:0	RXFRAMECNT	r	0x0000	Received-frame count.

25 Real-time clock (RTCx)

25.1 Overview

The real-time clock (RTCx) is a 32.768 kHz always-on wall clock with a one-shot alarm and a recurring periodic tick, delivered on a single combined interrupt. It keeps a 32-bit seconds count with a 15-bit subsecond prescaler, compares the seconds against a programmable alarm, and generates a subsecond-cadence periodic tick. The whole point of the block is to present that autonomous, slowly-ticking counter domain coherently to firmware on the memory bus. It has *no pins*. The main features are listed below:

- A 32-bit seconds counter with a 15-bit subsecond prescaler (RTCxSEC, RTCxSUB); the prescaler rolls at exactly $2^{15} = 32768$, so seconds is literally its carry-out and the clock keeps exact 1 Hz time
- A one-shot alarm: the alarm flag sets once when the seconds count first equals RTCxALM (full 32-bit equality); re-arm by writing a new compare value (RTCALMEN, RTCALMF)
- An independent periodic tick: a subsecond down-counter reloaded from RTCxPER generates a recurring tick without disturbing the wall-clock prescaler (RTCTICKEN, RTCTICKF)
- Coherent, side-effect-free reads: RTCxSEC and RTCxSUB return a double-buffered snapshot synchronized into the bus domain
- A single combined interrupt (RTCALMF, RTCTICKF, write-1-to-clear) demultiplexed by the interrupt service routine
- An always-on clock domain: the count survives clock reconfiguration and power-gating of the harts
- A reserved calibration slot (RTCxTRIM) for a future digital ppm-correction stage

25.2 Instances and signals

Table 25.1: RTCx instances

Instance	Base address	Interrupt vector(s)	Pins (AF)
RTC0	0x6500	114	none

Table 25.2: RTCx interrupt vectors

Vector	Instance	Description	Clear method
114	RTC0	RTC0 combined alarm/periodic-tick InterruptIRQB_RTC0	See chapter

25.3 Functional description

25.3.1 Always-On Clock Domain

Unlike the SPI, UART, I2C, QSPI, and I3C peripherals, RTCx does *not* run on SMCLK, and software must *not* apply the usual "write SYSCLKCR to 0 first" rule to it. The counter, alarm compare, and tick engine all

clock directly off the ungated LFXT crystal (the 32.768 kHz pad clock), so timekeeping is immune to the SMCLK source: it keeps running through a clock reconfiguration and through power-gating of a hart, and firmware cannot stop it by clearing the SMCLK enable. Because the LFXT counter and the memory bus are asynchronous, every value that crosses between them does so through a held-level or toggle handshake; there is no free-running FIFO, only quasi-static or once-per-LFXT-period data.

The clock-domain-crossing synchronizers, the sticky event flags, and the interrupt combiner ride the free-running fabric clock (the same always-running clock that hosts the shared-window slaves), *not* the gated bus clock. Consequently the flags set and the interrupt asserts autonomously while the bus is idle (exactly the wake-the-CPU use case), and interrupt *delivery* is alive whenever the chip is alive. If timekeeping alone matters, it survives everything else the chip can do to its clocks.

25.3.2 Reading the Time Coherently

A read of RTCxSEC or RTCxSUB returns a snapshot taken from the same instant, so a single read is never torn mid-carry. The snapshot is up to about one LFXT period (roughly 30.5 µs) plus a few fabric-clock cycles behind the live count, the inherent granularity of a 32 kHz wall clock. Reading the full 47-bit {seconds, subsecond} value takes two bus accesses, which can straddle one snapshot update; to get a guaranteed-coherent pair, firmware uses the standard idiom: read RTCxSEC, read RTCxSUB, read RTCxSEC again, and retry if the seconds value changed. This preserves the block's no-read-side-effects rule (a read never latches or clears anything).

25.4 Interrupts and events

The two event flags in RTCxSR, RTCALMF (alarm) and RTCTICKF (periodic tick), are write-1-to-clear: write a 1 to a bit to clear it; writing 0 leaves it unchanged, and a read never clears them. The alarm is one-shot (it fires once at the matching second and does not re-fire the next second), so firmware re-arms it by writing a new RTCxALM past the current time.

The single combined interrupt source occupies vector 114 (alarm or periodic tick), delivered through the interrupt router like every other shared-peripheral source; the interrupt service routine reads RTCxSR and demultiplexes RTCALMF versus RTCTICKF, clearing whichever fired before returning. This source sits above the external-interrupt (meip) delivery vector, which stays fixed at vector 85, above the I3C and NFC sources at vectors 86 through 97, and above the GPIOx port-4 and port-5 sources at vectors 98 through 113.

25.5 Programming

Set-time and alarm / period updates cross into the slow LFXT domain through a request / acknowledge handshake reported by RTCSYNC (the busy bit) in RTCxSR. Writing RTCxSEC atomically commits both seconds and the staged subsecond value (write RTCxSUB first if subseconds are also being set); on the applying LFXT edge the loaded value takes priority over the increment, so the count is never torn. Writing RTCxALM commits the alarm compare, and writing RTCxPER commits the periodic reload (16 bits, up to roughly a 2 s interval). Only one commit may be outstanding: firmware must poll RTCSYNC to 0 before the next committing write to RTCxSEC, RTCxALM, or RTCxPER.

25.6 Usage notes

In configurations that include it, RTC0 lives in the shared peripheral window behind the multi-core arbiter (see Section 5) in the mutex-bank page, sub-slot 5 at 0x6500, so any hart can use it.

25.7 Registers

Register offsets in this chapter are relative to the instance base address in Table 25.1 (RTC0 0x6500); the generated header names each base RTC0_BASE.

Table 25.3: RTCx register summary

Offset	Name	Access	Reset	Description
0x00	RTCxCR	rw	0x00000000	RTC control register
0x04	RTCxSEC	rw	0x00000000	Wall-clock seconds
0x08	RTCxSUB	rw	0x00000000	Subsecond prescaler, 0 to 32767 (2^15 - 1)
0x0C	RTCxALM	rw	0x00000000	Alarm compare value, in seconds
0x10	RTCxPER	rw	0x00000000	
0x14	RTCxSR	r/rw1	0x00000000	RTC status register
0x18	RTCxTRIM	r0	0x00000000	Reserved for digital fractional-prescaler / ppm calibration (deferred)

25.7.1 RTCxCR: RTC control register

Offset 0x00 **Reset** 0x00000000 **Width** 32 **Address** 0x6500

RTC control register. Enables the seconds/subsecond counter, the alarm compare, and the periodic-tick down-counter, and holds the two interrupt enables. The three enables cross into the LFXT domain as synchronized held levels; the interrupt enables stay in the bus/MCLK domain and gate the combined interrupt combinationally. Resets to 0. The reserved upper bits hold a future trim-enable field (RTC0TRIM, digital calibration, deferred).

Bits	Field	Type	Reset	Description
31:5	-	-	-	Reserved. Reads zero; write zero.
4	TICKIE	rw	0	Periodic-tick interrupt enable. When set, RTC0SR.TICKF drives the combined RTC interrupt (vector 114). 0: Interrupt disabled 1: Interrupt enabled
3	ALMIE	rw	0	Alarm interrupt enable. When set, RTC0SR.ALMF drives the combined RTC interrupt (vector 114). 0: Interrupt disabled 1: Interrupt enabled
2	TICKEN	rw	0	Periodic-tick enable. When set, the independent subsecond down-counter reloaded from RTC0PER runs and sets the tick flag on each underflow; when clear no tick event is generated. The tick counter never disturbs the wall-clock prescaler. 0: Tick disabled 1: Tick enabled
1	ALMEN	rw	0	Alarm-compare enable. When set, the alarm engine compares the seconds counter against RTC0ALM and sets the alarm flag on the match; when clear no alarm event is generated. 0: Alarm disabled 1: Alarm enabled

Bits	Field	Type	Reset	Description
0	EN	rw	0	Wall-clock enable. When set, the 47-bit seconds, subsecond counter advances on every LFXT edge; when clear the count is frozen (its value is preserved). 0: Counter frozen 1: Counting

25.7.2 RTCxSEC: Wall-clock seconds

Offset 0x04 **Reset** 0x00000000 **Width** 32 **Address** 0x6504

Wall-clock seconds. READ returns a coherent double-buffered snapshot synchronized into the bus domain (up to ~1 LFXT period, ~30.5 us, behind the live count; no read side effects). WRITE stages the seconds value and atomically commits set-time SEC, SUB into the LFXT domain through the SR.SYNC handshake (write SUB first if subseconds are also being set); the loaded value takes priority over the increment on the applying LFXT edge.

Bits	Field	Type	Reset	Description
31:0	SEC	rw	0x00000000	Seconds counter (0 to 2^32-1).

25.7.3 RTCxSUB: Subsecond prescaler, 0 to 32767 (2^15 - 1)

Offset 0x08 **Reset** 0x00000000 **Width** 32 **Address** 0x6508

Subsecond prescaler, 0 to 32767 (2^15 - 1). READ returns the coherent snapshot from the SAME instant as RTC0SEC (D7). WRITE only STAGES the subsecond value; it is committed by the following RTC0SEC write (to set subseconds alone, write SUB then SEC).

Bits	Field	Type	Reset	Description
31:15	-	-	-	Reserved. Reads zero; write zero.
14:0	SUB	rw	0x0000	Subsecond prescaler count (0 to 32767); wraps at 32768, carrying seconds by exactly one (exact 1 Hz).

25.7.4 RTCxALM: Alarm compare value, in seconds

Offset 0x0C **Reset** 0x00000000 **Width** 32 **Address** 0x650C

Alarm compare value, in seconds. The alarm flag (RTC0SR.ALMF) sets once when the seconds counter first equals this value (one-shot, full 32-bit equality); re-arm by writing a new value past the current second. READ returns the last written value (bus-domain staging readback, no CDC). WRITE stages and commits the new compare through the SR.SYNC handshake.

Bits	Field	Type	Reset	Description
31:0	ALM	rw	0x00000000	Alarm seconds compare value.

25.7.5 RTCxPER

Offset 0x10 **Reset** 0x00000000 **Width** 32 **Address** 0x6510

Periodic-tick reload, in subsecond (LFXT) ticks: a tick event fires every reload+1 LFXT ticks (~2 s max interval at 32.768 kHz, adjudication A4). READ returns the last written value (bus-domain staging readback). WRITE stages and commits the new reload through the SR.SYNC handshake.

Bits	Field	Type	Reset	Description
31:16	-	-	-	Reserved. Reads zero; write zero.
15:0	PER	rw	0x0000	Periodic-tick reload in LFXT ticks (0 to 65535).

25.7.6 RTCxSR: RTC status register

Offset 0x14 **Reset** 0x00000000 **Width** 32 **Address** 0x6514

RTC status register. SYNC is read-only; the two event flags (ALMF, TICKF) are write-1-to-clear (write a 1 to a bit to clear it; writing 0 leaves it unchanged) and are never cleared by a read. The combined RTC interrupt (vector 114) is (ALMF and RTCALMIE) or (TICKF and RTCTICKIE).

Bits	Field	Type	Reset	Description
31:3	-	-	-	Reserved. Reads zero; write zero.
2	TICKF	rw1	0	Periodic-tick flag. Set on each periodic-tick underflow; drives vector 114 when RTCTICKIE is set. Write 1 to clear. 0: No event 1: Tick fired
1	ALMF	rw1	0	Alarm flag. Set once when the seconds counter first matches RTC0ALM (one-shot); drives vector 114 when RTCALMIE is set. Write 1 to clear. 0: No event 1: Alarm fired
0	SYNC	r	0	Sync/busy. Reads 1 while a set-time / alarm / period commit is still crossing into the LFXT domain; software must poll this 0 before the next committing write (single outstanding commit). 0: Idle (safe to commit) 1: Commit in flight

25.7.7 RTCxTRIM: Reserved for digital fractional-prescaler / ppm calibration (deferred)

Offset 0x18 **Reset** 0x00000000 **Width** 32 **Address** 0x6518

Reserved for digital fractional-prescaler / ppm calibration (deferred). Reads 0, writes ignored. The slot and the RTC0CR trim-enable field are reserved so calibration bolts on without a register-map break.

Bits	Field	Type	Reset	Description
31:0	-	-	-	Reserved. Reads zero; write zero.

26 Pulse-width modulator (PWMx)

26.1 Overview

The PWM generator (PWMx) is a glitch-free two-channel edge-aligned pulse-width-modulation engine: a prescaled 16-bit up-counter with two 16-bit per-channel duty comparators, a double-buffered waveform update, per-channel polarity and an absolute programmable safe level, a software fault trip, and a period-event tick. Its reason to exist over the general-purpose timer compare path is the double buffering: a mid-period change to the period or a duty takes effect only at the next period boundary, so it can never produce a runt or double pulse. The whole engine runs on the free-running fabric clock (MCLK) through a power-of-two prescaler; there is no LFXT and no generated or gated clock anywhere in the block. It has *no input pins*. The main features are listed below:

- A 16-bit period (PWMxPER) and two 16-bit per-channel duties (PWMxDTY0, PWMxDTY1), all double-buffered; the waveform frequency is $f_{\mathrm{MCLK}}/2^{\mathrm{PSC}}/(\mathrm{PWMPER}+1)$
- A 4-bit power-of-two prescaler (PSC) producing a single-cycle tick enable, not a divided clock
- Glitch-free buffered update: waveform writes stage into shadow registers and commit atomically at the period boundary, tracked by UPDF
- Per-channel polarity (POL0, POL1) and an *absolute* programmable safe/off level (SAFE0, SAFE1)
- A software/mask-only fault trip (FLTEN, FLTTRIG) that forces both outputs safe within one clock, latched sticky in FLTF
- A recurring period event (PEVF) and two lean interrupts: fault (vector 115) and period event (vector 116)
- Reserved slots and control bits provisioned for deferred 4-channel, center-aligned, and deadtime/complementary bolt-ons without a register-map break

26.2 Instances and signals

Table 26.1: PWMx instances

Instance	Base address	Interrupt vector(s)	Pins (AF)
PWM0	0x6600	115 to 116	PWM0 P2.2 (AF2), PWM1 P2.3 (AF2)

Table 26.2: PWMx signals

Signal	Direction	Description	Pin (AF)
PWM0	Output	PWM0 channel 0 output (replaces the redundant T0CMP0 spread copy; alt plane AF2)	P2.2 (AF2)

Signal	Direction	Description	Pin (AF)
PWM1	Output	PWM0 channel 1 output (replaces the redundant T0CMP1 spread copy; alt plane AF2)	P2.3 (AF2)

Table 26.3: PWMx interrupt vectors

Vector	Instance	Description	Clear method
115	PWM0	PWM0 fault-trip InterruptIRQB_PWM0_FAULT	See chapter
116	PWM0	PWM0 period-event InterruptIRQB_PWM0_EVT	See chapter

26.3 Functional description

26.3.1 Buffered, Glitch-Free Update

Writes to PWMxPER, PWMxDTY0, and PWMxDTY1 do *not* change the live waveform. They land in shadow (staging) registers and are copied atomically into the active period and duties at the next period boundary, the counter wrap. UPDF in PWMxSR reads 1 from a buffered write until that write has been absorbed at the boundary, then 0, so firmware can confirm a coherent multi-word update has taken effect. A read of the buffered registers returns the staged value (the last written value), with no side effects. Polarity and the safe level, by contrast, are *immediate* (not buffered): program PWMxPOL before enabling a channel.

Each channel's raw waveform is active while the counter is below that channel's duty, so a channel is active for exactly the first PWMDTYn prescale ticks of each (PWMPER + 1)-tick period. The corner cases follow exactly from that relation: a duty of 0 gives a constant inactive level; a duty greater than the period gives a constant active level; and a period of 0 makes every prescale tick a period boundary (a legitimate DC/degenerate setting that pulses PEVF every tick).

26.3.2 Polarity and the Safe Level

Two distinct levels must not be conflated. The *inactive* level of an enabled, unfaulted channel is its polarity: with POLn= 0 the waveform is active-high and idles low. The *safe* level is an absolute pin level (SAFEn= 0 drives low, 1 drives high) forced whenever the channel is disabled (PWMEN= 0 or the channel enable is clear) or faulted, independent of polarity, so a half-bridge or heater has a board-deterministic off state regardless of a concurrent polarity setting. Both reset to 0, so at reset the safe level and the active-high inactive level coincide (both low) and the outputs are safe from cycle one.

26.3.3 The Software Fault

The fault is software/mask-only: there is no hardware fault pin. With FLTEN set, writing the self-clearing FLTTRIG command bit trips the fault: it latches the sticky FLTF and forces both outputs to their safe levels combinationally, within one clock (a half-bridge cannot wait a cycle). With FLTEN clear the FLTTRIG write is ignored. The fault overrides only the output stage; the prescaler, counter, shadow commit, and PEVF keep running, so the timebase is undisturbed and the waveform resumes phase-continuously on re-arm.

26.4 Interrupts and events

The fault interrupt is FLTF ∧ FLTIE and the period-event interrupt is PEVF ∧ PEVIE, both combinational and never latched.

The two interrupt sources occupy vectors 115 (fault) and 116 (period event); the fault takes the lower identifier so the interrupt router's fixed lowest-identifier-first priority dispatches the safety event first. Both are delivered through the interrupt router like every other shared-peripheral source. These sources sit above the external-interrupt (meip) delivery vector, which stays fixed at vector 85, above the I3C, NFC, GPIOx port-4/port-5, and RTC sources at vectors 86 through 114.

26.5 Programming

To re-arm after a fault, write 1 to FLTF (write-1-to-clear); the override releases and the outputs immediately resume tracking the still-running comparators.

26.6 Usage notes

In configurations that include it, PWM0 lives in the shared peripheral window behind the multi-core arbiter (see Section 5) in the mutex-bank page, sub-slot 6 at 0x6600, so any hart can use it. Because the engine clocks off the free-running fabric clock, its count is immune to clock reconfiguration, and software must *not* apply the "write SYSCLKCR to 0 first" rule that the SMCLK peripherals need.

The two channel outputs ride existing bonded pins with no new package pads: they take the alternate-function-2 plane of P2.2 and P2.3, adjacent to the T0CMP0/T0CMP1 compare homes on P2.0/P2.1, replacing two redundant timer-compare copies in the output-function spread (both compares remain reachable on many other pins).

26.7 Registers

Register offsets in this chapter are relative to the instance base address in Table 26.1 (PWM0 0x6600); the generated header names each base PWM0_BASE.

Table 26.4: PWMx register summary

Offset	Name	Access	Reset	Description
0x00	PWMxCR	rw/w1	0x00000000	PWM control register
0x04	PWMxPER	rw	0x00000000	PWM period modulus (buffered, D9)
0x08	PWMxDTY0	rw	0x00000000	Channel 0 duty compare (buffered, D9)
0x0C	PWMxDTY1	rw	0x00000000	Channel 1 duty compare (buffered, D9)
0x10	PWMxDTY2	r0	0x00000000	Reserved for the channel 2 duty compare (4-channel bolt-on, D16)
0x14	PWMxDTY3	r0	0x00000000	Reserved for the channel 3 duty compare (4-channel bolt-on, D16)
0x18	PWMxPOL	rw	0x00000000	
0x1C	PWMxDT	r0	0x00000000	
0x20	PWMxSR	r/rw1	0x00000000	PWM status register

26.7.1 PWMxCR: PWM control register

Offset 0x00 **Reset** 0x00000000 **Width** 32 **Address** 0x6600

PWM control register. Enables the prescaler + main counter (PWMEN) and each channel output (CH0EN/CH1EN), holds the two interrupt enables (PEVIE/FLTIE), the fault-system enable (FLTEN) and the write-1 software

fault trip (FLTTRIG), and the 4-bit prescaler (PSC, divide by 2^PSC). Resets to 0 (disabled, outputs safe). The reserved bits are provisioned for the 4-channel (CH2EN/CH3EN), center-aligned (CNTMODE), deadtime (DTEN) and HW-fault-polarity (FLTPOL) bolt-ons.

Bits	Field	Type	Reset	Description
31:20	-	-	-	Reserved. Reads zero; write zero.
19:16	PSC	rw	0b0000	Prescaler: the main counter advances once every 2^PSC MCLK cycles (PSC=0 -> every cycle). Frequency = f_MCLK / 2^PSC / (PER+1).
15	-	-	-	Reserved. Reads zero; write zero.
14	FLTTRIG	w1	0	Software fault trip (write-1 self-clearing command; reads 0). Writing 1 requests a trip: while FLTEN is set it latches SR.FLTF and forces both outputs safe within one clock. Re-arm by write-1-clearing SR.FLTF. 1: Trip the fault (if FLTEN set)
13	-	-	-	Reserved. Reads zero; write zero.
12	FLTEN	rw	0	Fault-system enable. Gates the software trip: a FLTTRIG write latches SR.FLTF only while FLTEN is set; with FLTEN clear the FLTTRIG write is ignored. 0: Fault system disabled (FLTTRIG ignored) 1: Fault system enabled
11:9	-	-	-	Reserved. Reads zero; write zero.
8	FLTIE	rw	0	Fault interrupt enable. When set, SR.FLTF drives the PWM0_FAULT interrupt (vector 115). 0: Interrupt disabled 1: Interrupt enabled
7	PEVIE	rw	0	Period-event interrupt enable. When set, SR.PEVF drives the PWM0_EVT interrupt (vector 116). 0: Interrupt disabled 1: Interrupt enabled
6:3	-	-	-	Reserved. Reads zero; write zero.
2	CH1EN	rw	0	Channel 1 output enable. When clear, CH1 drives its absolute safe level (POL.SAFE1) regardless of the waveform. 0: CH1 drives safe level 1: CH1 drives the waveform
1	CH0EN	rw	0	Channel 0 output enable. When clear, CH0 drives its absolute safe level (POL.SAFE0) regardless of the waveform. 0: CH0 drives safe level 1: CH0 drives the waveform
0	EN	rw	0	Master enable. When set, the prescaler and 16-bit main counter run; when clear the counter holds at 0 (clean restart on enable) and both outputs drive their safe levels. 0: Disabled (counter held, outputs safe) 1: Running

26.7.2 PWMxPER: PWM period modulus (buffered, D9)

Offset 0x04 **Reset** 0x00000000 **Width** 32 **Address** 0x6604

PWM period modulus (buffered, D9). The waveform period is (PER+1) prescale ticks: frequency = f_MCLK / 2^PSC / (PER+1). READ returns the last written value (bus-domain staging readback, no side effects). WRITE stages the value and arms SR.UPDF; it commits to the live waveform at the next period boundary (glitch-free).

Bits	Field	Type	Reset	Description
31:16	-	-	-	Reserved. Reads zero; write zero.
15:0	PER	rw	0x0000	Period modulus (0 to 65535); the counter wraps 0..PER, so the period is PER+1 ticks.

26.7.3 PWMxDTY0: Channel 0 duty compare (buffered, D9)

Offset 0x08 **Reset** 0x00000000 **Width** 32 **Address** 0x6608

Channel 0 duty compare (buffered, D9). CH0 is active for the first DTY0 prescale ticks of each period (DTY0=0 -> constant inactive; DTY0 >= PER+1 -> constant active). READ returns the staging value; WRITE stages and arms SR.UPDF (commits at the next period boundary).

Bits	Field	Type	Reset	Description
31:16	-	-	-	Reserved. Reads zero; write zero.
15:0	DTY0	rw	0x0000	Channel 0 duty compare (0 to 65535).

26.7.4 PWMxDTY1: Channel 1 duty compare (buffered, D9)

Offset 0x0C **Reset** 0x00000000 **Width** 32 **Address** 0x660C

Channel 1 duty compare (buffered, D9). CH1 is active for the first DTY1 prescale ticks of each period (same corner rules as DTY0). READ returns the staging value; WRITE stages and arms SR.UPDF (commits at the next period boundary).

Bits	Field	Type	Reset	Description
31:16	-	-	-	Reserved. Reads zero; write zero.
15:0	DTY1	rw	0x0000	Channel 1 duty compare (0 to 65535).

26.7.5 PWMxDTY2: Reserved for the channel 2 duty compare (4-channel bolt-on, D16)

Offset 0x10 **Reset** 0x00000000 **Width** 32 **Address** 0x6610

Reserved for the channel 2 duty compare (4-channel bolt-on, D16). Reads 0, writes ignored; provisioned so >2 channels bolt on without a register-map break.

Bits	Field	Type	Reset	Description
31:0	-	-	-	Reserved. Reads zero; write zero.

26.7.6 PWMxDTY3: Reserved for the channel 3 duty compare (4-channel bolt-on, D16)

Offset 0x14 **Reset** 0x00000000 **Width** 32 **Address** 0x6614

Reserved for the channel 3 duty compare (4-channel bolt-on, D16). Reads 0, writes ignored.

Bits	Field	Type	Reset	Description
31:0	-	-	-	Reserved. Reads zero; write zero.

26.7.7 PWMxPOL

Offset 0x18 **Reset** 0x00000000 **Width** 32 **Address** 0x6618
Per-channel polarity and absolute safe/off level (immediate, NOT buffered, D11; program before enabling). POLn inverts the channel waveform; SAFEn is the ABSOLUTE pin level (0 = drive low, 1 = drive high) forced when the channel is disabled or faulted, and it is polarity-independent, so the fault/idle physical state is deterministic. Resets to 0 (active-high, safe = low). The reserved bits hold the CH2/CH3 polarity and safe fields (4-channel bolt-on, D16).

Bits	Field	Type	Reset	Description
31:6	-	-	-	Reserved. Reads zero; write zero.
5	SAFE1	rw	0	Channel 1 absolute safe/off pin level (driven when CH1 is disabled or faulted). 0: Drive low 1: Drive high
4	SAFE0	rw	0	Channel 0 absolute safe/off pin level (driven when CH0 is disabled or faulted). 0: Drive low 1: Drive high
3:2	-	-	-	Reserved. Reads zero; write zero.
1	POL1	rw	0	Channel 1 polarity. 0: Active-high 1: Active-low (invert)
0	POL0	rw	0	Channel 0 polarity. 0: Active-high 1: Active-low (invert)

26.7.8 PWMxDT

Offset 0x1C **Reset** 0x00000000 **Width** 32 **Address** 0x661C
Reserved for the deadtime value (deadtime / complementary-pair bolt-on, D16). Reads 0, writes ignored; provisioned so deadtime bolts on without a register-map break.

Bits	Field	Type	Reset	Description
31:0	-	-	-	Reserved. Reads zero; write zero.

26.7.9 PWMxSR: PWM status register

Offset 0x20 **Reset** 0x00000000 **Width** 32 **Address** 0x6620
PWM status register. FLTF and PEVF are sticky write-1-to-clear event flags (write a 1 to a bit to clear it; writing 0 leaves it unchanged; never cleared by a read; a set arriving the same cycle as a clear survives). UPDF is read-only. PWM0_FAULT (vector 115) = FLTF and CR.FLTIE; PWM0_EVT (vector 116) = PEVF and CR.PEVIE (both combinational). The reserved DIR bit is the center-aligned count direction (D19).

Bits	Field	Type	Reset	Description
31:3	-	-	-	Reserved. Reads zero; write zero.

Bits	Field	Type	Reset	Description
2	UPDF	r	0	Buffered-update pending. Reads 1 from a staged PER/DTY0/DTY1 write until that write has been absorbed at the next period boundary. 0: Staging absorbed 1: Commit pending
1	PEVF	rw1	0	Period-event flag. Set at every active period boundary. Write 1 to clear. 0: No event 1: Period event
0	FLTF	rw1	0	Fault flag. Set when a FLTTRIG trip is accepted (FLTEN set); while set both outputs are forced to their safe levels. Write 1 to clear (re-arm). 0: No fault 1: Fault latched (outputs safe)

27 1-Wire interface (OWx)

27.1 Overview

The 1-Wire master (OWx) is a Dallas/Maxim 1-Wire link-layer controller for the cheap-sensor ecosystem (DS18B20-class thermometers, iButton / DS24xx ROM-ID chips). It runs the five microsecond-scale bus primitives in hardware (reset+presence, write-bit, read-bit, write-byte, read-byte) off a programmable time base, and leaves the higher-level ROM search and CRC-8 to firmware over those primitives. It is master-only and supports both standard and overdrive speeds. The whole engine runs on the free-running fabric clock (MCLK); the single open-drain DQ pin is sampled through a two-flop synchronizer, so no flip-flop is ever clocked by the bus pin. The main features are listed below:

- Five hardware link-layer primitives selected by OWxCMD: reset+presence detect, write-bit, read-bit, write-byte and read-byte, each launched by a single command write
- Byte transfers are LSB-first (the 1-Wire convention): OWxTX is shifted out least-significant-bit first, and read-byte assembles into OWxRX the same way
- Standard and overdrive timing sets, selected by OWODS and latched at each transaction launch so a mid-flight speed change never glitches a running slot
- A programmable time base (OWxDIV): one tick is (OWDIV + 1) MCLK cycles, giving a 0.5 µs tick at OWDIV = 11 with the nominal 24 MHz MCLK (all slot counts are integer half-microsecond ticks)
- Side-effect-free reads: the received byte/bit lands in OWxRX, never in the command register; a read never launches or clears anything
- Sticky write-1-to-clear status (OWTCIF, OWNOPRES, OWSHORT) and a single combined interrupt (vector 117)
- ROM search, ROM match and CRC-8 are done in firmware over the hardware bit primitives; the strong-pullup enable is a reserved stub (no driven-high phase this stage)

27.2 Instances and signals

Table 27.1: OWx instances

Instance	Base address	Interrupt vector(s)	Pins (AF)
OW0	0x6700	117	OW_DQ P3.7 (AF2)

Table 27.2: OWx signals

Signal	Direction	Description	Pin (AF)
OW_DQ	Bidirectional	OW0 1-Wire DQ, open-drain (replaces the redundant T0CMP1 spread copy; alt plane AF2)	P3.7 (AF2)

Table 27.3: OWx interrupt vectors

Vector	Instance	Description	Clear method
117	OW0	OW0 1-Wire combined transaction-complete/error Interrupt IRQB_OW0	See chapter

27.3 Functional description

27.3.1 The Time Base and the Two Speeds

The slot state machine counts *ticks*, never raw clock cycles, so all slot geometry is expressed in half-microsecond ticks independent of the actual MCLK rate. OWxDIV sets the tick period to (OWDIV + 1) MCLK cycles; program OWDIV = 11 for the nominal 0.5 µs tick at 24 MHz. OWODS selects between the standard slot-timing set and the overdrive (roughly ten-times-faster) set; it is latched into the transaction descriptor at launch, so writing a new OWxCR mid-transaction cannot disturb a slot already in flight. Because the engine clocks off the free-running fabric clock, its tick base is immune to clock reconfiguration, and software must *not* apply the "write SYSCLKCR to 0 first" rule that the SMCLK peripherals need. A simulation bench uses a small OWxDIV to compress the microsecond scale into a few clock cycles per tick.

27.3.2 Presence, Errors and Bus Faults

A reset transaction drives DQ low for the reset-low window, releases it, and samples the presence pulse: OWPRES sets if a device answered. OWNOPRES sets when a clean reset saw no presence pulse. OWSHORT sets when DQ is still low at the end of a recovery window (a stuck or shorted bus) after the master has released it. On a permanently-low bus during a reset, *SHORT wins*: OWSHORT sets and OWNOPRES is suppressed, because presence is unevaluable on a bus that never returns high. The three event flags are sticky and write-1-to-clear (write a 1 to a bit to clear it; a read never clears them).

27.4 Interrupts and events

The combined interrupt is (OWTCIF ∧ OWTCIE) ∨ ((OWNOPRES ∨ OWSHORT) ∧ OWERRIE), combinational and never latched; the interrupt service routine reads OWxSR, demultiplexes transaction-complete versus the error flags, and clears whichever fired before returning.

The single combined interrupt source occupies vector 117 (transaction complete or error), delivered through the interrupt router like every other shared-peripheral source. This source sits above the external-interrupt (meip) delivery vector, which stays fixed at vector 85, above the I3C, NFC, GPIOx port-4/port-5, RTC and PWM sources at vectors 86 through 116.

27.5 Programming

27.5.1 Launching a Transaction

A byte-lane-0 write to OWxCMD launches a transaction: it captures {operation, bit-value, current speed, current OWxTX byte} into a launch descriptor and starts the slot machine. The launch is suppressed when the master is disabled (OWEN= 0) or already busy; in that case the command content is still captured but no bus activity or completion occurs. Writes to OWxTX, OWxCR and OWxDIV never launch. OWBUSY covers the

launch instant: it reads 1 in the same cycle as the launching command write, so firmware may poll OWBUSY to 0 immediately after writing OWxCMD with no blind window. Only one transaction may be outstanding: poll OWBUSY clear before issuing the next command.

27.6 Usage notes

In configurations that include it, OW0 lives in the shared peripheral window behind the multi-core arbiter (see Section 5) in the mutex-bank page, sub-slot 7 at 0x6700, so any hart can use it.

The block has one pin, the open-drain DQ line, on the alternate-function-2 plane of P4.7 (GPIO31, the digital test port 3 pad) with its pull enabled at reset. That plane slot is one of the alternate-function output-spread copies of the TIMER0 compare-1 output, and DQ simply takes it over: compare 1 keeps its own primary pin P3.1, its two alternate-function-1 relocations, and more than twenty further spread copies, so nothing is lost. When OW0 is not instantiated the slot keeps its compare-1 copy and P4.7 behaves exactly as before. Software routes DQ by selecting alternate function 2 for that pin's field in the port's PxAFS register; until it does, the receiver reads an idle-high line and the pad remains a plain digital test port.

27.7 Registers

Register offsets in this chapter are relative to the instance base address in Table 27.1 (OW0 0x6700); the generated header names each base OW0_BASE.

Table 27.4: OWx register summary

Offset	Name	Access	Reset	Description
0x00	OWxCR	rw	0x00000000	1-Wire control register
0x04	OWxCMD	rw	0x00000000	1-Wire command register
0x08	OWxTX	rw	0x00000000	Next write byte, the write-byte (OP = 011) source, transmitted LSB-first
0x0C	OWxRX	r	0x00000000	
0x10	OWxDIV	rw	0x00000000	Time-base divisor
0x14	OWxSR	r/rw1	0x00000000	1-Wire status register
0x18	OWxSPU	r0	0x00000000	Reserved for the strong-pullup / parasite-power stage (D15)

27.7.1 OWxCR: 1-Wire control register

Offset 0x00 **Reset** 0x00000000 **Width** 32 **Address** 0x6700
1-Wire control register. Enables the master (OWEN), selects the bus speed (ODS: standard or overdrive, latched into the transaction descriptor at launch so a mid-flight change never glitches a running slot), holds the transaction-complete and error interrupt enables (TCIE, ERRIE), and carries the reserved strong-pullup enable stub (SPUEN, no effect this stage). Resets to 0 (master idle). The reserved upper bits reserve a future strong-pullup window and timing fields.

Bits	Field	Type	Reset	Description
31:5	-	-	-	Reserved. Reads zero; write zero.

Bits	Field	Type	Reset	Description
4	ERRIE	rw	0	Error interrupt enable. When set, OW0SR.NOPRES or OW0SR.SHORT drives the combined 1-Wire interrupt (vector 117). 0: Interrupt disabled 1: Interrupt enabled
3	TCIE	rw	0	Transaction-complete interrupt enable. When set, OW0SR.TCIF drives the combined 1-Wire interrupt (vector 117). 0: Interrupt disabled 1: Interrupt enabled
2	SPUEN	rw	0	Strong-pullup enable (reserved stub, D15). Writable but inert this stage: there is no driven-high strong-pullup phase (DQ is open-drain, never driven high). Reserved so a future parasite-power SPU bolts on without a register-map break. 0: No effect 1: No effect (reserved)
1	ODS	rw	0	Overdrive speed select, latched into the transaction descriptor at launch. 0 selects the standard slot-timing set, 1 the overdrive (~10x faster) set. (Named ODS; supersedes the spec sketch ODEN, adjudication A2.) 0: Standard speed 1: Overdrive speed
0	EN	rw	0	Master enable. When 0 the slot FSM is held idle and an OW0CMD launch is suppressed (the descriptor is still captured); RX and the status flags are preserved. Set to 1 to run transactions. 0: Disabled (FSM idle, launch suppressed) 1: Enabled

27.7.2 OWxCMD: 1-Wire command register

Offset 0x04 **Reset** 0x00000000 **Width** 32 **Address** 0x6704

1-Wire command register. A byte-lane-0 write LAUNCHES a transaction (D8): it always captures OP, BITVAL, current ODS, current OW0TX byte into the launch descriptor, and starts the slot FSM unless OWEN=0 or the master is busy (in which case the content is captured but no bus activity or completion occurs). OP selects the primitive (reset / write-bit / read-bit / write-byte / read-byte); BITVAL is the write-bit value. A read returns the last-written OP and BITVAL. Bytes are transmitted/received LSB-first.

Bits	Field	Type	Reset	Description
31:9	-	-	-	Reserved. Reads zero; write zero.
8	BITVAL	rw	0	Write-bit value for OP = write-bit (latched at launch; ignored by the other operations). 0: Write a 0 bit 1: Write a 1 bit
7:3	-	-	-	Reserved. Reads zero; write zero.
2:0	OP	rw	0b000	Operation: 000 = reset+presence, 001 = write-bit, 010 = read-bit, 011 = write-byte (OW0TX, LSB-first), 100 = read-byte (into OW0RX, LSB-first); 101-111 reserved (no bus activity, no launch).

27.7.3 OWxTX: Next write byte, the write-byte (OP = 011) source, transmitted LSB-first

Offset 0x08 **Reset** 0x00000000 **Width** 32 **Address** 0x6708
Next write byte, the write-byte (OP = 011) source, transmitted LSB-first. Writing this register never launches a transaction (D8); the byte is sampled into the descriptor at the next OW0CMD launch.

Bits	Field	Type	Reset	Description
31:8	-	-	-	Reserved. Reads zero; write zero.
7:0	TX	rw	0x00	Write byte (0 to 255).

27.7.4 OWxRX

Offset 0x0C **Reset** 0x00000000 **Width** 32 **Address** 0x670C
Last received data (read-only, side-effect-free, A3): a read-byte (OP = 100) assembles into [7:0] LSB-first; a read-bit (OP = 010) lands in [0]. Reset-cleared to 0; a read never clears or launches anything.

Bits	Field	Type	Reset	Description
31:8	-	-	-	Reserved. Reads zero; write zero.
7:0	RX	r	0x00	Received byte (read-byte) or received bit in [0] (read-bit).

27.7.5 OWxDIV: Time-base divisor

Offset 0x10 **Reset** 0x00000000 **Width** 32 **Address** 0x6710
Time-base divisor. The slot FSM counts ticks; one tick is (OW0DIV + 1) MCLK cycles. Program OW0DIV = 11 for a 0.5 microsecond tick at 24 MHz MCLK (adjudication A1: all slot counts are integer half-microsecond ticks); a bench uses a small divisor to compress simulation time. Resets to 0.

Bits	Field	Type	Reset	Description
31:16	-	-	-	Reserved. Reads zero; write zero.
15:0	DIV	rw	0x0000	Time-base divisor (0 to 65535); tick period = OW0DIV + 1 MCLK cycles.

27.7.6 OWxSR: 1-Wire status register

Offset 0x14 **Reset** 0x00000000 **Width** 32 **Address** 0x6714
1-Wire status register. BUSY and PRES are read-only levels; TCIF, NOPRES and SHORT are sticky write-1-to-clear flags (write a 1 to a bit to clear it; writing 0 leaves it unchanged; never cleared by a read; a set arriving the same cycle as a clear survives). BUSY covers the launch instant (it reads 1 the same cycle as the launching OW0CMD write, A6), so firmware may poll BUSY-clear immediately after CMD; observe the single-outstanding-transaction rule (poll BUSY before the next CMD). The combined interrupt (vector 117) is (TCIF and TCIE) or ((NOPRES or SHORT) and ERRIE). On a stuck-low bus SHORT wins and NOPRES is suppressed (A5).

Bits	Field	Type	Reset	Description
31:5	-	-	-	Reserved. Reads zero; write zero.

Bits	Field	Type	Reset	Description
4	SHORT	rw1	0	Bus-short error. Set when DQ is still low at the end of a recovery window (bus stuck/short) after the master has released it. On a reset, SHORT wins over NOPRES (A5). Drives vector 117 when ERRIE is set. Write 1 to clear. 0: No error 1: Bus stuck low
3	NOPRES	rw1	0	No-presence error. Set when the last reset saw no presence pulse (clean high release, no device); suppressed if SHORT sets on the same reset (A5). Drives vector 117 when ERRIE is set. Write 1 to clear. 0: No error 1: No presence pulse
2	PRES	r	0	Presence detected. Set when a device answered the last reset with a presence pulse (updated per reset transaction). 0: No device / not yet reset 1: Device present
1	TCIF	rw1	0	Transaction-complete flag. Set when the current transaction finishes; drives vector 117 when TCIE is set. Write 1 to clear. 0: No event 1: Transaction complete
0	BUSY	r	0	Transaction in progress. An OW0CMD launch is ignored while set. Reads 1 from the cycle of the launching write until completion (A6). 0: Idle 1: Busy

27.7.7 OWxSPU: Reserved for the strong-pullup / parasite-power stage (D15)

Offset 0x18 **Reset** 0x00000000 **Width** 32 **Address** 0x6718

Reserved for the strong-pullup / parasite-power stage (D15). Reads 0, writes ignored. The slot and the OW0CR.SPUEN field are reserved so a future driven-high strong-pullup (with its bounded firmware-armed safety window) bolts on without a register-map break.

Bits	Field	Type	Reset	Description
31:0	-	-	-	Reserved. Reads zero; write zero.

28 Direct memory access controller (DMAx)

28.1 Overview

The DMA controller (DMAx) is a configurable multi-channel single-shot direct-memory-access engine. It moves words from a source to a destination over the shared multi-core arbiter as a stream of single-word transactions, either flat-out under software command (memory-to-memory) or paced one word per peripheral data-ready event, with per-channel source and destination auto-increment, a two-level priority with word-granular round-robin between channels, an optional CRC16-CDMA2000 ride-along accumulated over the transferred data, and a hardware read-side-effect guard. The channel count is a build-time generic ({2, 4}); the register map is always the four-channel superset, so on a two-channel build the upper channel blocks and their status bits read 0. The whole transfer engine (the master-port state machine, the per-channel working counters, the round-robin picker, the CRC datapath, the pacing edge-detectors, the sticky event flags, and the two interrupt combiners) rides the free-running fabric clock (MCLK), while the register file rides the gated bus clock. It has *no pins*. The main features are listed below:

- Two or four independent single-shot channels, each programmed with a source byte address, a destination byte address, a word length, and a configuration word (DMAxCnSRC, DMAxCnDST, DMAxCnLEN, DMAxCnCFG)
- Memory-to-memory transfers under software command, or peripheral-paced transfers that move one word per data-ready event from the on-chip UARTx, QSPIx, or NFCx receivers (DMACnTRIG)
- Per-channel source and destination auto-increment (DMACnSINC, DMACnDINC): a fixed pointer drains or fills a peripheral data register, an incrementing pointer walks a buffer
- A two-level priority class (DMACnPRIO) with round-robin scheduling within a class, servicing one word per channel visit so no channel in a class starves
- An optional CRC16-CDMA2000 ride-along (DMACnCRCEN) folding each transferred word into the shared DMAxCRC accumulator, for an integrity check that costs no extra bus traffic
- A same-cycle DMABUSY status and sticky write-1-to-clear per-channel done and error flags (DMADONE, DMAERR), with two lean interrupts: combined done (vector 118) and error (vector 119)
- A hardware read-side-effect guard that refuses engine reads of the mutex bank or the interrupt-router claim word, plus an at-launch check for a zero length, a misaligned or out-of-window pointer, or a pointer into the private-TCM hole

28.2 Instances and signals

Table 28.1: DMAx instances

Instance	Base address	Interrupt vector(s)	Pins (AF)
DMA0	0x6800	118 to 119	none

Table 28.2: DMAx interrupt vectors

Vector	Instance	Description	Clear method
118	DMA0	DMA0 combined channels-done Interrupt IRQB_DMA0_DONE	Write 1 to the set DMA0SR.DMADONE channel bits
119	DMA0	DMA0 error (deny/LEN0/misalign/out-of-window) Interrupt IRQB_DMA0_ERR	Write 1 to the set DMA0SR.DMAERR channel bits

28.3 Functional description

28.3.1 Transaction Structure and Bus Behaviour

Each transferred word is a read transaction followed by a write transaction, issued as two separate single-word arbiter transactions with an observed idle gap between them. The engine never holds the arbiter grant and never bursts, so it can never pin the shared bus; it participates as one additional fair master and cannot starve the harts. A direct consequence, which firmware must respect, is that a DMA word copy is *not* atomic: another master may write the source or destination word between the read and the write of the same word. While a channel is active, ownership of its source and destination buffers is a software contract: do not let another agent race the region the DMA is walking.

DMABUSY in DMAxSR covers the launch instant with no blind window: it reads 1 in the same cycle as the launching DMAGO write (any channel active *or* a launch pending), and stays high through real engine activity, so firmware may write DMAGO and immediately spin on DMABUSY with no chance of falling through. DMAACTIVECH reports the index of the channel currently being serviced (0 when idle). Source and destination addresses read back the programmed values; DMAxCnLEN reads the current remaining word count, decrementing as the engine runs and reading 0 before the first launch, so after an abort the programmed pointers plus the remaining length show exactly where the transfer stopped.

28.3.2 Peripheral-Paced Transfers

Setting DMACnTRIG to a nonzero value paces the channel off a peripheral's receive data-ready event instead of running flat out: value 1 paces off the UARTx receive-complete event, 2 off the QSPIx receive-full event, and 3 off the NFCx payload-ready event. Each qualifying event authorizes exactly one word (one read/write pair), with the source pointer normally held fixed (DMACnSINC= 0) so successive events drain the same peripheral data register into a growing buffer. Pacing requires the source peripheral's own receive interrupt-enable bit to be set (the engine taps the peripheral's enabled interrupt level, not a raw flag); *routing* that peripheral's interrupt to a hart through the interrupt router is independent and optional; an enabled-but-unrouted source that only feeds the DMA is a normal and benign configuration.

The three sources differ in how their ready flag clears, and the engine handles each:

- UARTx receive is *read-to-clear*: the engine's own read of the receive register acknowledges the byte and re-arms the flag, so no extra transaction is needed. Each word carries one received byte in the low lane.
- QSPIx receive-full is *side-effect-free* on read, so after draining the word the engine issues one extra write transaction that writes-1-to-clear the receive-full bit in the QSPIx status register (status slot 5, at offset 0x14 within the peripheral) to re-arm the source. This is the only case that costs an extra bus transaction per word.

- NFCx payload-ready is *frame-granular*: it fires once per received reader frame, not once per byte, and a payload read auto-advances the peripheral's own buffer index rather than clearing the flag. So one frame event launches the channel's full programmed-length burst (the engine drains the whole length from the fixed payload register), then a single write-1-to-clear acknowledges the frame. Firmware sizes the length to the frame from the NFCx receive-status register.

28.3.3 The CRC Ride-Along

Setting DMACnCRCEN folds every word that channel transfers into the shared DMAxCRC accumulator (CRC16-CDMA2000, polynomial 0xC857, the same engine as the SYSTEM peripheral's CRC), feeding the four bytes of each word low byte first. There is one accumulator shared across all channels and no auto-seed: the firmware sequence is seed-then-go. Write DMAxCRC to the 0xFFFF seed, launch the channel, wait for its DMADONE, then read DMAxCRC for the result (no input or output reflection, no final XOR). Because the accumulator is shared and the engine interleaves channels word-by-word, a meaningful CRC requires exactly one DMACnCRCEN channel per measurement session; two enabled at once accumulate a CRC over the interleaved streams. One further caveat: if a DMACnCRCEN channel is aborted mid-transfer, its accumulator is left partway through and is not a meaningful check value.

28.3.4 Errors, Aborts, and the Read-Side-Effect Guard

The engine will not issue a read that would have a side effect. Before it would drive a read transaction it compares its own generated word address against a small deny table: the whole mutex-bank window (byte addresses 0x6000–0x60FF, where a read atomically claims a mutex) and the interrupt-router claim word (byte address 0x7800, where a read atomically claims an interrupt) are denied. A denied read is suppressed (no transaction is issued), the channel's DMAERR flag sets, the channel aborts, and the error interrupt fires. The router's pending, in-service, and enable registers are side-effect-free and remain legitimately DMA-readable; only the claim word is denied. The guard covers reads only. A DMA *write* into the mutex or router pages would also have a side effect (a mutex release, or an interrupt-complete), and those are *not* guarded in hardware this phase; keeping a destination pointer out of the mutex and router pages is a software contract.

DMAERR is also set, with no transfer issued, when a channel is launched with a zero length, with a source or destination that is not word-aligned, with a pointer outside the reachable shared window (at or above the extended-flash base), or with a pointer into the private tile-TCM hole (0x8000–0xBFFF, which is not an arbiter slave). These are checked at the launching DMAGO and reject the transfer outright rather than silently masking the pointer. There is no bus-fault input from the arbiter, so a bad-but-in-window pointer copies zeros or garbage exactly as a stray CPU load would; only the out-of-window, misaligned, TCM-hole, zero-length, and deny cases are caught.

A byte-lane-0 write to DMAxCR that sets a channel's DMAABORT command bit requests an orderly stop. If that channel owns an in-flight arbiter transaction the engine lets that single transaction complete normally (the handshake is never truncated), then drops the channel's busy *without* setting DMADONE or DMAERR (an abort is neither a completion nor an error). DMAABORT is self-clearing (reads 0), and the programmed pointers plus the remaining DMAxCnLEN show where the transfer stopped.

28.4 Interrupts and events

Both interrupts are combinational and never latched: the done interrupt (vector 118) is the OR of the DMADONE flags gated by DMADONEIE, and the error interrupt (vector 119) is the OR of the DMAERR flags gated

by DMAERRIE. The service routine reads DMAxSR, discovers which channels' DMADONE or DMAERR bits are set, and clears them write-1-to-clear (a set arriving the same cycle as a clear survives).

The two interrupt sources occupy vectors 118 (combined channels-done) and 119 (error); the done source takes the lower identifier, and both are delivered through the interrupt router like every other shared-peripheral source. These sources sit above the external-interrupt (meip) delivery vector, which stays fixed at vector 85, above the I3C, NFC, GPIOx port-4/port-5, RTC, PWM, and 1-Wire sources at vectors 86 through 117.

28.5 Programming

A channel is programmed by writing its source and destination byte addresses, its word length, and its configuration word, then launched by a byte-lane-0 write to DMAxCR that sets the channel's DMAGO command bit. DMAGO is a self-clearing command (it reads 0); the write captures the channel's programmed {source, destination, length, configuration} into the engine's working registers and starts it. The launch is suppressed (the content is still captured but no transfer begins) when the master enable DMAEN is clear or when that channel is already busy, so a stray second DMAGO cannot reload a channel mid-transfer. The rule is one outstanding DMAGO per channel: poll DMABUSY clear (or wait for the channel's DMADONE) before re-arming it.

28.6 Usage notes

A separate note applies to the shared NPU staging RAM: while an NPU computation is running the NPU owns that RAM, so firmware must not aim a DMA source or destination into the NPU staging window until the NPU reports idle, a software contract, not a hardware guard.

In configurations that include it, DMA0 lives in the shared peripheral window behind the multi-core arbiter (see Section 5) in the mutex-bank page, sub-slot 8 at 0x6800, so any hart can program and launch it. Because the engine clocks off the free-running fabric clock, its progress is immune to clock reconfiguration, and software must *not* apply the "write SYSCLKCR to 0 first" rule that the SMCLK peripherals need.

DMA0 is unique among the shared-window peripherals in also being an arbiter *master*: its transfer engine is an additional bus master alongside the harts, so enabling the block widens the shared fabric by one master. It participates in the same fair round-robin as the harts, issuing single-word transactions with an idle gap between each, so it is starvation-safe by the arbiter's own fairness and cannot lock the bus.

28.7 Registers

Register offsets in this chapter are relative to the instance base address in Table 28.1 (DMA0 0x6800); the generated header names each base DMA0_BASE.

Table 28.3: DMAx register summary

Offset	Name	Access	Reset	Description
0x00	DMAxCR	rw/w	0x00000000	DMA control register
0x04	DMAxSR	r/rw1	0x00000000	DMA status register
0x08	DMAxC0SRC	rw	0x00000000	
0x0C	DMAxC0DST	rw	0x00000000	
0x10	DMAxC0LEN	rw	0x00000000	Channel 0 transfer length in WORDS
0x14	DMAxC0CFG	rw	0x00000000	

Offset	Name	Access	Reset	Description
0x18	DMAxC1SRC	rw	0x00000000	
0x1C	DMAxC1DST	rw	0x00000000	
0x20	DMAxC1LEN	rw	0x00000000	Channel 1 transfer length in WORDS
0x24	DMAxC1CFG	rw	0x00000000	
0x28	DMAxC2SRC	rw	0x00000000	
0x2C	DMAxC2DST	rw	0x00000000	
0x30	DMAxC2LEN	rw	0x00000000	Channel 2 transfer length in WORDS
0x34	DMAxC2CFG	rw	0x00000000	
0x38	DMAxC3SRC	rw	0x00000000	
0x3C	DMAxC3DST	rw	0x00000000	
0x40	DMAxC3LEN	rw	0x00000000	Channel 3 transfer length in WORDS
0x44	DMAxC3CFG	rw	0x00000000	
0x48	DMAxCRC	rw	0x0000FFFF	CRC16-CDMA2000 (poly 0xC857) accumulator / seed, resets to 0xFFFF
0x4C	DMAxDESC	r0	0x00000000	

28.7.1 DMAxCR: DMA control register

Offset 0x00 **Reset** 0x00000000 **Width** 32 **Address** 0x6800

DMA control register. Holds the master enable (DMAEN), the per-channel arm/launch and orderly-abort command strobes (CHnGO / CHnABORT, self-clearing, read 0), and the two interrupt enables (DONEIE / ERRIE). Resets to 0. A byte-lane-0 write setting CHnGO[n] captures the channel n SRC,DST,LEN,CFG into the working registers and launches it (the launch is suppressed while DMAEN=0 or channel n is already busy). CHnGO/CHnABORT bits for n >= NCH read 0 and ignore writes.

Bits	Field	Type	Reset	Description
31:14	-	-	-	Reserved. Reads zero; write zero.
13	ERRIE	rw	0	Error interrupt enable. When set, DMA0SR.CHnERR (OR over channels) drives the DMA0_ERR interrupt (vector 119). 0: Interrupt disabled 1: Interrupt enabled
12	DONEIE	rw	0	Combined-done interrupt enable. When set, DMA0SR.CHnDONE (OR over channels) drives the DMA0_DONE interrupt (vector 118). 0: Interrupt disabled 1: Interrupt enabled
11:9	-	-	-	Reserved. Reads zero; write zero.
8:5	ABORT	w	0b0000	Per-channel orderly-abort command (CHnABORT[3:0], bit 5+n = channel n). Write 1 to request channel n stop; any in-flight arbiter transaction completes normally (the handshake is never truncated), then the channel's busy drops WITHOUT setting CHnDONE or CHnERR (abort is neither done nor error, D15). Self-clearing command (reads 0). Bits for n >= NCH read 0 / ignore writes.
4:1	GO	w	0b0000	Per-channel arm+launch command (CHnGO[3:0], bit 1+n = channel n). Write 1 to a bit to capture that channel's programmed descriptor and start the transfer; self-clearing command (reads 0, D8). The launch is suppressed while DMAEN=0 or that channel is busy. Bits for n >= NCH read 0 / ignore writes.

Bits	Field	Type	Reset	Description
0	EN	rw	0	Master enable. When 0 all channels are held idle and every CHnGO launch is suppressed. Set to 1 before arming a channel. 0: Disabled 1: Enabled

28.7.2 DMAxSR: DMA status register

Offset 0x04 **Reset** 0x00000000 **Width** 32 **Address** 0x6804

DMA status register. BUSY and ACTIVECH are read-only; CHnDONE and CHnERR are sticky write-1-to-clear event flags (write a 1 to a bit to clear it; writing 0 leaves it unchanged; never cleared by a read; a set arriving the same cycle as a clear survives). BUSY asserts the SAME cycle as the triggering CHnGO write (busy OR a GO pending, D8/D16), so a driver may write GO then immediately spin on BUSY. DMA0_DONE (vector 118) = (OR of CHnDONE) and CR.DONEIE; DMA0_ERR (vector 119) = (OR of CHnERR) and CR.ERRIE, both combinational. CHnDONE/CHnERR/ACTIVECH bits for n >= NCH read 0.

Bits	Field	Type	Reset	Description
31:12	-	-	-	Reserved. Reads zero; write zero.
11:9	ACTIVECH	r	0b000	Index of the currently-serviced channel (0 when idle, engine-set, D16).
8:5	ERR	rw1	0b0000	Per-channel error flags (CHnERR[3:0], bit 5+n = channel n). Set on a deny-guard hit, LEN=0 at GO, misaligned/out-of-window/TCM-hole SRC or DST (D13/A18); drives vector 119 when ERRIE is set. Write 1 to clear. Bits for n >= NCH read 0.
4:1	DONE	rw1	0b0000	Per-channel done flags (CHnDONE[3:0], bit 1+n = channel n). Set when channel n reaches LEN=0 (clean completion); drives vector 118 when DONEIE is set. Write 1 to clear. Bits for n >= NCH read 0.
0	BUSY	r	0	Any channel active OR a GO pending. Asserts the same cycle as the CHnGO write and stays high through real engine activity (no blind window, D8). 0: Idle 1: Busy

28.7.3 DMAxC0SRC

Offset 0x08 **Reset** 0x00000000 **Width** 32 **Address** 0x6808

Channel 0 source byte address in the 0x0-0x1FFFF shared window (word-aligned: bits[1:0] must be 0; bits[31:17] must be 0). The full written 32-bit value reads back; the engine presents byte_ptr(16:2) as the 15-bit arbiter word address. A misaligned, out-of-window (>= 0x20000), or TCM-hole (0x8000-0xBFFF) SRC is rejected at GO with CHnERR (D13/A18).

Bits	Field	Type	Reset	Description
31:0	C0SRC	rw	0x00000000	Source byte address (word-aligned).

28.7.4 DMAxC0DST

Offset 0x0C **Reset** 0x00000000 **Width** 32 **Address** 0x680C
Channel 0 destination byte address (same word-aligned / in-window / TCM-hole rules as C0SRC, D13/A18).

Bits	Field	Type	Reset	Description
31:0	C0DST	rw	0x00000000	Destination byte address (word-aligned).

28.7.5 DMAxC0LEN: Channel 0 transfer length in WORDS

Offset 0x10 **Reset** 0x00000000 **Width** 32 **Address** 0x6810
Channel 0 transfer length in WORDS. Decrements as the engine runs (a read returns the current remaining count, so pointers/LEN show where an aborted transfer stopped, D15); reads 0 before the first GO. LEN=0 at GO is a programming error (CHnERR, no transfer issued, D13).

Bits	Field	Type	Reset	Description
31:0	C0LEN	rw	0x00000000	Remaining transfer length in words.

28.7.6 DMAxC0CFG

Offset 0x14 **Reset** 0x00000000 **Width** 32 **Address** 0x6814
Channel 0 configuration: source/dest auto-increment (SINC/DINC, +4 per word when set, else a fixed peripheral data register), trigger source (TRIG), channel priority class (PRIO) and CRC ride-along enable (CRCEN). Program before GO.

Bits	Field	Type	Reset	Description
31:8	-	-	-	Reserved. Reads zero; write zero.
7	C0CRCEN	rw	0	CRC ride-along enable. When set, each transferred word is folded (bytes low-to-high) into the shared DMA0CRC accumulator (CRC16-CDMA2000, D14). Use one CRCEN channel per measurement session (a single shared accumulator). 0: No CRC 1: Accumulate CRC
6	C0PRIO	rw	0	Priority class. Among serviceable channels the high class (1) is picked before the low class (0); within a class a round-robin pointer prevents starvation (D7). A continuously-serviceable PRIO=1 channel can hold off PRIO=0 channels (standard DMA priority semantic). 0: Low priority class 1: High priority class

Bits	Field	Type	Reset	Description
5:2	C0TRIG	rw	0b0000	Trigger source (D9/D10/A8). 0 = software / mem-to-mem (continuously serviceable); 1 = UART0 RC (one word per received byte, read-to-clear); 2 = QSPI0 RX-full (one word per event + an extra W1C ack txn); 3 = NFC0 payload-ready (a frame event launches the full LEN-word burst + one ack); 4-15 reserved. Pacing requires the source peripheral's own receive interrupt-enable bit set (A8). 0b0000: Software / mem-to-mem 0b0001: UART0 receive-complete 0b0010: QSPI0 RX-full 0b0011: NFC0 payload-ready
1	C0DINC	rw	0	Destination auto-increment. 1 = dst += 4 per word; 0 = dst held (peripheral fill register, D11). 0: Fixed destination 1: Increment destination
0	C0SINC	rw	0	Source auto-increment. 1 = src += 4 per word (block copy); 0 = src held (peripheral data register, the paced-drain case, D11). 0: Fixed source 1: Increment source

28.7.7 DMAxC1SRC

Offset 0x18 **Reset** 0x00000000 **Width** 32 **Address** 0x6818

Channel 1 source byte address in the 0x0-0x1FFFF shared window (word-aligned: bits[1:0] must be 0; bits[31:17] must be 0). The full written 32-bit value reads back; the engine presents byte_ptr(16:2) as the 15-bit arbiter word address. A misaligned, out-of-window (>= 0x20000), or TCM-hole (0x8000-0xBFFF) SRC is rejected at GO with CHnERR (D13/A18).

Bits	Field	Type	Reset	Description
31:0	C1SRC	rw	0x00000000	Source byte address (word-aligned).

28.7.8 DMAxC1DST

Offset 0x1C **Reset** 0x00000000 **Width** 32 **Address** 0x681C

Channel 1 destination byte address (same word-aligned / in-window / TCM-hole rules as C1SRC, D13/A18).

Bits	Field	Type	Reset	Description
31:0	C1DST	rw	0x00000000	Destination byte address (word-aligned).

28.7.9 DMAxC1LEN: Channel 1 transfer length in WORDS

Offset 0x20 **Reset** 0x00000000 **Width** 32 **Address** 0x6820

Channel 1 transfer length in WORDS. Decrements as the engine runs (a read returns the current remaining count, so pointers/LEN show where an aborted transfer stopped, D15); reads 0 before the first GO. LEN=0 at GO is a programming error (CHnERR, no transfer issued, D13).

Bits	Field	Type	Reset	Description
31:0	C1LEN	rw	0x00000000	Remaining transfer length in words.

28.7.10 DMAxC1CFG

Offset 0x24 **Reset** 0x00000000 **Width** 32 **Address** 0x6824
Channel 1 configuration: source/dest auto-increment (SINC/DINC, +4 per word when set, else a fixed peripheral data register), trigger source (TRIG), channel priority class (PRIO) and CRC ride-along enable (CRCEN). Program before GO.

Bits	Field	Type	Reset	Description
31:8	-	-	-	Reserved. Reads zero; write zero.
7	C1CRCEN	rw	0	CRC ride-along enable. When set, each transferred word is folded (bytes low-to-high) into the shared DMA0CRC accumulator (CRC16-CDMA2000, D14). Use one CRCEN channel per measurement session (a single shared accumulator). 0: No CRC 1: Accumulate CRC
6	C1PRIO	rw	0	Priority class. Among serviceable channels the high class (1) is picked before the low class (0); within a class a round-robin pointer prevents starvation (D7). A continuously-serviceable PRIO=1 channel can hold off PRIO=0 channels (standard DMA priority semantic). 0: Low priority class 1: High priority class
5:2	C1TRIG	rw	0b0000	Trigger source (D9/D10/A8). 0 = software / mem-to-mem (continuously serviceable); 1 = UART0 RC (one word per received byte, read-to-clear); 2 = QSPI0 RX-full (one word per event + an extra W1C ack txn); 3 = NFC0 payload-ready (a frame event launches the full LEN-word burst + one ack); 4-15 reserved. Pacing requires the source peripheral's own receive interrupt-enable bit set (A8). 0b0000: Software / mem-to-mem 0b0001: UART0 receive-complete 0b0010: QSPI0 RX-full 0b0011: NFC0 payload-ready
1	C1DINC	rw	0	Destination auto-increment. 1 = dst += 4 per word; 0 = dst held (peripheral fill register, D11). 0: Fixed destination 1: Increment destination
0	C1SINC	rw	0	Source auto-increment. 1 = src += 4 per word (block copy); 0 = src held (peripheral data register, the paced-drain case, D11). 0: Fixed source 1: Increment source

28.7.11 DMAxC2SRC

Offset 0x28 **Reset** 0x00000000 **Width** 32 **Address** 0x6828
Channel 2 source byte address in the 0x0-0x1FFFF shared window (word-aligned: bits[1:0] must be 0; bits[31:17] must be 0). The full written 32-bit value reads back; the engine presents byte_ptr(16:2) as the 15-bit arbiter word address. A misaligned, out-of-window (>= 0x20000), or TCM-hole (0x8000-0xBFFF) SRC is rejected at GO with CHnERR (D13/A18). (reads 0 / ignores writes when the build has NCH=2)

Bits	Field	Type	Reset	Description
31:0	C2SRC	rw	0x00000000	Source byte address (word-aligned).

28.7.12 DMAxC2DST

Offset 0x2C **Reset** 0x00000000 **Width** 32 **Address** 0x682C
Channel 2 destination byte address (same word-aligned / in-window / TCM-hole rules as C2SRC, D13/A18). (reads 0 / ignores writes when the build has NCH=2)

Bits	Field	Type	Reset	Description
31:0	C2DST	rw	0x00000000	Destination byte address (word-aligned).

28.7.13 DMAxC2LEN: Channel 2 transfer length in WORDS

Offset 0x30 **Reset** 0x00000000 **Width** 32 **Address** 0x6830
Channel 2 transfer length in WORDS. Decrements as the engine runs (a read returns the current remaining count, so pointers/LEN show where an aborted transfer stopped, D15); reads 0 before the first GO. LEN=0 at GO is a programming error (CHnERR, no transfer issued, D13). (reads 0 / ignores writes when the build has NCH=2)

Bits	Field	Type	Reset	Description
31:0	C2LEN	rw	0x00000000	Remaining transfer length in words.

28.7.14 DMAxC2CFG

Offset 0x34 **Reset** 0x00000000 **Width** 32 **Address** 0x6834
Channel 2 configuration: source/dest auto-increment (SINC/DINC, +4 per word when set, else a fixed peripheral data register), trigger source (TRIG), channel priority class (PRIO) and CRC ride-along enable (CRCEN). Program before GO. (reads 0 / ignores writes when the build has NCH=2)

Bits	Field	Type	Reset	Description
31:8	-	-	-	Reserved. Reads zero; write zero.
7	C2CRCEN	rw	0	CRC ride-along enable. When set, each transferred word is folded (bytes low-to-high) into the shared DMA0CRC accumulator (CRC16-CDMA2000, D14). Use one CRCEN channel per measurement session (a single shared accumulator). 0: No CRC 1: Accumulate CRC
6	C2PRIO	rw	0	Priority class. Among serviceable channels the high class (1) is picked before the low class (0); within a class a round-robin pointer prevents starvation (D7). A continuously-serviceable PRIO=1 channel can hold off PRIO=0 channels (standard DMA priority semantic). 0: Low priority class 1: High priority class

Bits	Field	Type	Reset	Description
5:2	C2TRIG	rw	0b0000	Trigger source (D9/D10/A8). 0 = software / mem-to-mem (continuously serviceable); 1 = UART0 RC (one word per received byte, read-to-clear); 2 = QSPI0 RX-full (one word per event + an extra W1C ack txn); 3 = NFC0 payload-ready (a frame event launches the full LEN-word burst + one ack); 4-15 reserved. Pacing requires the source peripheral's own receive interrupt-enable bit set (A8). 0b0000: Software / mem-to-mem 0b0001: UART0 receive-complete 0b0010: QSPI0 RX-full 0b0011: NFC0 payload-ready
1	C2DINC	rw	0	Destination auto-increment. 1 = dst += 4 per word; 0 = dst held (peripheral fill register, D11). 0: Fixed destination 1: Increment destination
0	C2SINC	rw	0	Source auto-increment. 1 = src += 4 per word (block copy); 0 = src held (peripheral data register, the paced-drain case, D11). 0: Fixed source 1: Increment source

28.7.15 DMAxC3SRC

Offset 0x38 **Reset** 0x00000000 **Width** 32 **Address** 0x6838

Channel 3 source byte address in the 0x0-0x1FFFF shared window (word-aligned: bits[1:0] must be 0; bits[31:17] must be 0). The full written 32-bit value reads back; the engine presents byte_ptr(16:2) as the 15-bit arbiter word address. A misaligned, out-of-window (>= 0x20000), or TCM-hole (0x8000-0xBFFF) SRC is rejected at GO with CHnERR (D13/A18). (reads 0 / ignores writes when the build has NCH=2)

Bits	Field	Type	Reset	Description
31:0	C3SRC	rw	0x00000000	Source byte address (word-aligned).

28.7.16 DMAxC3DST

Offset 0x3C **Reset** 0x00000000 **Width** 32 **Address** 0x683C

Channel 3 destination byte address (same word-aligned / in-window / TCM-hole rules as C3SRC, D13/A18). (reads 0 / ignores writes when the build has NCH=2)

Bits	Field	Type	Reset	Description
31:0	C3DST	rw	0x00000000	Destination byte address (word-aligned).

28.7.17 DMAxC3LEN: Channel 3 transfer length in WORDS

Offset 0x40 **Reset** 0x00000000 **Width** 32 **Address** 0x6840

Channel 3 transfer length in WORDS. Decrements as the engine runs (a read returns the current remaining count, so pointers/LEN show where an aborted transfer stopped, D15); reads 0 before the first GO. LEN=0 at GO is a programming error (CHnERR, no transfer issued, D13). (reads 0 / ignores writes when the build has NCH=2)

Bits	Field	Type	Reset	Description
31:0	C3LEN	rw	0x00000000	Remaining transfer length in words.

28.7.18 DMAxC3CFG

Offset 0x44 **Reset** 0x00000000 **Width** 32 **Address** 0x6844

Channel 3 configuration: source/dest auto-increment (SINC/DINC, +4 per word when set, else a fixed peripheral data register), trigger source (TRIG), channel priority class (PRIO) and CRC ride-along enable (CRCEN). Program before GO. (reads 0 / ignores writes when the build has NCH=2)

Bits	Field	Type	Reset	Description
31:8	-	-	-	Reserved. Reads zero; write zero.
7	C3CRCEN	rw	0	CRC ride-along enable. When set, each transferred word is folded (bytes low-to-high) into the shared DMA0CRC accumulator (CRC16-CDMA2000, D14). Use one CRCEN channel per measurement session (a single shared accumulator). 0: No CRC 1: Accumulate CRC
6	C3PRIO	rw	0	Priority class. Among serviceable channels the high class (1) is picked before the low class (0); within a class a round-robin pointer prevents starvation (D7). A continuously-serviceable PRIO=1 channel can hold off PRIO=0 channels (standard DMA priority semantic). 0: Low priority class 1: High priority class
5:2	C3TRIG	rw	0b0000	Trigger source (D9/D10/A8). 0 = software / mem-to-mem (continuously serviceable); 1 = UART0 RC (one word per received byte, read-to-clear); 2 = QSPI0 RX-full (one word per event + an extra W1C ack txn); 3 = NFC0 payload-ready (a frame event launches the full LEN-word burst + one ack); 4-15 reserved. Pacing requires the source peripheral's own receive interrupt-enable bit set (A8). 0b0000: Software / mem-to-mem 0b0001: UART0 receive-complete 0b0010: QSPI0 RX-full 0b0011: NFC0 payload-ready
1	C3DINC	rw	0	Destination auto-increment. 1 = dst += 4 per word; 0 = dst held (peripheral fill register, D11). 0: Fixed destination 1: Increment destination
0	C3SINC	rw	0	Source auto-increment. 1 = src += 4 per word (block copy); 0 = src held (peripheral data register, the paced-drain case, D11). 0: Fixed source 1: Increment source

28.7.19 DMAxCRC: CRC16-CDMA2000 (poly 0xC857) accumulator / seed, resets to 0xFFFF

Offset 0x48 **Reset** 0x0000FFFF **Width** 32 **Address** 0x6848

CRC16-CDMA2000 (poly 0xC857) accumulator / seed, resets to 0xFFFF. Firmware writes the 0xFFFF seed before GO (the SYSTEM0 seed-via-state convention, no auto-seed) and reads the result after DONE; the engine accumulates in the MCLK domain, feeding each CRCEN word's four bytes low-to-high. No input/output reflection, no final XOR (the work.CRC16 contract). Upper bits read 0.

Bits	Field	Type	Reset	Description
31:16	-	-	-	Reserved. Reads zero; write zero.
15:0	CRC	rw	0xFFFF	CRC16-CDMA2000 accumulator/seed (reset 0xFFFF).

28.7.20 DMAxDESC

Offset 0x4C **Reset** 0x00000000 **Width** 32 **Address** 0x684C
Reserved for a descriptor-chain head pointer (out of scope this single-shot phase). Reads 0, writes ignored, provisioned so scatter-gather bolts on at this slot without a register-map break. Slots >= 20 also read 0.

Bits	Field	Type	Reset	Description
31:0	-	-	-	Reserved. Reads zero; write zero.

29 True random number generator (TRNGx)

29.1 Overview

The TRNG (TRNGx) is a ring-oscillator entropy source and harvest engine: a free-running ensemble of ring oscillators is XOR-reduced to one noisy bit, sampled by the free-running fabric clock (MCLK) through a two-flop synchronizer, decimated to let independent ring jitter accumulate between samples, and direct-packed 32 raw bits at a time into a data register that a driver drains one word at a time. A lightweight repetition-count health test watches the raw stream and raises a sticky alarm (which automatically halts harvesting) if the source looks stuck. The block has zero pins (the ring oscillators are internal combinational fabric, never a pad) and delivers one combined interrupt. The main features are listed below:

- A free-running ring-oscillator ensemble (TRNGROSEL selects/masks which rings contribute to the XOR reduction; 0000 selects every ring, the safe default-on encoding), gated on and off by TRNGEN (the power/leakage lever)
- A programmable decimator (TRNGDECIM): one raw sample is taken every $2^{\text{TRNGDECIM}}$ MCLK cycles, so software can trade harvest rate for per-bit entropy accumulation
- Direct-pack whitening into a 32-bit word (TRNGxDR), with a **read-consumes** data register: a qualified read returns the word and consumes it in the SAME access; TRNGDRDY clears the same cycle, so no read ever returns a stale or already-consumed word
- An SP 800-90B-style Repetition Count Test (RCT) health check (TRNGRCTC cutoff, default 32) that sets a sticky alarm (TRNGALMF) and **auto-halts harvesting** while it is set
- A read-only run-length diagnostic (TRNGRUNLEN) for tuning the health-test cutoff or observing ring health directly
- One combined interrupt (data-ready or health-alarm) on the router

29.1.1 The Entropy Caveat

TRNGx derives entropy from free-running digital ring-oscillator jitter sampled by MCLK on a standard-cell synthesized flow with dont_touch preservation. **This is a BRING-UP-GRADE entropy source, NOT a certified one.** RO jitter on a synthesized digital flow is sensitive to placement, temperature, voltage, and on-die coupling; the ensemble XOR + decimation + repetition-count health test raise the bar but do NOT constitute an SP 800-90B / AIS-31 validated design, and no conditioning component (DRBG) is included in hardware. Firmware MUST run the raw output through a vetted DRBG (e.g. the X3 SHA path) before using it as cryptographic key material, MUST honor the TRNGALMF health alarm (discard output while it is set), and MUST NOT treat the raw 32-bit words as directly usable keys. Certification would require an analog-assisted source and a formal entropy assessment, explicitly out of scope for this all-digital bring-up block.

29.2 Instances and signals

Table 29.1: TRNGx instances

Instance	Base address	Interrupt vector(s)	Pins (AF)
TRNG0	0x6900	121	none

Table 29.2: TRNGx interrupt vectors

Vector	Instance	Description	Clear method
121	TRNG0	TRNG0 combined data-ready/health-alarm InterruptIRQB_TRNG0	See chapter

29.3 Functional description

29.3.1 Enabling, Configuring, and Draining the Harvest

Setting TRNGEN lets the ring-oscillator ensemble oscillate and starts the decimator, the 32-bit assembler and the health test; clearing it (or an active health alarm, Section 29.3.2) parks the rings and freezes the harvest engine, preserving the last assembled word and every status flag. TRNGROSEL selects which rings in the ensemble feed the XOR reduction (0000 = every ring); TRNGDECIM sets how many MCLK cycles separate two raw samples, from every cycle (0) up to 2^{15} cycles apart; a nonzero value is recommended so consecutive samples reflect independently-accumulated jitter rather than the same ring phase. Once 32 raw samples have been packed, TRNGDRDY sets (and the data-ready interrupt fires if TRNGDRDYIE is set); software either polls TRNGDRDY or waits on the interrupt, then reads TRNGxDR. That read is the one place in this register library where reading has a side effect: a qualifying read (TRNGDRDY = 1 at the time of the access) returns the word and **consumes** it: TRNGDRDY clears in the SAME cycle as the consuming read, so a status read issued immediately afterward correctly reports "no word ready" rather than a stale "still ready". A read while TRNGDRDY = 0 (nothing assembled yet, or the previous word is still being retired internally) simply returns 0 and does not consume anything; the next genuinely new word is still delivered intact on a later read. The engine holds only one assembled word at a time (no FIFO): if a new word finishes assembling before the previous one is drained, the engine holds it internally without dropping bits and promotes it into TRNGxDR the instant the register frees up.

29.3.2 Health Test

A repetition-count run-length counter watches the raw (pre-whitening) sample stream: it increments while consecutive raw samples are identical and resets to 1 on any change. When the run length reaches the cutoff in TRNGRCTC (or the hardware default of 32 when TRNGRCTC is left at its reset value of 0), the sticky TRNGALMF flag sets and harvesting **automatically halts**: no new sample feeds the assembler, TRNGDRDY cannot re-assert, and TRNGRUN reads 0, because continuing to emit words from a source that just failed its health check is the unsafe choice. Firmware clears TRNGALMF by writing a 1 to it; harvesting resumes as soon as the raw stream produces a healthy run. TRNGRUNLEN is a read-only saturating diagnostic that exposes the CURRENT run length at any time, letting firmware watch how close the source is running to the cutoff without waiting for an alarm. Only the Repetition Count Test is implemented in this block; an Adaptive Proportion Test is a possible future addition and the register map reserves headroom for it.

29.3.3 Clocking

The whole harvest engine (the RO synchronizer, the decimator, the assembler, the health test, and the interrupt combiner) rides the free-running fabric clock (MCLK), so its timing is immune to clock reconfig-

uration and, unlike the SMCLK-domain serial peripherals, software does *not* need to write SYSCLKCR to 0 before using it. The ring oscillators themselves are independent of every chip clock: they self-oscillate from their own inverter delays whenever TRNGEN is set (which is precisely why they are usable as an entropy source), and MCLK only samples them; it never drives them.

29.4 Interrupts and events

The one combined interrupt is (TRNGDRDY ∧ TRNGDRDYIE) ∨ (TRNGALMF ∧ TRNGALMIE), combinational and never latched. A service routine reads TRNGxSR to demultiplex the source, then either drains TRNGxDR (which clears TRNGDRDY as a side effect of the read) or writes 1 to TRNGALMF to clear the alarm, before returning.

29.5 Programming

Bring-up sequence:

1. Program TRNGROSEL (0000 selects every ring) and TRNGDECIM (nonzero, so consecutive samples accumulate independent jitter), and set the health-test cutoff TRNGRCTC if the hardware default of 32 is not wanted.
2. Set TRNGEN. The rings start oscillating and the decimator, assembler and health test start.
3. Poll TRNGDRDY, or enable TRNGDRDYIE and take the interrupt.
4. Read TRNGxDR. The read returns the word and consumes it in the same access.
5. On TRNGALMF, discard the harvest and write 1 to the flag to clear it. Harvesting stays halted until it is cleared.

29.6 Usage notes

Treat every word out of TRNGx as raw, unconditioned entropy: run it through a vetted DRBG before using it as key material, seed material, or a nonce; never use it directly. Discard anything read while TRNGALMF is set (or, better, avoid reading while the alarm is set at all; TRNGDRDY will not assert during a halt). Because the engine holds only one word at a time, poll TRNGDRDY (or take the interrupt) between reads rather than reading on a fixed schedule. A nonzero TRNGDECIM is recommended for real accumulation between samples; see Section 29.3.2 and the entropy caveat above.

29.7 Registers

Register offsets in this chapter are relative to the instance base address in Table 29.1 (TRNG0 0x6900); the generated header names each base TRNG0_BASE.

Table 29.3: TRNGx register summary

Offset	Name	Access	Reset	Description
0x00	TRNGxCR	rw	0x00000000	TRNG control register
0x04	TRNGxSR	r/rw1	0x00000000	TRNG status register

Offset	Name	Access	Reset	Description
0x08	TRNGxDR	r	0x00000000	Entropy data register
0x0C	TRNGxHT	rw/r	0x00000000	Health-test control/diagnostic register

29.7.1 TRNGxCR: TRNG control register

Offset 0x00 **Reset** 0x00000000 **Width** 32 **Address** 0x6900

TRNG control register. EN gates the entropy source (0 parks the RO rings, the power/leakage lever, and freezes harvesting; flags and the last assembled word are preserved). DRDYIE/ALMIE independently gate the two halves of the combined interrupt (vector 121). ROSEL selects/masks which rings in the ensemble contribute to the XOR reduction (0000 = all rings, the safe default-on encoding, D6 ruling 4). DECIM sets the decimation: one raw sample is taken every 2^DECIM MCLK cycles (0 = every cycle); a larger DECIM lets more independent ring jitter accumulate between samples. Resets to 0 (disabled).

Bits	Field	Type	Reset	Description
31:12	-	-	-	Reserved. Reads zero; write zero.
11:8	DECIM	rw	0b0000	Decimation exponent. One raw sample is taken every 2^TRNGDECIM MCLK cycles (0 = every MCLK cycle, up to 32768 cycles at 15). A nonzero value is recommended for real accumulation between independent samples (D7).
7:4	ROSEL	rw	0b0000	Ring-oscillator ensemble contribution select/mask, forwarded to the entropy source. 0000 selects every ring in the ensemble (the reset-safe all-on encoding); a nonzero value masks a subset of the NRO rings into the XOR reduction (D6 ruling 4).
3	-	-	-	Reserved. Reads zero; write zero.
2	ALMIE	rw	0	Health-alarm interrupt enable. When set, TRNG0SR.ALMF drives the combined interrupt (vector 121). 0: Interrupt disabled 1: Interrupt enabled
1	DRDYIE	rw	0	Data-ready interrupt enable. When set, TRNG0SR.DRDY drives the combined interrupt (vector 121). 0: Interrupt disabled 1: Interrupt enabled
0	EN	rw	0	Entropy engine enable. 0 parks the RO ensemble and freezes the decimator/assembler/health test; the last assembled word, DRDY, and ALMF are preserved. 1 lets the rings oscillate and harvesting proceed (subject to the health-test auto-halt, D8/ruling 3). 0: Disabled (rings parked) 1: Enabled

29.7.2 TRNGxSR: TRNG status register

Offset 0x04 **Reset** 0x00000000 **Width** 32 **Address** 0x6904

TRNG status register. DRDY and RUN are read-only levels; ALMF is a sticky write-1-to-clear health-alarm flag (write a 1 to clear it; writing 0 leaves it unchanged; never cleared by a read; a set arriving the same cycle as a clear survives, D8). DRDY reflects the D9 blind-window-corrected level: a qualifying TRNG0DR read clears DRDY in the SAME cycle as the consuming read (an SR read issued immediately after a DR read observes DRDY=0 even though the harvest engine has not yet finished tearing the word down internally),

so polling DRDY right after draining TRNG0DR is always accurate. The combined interrupt (vector 121) is (DRDY and TRNGDRDYIE) or (ALMF and TRNGALMIE).

Bits	Field	Type	Reset	Description
31:3	-	-	-	Reserved. Reads zero; write zero.
2	RUN	r	0	Entropy engine running: TRNGEN is set and no health alarm is currently latched (D12). Reads 0 whenever the rings are parked, either because TRNGEN=0 or because TRNGALMF is set (auto-halt). 0: Idle / halted 1: Running
1	ALMF	rw1	0	Health-test alarm flag (sticky). Set when the repetition-count health test (TRNG0HT.RCTC, or the hardware default of 32) reaches its cutoff; while set, harvesting is AUTO-HALTED (D8/ruling 3: no new sample feeds the assembler, DRDY cannot re-assert, RUN reads 0), write 1 to clear and let harvesting resume once the raw stream is healthy again. Drives vector 121 when TRNGALMIE is set. Write 1 to clear. 0: No alarm 1: Repetition-count cutoff reached
0	DRDY	r	0	Data ready: a fresh 32-bit entropy word is available in TRNG0DR. Clears the SAME cycle as the qualifying TRNG0DR read that consumes the word (D9 blind-window fix), and re-asserts only once the harvest engine assembles the NEXT word. 0: No word ready (a TRNG0DR read now returns 0 and does not consume) 1: Word ready

29.7.3 TRNGxDR: Entropy data register

Offset 0x08 **Reset** 0x00000000 **Width** 32 **Address** 0x6908

Entropy data register. [31:0] is the most recently assembled 32-bit entropy word, direct-packed MSB-first from decimated ring-oscillator samples (D7, ruling 1: no hardware whitening, firmware DRBGs the output, D16). READ SIDE EFFECT (D9, the one exception to "no read side effects" in this register library): a qualifying read (TRNGDRDY=1 at the time of the access) CONSUMES the word, TRNG0SR.DRDY clears the SAME cycle, and the harvest engine begins assembling the next word. The consume is gated to fire EXACTLY ONCE per real bus access (a repeated internal edge in the same access cannot double-pop). A read while TRNGDRDY=0 (no word ready, or the previous word is still being retired) returns 0 and does NOT consume anything, the next real word is still delivered intact on a later read. Writes are ignored.

Bits	Field	Type	Reset	Description
31:0	DR	r	0x00000000	Entropy word. A read while TRNGDRDY=1 returns the word and consumes it (DRDY clears the same cycle, D9); a read while TRNGDRDY=0 returns 0 and does not consume.

29.7.4 TRNGxHT: Health-test control/diagnostic register

Offset 0x0C **Reset** 0x00000000 **Width** 32 **Address** 0x690C

Health-test control/diagnostic register. RCTC is the repetition-count cutoff: the number of consecutive identical raw samples that trips the alarm (TRNG0SR.ALMF). RCTC=0 selects the hardware default of 32

(ruling 3), so the reset value is a safe default-on cutoff; a nonzero value is a firmware override. RUNLEN is a read-only saturating diagnostic exposing the CURRENT consecutive-identical-sample run length (resets to 1 on any change, saturates at 63), useful for tuning RCTC or observing ring health without waiting for an alarm. Other bits reserved, read 0. Resets to 0.

Bits	Field	Type	Reset	Description
31:22	-	-	-	Reserved. Reads zero; write zero.
21:16	RUNLEN	r	0x00	Current consecutive-identical-raw-sample run length (0 to 63, saturating diagnostic). Resets to 1 (via have_prev) on the first sample after any change; does not itself trigger the alarm, compare against TRNGRCTC to see how close the raw stream is to tripping it.
15:8	-	-	-	Reserved. Reads zero; write zero.
7:0	RCTC	rw	0x00	Repetition-count cutoff (0 to 255). 0 selects the hardware default of 32 consecutive identical raw samples (D8/ruling 3); a nonzero value overrides it.

30 I2C target (I2CTx)

30.1 Overview

The hardware-autonomous I2C target (I2CTx) is an I2C slave engine that handles the wire protocol entirely in hardware: it matches its 7-bit address (with a per-bit wildcard mask and an optional general-call response), receives and transmits one byte at a time with ready/empty status, stretches the clock for lossless flow control, and flags every framing event (START, STOP, repeated-START, NACK) and a stuck-bus condition. It complements the software-serviced slave-mode registers of I2C0/I2C1: where those cores expose the slave state to firmware for per-byte servicing, I2CT0 needs no bit-banging: it answers its address, ACKs, buffers a byte, and raises an interrupt, all in hardware. The whole engine runs on the free-running fabric clock (MCLK); the SDA and SCL pins are each sampled through an independent two-flop synchronizer, so no flip-flop is ever clocked by a bus pin (the deliberate contrast with the pad-clocked slave path of I2C0). The main features are listed below:

- 7-bit address match with a wildcard mask (I2CTSAD/I2CTSADM, a mask bit makes that address bit a don't-care) and an optional general-call response (I2CTGCEN)
- Byte-at-a-time receive (I2CTxRX + I2CTRXF) and transmit (I2CTxTX + I2CTTXE), MSB-first per the I2C convention, with a side-effect-free received-byte read
- Hardware clock stretching (I2CTCSEN): the target holds SCL low after an ACK until firmware services the transfer, giving lossless flow control; with stretching disabled the target never holds SCL and firmware must service within one bit-time
- Full framing detection (I2CTSTOPF, I2CTRSTARTF, I2CTNACKF, I2CTAMF/I2CTGCF) plus a read-only bus-busy (I2CTBUSY) and transfer-direction (I2CTTM) status
- Receive-overrun protection: a byte arriving while the previous one is unread sets I2CTOVF, is auto-NACKed and dropped
- A configurable stuck-SCL watchdog (I2CTxWDG): a timeout on SCL held low while busy sets I2CTERRF and aborts the transaction; the reset value 0 disables it (default-off)
- Sticky write-1-to-clear status flags and two combined interrupts (vector 122 address/error, vector 123 tx-ready/rx-full)

30.2 Instances and signals

Table 30.1: I2CTx instances

Instance	Base address	Interrupt vector(s)	Pins (AF)
I2CT0	0x6A00	122 to 123	none

Table 30.2: I2CTx interrupt vectors

Vector	Instance	Description	Clear method
122	I2CT0	I2CT0 combined address-match/error InterruptIRQB_I2CT0_AE	See chapter
123	I2CT0	I2CT0 combined tx-ready/rx-full InterruptIRQB_I2CT0_DATA	See chapter

30.3 Functional description

30.3.1 Address Match, Direction, and Framing

On a START the target shifts in the address byte MSB-first and matches it against I2CTSAD masked by I2CTSADM (a mask bit is a don't-care); with I2CTGCEN set it also answers the general-call address (0x00, write direction) and sets I2CTGCF alongside I2CTAMF. A match drives the address ACK (SDA pulled low; the target never drives SDA high, open-drain) and latches the transfer direction into I2CTTM (1 = target-transmitter / host read, 0 = target-receiver / host write); a mismatch is silently ignored. I2CTBUSY tracks the transaction from START to STOP and distinguishes an initial START from a repeated-START (a START while busy sets I2CTRSTARTF and re-enters the address phase without dropping busy, the classic write-address-then-read reversal). Every framing event releases the bus immediately, so the target never holds SDA or SCL across a boundary.

The wire protocol itself (START, address and direction, ACK, data, STOP) is the same one shown in Figure 20.1 in the I2Cx chapter.

30.3.2 Receive, Transmit, and Clock Stretching

On a host write the target shifts the byte in, and if the receive buffer is free it ACKs, latches the byte into I2CTxRX and sets I2CTRXF; firmware learns via the vector-123 interrupt, reads I2CTxRX (side-effect-free) and writes 1 to I2CTRXF to free the buffer. On a host read the target needs bytes from firmware: I2CTTXE asserts when the transmit buffer is empty, firmware loads I2CTxTX (a write clears I2CTTXE in the same cycle it loads a byte, so a poll right after the write is never blind), and the byte shifts out MSB-first. When clock stretching is enabled (I2CTCSEN) the target holds SCL low after the ACK until firmware services the transfer: the host, waiting to raise SCL for the next bit, simply stalls; release is synchronous in MCLK, which is safe because the host is already stopped. With stretching disabled the target never holds SCL: firmware must keep up within one bit-time or accept an overrun (I2CTOVF, auto-NACK) on receive, or a defined stale 0xFF on transmit.

30.3.3 Bus Speed and the MCLK Ratio

The block is purely edge-reactive (it never counts raw clock cycles for protocol timing), so nothing in it scales with the actual bus frequency; only the synchronizer and edge-detect latency (a few MCLK cycles per SDA/SCL event) sets a floor on how fast SCL may run. The guaranteed floor is $f_{SCL} \leq \text{MCLK}/24$: at the nominal 24 MHz MCLK both Standard-mode (100 kHz) and Fast-mode (400 kHz) are guaranteed with ample margin.

30.4 Interrupts and events

The two combined interrupts are (I2CTAMF ∨ I2CTGCF ∨ I2CTOVF ∨ I2CTNACKF ∨ I2CTSTOPF ∨ I2CTRSTARTF ∨ I2CTERRF) ∧ I2CTAEIE on vector 122 (address/error) and (I2CTRXF ∨ I2CTTXE) ∧ I2CTDATAIE on vector 123 (tx-ready/rx-full), both combinational and never latched; the service routine reads I2CTxSR, demultiplexes

the source and clears whichever event flags fired before returning. The two sources sit above the external-interrupt (meip) delivery vector fixed at 85 and above every other shared-peripheral source.

30.5 Programming

Program I2CTSAD and I2CTSADM with the address and its wildcard mask, choose the general-call response with I2CTGCEN, select clock stretching with I2CTCSEN, and set the stuck-SCL timeout in I2CTxWDG if the bus needs one (the reset value 0 disables it). Service on receive is: take the vector-123 interrupt, read I2CTxRX, write 1 to I2CTRXF. Service on transmit is: take the same interrupt on I2CTTXE and write the next byte to I2CTxTX. Framing and error flags are cleared the same way, by writing 1 to the bit that fired.

30.6 Usage notes

Because the engine clocks off the free-running fabric clock, its timing is immune to clock reconfiguration, and software must *not* apply the "write SYSCLKCR to 0 first" rule that the SMCLK-domain I2C0/I2C1 cores need; I2CT0 is unaffected by SMCLK reconfiguration.

In configurations that include it, I2CT0 lives in the shared peripheral window behind the multi-core arbiter (see Section 5) in the mutex-bank page, sub-slot 10 at 0x6A00, so any hart can use it.

The block adds no pins: it shares the open-drain SDA0/SCL0 pads with I2C0 through a wired-AND merge on the drive-low (DIR) plane (the SCL merge realizes the target's clock stretching and the SDA merge its ACK/data driving), so the two engines can share one bus, including the on-die host-side loopback used for self-test. When I2CT0 is not instantiated, those pads carry only I2C0 exactly as before.

30.7 Registers

Register offsets in this chapter are relative to the instance base address in Table 30.1 (I2CT0 0x6A00); the generated header names each base I2CT0_BASE.

Table 30.3: I2CTx register summary

Offset	Name	Access	Reset	Description
0x00	I2CTxCR	rw	0x00000000	I2C target control register
0x04	I2CTxSR	r/rw1	0x00000000	I2C target status register
0x08	I2CTxTX	rw	0x00000000	Transmit byte buffer
0x0C	I2CTxRX	r	0x00000000	
0x10	I2CTxWDG	rw	0x00000000	SCL-low watchdog timeout

30.7.1 I2CTxCR: I2C target control register

Offset 0x00 **Reset** 0x00000000 **Width** 32 **Address** 0x6A00

I2C target control register. Enables the target (EN), the general-call response (GCEN), and hardware clock stretching (CSEN); holds the two interrupt enables (AEIE for the address/error IRQ at vector 122, DATAIE for the tx-ready/rx-full IRQ at vector 123); and carries the 7-bit target address (SAD) with a per-bit wildcard mask (SADM, a 1 makes that address bit dont-care). Resets to 0 (target idle, SDA/SCL released).

Bits	Field	Type	Reset	Description
31:23	-	-	-	Reserved. Reads zero; write zero.

Bits	Field	Type	Reset	Description
22:16	SADM	rw	0x00	Address match mask: a 1 in a bit position makes the corresponding SAD address bit a wildcard (dont-care) during the match.
15	-	-	-	Reserved. Reads zero; write zero.
14:8	SAD	rw	0x00	7-bit target address matched against the incoming address byte (masked by SADM).
7:5	-	-	-	Reserved. Reads zero; write zero.
4	DATAIE	rw	0	Data interrupt enable. When set, the OR of RXF and TXE drives the combined interrupt at vector 123. 0: Interrupt disabled 1: Interrupt enabled
3	AEIE	rw	0	Address/error interrupt enable. When set, the OR of the address/error status flags (AMF, GCF, OVF, NACKF, STOPF, RSTARTF, ERRF) drives the combined interrupt at vector 122. 0: Interrupt disabled 1: Interrupt enabled
2	CSEN	rw	0	Clock-stretch enable. When set, the target holds SCL low after an ACK until firmware services the transfer (reads I2CTRX / writes I2CTTX), giving lossless flow control. When 0 the target never stretches; firmware must service within one bit-time or accept an RX overrun / a stale 0xFF transmit byte. 0: No clock stretching 1: Clock stretching enabled
1	GCEN	rw	0	General-call enable. When set, the target also matches the general-call address (0x00, write direction) and sets GCF alongside AMF. 0: General call ignored 1: General call answered
0	EN	rw	0	Target enable. When 0 the FSM is held idle with SDA/SCL released; the received byte and the status flags are preserved. Set to 1 to respond on the bus. 0: Disabled (FSM idle, bus released) 1: Enabled

30.7.2 I2CTxSR: I2C target status register

Offset 0x04 **Reset** 0x00000000 **Width** 32 **Address** 0x6A04

I2C target status register. BUSY, TM and TXE are read-only levels; AMF, GCF, RXF, OVF, NACKF, STOPF, RSTARTF and ERRF are sticky write-1-to-clear event flags (write a 1 to a bit to clear it; writing 0 leaves it unchanged; never cleared by a read; a set arriving the same cycle as a clear survives). The address/error interrupt (vector 122) is (AMF or GCF or OVF or NACKF or STOPF or RSTARTF or ERRF) and AEIE; the data interrupt (vector 123) is (RXF or TXE) and DATAIE.

Bits	Field	Type	Reset	Description
31:11	-	-	-	Reserved. Reads zero; write zero.

Bits	Field	Type	Reset	Description
10	ERRF	rw1	0	Error flag: a protocol error or the SCL-low watchdog timeout (I2CTWDG). On expiry the transaction is aborted and BUSY drops. Write 1 to clear. 0: No error 1: Protocol error / watchdog timeout
9	RSTARTF	rw1	0	Repeated-START flag: a repeated-START was detected (START while BUSY); the target re-enters the address phase without dropping BUSY. Write 1 to clear. 0: No event 1: Repeated-START detected
8	STOPF	rw1	0	STOP flag: a STOP condition was detected. Write 1 to clear. 0: No event 1: STOP detected
7	NACKF	rw1	0	NACK flag: the host NACKed a transmitted byte (normal read termination), or the target auto-NACKed a received byte on overrun (disambiguate with TM/OVF). Write 1 to clear. 0: No event 1: NACK handshake
6	OVF	rw1	0	Receive-overrun flag: a byte arrived while RXF was still set (previous byte unread); the target auto-NACKed and dropped the byte. Write 1 to clear. 0: No overrun 1: RX overrun
5	TXE	r	0	Transmit-empty level: asserted while the target is in transmit mode and needs the next byte loaded into I2CTTX. Reads clear (0) in the same cycle as the I2CTTX write that loads a byte (BUSY-same-cycle rule). 0: Byte loaded / not transmitting 1: Load I2CTTX
4	RXF	rw1	0	Receive-full flag: a received byte is available in I2CTRX. Read I2CTRX (side-effect-free) then write 1 here to free the buffer. Write 1 to clear. 0: Buffer empty 1: Byte available
3	GCF	rw1	0	General-call flag: the general-call address matched (set alongside AMF when GCEN is set). Write 1 to clear. 0: No event 1: General call matched
2	AMF	rw1	0	Address-match flag: our address (masked) matched this transaction. Write 1 to clear. 0: No event 1: Address matched
1	TM	r	0	Transfer direction (latched R/W from the matched address). 1 = target-transmitter (host read); 0 = target-receiver (host write). 0: Target-receiver (host write) 1: Target-transmitter (host read)

Bits	Field	Type	Reset	Description
0	BUSY	r	0	Bus busy: set on START, cleared on STOP or a watchdog abort. Distinguishes an initial START from a repeated-START. 0: Bus idle 1: Transaction in progress

30.7.3 I2CTxTX: Transmit byte buffer

Offset 0x08 **Reset** 0x00000000 **Width** 32 **Address** 0x6A08

Transmit byte buffer. A byte-lane-0 write loads the next transmit byte and clears TXE (the loaded byte is shifted out MSB-first on the next host read). Reads back the last-written value.

Bits	Field	Type	Reset	Description
31:8	-	-	-	Reserved. Reads zero; write zero.
7:0	TX	rw	0x00	Next transmit byte (0 to 255).

30.7.4 I2CTxRX

Offset 0x0C **Reset** 0x00000000 **Width** 32 **Address** 0x6A0C

Last received byte (read-only, side-effect-free, D9): the most recent host-written byte, valid while RXF is set. A read never clears RXF (write 1 to I2CTSR.RXF to free the buffer). Reset 0.

Bits	Field	Type	Reset	Description
31:8	-	-	-	Reserved. Reads zero; write zero.
7:0	RX	r	0x00	Received byte.

30.7.5 I2CTxWDG: SCL-low watchdog timeout

Offset 0x10 **Reset** 0x00000000 **Width** 32 **Address** 0x6A10

SCL-low watchdog timeout. A counter increments while SCL is held low and the bus is busy; on reaching WDTO x 256 MCLK cycles it sets ERRF, aborts the transaction and drops BUSY. WDTO = 0 disables the watchdog (the reset value, so the default is off, which is identity-safe). The counter resets on every SCL rising edge and when not busy, so a healthy idle bus never trips it.

Bits	Field	Type	Reset	Description
31:16	-	-	-	Reserved. Reads zero; write zero.
15:0	WDTO	rw	0x0000	SCL-low watchdog timeout in units of 256 MCLK cycles (0 to 65535); 0 disables the watchdog.

31 Event fabric (EVFAB)

31.1 Overview

The event/trigger fabric (EVFAB) is a PPI-style crossbar that lets peripherals command each other with no processor in the loop. Eight independent channels each hold one {EVSEL, TASKSEL} pair; when the event a channel selects fires and the channel is enabled, the fabric emits a registered one-MCLK pulse on the task it selects, one MCLK after the event. The whole block (the event front end, the crossbar, the output register, and the sticky status words) rides the free-running fabric clock (MCLK) in the always-on shared domain: WFI keeps it alive, field-power mode only slows it down, and PWRCTRL never gates it. A chain therefore keeps firing with every hart asleep and the bus completely idle, which is the entire point of the block. EVFAB is never a bus master, never stalls, and never rate-limits: it only ever adds one MCLK of latency between an event and the task pulse it drives. The main features are listed below:

- Eight channels (EVFCH0CFG…EVFCH7CFG), each a fixed {EVSEL, TASKSEL} pair; the channel-configuration array reserves sixteen addresses for future growth, so slots 8–15 read 0 and ignore writes on this build
- Sixteen wired event lines (EVFEVSELn encodes 5 bits; code 31 is the documented NONE), each carried into the fabric as a pulse, a toggle, or a level according to its source's clock domain; the fabric owns *all* clock-domain crossing, so no producer peripheral needs its own synchronizer for this feature
- Ten wired task lines (EVFTASKSELn encodes 4 bits), delivered as clean registered one-MCLK pulses; a consumer never sees a held level from the fabric
- Write-1-to-set/write-1-to-clear channel-enable aliases (EVFCHENSET/EVFCHENCLR) so two harts can enable or disable different channels without racing a read-modify-write of EVFCHEN
- Sticky, write-1-to-clear EVFFIRED (channel fired), EVFOVR (a pulse was degraded), and EVFEVSTAT (a raw event was seen, recorded even while the fabric is disabled) status words, all set-wins over a same-cycle clear
- Firmware injection of a channel firing (EVFCHTRIG) or a raw event (EVFEVTRIG) with no producer hardware required at all, for bring-up and self-test
- Vectorless in this version: the interrupt output is a constant 0 and the EVFIE slot is reserved, so firmware polls EVFSR, whose two bits are live OR-reductions of EVFFIRED and EVFOVR

31.2 Instances and signals

Table 31.1: EVFAB instances

Instance	Base address	Interrupt vector(s)	Pins (AF)
EVFAB	0x6B00	none	none

31.3 Functional description

31.3.1 Producers: the Event Lines

Table 31.2 lists every wired EVFEVSELn code. Every tap is taken from its source peripheral's flag-*set* condition, before any interrupt mask is applied, so a chain works even with every relevant interrupt disabled at the source, and EVFEVSTAT can directly catch a tap that was mistakenly wired after a mask instead of before one. Each line carries one of three front-end modes, selected once per line at synthesis time and fixed for this build:

- **Pulse (P):** the source is already a clean one-MCLK pulse in the MCLK domain, and the fabric passes it through combinationally (no flop, so the in-fabric latency for a pulse input is exactly the one output-register stage). A pulse wider than one MCLK cycle fires its channels once per cycle it is high, a producer contract, not a fabric limitation.
- **Toggle (T):** the source is a level that flips state in its own clock domain (a domain the fabric cannot assume is MCLK, or even free-running). The fabric two-flop synchronizes it into MCLK and XORs consecutive samples, producing exactly one MCLK pulse per flip in *either* direction. A toggle input costs two extra MCLK cycles of latency versus a pulse input (three total from the flip to the task pulse).
- **Level (L):** the source is a level that the fabric two-flop synchronizes and edge-detects on the *rising* edge only; a level that stays high forever fires exactly once, and the falling edge is silent.

EVSEL	Source	Mode	Notes
0	RTC0 tick	P	one-second periodic tick
1	RTC0 alarm	P	one-shot compare match
2	PWM0 period	P	period-boundary tick
3	PWM0 fault	P	FLTF set condition
4	TIMER0 compare0	T	T0CMP0 match toggle
5	TIMER0 overflow	T	counter-overflow toggle
6	TIMER1 compare0	T	same shape as event 4, second instance
7	UART0 receive	P	receive-complete pulse
8	NFC0 field-detect	T	carrier-presence toggle
9	NFC0 rx-frame	T	reader-frame-received toggle
10	DMA0 channel-0 done	P	DMADONE[0] set pulse
11	DMA0 channel-1 done	P	DMADONE[1] set pulse
12	DMA0 error (combined)	P	OR of all channel error-set pulses
13	TRNG0 data-ready	L	TRNGDRDY level
14	I2CT0 address-match	P	I2CTAMF set pulse
15	GPIO0 masked pad edge	(special)	see below
16–31	reserved	–	always inert (no line drives them)

Table 31.2: EVFEVSELn event lines

Event 15 does not come from a plain input line: it is built *inside* the fabric from GPIO0's eight raw pad levels (the pre-mask value, independent of every pin's PxIE interrupt enable). Each of the eight pads is independently two-flop synchronized and rising-edge detected, ANDed against its own bit in EVFGPIOMASK, and the eight results are OR-reduced onto event 15; a bit clear in EVFGPIOMASK means that pad's edges

never reach the fabric at all, which also means a masked-off, floating, or unbonded pad can never inject an unknown value into the crossbar. EVFGPIOMASK resets to 0, so this path is completely inert out of reset.

Events whose source peripheral is a config-droppable block are only wired in configurations that include that block: RTC0 (event 0/1), PWM0 (event 2/3), TIMER1 (event 6), NFC0 (event 8/9), DMA0 (event 10/11/12), TRNG0 (event 13), and I2CT0 (event 14). In a configuration that drops one of these blocks its event line is tied inactive, so a channel programmed with that EVFEVSELn code is simply and permanently inert, exactly as if it had been programmed with the reserved NONE code. TIMER0, UART0, and GPIO0 are always present, so events 4, 5, 7, and 15 are wired in every configuration.

The EVFEVSELn codes are an ABI: once a producer ships, its code number never moves, and a firmware image that only knows the codes it uses is unaffected by every future addition to this table. Codes 16 through 31 are reserved for future producers (deferred TIMER0/TIMER1 input captures, the remaining GPIOx ports, QSPI0, OW0, I3C0, UART0/UART1 transmit events, NFC0 transmit-done/CRC-error, further DMA0 channels, and a TRNG0 health alarm); programming one of them, or code 31 (the documented NONE encoding), leaves a channel structurally inert with no special-case behavior anywhere in the fabric.

31.3.2 Consumers: the Task Lines

Table 31.3 lists every wired EVFTASKSELn code. Every task input is, by construction, a clean registered one-MCLK pulse: the single output-register stage described in Section 31.3.3 is the *only* flop between an event and every task line, so a consumer can never observe a held level from the fabric and never needs its own pulse-stretcher. Where a task's own action would be hazardous if delivered as a held level (for example, a toggle that would produce a continuous square wave rather than one event), that shape is simply not offered as a task in this version.

TASKSEL	Consumer	Action
0	DMA0 channel-0 GO	launches channel 0 exactly like a DMAGO write
1	DMA0 channel-1 GO	launches channel 1 exactly like a DMAGO write
2	TIMER0 START	sets the timer's run bit
3	TIMER0 STOP	clears the timer's run bit
4	PWM0 fault trip	equivalent to a FLTTRIG write
5	PWRCTRL tile wake	clears the addressed tile's held-in-gate request
6	NPU0 THINK	starts a computation exactly like an NPUCR bit-16 write
7	GPIO0 OUT-SET	sets every pin selected in PxTASK
8	GPIO0 OUT-CLR	clears every pin selected in PxTASK
9–15	reserved	EVFFIRED still sets; no task line exists

Table 31.3: EVFTASKSELn task lines

Tasks 0/1 (DMA0), 4 (PWM0), and 6 (NPU0) are only wired in configurations that include those blocks; TIMER0, PWRCTRL, and GPIO0 are always present, so tasks 2, 3, 5, 7, and 8 are wired in every configuration. A channel programmed with EVFTASKSELn 9 through 15, or targeting a task whose consumer block is absent from the configuration, still sets EVFFIRED on a matching event (the fabric decoded a real firing) but drives no task line and can never set EVFOVR for that channel; there is no consumer busy line to compare against, so the fabric correctly reports nothing was degraded.

A firing channel launches its task action *in addition to*, not instead of, the consumer's ordinary software-driven path: a task-driven DMA0 GO behaves exactly like a register DMAGO write (subject to the same busy/enable checks), a task-driven TIMER0 START or STOP behaves exactly like the corresponding control-register write, and so on. Where a task and a same-cycle CPU write could disagree on the same bit, the

peripheral's own chapter states which one wins (GPIO0's PxTASK outputs, for example, apply after the CPU's write, and a same-cycle SET+CLEAR on the same pin resolves to CLEAR).

No task consumer counts firings. Two or more channels landing on the same task in the same cycle merge into exactly one task pulse (recorded as EVFOVR on every colliding channel, Section 31.3.3); every wired task line is safe to receive that merged single pulse, because none of them has counting semantics: a GO, a START, a STOP, a fault trip, a tile-wake, a THINK, or a pin set/clear are all idempotent-per-cycle actions. This is a binding property of the six touched blocks, not something a driver must additionally guard: if a future task addition ever needed to count distinct firings rather than react to "at least one fired," it could not use this fabric's task interface as-is.

31.3.3 Channel Enable, Status, and Injection

The fabric lives at 0x6B00 (Section 31.6.1), decoded as sixty-four consecutive words. EVFCR bit 0 is the single global enable; EVFCHEN then gates each of the eight channels independently. A channel fires only when EVFCR.EN, its own EVFCHEN bit, and its selected event are all true; clearing either gate makes the channel *completely* inert: no task pulse, no EVFFIRED, no EVFOVR. EVFEVSTAT sits upstream of both gates and keeps recording regardless, which is deliberate: it lets firmware (or a bench) prove that an event tap was taken *before* any mask, by disabling everything downstream and still observing EVFEVSTAT set.

EVFCHEN may be written directly, but firmware normally uses the write-1-to-set/write-1-to-clear aliases EVFCHENSET and EVFCHENCLR instead: each is a mask where writing a 1 to bit n sets or clears EVFCHEN[n] and writing 0 leaves that bit untouched, and both read back the live EVFCHEN value. Two harts enabling or disabling *different* channels can therefore each issue one write with no read-modify-write race and no risk of one hart's write clobbering the other's in-flight change, the same convention as PxOUTS/PxOUTC and the IRQROUTER claim mechanism. Each EVFCHnCFG word carries that channel's EVFEVSELn field, its EVFTASKSELn field, and a read-only EVFENRn mirror of its live EVFCHEN bit, so a driver can read a channel's full configuration and enable state back in one access.

EVFFIRED (one sticky bit per channel) sets whenever an enabled channel fires and clears only on an explicit write-1; a firing that lands in the same cycle as a write-1-to-clear leaves the bit *set* (set-wins), so a status word being drained can never silently swallow an event that arrives mid-clear. EVFOVR sets, per firing channel, when that channel's task consumer was already busy at the moment of firing, or when two or more enabled channels landed on the same task in the same cycle (Section 31.3.2). In every such case the task pulse is still emitted normally; EVFOVR only records that the pulse was *degraded* in some way, never that it was withheld. Both are drained the same way: read to see which channels are set, write 1 to the bits just handled.

EVFCHTRIG and EVFEVTRIG are write-only injection registers (both read 0) that let firmware exercise the fabric with no producer hardware involved at all: a 1 written to bit n of EVFCHTRIG injects one firing directly at channel n, honoring that channel's EVFCR/EVFCHEN gates exactly like a real event (a CHTRIG to a disabled channel does nothing); a 1 written to bit e of EVFEVTRIG injects one occurrence of raw event e, which is recorded in EVFEVSTAT even with the fabric globally disabled and reaches the crossbar exactly like a genuine tap. Both share one hardware property that also protects real bus writes: the access itself may be held on the bus for several clock edges (a slow or contended master), but the fabric decodes the write *once* per select window, so a single CPU store, however long the underlying bus access takes to retire, produces exactly one injection, never one per bus-clock edge it happens to span.

EVFCAP is a read-only word that reports the *live* counts this build implements (8 channels, 16 events, 10 tasks, fabric version 1), so a driver can size its loops from the register instead of a compile-time constant, and instead of the wider *encode spaces* that EVFEVSELn/EVFTASKSELn reserve for growth.

31.4 Interrupts and events

This version spends no interrupt vector at all: `irq_evfab` is wired to a constant 0 and the `EVFIE` slot is reserved and reads 0, so `EVFAB` never appears in the `IRQROUTER`'s vector list and the frozen interrupt-vector numbering is completely undisturbed by this peripheral. Firmware polls `EVFSR`, whose two bits are live OR-reductions of `EVFFIRED` and `EVFOVR`. A future version that spends a vector for `EVFFIRED`/`EVFOVR` would be a purely additive change (implement `EVFIE`, drive `irq_evfab`, and route the new vector) with no register-map break.

31.5 Programming

31.5.1 Driver Contract

- **Configure before enabling.** `EVFCHnCFG.EVFEVSELn` and `EVFCHnCFG.EVFTASKSELn` must only be changed while that channel's `EVFCHEN` bit is 0; the fabric does not interlock a live reconfiguration against an in-flight firing, so the sequence is: clear the channel (`EVFCHENCLR`), write its new `EVFCHnCFG`, then re-enable it (`EVFCHENSET`).
- **Action-visibility latency.** A write to `EVFCHTRIG`, `EVFEVTRIG`, or any of the write-1-to-clear status words (`EVFFIRED`, `EVFOVR`, `EVFEVSTAT`) takes effect three `MCLK` cycles after its bus access opens; the register file itself is on the gated bus clock, but these particular actions are non-idempotent and are decoded once, in the free-running `MCLK` domain, by a dedicated edge detector. A status read issued immediately after such a write (only possible from a master faster than the shared multi-core bus) can observe stale state; on PentaWoundPt2mm a shared-window access already costs several `MCLK` cycles end to end, so this window is not observable in practice, and no special handling is needed beyond not assuming same-instruction visibility.
- **No consumer may count firings.** Section 31.3.2's merge rule means a driver must not build a protocol on top of a task line that depends on receiving one pulse per channel firing when two channels share that task. Design channel-to-task assignments so every task is either driven by one channel, or every colliding channel's action is genuinely idempotent (the case for all ten wired tasks today).
- **The NPU staging-RAM contract still applies to task 6.** A fabric-driven `NPU0 THINK` obeys exactly the same rule as a register-driven one: no hart may touch `0xC000–0xFFFF` while a `THINK` may be in flight. Because the fabric can start a `THINK` with no CPU instruction executing at all, a design that uses task 6 must guarantee by construction, not by polling right before the event, that nothing else is touching the staging RAM when the triggering event might occur; there is no hardware interlock.
- **Channel ownership across harts is a software convention.** There is no `OWNER` field anywhere in the register map. Multiple harts sharing the fabric should partition the eight channels by index (for example, by convention rather than by any register), using `EVFCHENSET`/`EVFCHENCLR` so that enabling or disabling one hart's channels never touches another's, and never writing another hart's `EVFCHnCFG` without an out-of-band handshake (a mutex, or simply "this channel belongs to hart 2").

31.5.2 Worked Example: An Autonomous Sensing Chain

The following configuration lets `TIMER0` trigger a `UART0` transmission through `DMA0` with the CPU asleep the entire time, and is the reference chain proven by the `shevfab` test in the behavioral regression:

TIMER0 compare0 (EV4) → channel 0 → DMA0 channel-0 GO (TASK 0) → DMA0 fills UART0TX through the arbiter → DMA0 channel-0 done (vector 118) → IRQROUTER → meip → hart wakes

Firmware programs EVFCH0CFG with EVFEVSELn = 4 (TIMER0 compare0) and EVFTASKSELn = 0 (DMA0 channel-0 GO) while channel 0 is disabled, arms DMA0 channel 0 with its source/destination/length pointed at the UART0 transmit path and *its own* DMAGO bit left clear (so the fabric task pulse is the *only* way the transfer launches; a broken event path cannot be masked by an accidental register GO left over from a previous test), sets EVFCR.EN and EVFCHENSET bit 0 (arming TIMER0's own compare0 interrupt path is not required at all; the fabric taps the compare event directly, before any interrupt enable), enables DMA0's done interrupt, routes vector 118 to itself in IRQROUTER, starts TIMER0, and executes WFI.

From that point the hart executes nothing until the chain completes: TIMER0's compare0 toggle reaches the fabric three MCLK cycles after it flips (Section 31.3.1, toggle mode), fires channel 0, and one MCLK later the fabric drives a task pulse into DMA0's channel-0 GO input exactly as if firmware had written DMAGO itself. DMA0 runs its transfer over the shared arbiter, sets its channel-0 done flag on completion, the combined done interrupt reaches the router, the router raises meip to the hart, and the hart's interrupt-service routine clears DMADONE[0] and resumes normal execution. Every link in that path (the event tap, the fabric crossbar, the task action, the DMA transfer, and the interrupt delivery) is independently a documented, testable contract; the chain is simply those contracts composed with nobody polling in between.

A companion pattern from the same design worth noting, though it is not shipped as a single test: RTC0's periodic tick (event 0) driving TIMER0 START (task 2), with a second channel wiring TIMER0's own compare event back to TIMER0 STOP (task 3), produces an autonomous, CPU-free sensing window (the real-time clock opens a timed measurement interval and the timer itself closes it) entirely inside the fabric.

31.6 Usage notes

31.6.1 Shared Access on PentaWoundPt2mm

In configurations that include it, EVFAB lives in the shared peripheral window behind the multi-core arbiter (see Section 5) in the mutex-bank page, sub-slot 11 at 0x6B00, so any hart can configure it and any hart's channels can drive any wired task. EVFAB has no pins and, because its datapath and register file both ultimately ride the free-running fabric clock, its operation is immune to clock reconfiguration: software does *not* need to write SYSCLKCR to 0 before using it, unlike the SMCLK-domain serial peripherals.

31.7 Registers

Register offsets in this chapter are relative to the instance base address in Table 31.1 (EVFAB 0x6B00); the generated header names each base EVFAB_BASE.

Table 31.4: EVFAB register summary

Offset	Name	Access	Reset	Description
0x00	EVFCR	rw	0x00000000	Event fabric control register
0x04	EVFSR	r	0x00000000	Event fabric status register
0x08	EVFIE	r0	0x00000000	Reserved interrupt-enable register
0x0C	EVFCAP	r	0x010A1008	Capability register (read-only constant)
0x10	EVFCHEN	rw	0x00000000	Channel enable register
0x14	EVFCHENSET	rw1	0x00000000	Channel enable SET alias
0x18	EVFCHENCLR	rw1	0x00000000	Channel enable CLEAR alias

Offset	Name	Access	Reset	Description
0x1C	EVFCHTRIG	w	0x00000000	Channel trigger injection register
0x20	EVFFIRED	rw1	0x00000000	Sticky channel-fired flags
0x24	EVFOVR	rw1	0x00000000	Sticky channel-overrun flags
0x28	EVFEVSTAT	rw1	0x00000000	Sticky raw-event record
0x2C	EVFEVTRIG	w	0x00000000	Raw event injection register
0x30 to 0x3B	-	-	-	Reserved
0x3C	EVFGPIOMASK	rw	0x00000000	GPIO0 edge-path mask
0x40	EVFCH0CFG	rw/r	0x00000000	Channel 0 configuration
0x44	EVFCH1CFG	rw/r	0x00000000	Channel 1 configuration
0x48	EVFCH2CFG	rw/r	0x00000000	Channel 2 configuration
0x4C	EVFCH3CFG	rw/r	0x00000000	Channel 3 configuration
0x50	EVFCH4CFG	rw/r	0x00000000	Channel 4 configuration
0x54	EVFCH5CFG	rw/r	0x00000000	Channel 5 configuration
0x58	EVFCH6CFG	rw/r	0x00000000	Channel 6 configuration
0x5C	EVFCH7CFG	rw/r	0x00000000	Channel 7 configuration
0x60	EVFCH8CFG	r0	0x00000000	Reserved channel-8 configuration slot
0x64	EVFCH9CFG	r0	0x00000000	Reserved channel-9 configuration slot
0x68	EVFCH10CFG	r0	0x00000000	Reserved channel-10 configuration slot
0x6C	EVFCH11CFG	r0	0x00000000	Reserved channel-11 configuration slot
0x70	EVFCH12CFG	r0	0x00000000	Reserved channel-12 configuration slot
0x74	EVFCH13CFG	r0	0x00000000	Reserved channel-13 configuration slot
0x78	EVFCH14CFG	r0	0x00000000	Reserved channel-14 configuration slot
0x7C	EVFCH15CFG	r0	0x00000000	Reserved channel-15 configuration slot

31.7.1 EVFCR: Event fabric control register

Offset 0x00 **Reset** 0x00000000 **Width** 32 **Address** 0x6B00

Event fabric control register. EN is the global kill switch: with EN clear NO channel can fire, no FIRED or OVR flag can set, and no task pulse can be emitted, whatever the channel enables and configuration words say (raw events are still recorded in EVFEVSTAT, which sits upstream of the gate). Resets to 0, so the fabric comes out of reset completely inert; configure the channels first, then set EN. Other bits reserved, read 0.

Bits	Field	Type	Reset	Description
31:1	-	-	-	Reserved. Reads zero; write zero.
0	EN	rw	0	Global fabric enable. Gates every channel together with its own EVFCHEN bit (both must be set for a channel to fire). 0: Fabric disabled (no pulses, no flags) 1: Fabric enabled

31.7.2 EVFSR: Event fabric status register

Offset 0x04 **Reset** 0x00000000 **Width** 32 **Address** 0x6B04

Event fabric status register. Both bits are LIVE read-only reductions of the sticky words, not state of their own: firmware polls this one word to learn whether anything at all has fired or been degraded, then reads EVFFIRED / EVFOVR to find out which channels. There is no interrupt in this version of the fabric (EVFIE is reserved), so this register is the whole notification mechanism. Other bits reserved, read 0.

Bits	Field	Type	Reset	Description
31:2	-	-	-	Reserved. Reads zero; write zero.
1	OVRIF	r	0	Set while ANY bit of EVFOVR is set (a logical OR of the sticky per-channel overrun flags). 0: No overrun recorded 1: At least one overrun recorded
0	FIREDIF	r	0	Set while ANY bit of EVFFIRED is set (a logical OR of the sticky per-channel fired flags). Clears only when every EVFFIRED bit has been written back to 0. 0: No channel has fired since the flags were cleared 1: At least one channel has fired

31.7.3 EVFIE: Reserved interrupt-enable register

Offset 0x08 **Reset** 0x00000000 **Width** 32 **Address** 0x6B08
Reserved interrupt-enable register. This version of the event fabric is deliberately VECTORLESS: it spends no interrupt vector, its interrupt output is tied inactive, and this slot reads 0 and ignores writes. The slot is reserved so that a later revision can add per-flag interrupt enables without moving any other register.

Bits	Field	Type	Reset	Description
31:0	-	-	-	Reserved. Reads zero; write zero.

31.7.4 EVFCAP: Capability register (read-only constant)

Offset 0x0C **Reset** 0x010A1008 **Width** 32 **Address** 0x6B0C
Capability register (read-only constant). Reports the LIVE line counts of this build so a driver can size its loops instead of hardcoding them: the number of channels, event lines and task lines actually implemented, plus a version number. At the Castalia configuration it reads 0x010A1008.

Bits	Field	Type	Reset	Description
31:24	VER	r	0x01	Fabric version (1).
23:16	NTASK	r	0x0A	Number of implemented task lines (10). A TASKSEL at or above this count still sets EVFFIRED but drives no task line and can never set EVFOVR.
15:8	NEV	r	0x10	Number of implemented event lines (16). EVSEL codes at or above this count select nothing, which makes the channel inert (EVSEL 31 is the documented NONE encoding).
7:0	NCH	r	0x08	Number of implemented channels (8). Channel registers above this count read 0 and ignore writes.

31.7.5 EVFCHEN: Channel enable register

Offset 0x10 **Reset** 0x00000000 **Width** 32 **Address** 0x6B10
Channel enable register. Bit n enables channel n; a channel fires only when this bit AND EVFCR.EN are both set, and a disabled channel is completely inert (no task pulse, no EVFFIRED, no EVFOVR). Resets to 0. Writing this register directly is safe only for a single owner: when several harts share the fabric, use the EVFCHENSET / EVFCHENCLR aliases instead, which set or clear individual bits without a read-modify-write. Bits above the implemented channel count read 0.

Bits	Field	Type	Reset	Description
31:8	-	-	-	Reserved. Reads zero; write zero.
7:0	CHEN	rw	0x00	Per-channel enable bits (bit n = channel n).

31.7.6 EVFCHENSET: Channel enable SET alias

Offset 0x14 **Reset** 0x00000000 **Width** 32 **Address** 0x6B14

Channel enable SET alias. Writing a 1 to bit n sets EVFCHEN bit n; writing 0 leaves that bit alone, so a hart can enable its own channels without disturbing another hart's. Reading this address returns the current EVFCHEN value.

Bits	Field	Type	Reset	Description
31:8	-	-	-	Reserved. Reads zero; write zero.
7:0	CHENSET	rw1	0x00	Write 1 to enable channel n (write 0 = no effect). Reads back EVFCHEN.

31.7.7 EVFCHENCLR: Channel enable CLEAR alias

Offset 0x18 **Reset** 0x00000000 **Width** 32 **Address** 0x6B18

Channel enable CLEAR alias. Writing a 1 to bit n clears EVFCHEN bit n; writing 0 leaves that bit alone. Reading this address returns the current EVFCHEN value. Disable a channel through this register before changing its EVFCHnCFG selectors.

Bits	Field	Type	Reset	Description
31:8	-	-	-	Reserved. Reads zero; write zero.
7:0	CHENCLR	rw1	0x00	Write 1 to disable channel n (write 0 = no effect). Reads back EVFCHEN.

31.7.8 EVFCHTRIG: Channel trigger injection register

Offset 0x1C **Reset** 0x00000000 **Width** 32 **Address** 0x6B1C

Channel trigger injection register. Writing a 1 to bit n makes channel n fire exactly once, as if its selected event had occurred: the task pulse, EVFFIRED and EVFOVR all behave identically to a hardware firing. The injection still honours the enables, a trigger written to a disabled channel, or with EVFCR.EN clear, does nothing. However long the bus write is held, exactly one firing is injected. Reads 0.

Bits	Field	Type	Reset	Description
31:8	-	-	-	Reserved. Reads zero; write zero.
7:0	CHTRIG	w	0x00	Write 1 to inject one firing on channel n.

31.7.9 EVFFIRED: Sticky channel-fired flags

Offset 0x20 **Reset** 0x00000000 **Width** 32 **Address** 0x6B20

Sticky channel-fired flags. Bit n sets whenever channel n fires (from a hardware event or an EVFCHTRIG injection) and stays set until firmware writes a 1 to it. A firing arriving in the same cycle as the clearing write WINS, the flag survives, so no event can be lost between a read and a clear. Bits above the implemented channel count read 0.

Bits	Field	Type	Reset	Description
31:8	-	-	-	Reserved. Reads zero; write zero.
7:0	FIRED	rw1	0x00	Channel n has fired. Write 1 to clear.

31.7.10 EVFOVR: Sticky channel-overrun flags

Offset 0x24 **Reset** 0x00000000 **Width** 32 **Address** 0x6B24
Sticky channel-overrun flags. Bit n sets when channel n fires but its task pulse was DEGRADED: either the target consumer was already busy at that moment, or another enabled channel fired onto the SAME task line in the same cycle (the two firings merge into one pulse and BOTH channels record an overrun, the fabric cannot say which one won). An overrun never suppresses the pulse and never applies backpressure; it is a diagnostic. A channel whose TASKSEL selects a reserved code can never set this flag, since no task line exists to be busy. Set wins over a same-cycle clear. Bits above the implemented channel count read 0.

Bits	Field	Type	Reset	Description
31:8	-	-	-	Reserved. Reads zero; write zero.
7:0	OVR	rw1	0x00	Channel n fired while its task was busy, or merged with another channel onto the same task in the same cycle. Write 1 to clear.

31.7.11 EVFEVSTAT: Sticky raw-event record

Offset 0x28 **Reset** 0x00000000 **Width** 32 **Address** 0x6B28
Sticky raw-event record. Bit e sets whenever event line e is seen by the fabric front end, INDEPENDENTLY of EVFCR.EN and of any channel enable or configuration, it is upstream of every gate. That makes it both a bring-up aid (does this producer actually pulse?) and a direct test of tap discipline: because every event is tapped from its source flag's SET condition rather than from a masked interrupt line, clearing a producer's interrupt enable must NOT stop its bit appearing here. Set wins over a same-cycle clear. Bits above the implemented event count read 0.

Bits	Field	Type	Reset	Description
31:16	-	-	-	Reserved. Reads zero; write zero.
15:0	EVSTAT	rw1	0x0000	Event line e has been seen. Write 1 to clear.

31.7.12 EVFEVTRIG: Raw event injection register

Offset 0x2C **Reset** 0x00000000 **Width** 32 **Address** 0x6B2C
Raw event injection register. Writing a 1 to bit e injects one occurrence of event line e exactly as if the producer had generated it: it is recorded in EVFEVSTAT and offered to every channel selecting that event. However long the bus write is held, exactly one event is injected. With this register the entire crossbar, every event to every task, including events whose producer block is absent from the configuration, is testable from firmware with no producer hardware at all. Reads 0.

Bits	Field	Type	Reset	Description
31:16	-	-	-	Reserved. Reads zero; write zero.
15:0	EVTRIG	w	0x0000	Write 1 to inject one occurrence of event line e.

31.7.13 EVFGPIOMASK: GPIO0 edge-path mask

Offset 0x3C **Reset** 0x00000000 **Width** 32 **Address** 0x6B3C

GPIO0 edge-path mask. Event line 15 is generated inside the fabric from GPIO0's eight raw pad edges: each pad is synchronized, edge-detected, ANDed with its bit here, and the eight results are ORed into one event. A bit clear means that pad contributes nothing, and, because the mask is applied BEFORE the OR, a masked-off pad also absorbs an undriven or unbonded pin instead of poisoning the event. The port's own PxIE interrupt enables are never consulted, so a pin can drive the fabric without ever raising an interrupt. Resets to 0, leaving the whole path inert. Other bits reserved, read 0.

Bits	Field	Type	Reset	Description
31:8	-	-	-	Reserved. Reads zero; write zero.
7:0	GPIOMASK	rw	0x00	Per-pad enable for the GPIO0 edge path (bit i = GPIO0 pin i).

31.7.14 EVFCH0CFG: Channel 0 configuration

Offset 0x40 **Reset** 0x00000000 **Width** 32 **Address** 0x6B40

Channel 0 configuration. EVSEL picks which event line feeds the channel and TASKSEL picks which task line it drives; ENR is a read-only mirror of this channel's EVFCHEN bit, so one read shows a channel's whole state. Selector encoding is deliberately forgiving: EVSEL 31 is the documented NONE code, and any EVSEL or TASKSEL value with no line behind it simply makes the channel (or its output) inert rather than aliasing onto a real one. Change EVSEL or TASKSEL only while the channel is disabled, the fabric does not interlock a live reconfiguration. Resets to 0, which is harmless because a channel is double-gated by EVFCR.EN and EVFCHEN.

Bits	Field	Type	Reset	Description
31	ENR0	r	0	Read-only mirror of EVFCHEN bit 0 (this channel's enable). 0: Channel disabled 1: Channel enabled
30:12	-	-	-	Reserved. Reads zero; write zero.
11:8	TASKSEL0	rw	0b0000	Task select for channel 0: the task line this channel pulses. 0 = DMA0 channel-0 GO, 1 = DMA0 channel-1 GO, 2 = TIMER0 START, 3 = TIMER0 STOP, 4 = PWM0 fault trip, 5 = PWRCTRL tile wake, 6 = NPU0 THINK, 7 = GPIO0 output SET, 8 = GPIO0 output CLEAR. Codes 9 and above drive no line (the channel still records EVFFIRED). A task whose consumer block is absent from this configuration is left unconnected.
7:5	-	-	-	Reserved. Reads zero; write zero.

Bits	Field	Type	Reset	Description
4:0	EVSEL0	rw	0x00	Event select for channel 0: the event line whose occurrence fires this channel. An event whose producer block is absent from this configuration is tied inactive and simply never fires. 0x00 (0): RTC0 tick, 1 = RTC0 alarm, 2 = PWM0 period, 3 = PWM0 fault, 4 = TIMER0 compare0, 5 = TIMER0 overflow, 6 = TIMER1 compare0, 7 = UART0 receive, 8 = NFC0 field-detect, 9 = NFC0 rx-frame, 10 = DMA0 channel-0 done, 11 = DMA0 channel-1 done, 12 = DMA0 error, 13 = TRNG0 data-ready, 14 = I2CT0 address-match, 15 = masked GPIO0 pad edge (see EVFGPIOMASK) 0x1F (31): NONE; other codes select nothing

31.7.15 EVFCH1CFG: Channel 1 configuration

Offset 0x44 **Reset** 0x00000000 **Width** 32 **Address** 0x6B44

Channel 1 configuration. EVSEL picks which event line feeds the channel and TASKSEL picks which task line it drives; ENR is a read-only mirror of this channel's EVFCHEN bit, so one read shows a channel's whole state. Selector encoding is deliberately forgiving: EVSEL 31 is the documented NONE code, and any EVSEL or TASKSEL value with no line behind it simply makes the channel (or its output) inert rather than aliasing onto a real one. Change EVSEL or TASKSEL only while the channel is disabled, the fabric does not interlock a live reconfiguration. Resets to 0, which is harmless because a channel is double-gated by EVFCR.EN and EVFCHEN.

Bits	Field	Type	Reset	Description
31	ENR1	r	0	Read-only mirror of EVFCHEN bit 1 (this channel's enable). 0: Channel disabled 1: Channel enabled
30:12	-	-	-	Reserved. Reads zero; write zero.
11:8	TASKSEL1	rw	0b0000	Task select for channel 1: the task line this channel pulses. 0 = DMA0 channel-0 GO, 1 = DMA0 channel-1 GO, 2 = TIMER0 START, 3 = TIMER0 STOP, 4 = PWM0 fault trip, 5 = PWRCTRL tile wake, 6 = NPU0 THINK, 7 = GPIO0 output SET, 8 = GPIO0 output CLEAR. Codes 9 and above drive no line (the channel still records EVFFIRED). A task whose consumer block is absent from this configuration is left unconnected.
7:5	-	-	-	Reserved. Reads zero; write zero.
4:0	EVSEL1	rw	0x00	Event select for channel 1: the event line whose occurrence fires this channel. An event whose producer block is absent from this configuration is tied inactive and simply never fires. 0x00 (0): RTC0 tick, 1 = RTC0 alarm, 2 = PWM0 period, 3 = PWM0 fault, 4 = TIMER0 compare0, 5 = TIMER0 overflow, 6 = TIMER1 compare0, 7 = UART0 receive, 8 = NFC0 field-detect, 9 = NFC0 rx-frame, 10 = DMA0 channel-0 done, 11 = DMA0 channel-1 done, 12 = DMA0 error, 13 = TRNG0 data-ready, 14 = I2CT0 address-match, 15 = masked GPIO0 pad edge (see EVFGPIOMASK) 0x1F (31): NONE; other codes select nothing

31.7.16 EVFCH2CFG: Channel 2 configuration

Offset 0x48 **Reset** 0x00000000 **Width** 32 **Address** 0x6B48

Channel 2 configuration. EVSEL picks which event line feeds the channel and TASKSEL picks which task line it drives; ENR is a read-only mirror of this channel's EVFCHEN bit, so one read shows a channel's whole state. Selector encoding is deliberately forgiving: EVSEL 31 is the documented NONE code, and any EVSEL or TASKSEL value with no line behind it simply makes the channel (or its output) inert rather than aliasing onto a real one. Change EVSEL or TASKSEL only while the channel is disabled, the fabric does not interlock a live reconfiguration. Resets to 0, which is harmless because a channel is double-gated by EVFCR.EN and EVFCHEN.

Bits	Field	Type	Reset	Description
31	ENR2	r	0	Read-only mirror of EVFCHEN bit 2 (this channel's enable). 0: Channel disabled 1: Channel enabled
30:12	-	-	-	Reserved. Reads zero; write zero.
11:8	TASKSEL2	rw	0b0000	Task select for channel 2: the task line this channel pulses. 0 = DMA0 channel-0 GO, 1 = DMA0 channel-1 GO, 2 = TIMER0 START, 3 = TIMER0 STOP, 4 = PWM0 fault trip, 5 = PWRCTRL tile wake, 6 = NPU0 THINK, 7 = GPIO0 output SET, 8 = GPIO0 output CLEAR. Codes 9 and above drive no line (the channel still records EVFFIRED). A task whose consumer block is absent from this configuration is left unconnected.
7:5	-	-	-	Reserved. Reads zero; write zero.
4:0	EVSEL2	rw	0x00	Event select for channel 2: the event line whose occurrence fires this channel. An event whose producer block is absent from this configuration is tied inactive and simply never fires. 0x00 (0): RTC0 tick, 1 = RTC0 alarm, 2 = PWM0 period, 3 = PWM0 fault, 4 = TIMER0 compare0, 5 = TIMER0 overflow, 6 = TIMER1 compare0, 7 = UART0 receive, 8 = NFC0 field-detect, 9 = NFC0 rx-frame, 10 = DMA0 channel-0 done, 11 = DMA0 channel-1 done, 12 = DMA0 error, 13 = TRNG0 data-ready, 14 = I2CT0 address-match, 15 = masked GPIO0 pad edge (see EVFGPIOMASK) 0x1F (31): NONE; other codes select nothing

31.7.17 EVFCH3CFG: Channel 3 configuration

Offset 0x4C **Reset** 0x00000000 **Width** 32 **Address** 0x6B4C

Channel 3 configuration. EVSEL picks which event line feeds the channel and TASKSEL picks which task line it drives; ENR is a read-only mirror of this channel's EVFCHEN bit, so one read shows a channel's whole state. Selector encoding is deliberately forgiving: EVSEL 31 is the documented NONE code, and any EVSEL or TASKSEL value with no line behind it simply makes the channel (or its output) inert rather than aliasing onto a real one. Change EVSEL or TASKSEL only while the channel is disabled, the fabric does not interlock a live reconfiguration. Resets to 0, which is harmless because a channel is double-gated by EVFCR.EN and EVFCHEN.

Bits	Field	Type	Reset	Description
31	ENR3	r	0	Read-only mirror of EVFCHEN bit 3 (this channel's enable). 0: Channel disabled 1: Channel enabled
30:12	-	-	-	Reserved. Reads zero; write zero.
11:8	TASKSEL3	rw	0b0000	Task select for channel 3: the task line this channel pulses. 0 = DMA0 channel-0 GO, 1 = DMA0 channel-1 GO, 2 = TIMER0 START, 3 = TIMER0 STOP, 4 = PWM0 fault trip, 5 = PWRCTRL tile wake, 6 = NPU0 THINK, 7 = GPIO0 output SET, 8 = GPIO0 output CLEAR. Codes 9 and above drive no line (the channel still records EVFFIRED). A task whose consumer block is absent from this configuration is left unconnected.
7:5	-	-	-	Reserved. Reads zero; write zero.
4:0	EVSEL3	rw	0x00	Event select for channel 3: the event line whose occurrence fires this channel. An event whose producer block is absent from this configuration is tied inactive and simply never fires. 0x00 (0): RTC0 tick, 1 = RTC0 alarm, 2 = PWM0 period, 3 = PWM0 fault, 4 = TIMER0 compare0, 5 = TIMER0 overflow, 6 = TIMER1 compare0, 7 = UART0 receive, 8 = NFC0 field-detect, 9 = NFC0 rx-frame, 10 = DMA0 channel-0 done, 11 = DMA0 channel-1 done, 12 = DMA0 error, 13 = TRNG0 data-ready, 14 = I2CT0 address-match, 15 = masked GPIO0 pad edge (see EVFGPIOMASK) 0x1F (31): NONE; other codes select nothing

31.7.18 EVFCH4CFG: Channel 4 configuration

Offset 0x50 **Reset** 0x00000000 **Width** 32 **Address** 0x6B50

Channel 4 configuration. EVSEL picks which event line feeds the channel and TASKSEL picks which task line it drives; ENR is a read-only mirror of this channel's EVFCHEN bit, so one read shows a channel's whole state. Selector encoding is deliberately forgiving: EVSEL 31 is the documented NONE code, and any EVSEL or TASKSEL value with no line behind it simply makes the channel (or its output) inert rather than aliasing onto a real one. Change EVSEL or TASKSEL only while the channel is disabled, the fabric does not interlock a live reconfiguration. Resets to 0, which is harmless because a channel is double-gated by EVFCR.EN and EVFCHEN.

Bits	Field	Type	Reset	Description
31	ENR4	r	0	Read-only mirror of EVFCHEN bit 4 (this channel's enable). 0: Channel disabled 1: Channel enabled
30:12	-	-	-	Reserved. Reads zero; write zero.
11:8	TASKSEL4	rw	0b0000	Task select for channel 4: the task line this channel pulses. 0 = DMA0 channel-0 GO, 1 = DMA0 channel-1 GO, 2 = TIMER0 START, 3 = TIMER0 STOP, 4 = PWM0 fault trip, 5 = PWRCTRL tile wake, 6 = NPU0 THINK, 7 = GPIO0 output SET, 8 = GPIO0 output CLEAR. Codes 9 and above drive no line (the channel still records EVFFIRED). A task whose consumer block is absent from this configuration is left unconnected.

Bits	Field	Type	Reset	Description
7:5	-	-	-	Reserved. Reads zero; write zero.
4:0	EVSEL4	rw	0x00	Event select for channel 4: the event line whose occurrence fires this channel. An event whose producer block is absent from this configuration is tied inactive and simply never fires. 0x00 (0): RTC0 tick, 1 = RTC0 alarm, 2 = PWM0 period, 3 = PWM0 fault, 4 = TIMER0 compare0, 5 = TIMER0 overflow, 6 = TIMER1 compare0, 7 = UART0 receive, 8 = NFC0 field-detect, 9 = NFC0 rx-frame, 10 = DMA0 channel-0 done, 11 = DMA0 channel-1 done, 12 = DMA0 error, 13 = TRNG0 data-ready, 14 = I2CT0 address-match, 15 = masked GPIO0 pad edge (see EVFGPIOMASK) 0x1F (31): NONE; other codes select nothing

31.7.19 EVFCH5CFG: Channel 5 configuration

Offset 0x54 **Reset** 0x00000000 **Width** 32 **Address** 0x6B54

Channel 5 configuration. EVSEL picks which event line feeds the channel and TASKSEL picks which task line it drives; ENR is a read-only mirror of this channel's EVFCHEN bit, so one read shows a channel's whole state. Selector encoding is deliberately forgiving: EVSEL 31 is the documented NONE code, and any EVSEL or TASKSEL value with no line behind it simply makes the channel (or its output) inert rather than aliasing onto a real one. Change EVSEL or TASKSEL only while the channel is disabled, the fabric does not interlock a live reconfiguration. Resets to 0, which is harmless because a channel is double-gated by EVFCR.EN and EVFCHEN.

Bits	Field	Type	Reset	Description
31	ENR5	r	0	Read-only mirror of EVFCHEN bit 5 (this channel's enable). 0: Channel disabled 1: Channel enabled
30:12	-	-	-	Reserved. Reads zero; write zero.
11:8	TASKSEL5	rw	0b0000	Task select for channel 5: the task line this channel pulses. 0 = DMA0 channel-0 GO, 1 = DMA0 channel-1 GO, 2 = TIMER0 START, 3 = TIMER0 STOP, 4 = PWM0 fault trip, 5 = PWRCTRL tile wake, 6 = NPU0 THINK, 7 = GPIO0 output SET, 8 = GPIO0 output CLEAR. Codes 9 and above drive no line (the channel still records EVFFIRED). A task whose consumer block is absent from this configuration is left unconnected.
7:5	-	-	-	Reserved. Reads zero; write zero.

Bits	Field	Type	Reset	Description
4:0	EVSEL5	rw	0x00	Event select for channel 5: the event line whose occurrence fires this channel. An event whose producer block is absent from this configuration is tied inactive and simply never fires. 0x00 (0): RTC0 tick, 1 = RTC0 alarm, 2 = PWM0 period, 3 = PWM0 fault, 4 = TIMER0 compare0, 5 = TIMER0 overflow, 6 = TIMER1 compare0, 7 = UART0 receive, 8 = NFC0 field-detect, 9 = NFC0 rx-frame, 10 = DMA0 channel-0 done, 11 = DMA0 channel-1 done, 12 = DMA0 error, 13 = TRNG0 data-ready, 14 = I2CT0 address-match, 15 = masked GPIO0 pad edge (see EVFGPIOMASK) 0x1F (31): NONE; other codes select nothing

31.7.20 EVFCH6CFG: Channel 6 configuration

Offset 0x58 **Reset** 0x00000000 **Width** 32 **Address** 0x6B58

Channel 6 configuration. EVSEL picks which event line feeds the channel and TASKSEL picks which task line it drives; ENR is a read-only mirror of this channel's EVFCHEN bit, so one read shows a channel's whole state. Selector encoding is deliberately forgiving: EVSEL 31 is the documented NONE code, and any EVSEL or TASKSEL value with no line behind it simply makes the channel (or its output) inert rather than aliasing onto a real one. Change EVSEL or TASKSEL only while the channel is disabled, the fabric does not interlock a live reconfiguration. Resets to 0, which is harmless because a channel is double-gated by EVFCR.EN and EVFCHEN.

Bits	Field	Type	Reset	Description
31	ENR6	r	0	Read-only mirror of EVFCHEN bit 6 (this channel's enable). 0: Channel disabled 1: Channel enabled
30:12	-	-	-	Reserved. Reads zero; write zero.
11:8	TASKSEL6	rw	0b0000	Task select for channel 6: the task line this channel pulses. 0 = DMA0 channel-0 GO, 1 = DMA0 channel-1 GO, 2 = TIMER0 START, 3 = TIMER0 STOP, 4 = PWM0 fault trip, 5 = PWRCTRL tile wake, 6 = NPU0 THINK, 7 = GPIO0 output SET, 8 = GPIO0 output CLEAR. Codes 9 and above drive no line (the channel still records EVFFIRED). A task whose consumer block is absent from this configuration is left unconnected.
7:5	-	-	-	Reserved. Reads zero; write zero.
4:0	EVSEL6	rw	0x00	Event select for channel 6: the event line whose occurrence fires this channel. An event whose producer block is absent from this configuration is tied inactive and simply never fires. 0x00 (0): RTC0 tick, 1 = RTC0 alarm, 2 = PWM0 period, 3 = PWM0 fault, 4 = TIMER0 compare0, 5 = TIMER0 overflow, 6 = TIMER1 compare0, 7 = UART0 receive, 8 = NFC0 field-detect, 9 = NFC0 rx-frame, 10 = DMA0 channel-0 done, 11 = DMA0 channel-1 done, 12 = DMA0 error, 13 = TRNG0 data-ready, 14 = I2CT0 address-match, 15 = masked GPIO0 pad edge (see EVFGPIOMASK) 0x1F (31): NONE; other codes select nothing

31.7.21 EVFCH7CFG: Channel 7 configuration

Offset 0x5C **Reset** 0x00000000 **Width** 32 **Address** 0x6B5C

Channel 7 configuration. EVSEL picks which event line feeds the channel and TASKSEL picks which task line it drives; ENR is a read-only mirror of this channel's EVFCHEN bit, so one read shows a channel's whole state. Selector encoding is deliberately forgiving: EVSEL 31 is the documented NONE code, and any EVSEL or TASKSEL value with no line behind it simply makes the channel (or its output) inert rather than aliasing onto a real one. Change EVSEL or TASKSEL only while the channel is disabled, the fabric does not interlock a live reconfiguration. Resets to 0, which is harmless because a channel is double-gated by EVFCR.EN and EVFCHEN.

Bits	Field	Type	Reset	Description
31	ENR7	r	0	Read-only mirror of EVFCHEN bit 7 (this channel's enable). 0: Channel disabled 1: Channel enabled
30:12	-	-	-	Reserved. Reads zero; write zero.
11:8	TASKSEL7	rw	0b0000	Task select for channel 7: the task line this channel pulses. 0 = DMA0 channel-0 GO, 1 = DMA0 channel-1 GO, 2 = TIMER0 START, 3 = TIMER0 STOP, 4 = PWM0 fault trip, 5 = PWRCTRL tile wake, 6 = NPU0 THINK, 7 = GPIO0 output SET, 8 = GPIO0 output CLEAR. Codes 9 and above drive no line (the channel still records EVFFIRED). A task whose consumer block is absent from this configuration is left unconnected.
7:5	-	-	-	Reserved. Reads zero; write zero.
4:0	EVSEL7	rw	0x00	Event select for channel 7: the event line whose occurrence fires this channel. An event whose producer block is absent from this configuration is tied inactive and simply never fires. 0x00 (0): RTC0 tick, 1 = RTC0 alarm, 2 = PWM0 period, 3 = PWM0 fault, 4 = TIMER0 compare0, 5 = TIMER0 overflow, 6 = TIMER1 compare0, 7 = UART0 receive, 8 = NFC0 field-detect, 9 = NFC0 rx-frame, 10 = DMA0 channel-0 done, 11 = DMA0 channel-1 done, 12 = DMA0 error, 13 = TRNG0 data-ready, 14 = I2CT0 address-match, 15 = masked GPIO0 pad edge (see EVFGPIOMASK) 0x1F (31): NONE; other codes select nothing

31.7.22 EVFCH8CFG: Reserved channel-8 configuration slot

Offset 0x60 **Reset** 0x00000000 **Width** 32 **Address** 0x6B60

Reserved channel-8 configuration slot. This build implements 8 channels, so this address reads 0 and ignores writes; the map reserves 16 channel addresses so a wider fabric needs no register move.

Bits	Field	Type	Reset	Description
31:0	-	-	-	Reserved. Reads zero; write zero.

31.7.23 EVFCH9CFG: Reserved channel-9 configuration slot

Offset 0x64 **Reset** 0x00000000 **Width** 32 **Address** 0x6B64

Reserved channel-9 configuration slot. This build implements 8 channels, so this address reads 0 and ignores writes; the map reserves 16 channel addresses so a wider fabric needs no register move.

Bits	Field	Type	Reset	Description
31:0	-	-	-	Reserved. Reads zero; write zero.

31.7.24 EVFCH10CFG: Reserved channel-10 configuration slot

Offset 0x68 **Reset** 0x00000000 **Width** 32 **Address** 0x6B68
Reserved channel-10 configuration slot. This build implements 8 channels, so this address reads 0 and ignores writes; the map reserves 16 channel addresses so a wider fabric needs no register move.

Bits	Field	Type	Reset	Description
31:0	-	-	-	Reserved. Reads zero; write zero.

31.7.25 EVFCH11CFG: Reserved channel-11 configuration slot

Offset 0x6C **Reset** 0x00000000 **Width** 32 **Address** 0x6B6C
Reserved channel-11 configuration slot. This build implements 8 channels, so this address reads 0 and ignores writes; the map reserves 16 channel addresses so a wider fabric needs no register move.

Bits	Field	Type	Reset	Description
31:0	-	-	-	Reserved. Reads zero; write zero.

31.7.26 EVFCH12CFG: Reserved channel-12 configuration slot

Offset 0x70 **Reset** 0x00000000 **Width** 32 **Address** 0x6B70
Reserved channel-12 configuration slot. This build implements 8 channels, so this address reads 0 and ignores writes; the map reserves 16 channel addresses so a wider fabric needs no register move.

Bits	Field	Type	Reset	Description
31:0	-	-	-	Reserved. Reads zero; write zero.

31.7.27 EVFCH13CFG: Reserved channel-13 configuration slot

Offset 0x74 **Reset** 0x00000000 **Width** 32 **Address** 0x6B74
Reserved channel-13 configuration slot. This build implements 8 channels, so this address reads 0 and ignores writes; the map reserves 16 channel addresses so a wider fabric needs no register move.

Bits	Field	Type	Reset	Description
31:0	-	-	-	Reserved. Reads zero; write zero.

31.7.28 EVFCH14CFG: Reserved channel-14 configuration slot

Offset 0x78 **Reset** 0x00000000 **Width** 32 **Address** 0x6B78
Reserved channel-14 configuration slot. This build implements 8 channels, so this address reads 0 and ignores writes; the map reserves 16 channel addresses so a wider fabric needs no register move.

Bits	Field	Type	Reset	Description
31:0	-	-	-	Reserved. Reads zero; write zero.

31.7.29 EVFCH15CFG: Reserved channel-15 configuration slot

Offset 0x7C **Reset** 0x00000000 **Width** 32 **Address** 0x6B7C

Reserved channel-15 configuration slot. This build implements 8 channels, so this address reads 0 and ignores writes; the map reserves 16 channel addresses so a wider fabric needs no register move.

Bits	Field	Type	Reset	Description
31:0	-	-	-	Reserved. Reads zero; write zero.

32 AFEx

32.1 Overview

The analog front-end site (AFEx) is the digital half of one electrochemical channel. Four identical sites, one per channel tile, drive the four anatop_pixel cells of the analog macro: a grounded-working-electrode potentiostat (control amplifier into the counter electrode, transimpedance amplifier holding the working electrode at the common mode), a 320-code programmable feedback ladder, a 12-bit potential-program DAC, a 12-bit common-mode DAC, a local bias generator, a 16:1 converter input multiplexer, a 16:1 analog test-port multiplexer and a 10-bit SAR converter. The site owns the 50 control bits of that cell and sequences its converter. The main features are listed below:

- The 50 control bits as direct register outputs: AFExTIA (ladder code), AFExDACVP and AFExDACVCM (DAC codes and enables), AFExBIAS (bias trim), AFExMUX (converter and test-port selects)
- A converter sequencer: trigger clock at MCLK / 2(AFECLKDIV+1), twelve falling edges per conversion in 16 + AFESAMPLESTEP ticks, reset release with a 128-cycle quiet period
- Result capture inside the READY window with the converter's inverted bit 9 corrected, into a four-entry FIFO tagged with the input slot and the excitation phase (AFExDATA)
- Single-shot and continuous conversion (AFESTART, AFECONT), a simultaneous-sample trigger shared by the four sites (AFESYNC, AFESYNCEN), and a two-code excitation swap engine (AFExSWAP, AFESWAPEN)
- One level interrupt per site, combined onto vector 124 (AFEDRDYIE, AFEERRIE)
- Per-site ownership: a site answers its channel hart or hart 0; every other master reads 0 and its writes are dropped

32.2 Instances and signals

Table 32.1: AFEx instances

Instance	Base address	Interrupt vector(s)	Pins (AF)
AFE0	0x6C00	124	none
AFE1	0x6D00	none	none
AFE2	0x6E00	none	none
AFE3	0x6F00	none	none

Table 32.2: AFEx interrupt vectors

Vector	Instance	Description	Clear method
124	AFE0	AFE0-3 combined result-ready / error Interrupt (OR of the four sites) IRQB_AFE	Per site: write 1 to AFExSR.AFEDRDY (pop) until AFECNT = 0, and to AFEOVF / AFETO

32.3 Functional description

32.3.1 Control surface

Every field of AFExTIA, AFExDACVP, AFExDACVCM, AFExBIAS and AFExMUX reaches the analog macro directly as 1.0 V CMOS; the 1.0 V to 2.5 V level shifters are inside the macro. The bits reset to zero, which is the minimum-gain, converter-on-the-transimpedance-output state, and two of the reset values are not usable operating points: AFEBIASADJ = 0 is not the nominal bias trim, and AFEATPSEL = 0 selects an undriven test-port slot. The power-up sequence in the Programming section loads them before any measurement.

The feedback resistance follows $R_f = 18\,\mathrm{k\Omega} + 6\,\mathrm{k\Omega}\,N$ with $N =$ AFERESEN + 64 popcount(AFETHEN), $N = 0 \ldots 319$; the law is 9 % high at $N = 0$ and within 0.5 % from $N = 20$ up, so gain constants are stored per code, never computed. Both DACs step 610.35 µV per code.

32.3.2 Converter sequencer

AFEEN releases the converter reset (SARADC_rst = not EN) and starts a 128-cycle quiet period; AFEREL reports its end and AFESTART is ignored before it. A conversion is 16 + AFESAMPLESTEP ticks of the trigger clock: one idle tick, the clear pulse (one tick), one low tick, the sample pulse (1 + AFESAMPLESTEP ticks; its falling edge is the aperture), one low tick, ten conversion pulses (one falling edge per bit trial) and one low tick. With AFESAMPLESTEP = 7 and a 20 MHz trigger clock that is 23 ticks, 1.150 µs, 869.6 kSa/s, the pattern the macro is characterised with. The trigger clock is MCLK / 2(AFECLKDIV + 1): a 40 MHz MCLK at AFECLKDIV = 0 gives 20 MHz; a 24 MHz MCLK gives 12 MHz, which is slower than characterised and inside the macro's synchronous margin. The output is a register, so the gated clock is glitch-free.

The macro raises READY after the tenth conversion falling edge and holds the data bus valid for about 100 ns. The site captures the bus on the MCLK edge after the one that first samples READY high (25 to 50 ns after the rise at 40 MHz), inverts bit 9, and pushes the code into the FIFO with the AFEADCSEL value and the swap phase latched at the aperture. A READY missing for 2048 MCLK after an aperture, or a new aperture with the previous result still outstanding, sets AFETO; a capture into a full FIFO sets AFEOVF and drops the sample.

32.3.3 Result FIFO

AFExDATA is the FIFO head and reading it has no side effect: AFEVALID says whether it holds a result and AFECNT counts the entries. Writing 1 to AFEDRDY pops the head; a pop on an empty FIFO does nothing. The tags let firmware interleave the current channel (AFEADCSEL = 0) and the potential channel (AFEADCSEL = 2) in one continuous run and sort the results afterwards.

32.3.4 Simultaneous sampling and the swap engine

A write of 1 to AFESYNC on any site pulses a trigger shared by the four sites, and every site with AFESYNCEN set starts on the same MCLK edge; sites programmed with the same AFECLKDIV then run tick-aligned. With AFESWAPEN and AFECONT set, A_Dac_Vp alternates between AFEVP and AFEVP2 every AFEPERIOD + 1 conversions, switching at a conversion boundary, so the square-wave excitation and the converter strobe share one divider and the sampling grid is commensurate with the excitation. The phase at each aperture is recorded in AFEPHTAG.

32.4 Interrupts and events

Each site drives one level interrupt, (AFEDRDY ∧ AFEDRDYIE) ∨ ((AFEOVF ∨ AFETO) ∧ AFEERRIE), and the four lines are ORed onto vector 124 (the interrupt router caps the source count at 127, which leaves no room for four). Every channel hart routes vector 124; the service routine reads its own site's AFExSR, returns if nothing is set there, otherwise pops the FIFO until AFECNT is 0 and clears AFEOVF and AFETO with a write of 1. The block produces no event-fabric taps.

32.5 Programming

32.5.1 Power-up

1. Write the stored bias trim to AFEBIASADJ; nothing else on the channel is meaningful before the bias rails are up.
2. Enable and program the common-mode DAC (AFEVCMEN, AFEVCM) and the potential DAC (AFEVPEN, AFEVP).
3. Set the gain code in AFExTIA and AFEADCSEL = 0.
4. Set AFEEN and poll AFEREL.

32.5.2 One conversion

Write AFESTART; wait for AFEDRDY (or the interrupt with AFEDRDYIE); read AFExDATA; write 1 to AFEDRDY to pop. Wait 0.54 µs after any AFEADCSEL change before the next start.

32.5.3 Continuous record

Set AFECONT with AFESTART (or arm AFESYNCEN on every site and write AFESYNC once for a simultaneous record). Drain the FIFO faster than one conversion per 1.15 µs at the design rate, or set AFEERRIE and treat AFEOVF as a lost sample. Clear AFECONT to stop at the next conversion boundary; clear AFEEN to abort and reset the converter.

32.5.4 Square-wave impedance record

Program AFEVP and AFEVP2 as a code pair inside one octave, AFEPERIOD for the excitation half period in conversions, then AFESWAPEN with AFECONT and AFESTART. Discard the electrode's own transient (at least 100 time constants), then correlate the tagged results over an integer number of periods.

32.6 Usage notes

- Reads of every register are side-effect-free; the FIFO is popped by AFEDRDY only, so LR/SC and AMO sequences to a site address are harmless.
- AFESTART and AFESYNC read 0. A AFESTART written in the same access that sets AFEEN is lost, because AFEREL is not yet set.
- The converter's raw bus carries bit 9 inverted; AFECODE is already the straight code, offset-binary about the common-mode code. Do not invert again.

- Converter input slots 3 and 6 to 15 are undriven in the tapeout cell; select only 0, 1, 2, 4 and 5.
- The data hold time of the converter bus after READY is not characterised; the capture point assumes at least two MCLK periods and is to be confirmed on the extracted macro before tapeout.

32.7 Registers

Register offsets in this chapter are relative to the instance base address in Table 32.1 (AFE0 0x6C00, AFE1 0x6D00, AFE2 0x6E00, AFE3 0x6F00); the generated header names each base AFE0_BASE and so on.

Table 32.3: AFEx register summary

Offset	Name	Access	Reset	Description
0x00	AFExCR	rw/w1	0x00000700	Control
0x04	AFExSR	r/rw1	0x00000000	Status
0x08	AFExDATA	r	0x00000000	Result FIFO head, side-effect-free
0x0C	AFExTIA	rw	0x00000000	
0x10	AFExDACVP	rw	0x00000000	Potential-program DAC (A_Dac_Vp, 610.35 uV per code) and its enable
0x14	AFExDACVCM	rw	0x00000000	Common-mode DAC (A_Dac_Vcm, 610.35 uV per code) and its enable
0x18	AFExBIAS	rw	0x00000000	Local bias generator trim
0x1C	AFExMUX	rw	0x000000F0	Converter input and analog test-port selects
0x20	AFExSWAP	rw	0x00000000	
0x24	AFExBIASG0	rw	0x00000000	
0x28	AFExBIASG1	rw	0x00000000	
0x2C	AFExBIASG2	rw	0x00000000	
0x30	AFExBIASG3	rw	0x00000000	
0x34	AFExBIASGCR	rw	0x00000000	Shared bias generator mode, enables and trim

32.7.1 AFExCR: Control

Offset 0x00 **Reset** 0x00000700 **Width** 32 **Address** 0x6C00 (AFE0), 0x6D00 (AFE1), 0x6E00 (AFE2), 0x6F00 (AFE3)
Control. EN releases the converter reset (SARADC_rst = not EN) and, after 128 MCLK of quiet, allows a start; START and SYNC are write-1 pulses and read 0. SAMPLESTEP resets to 7 (the characterised 400 ns sample phase at 20 MHz); every other bit resets to 0.

Bits	Field	Type	Reset	Description
31:16	-	-	-	Reserved. Reads zero; write zero.
15:12	CLKDIV	rw	0b0000	Trigger-clock divider: SARADC_clk = MCLK / (2 (CLKDIV + 1)). 0 gives 20 MHz from a 40 MHz MCLK (the characterised rate) and 12 MHz from 24 MHz.
11:8	SAMPLESTEP	rw	0b0111	Extra sample-phase ticks: the sample pulse is 1 + SAMPLESTEP ticks and a conversion is 16 + SAMPLESTEP ticks. Resets to 7.

Bits	Field	Type	Reset	Description
7	SWAPEN	rw	0	Excitation swap engine: with CONT set, A_Dac_Vp alternates between DACVP.VP and SWAP.VP2 every PERIOD + 1 conversions, switching at a conversion boundary. 0: A_Dac_Vp = VP 1: A_Dac_Vp alternates VP / VP2
6	ERRIE	rw	0	Interrupt enable for OVF and TO.
5	DRDYIE	rw	0	Interrupt enable for DRDY (result FIFO not empty).
4	SYNCEN	rw	0	Start on the shared trigger: a SYNC written on any site starts every site with SYNCEN set on the same MCLK edge.
3	SYNC	w1	0	Broadcast the simultaneous-sample trigger (one MCLK pulse to all four sites). Self-clearing, reads 0.
2	START	w1	0	Start a conversion (a continuous run when CONT is set). Ignored unless REL is set and the site is not BUSY. Self-clearing, reads 0.
1	CONT	rw	0	Continuous conversion: the pattern reloads at every conversion boundary until CONT is cleared, when the conversion in flight completes.
0	EN	rw	0	Converter enable. 0 holds SARADC_rst high, stops the trigger clock and aborts a run; 1 releases the reset and starts the 128 MCLK quiet period. 0: Converter in reset 1: Converter released

32.7.2 AFExSR: Status

Offset 0x04 **Reset** 0x00000000 **Width** 32 **Address** 0x6C04 (AFE0), 0x6D04 (AFE1), 0x6E04 (AFE2), 0x6F04 (AFE3)

Status. DRDY, OVF and TO take a write-1: DRDY pops the FIFO head, OVF and TO clear. BUSY, REL and CNT are read-only live status.

Bits	Field	Type	Reset	Description
31:12	-	-	-	Reserved. Reads zero; write zero.
11:8	CNT	r	0b0000	Result FIFO occupancy, 0 to 4.
7:5	-	-	-	Reserved. Reads zero; write zero.
4	TO	rw1	0	Missed READY: no READY inside 2048 MCLK of an aperture, or a new aperture with the previous conversion still outstanding. Write 1 to clear.
3	REL	r	0	Converter released: SARADC_rst low and the quiet period elapsed; START is accepted.
2	OVF	rw1	0	A result arrived with the FIFO full and was dropped. Write 1 to clear.
1	DRDY	rw1	0	Result FIFO not empty (DATA holds the oldest result). Write 1 to pop the head; a pop on an empty FIFO does nothing.
0	BUSY	r	0	A conversion pattern is running.

32.7.3 AFExDATA: Result FIFO head, side-effect-free

Offset 0x08 **Reset** 0x00000000 **Width** 32 **Address** 0x6C08 (AFE0), 0x6D08 (AFE1), 0x6E08 (AFE2), 0x6F08 (AFE3)

Result FIFO head, side-effect-free. Bit 9 of the converter bus is already inverted, so CODE is the straight 10-bit code, offset-binary about the common-mode code. Reads 0 with VALID clear when the FIFO is empty.

Bits	Field	Type	Reset	Description
31:16	-	-	-	Reserved. Reads zero; write zero.
15	VALID	r	0	The entry is valid (FIFO not empty at the read).
14	PHTAG	r	0	Excitation swap phase at this conversion's aperture (0 = VP, 1 = VP2).
13:10	SELTAG	r	0b0000	SARADC_SEL (MUX.ADCSEL) at this conversion's aperture.
9:0	CODE	r	0x000	Converter code, bit 9 corrected (code = raw xor 0x200).

32.7.4 AFExTIA

Offset 0x0C **Reset** 0x00000000 **Width** 32 **Address** 0x6C0C (AFE0), 0x6D0C (AFE1), 0x6E0C (AFE2), 0x6F0C (AFE3)

Transimpedance feedback ladder: Rf = 18 kOhm + 6 kOhm N, N = RESEN + 64 popcount(THEN), 0 to 319. Active-high = insert. Resets to 0 (minimum gain).

Bits	Field	Type	Reset	Description
31:10	-	-	-	Reserved. Reads zero; write zero.
9:6	THEN	rw	0b0000	ThEn<3:0>: thermometer segments, 64 units each.
5:0	RESEN	rw	0x00	ResEn<5:0>: binary segments (32/16/8/4/2/1 units).

32.7.5 AFExDACVP: Potential-program DAC (A_Dac_Vp, 610.35 uV per code) and its enable

Offset 0x10 **Reset** 0x00000000 **Width** 32 **Address** 0x6C10 (AFE0), 0x6D10 (AFE1), 0x6E10 (AFE2), 0x6F10 (AFE3)

Bits	Field	Type	Reset	Description
31:13	-	-	-	Reserved. Reads zero; write zero.
12	VPEN	rw	0	En_Dac_Vp.
11:0	VP	rw	0x000	A_Dac_Vp<11:0>, straight binary. With SWAPEN set this is the phase-0 code.

32.7.6 AFExDACVCM: Common-mode DAC (A_Dac_Vcm, 610.35 uV per code) and its enable

Offset 0x14 **Reset** 0x00000000 **Width** 32 **Address** 0x6C14 (AFE0), 0x6D14 (AFE1), 0x6E14 (AFE2), 0x6F14 (AFE3)

Bits	Field	Type	Reset	Description
31:13	-	-	-	Reserved. Reads zero; write zero.
12	VCMEN	rw	0	En_Dac_Vcm.
11:0	VCM	rw	0x000	A_Dac_Vcm<11:0>, straight binary.

32.7.7 AFExBIAS: Local bias generator trim

Offset 0x18 **Reset** 0x00000000 **Width** 32 **Address** 0x6C18 (AFE0), 0x6D18 (AFE1), 0x6E18 (AFE2), 0x6F18 (AFE3)

Local bias generator trim. Higher code = smaller bias current, 2.7 percent per code. Reset 0 is not the nominal trim: load the stored code first in every power-up sequence.

Bits	Field	Type	Reset	Description
31:6	-	-	-	Reserved. Reads zero; write zero.
5:0	BIASADJ	rw	0x00	Bias_Adj<5:0>.

32.7.8 AFExMUX: Converter input and analog test-port selects

Offset 0x1C **Reset** 0x000000F0 **Width** 32 **Address** 0x6C1C (AFE0), 0x6D1C (AFE1), 0x6E1C (AFE2), 0x6F1C (AFE3)

Converter input and analog test-port selects. ADCSEL: 0 = TIA output, 1 = common mode, 2 = potential-program DAC, 4 = test port, 5 = ground; other slots are undriven, do not select. ATPSEL slot 0 is undriven at reset.

Bits	Field	Type	Reset	Description
31:8	-	-	-	Reserved. Reads zero; write zero.
7:4	ATPSEL	rw	0b1111	ATP_SEL<3:0>: analog test-port multiplexer select.
3:0	ADCSEL	rw	0b0000	SARADC_SEL<3:0>: converter input multiplexer select. Wait 0.54 us after a change before a result is meaningful.

32.7.9 AFExSWAP

Offset 0x20 **Reset** 0x00000000 **Width** 32 **Address** 0x6C20 (AFE0), 0x6D20 (AFE1), 0x6E20 (AFE2), 0x6F20 (AFE3)

Excitation swap engine: the second potential code and the half period, in conversions. The swap and the converter strobe share one divider, so the excitation grid is commensurate with sampling.

Bits	Field	Type	Reset	Description
31:16	PERIOD	rw	0x0000	Half period minus 1, in conversions: A_Dac_Vp toggles every PERIOD + 1 conversions.
15:12	-	-	-	Reserved. Reads zero; write zero.
11:0	VP2	rw	0x000	Phase-1 potential code.

32.7.10 AFExBIASG0

Offset 0x24 **Reset** 0x00000000 **Width** 32 **Address** 0x6C24 (AFE0), 0x6D24 (AFE1), 0x6E24 (AFE2), 0x6F24 (AFE3)

Shared bias generator (anatop_biasgen_g) R-2R code for rail bn, the NMOS cascode rail, driving the macro's code_bn pins. SITE 0 ONLY: sites 1-3 read 0 here and drop writes. The four rails are independent voltages from four independent ladders, so each carries its own code. Resets to 0, and the value is inert at reset by construction: BIASGCR resets with EN = 0 (generator off) and USEDAC = 0 (the generator core, not the ladders, sets the rails), so nothing reads this word until firmware writes it. The shipped mission mode is the generator path, which needs no DAC code at all. Present only in the per-tile AFE topology.

Bits	Field	Type	Reset	Description
31:14	-	-	-	Reserved. Reads zero; write zero.
13:0	BGCODE0	rw	0x0000	14-bit R-2R code setting rail bn when BIASGCR.USEDAC is set. Bench value 4628 (0x1214) at 25 C, AVDD 2.5 V, which forces the rail onto its in-pixel target to better than 0.4 mV (B9); at 37 C the same target needs 4688. Reset 0; write all four before setting USEDAC, or the ladders drive every rail to 0 V and the macro holds 1.20 mA, 2.7x nominal, until USEDAC is cleared. The code moves 4.8 to 22.2 LSB per kelvin, so this value is a bench setting at room temperature, not a shipped constant.

32.7.11 AFExBIASG1

Offset 0x28 **Reset** 0x00000000 **Width** 32 **Address** 0x6C28 (AFE0), 0x6D28 (AFE1), 0x6E28 (AFE2), 0x6F28 (AFE3)

Shared bias generator (anatop_biasgen_g) R-2R code for rail bnc, the NMOS cascode gate rail, driving the macro's code_bnc pins. SITE 0 ONLY: sites 1-3 read 0 here and drop writes. The four rails are independent voltages from four independent ladders, so each carries its own code. Resets to 0, and the value is inert at reset by construction: BIASGCR resets with EN = 0 (generator off) and USEDAC = 0 (the generator core, not the ladders, sets the rails), so nothing reads this word until firmware writes it. The shipped mission mode is the generator path, which needs no DAC code at all. Present only in the per-tile AFE topology.

Bits	Field	Type	Reset	Description
31:14	-	-	-	Reserved. Reads zero; write zero.
13:0	BGCODE1	rw	0x0000	14-bit R-2R code setting rail bnc when BIASGCR.USEDAC is set. Bench value 5895 (0x1707) at 25 C, AVDD 2.5 V, which forces the rail onto its in-pixel target to better than 0.4 mV (B9); at 37 C the same target needs 5972. Reset 0; write all four before setting USEDAC, or the ladders drive every rail to 0 V and the macro holds 1.20 mA, 2.7x nominal, until USEDAC is cleared. The code moves 4.8 to 22.2 LSB per kelvin, so this value is a bench setting at room temperature, not a shipped constant.

32.7.12 AFExBIASG2

Offset 0x2C **Reset** 0x00000000 **Width** 32 **Address** 0x6C2C (AFE0), 0x6D2C (AFE1), 0x6E2C (AFE2), 0x6F2C (AFE3)

Shared bias generator (anatop_biasgen_g) R-2R code for rail bp, the PMOS cascode rail, driving the macro's code_bp pins. SITE 0 ONLY: sites 1-3 read 0 here and drop writes. The four rails are independent voltages from four independent ladders, so each carries its own code. Resets to 0, and the value is inert at reset by construction: BIASGCR resets with EN = 0 (generator off) and USEDAC = 0 (the generator core, not the ladders, sets the rails), so nothing reads this word until firmware writes it. The shipped mission mode is the generator path, which needs no DAC code at all. Present only in the per-tile AFE topology.

Bits	Field	Type	Reset	Description
31:14	-	-	-	Reserved. Reads zero; write zero.

Bits	Field	Type	Reset	Description
13:0	BGCODE2	rw	0x0000	14-bit R-2R code setting rail bp when BIASGCR.USEDAC is set. Bench value 14058 (0x36EA) at 25 C, AVDD 2.5 V, which forces the rail onto its in-pixel target to better than 0.4 mV (B9); at 37 C the same target needs 14241. Reset 0; write all four before setting USEDAC, or the ladders drive every rail to 0 V and the macro holds 1.20 mA, 2.7x nominal, until USEDAC is cleared. The code moves 4.8 to 22.2 LSB per kelvin, so this value is a bench setting at room temperature, not a shipped constant.

32.7.13 AFExBIASG3

Offset 0x30 **Reset** 0x00000000 **Width** 32 **Address** 0x6C30 (AFE0), 0x6D30 (AFE1), 0x6E30 (AFE2), 0x6F30 (AFE3)

Shared bias generator (anatop_biasgen_g) R-2R code for rail bpc, the PMOS cascode gate rail, driving the macro's code_bpc pins. SITE 0 ONLY: sites 1-3 read 0 here and drop writes. The four rails are independent voltages from four independent ladders, so each carries its own code. Resets to 0, and the value is inert at reset by construction: BIASGCR resets with EN = 0 (generator off) and USEDAC = 0 (the generator core, not the ladders, sets the rails), so nothing reads this word until firmware writes it. The shipped mission mode is the generator path, which needs no DAC code at all. Present only in the per-tile AFE topology.

Bits	Field	Type	Reset	Description
31:14	-	-	-	Reserved. Reads zero; write zero.
13:0	BGCODE3	rw	0x0000	14-bit R-2R code setting rail bpc when BIASGCR.USEDAC is set. Bench value 12294 (0x3006) at 25 C, AVDD 2.5 V, which forces the rail onto its in-pixel target to better than 0.4 mV (B9); at 37 C the same target needs 12454. Reset 0; write all four before setting USEDAC, or the ladders drive every rail to 0 V and the macro holds 1.20 mA, 2.7x nominal, until USEDAC is cleared. The code moves 4.8 to 22.2 LSB per kelvin, so this value is a bench setting at room temperature, not a shipped constant.

32.7.14 AFExBIASGCR: Shared bias generator mode, enables and trim

Offset 0x34 **Reset** 0x00000000 **Width** 32 **Address** 0x6C34 (AFE0), 0x6D34 (AFE1), 0x6E34 (AFE2), 0x6F34 (AFE3)

Shared bias generator mode, enables and trim. SITE 0 ONLY. The four mode bits are in the macro's own pin order (use_dac, en_gen, en_buf_int, en) so the register and the LEF can be compared by eye. Every bit resets to 0: the generator is off and its four rails undriven, so a chip that never writes this word has four dead channels rather than four mis-biased ones.

Bits	Field	Type	Reset	Description
31:14	-	-	-	Reserved. Reads zero; write zero.

Bits	Field	Type	Reset	Description
13:8	BGADJ	rw	0x00	Generator trim, moving all four rails together in 1.4 to 4.5 mV steps over 45 to 141 mV. RESET 0 IS THE adj0 END OF THE RANGE, NOT THE NOMINAL: at 0 the generator draws 56.6 uA and bn sits at 0.678 V against a 0.6147 V nominal, so the power-up sequence must write a trim code before any channel is enabled. WRITE 37 for operation at 37 C: measured, it holds all four buffered rails within 0.68 mV of the in-pixel targets with no DAC code at all, at 419.2 uA, and on the topology-A local generator it reproduces those rails to 0.01 mV (B9, firmware contract R15). Code 33 is the same generator at 27 C, where it reproduces the published standalone rails and their 24.46 uA reference current; the two codes are 4.7 percent apart in reference current and are not alternatives but two temperatures. The trim is temperature-specific: the in-pixel target set is itself a 37 C set.
7:4	-	-	-	Reserved. Reads zero; write zero.
3	BGEN	rw	0	Master enable for the generator and its four output buffers (macro pin en). 0 leaves all four rails undriven and every channel unusable. 0: Generator off 1: Generator on
2	BGENBUFINT	rw	0	Enable the macro's four INTERNAL buffers (macro pin en_buf_int). Ship 0: measured at nominal they move the rails by 0.3 mV and the offset by 2 uV behind the outer buffer, and cost four OTA bias currents. 0: Internal buffers off (shipped) 1: Internal buffers on
1	BGENGEN	rw	0	Enable the wide-swing cascode generator core (macro pin en_gen).
0	BGUSEDAC	rw	0	Rail source select (macro pin use_dac). 0 = the generator core sets all four rails and BIASG0-3 are inert; 1 = the four R-2R ladders set them from BIASG0-3. SHIP 0. The DAC path is a bench override valid only near nominal: with the 25 C codes applied the four output buffers rail to AVDD for T <= -25 C and for AVDD <= 2.275 V, the supply current reaches 1325 uA at 85 C and 2035 uA at AVDD 2.75 V against 435.8 uA in generator mode, and the transfer is non-monotonic at the LSB level so a step-and-keep firmware search will not converge on it (B9, firmware contract R15). There is no ordering constraint against ENGEN with valid codes; with any code at 0 there is. 0: Generator sets the rails (shipped) 1: BIASG0-3 set the rails

33 Interrupt router (IRQROUTER)

33.1 Overview

The interrupt router (IRQROUTER) is the chip's peripheral interrupt controller. Every deglitched peripheral interrupt level terminates here, and delivery follows the claim/complete model of a RISC-V platform-level interrupt controller: the router raises one external-interrupt wire (meip, vector 85) to each hart, and the servicing hart reads CLAIM to discover and atomically claim the pending source. Routing is per hart and per vector, so a peripheral's interrupt can be steered to whichever hart currently owns that peripheral.

- One row of four 32-bit enable words per hart (HxENL, HxENM, HxENU, HxENX), one bit per vector, covering vectors 0 to 124
- One meip wire per hart (vector 85); CLAIM read claims the lowest pending enabled vector, CLAIM write completes it
- Exactly-once delivery: a claimed vector is hidden from every hart until completed
- Fixed priority: the lowest pending vector number wins
- Read-only raw-level words (PENDL, PENDM, PENDU, PENDX) and under-service words (INSVCL, INSVCM, INSVCU, INSVCX)
- Resets fully masked, so the router is inert until programmed
- Runs on the free-running MCLK, so a routed interrupt wakes a hart whose clock is stopped by sleep

The CLINT vectors (83 and 84) are never delivered through meip: each hart receives its own software and timer interrupts on dedicated wires, so the corresponding row bits are writable but have no effect.

33.2 Instances and signals

Table 33.1: IRQROUTER instances

Instance	Base address	Interrupt vector(s)	Pins (AF)
IRQROUTER	0x7000	none	none

33.3 Block diagram

Figure 33.1 draws the router in its place in the chip: every peripheral interrupt source on one side, the per-hart enable rows and the claim/complete stage in the middle, and the harts on the other. The two CLINT levels reach the harts on their own wires and never pass through the router.

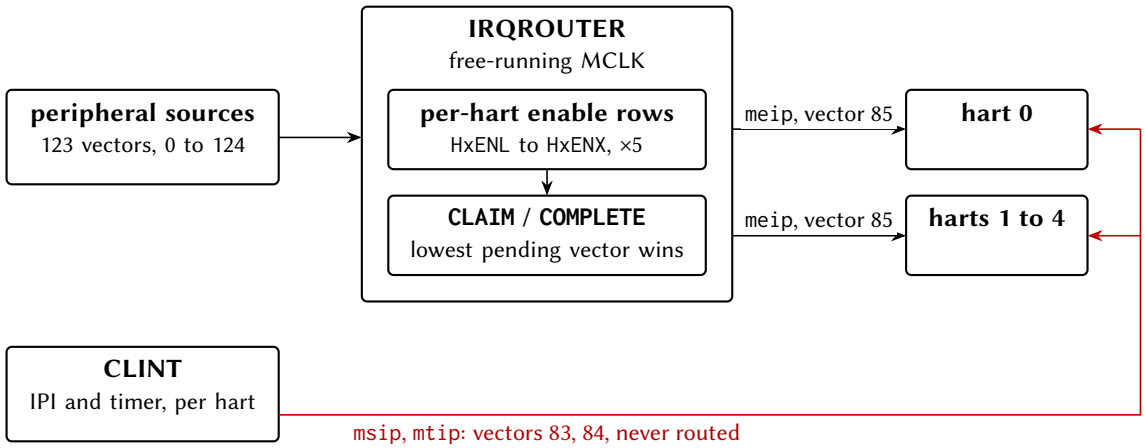


Figure 33.1: The interrupt fabric: peripheral sources, the router's enable rows and claim/complete stage, the harts, and the CLINT pair in red because it bypasses the router.

33.4 Functional description

33.4.1 Routing rows

Hart h's row is the four words HxENL (vectors 0 to 31), HxENM (32 to 63), HxENU (64 to 95) and HxENX (96 to 124), with $x = h$. Bit v mod 32 of the word selected by $v \div 32$ enables vector v for that hart. The packing is contiguous, so bit b of HxENU is vector $64 + b$. Any hart may write any row through the shared window, which is what allows a supervising hart to hand a peripheral to another hart. The rows start at the block base, 16 bytes per hart; CLAIM and the status words sit at the fixed offsets 0x800, 0x810 and 0x820, independent of the hart count.

33.4.2 Claim and complete

Hart h's meip wire is asserted while some vector v is pending (deglitched level high), enabled in row h, and not under service. A read of CLAIM by hart h returns the lowest such v and marks it under service, which hides it from every hart's meip until it is completed; the arbiter attributes the read to the requesting hart, so the router evaluates the reader's own row. A read that finds nothing pending for the reader returns 0xFFFFFFFF, which happens when another hart claimed the only pending source first. A write of v to CLAIM completes it: the vector becomes deliverable again, and if its level is still asserted it pends again immediately. Figure 33.2 shows one delivery.

The handler cell in the figure stands for many mclk of software. The handler clears the level at the peripheral before it completes, because a level still high at completion pends the source again at once. Completion is not qualified by owner, so that a supervising hart can complete a vector on behalf of a hart that died while servicing it; the under-service words show which vectors are stuck. Every hart, hart 0 included, uses this path; row 0 is hart 0's live routing row.

33.4.3 Status words

The four PENDx words carry the raw deglitched level of every vector, whether enabled or not, and the four INSVCx words carry the under-service state. Both sets are read-only and exist for diagnosis and recovery.

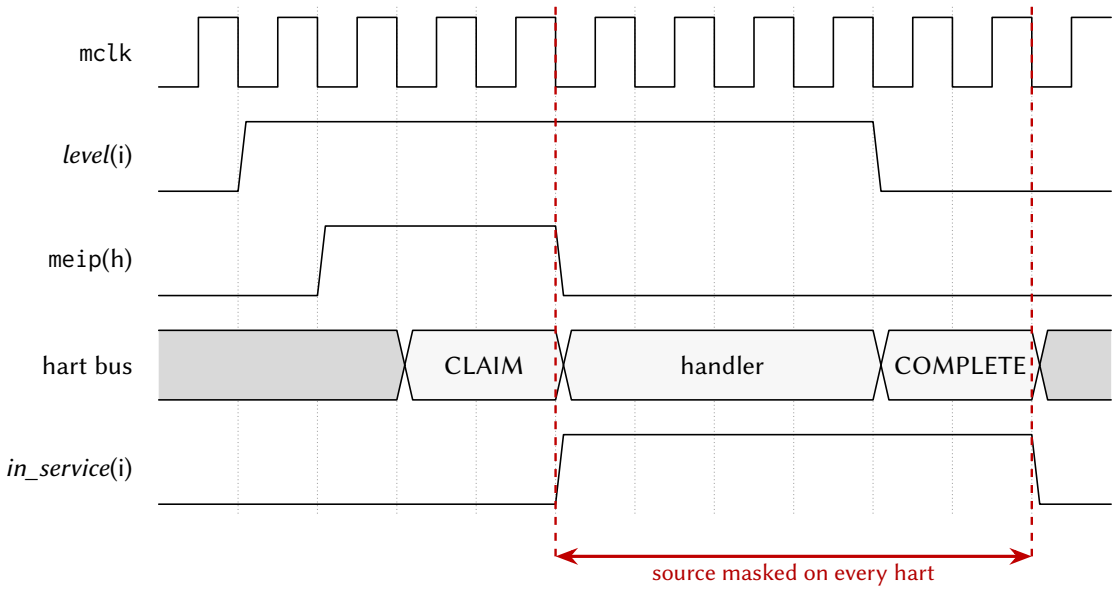


Figure 33.2: Claim/complete delivery of one peripheral interrupt.

33.5 Interrupts and events

The router does not raise a vector of its own; it drives vector 85 (`meip`) into each hart. Table 33.2 states the condition.

Vector	Condition	Set by	Cleared by
85	A vector pending, enabled in this hart's row and not under service	The peripheral asserting its interrupt level	Reading `CLAIM` (hides the vector until completed)

Table 33.2: `IRQROUTER` delivery into the harts.

33.6 Programming

33.6.1 Routing a peripheral to hart *h*

1. Clear the peripheral's vector bits in the previous owner's row (`HxENL` to `HxENX` with x the previous owner).
2. Set the same bits in hart h's row.
3. Enable vector 85 in hart h's interrupt controller.

33.6.2 Servicing a routed interrupt

1. Read `CLAIM`; if the value is `0xFFFFFFFF`, return, because the interrupt was spurious.
2. Service the source and clear the interrupt flag at the peripheral.
3. Write the claimed vector number to `CLAIM`.

33.7 Usage notes

Clear the level at the peripheral before completing. A completed vector whose level is still asserted pends again at once, so a handler that completes first re-enters immediately.

Treat `0xFFFFFFFF` from `CLAIM` as spurious. It is the normal outcome when several harts enable the same vector and another hart claimed it first.

Route a vector to one hart at a time unless the sources are idempotent. Exactly-once delivery guarantees that one hart services each event, not which hart.

Recover a stuck vector from a supervisor. Read the `INSVCx` words to find a vector left under service by a dead hart and write its number to `CLAIM` from any hart.

33.8 Registers

Register offsets in this chapter are relative to the instance base address in Table 33.1 (`IRQROUTER` `0x7000`); the generated header names each base `IRQROUTER_BASE`.

Table 33.3: `IRQROUTER` register summary

Offset	Name	Access	Reset	Description
`0x00 + 16n`	`HnENL (n = 0..4)`	`rw`	`0x00000000`	Hart n interrupt routing register, vectors 31:0
`0x04 + 16n`	`HnENM (n = 0..4)`	`rw`	`0x00000000`	Hart n interrupt routing register, vectors 63:32
`0x08 + 16n`	`HnENU (n = 0..4)`	`rw`	`0x00000000`	Hart n interrupt routing register, vectors 95:64
`0x0C + 16n`	`HnENX (n = 0..4)`	`rw`	`0x00000000`	
`0x50` to `0x7FF`	-	-	-	Reserved
`0x800`	`CLAIM`	`rw`	`0x00000000`	Interrupt claim/complete register (M19)
`0x804` to `0x80F`	-	-	-	Reserved
`0x810`	`PENDL`	`r`	`0x00000000`	
`0x814`	`PENDM`	`r`	`0x00000000`	Raw pending interrupt levels, vectors 63:32 (read-only)
`0x818`	`PENDU`	`r`	`0x00000000`	Raw pending interrupt levels, vectors 95:64 (read-only)
`0x81C`	`PENDX`	`r`	`0x00000000`	
`0x820`	`INSVCL`	`r`	`0x00000000`	
`0x824`	`INSVCM`	`r`	`0x00000000`	Under-service flags, vectors 63:32 (read-only)
`0x828`	`INSVCU`	`r`	`0x00000000`	Under-service flags, vectors 95:64 (read-only)
`0x82C`	`INSVCX`	`r`	`0x00000000`	

33.8.1 HnENL (n = 0..4): Hart n interrupt routing register, vectors 31:0

Offset 0x00 + 16n **Reset** 0x00000000 **Width** 32
Hart n interrupt routing register, vectors 31:0. Each bit enables delivery of the corresponding interrupt vector to hart n via its meip wire (vector 85) and the CLAIM/COMPLETE mechanism.

Bits	Field	Type	Reset	Description
31:0	HnENL	rw	0x00000000	Interrupt enable bits for vectors 31:0, routed to hart n.

33.8.2 HnENM (n = 0..4): Hart n interrupt routing register, vectors 63:32

Offset 0x04 + 16n **Reset** 0x00000000 **Width** 32

Bits	Field	Type	Reset	Description
31:0	HnENM	rw	0x00000000	Interrupt enable bits for vectors 63:32, routed to hart n.

33.8.3 HnENU (n = 0..4): Hart n interrupt routing register, vectors 95:64

Offset 0x08 + 16n **Reset** 0x00000000 **Width** 32
Hart n interrupt routing register, vectors 95:64. Bits 19 and 20 correspond to the CLINT vectors 83 and 84, which are delivered on dedicated hardwired wires and never through meip: these two bits are writable but have no effect.

Bits	Field	Type	Reset	Description
31:0	HnENU	rw	0x00000000	Interrupt enable bits for vectors 95:64, routed to hart n.

33.8.4 HnENX (n = 0..4)

Offset 0x0C + 16n **Reset** 0x00000000 **Width** 32
Hart n interrupt routing register, vectors 124:96 (bits 28:0; upper bits read as 0).

Bits	Field	Type	Reset	Description
31:29	-	-	-	Reserved. Reads zero; write zero.
28:0	HnENX	rw	0x00000000	Interrupt enable bits for vectors 124:96, routed to hart n.

33.8.5 CLAIM: Interrupt claim/complete register (M19)

Offset 0x800 **Reset** 0x00000000 **Width** 32 **Address** 0x7800
Interrupt claim/complete register (M19). READ = claim: atomically returns the lowest pending interrupt vector that is enabled in the READING hart's routing row and not already under service, and marks it under service (masking it from every hart's meip). Returns 0xFFFFFFFF if nothing is pending for the reader; a handler seeing that value treats the interrupt as spurious and simply returns. WRITE = complete: write the claimed vector number to end its service; the vector becomes deliverable again (and pends immediately if its level is still asserted, so clear the level at the peripheral BEFORE completing). Completion is deliberately not qualified by owner, so a supervisor hart can complete on behalf of a hung hart (recovery); written values that are not valid vector numbers, including a stored 0xFFFFFFFF, are ignored.

Bits	Field	Type	Reset	Description
31:0	CLAIM	rw	0x00000000	Read: claimed vector number (0xFFFFFFFF = none pending for this hart). Write: vector number to complete.

33.8.6 PENDL

Offset 0x810 **Reset** 0x00000000 **Width** 32 **Address** 0x7810

Raw pending interrupt levels, vectors 31:0 (read-only; deglitched peripheral levels before enable/claim masking). Debug and polling aid.

Bits	Field	Type	Reset	Description
31:0	PENDL	r	0x00000000	Deglitched interrupt levels for vectors 31:0.

33.8.7 PENDM: Raw pending interrupt levels, vectors 63:32 (read-only)

Offset 0x814 **Reset** 0x00000000 **Width** 32 **Address** 0x7814

Bits	Field	Type	Reset	Description
31:0	PENDM	r	0x00000000	Deglitched interrupt levels for vectors 63:32.

33.8.8 PENDU: Raw pending interrupt levels, vectors 95:64 (read-only)

Offset 0x818 **Reset** 0x00000000 **Width** 32 **Address** 0x7818

Bits	Field	Type	Reset	Description
31:0	PENDU	r	0x00000000	Deglitched interrupt levels for vectors 95:64.

33.8.9 PENDX

Offset 0x81C **Reset** 0x00000000 **Width** 32 **Address** 0x781C

Raw pending interrupt levels, vectors 124:96 (bits 28:0, read-only; upper bits read as 0). Completes the raw-level readback for the fourth enable word (added with the peripheral-library program; before it, sources above 95 were routable and claimable but absent from the debug readback).

Bits	Field	Type	Reset	Description
31:29	-	-	-	Reserved. Reads zero; write zero.
28:0	PENDX	r	0x00000000	Deglitched interrupt levels for vectors 124:96.

33.8.10 INSVCL

Offset 0x820 **Reset** 0x00000000 **Width** 32 **Address** 0x7820

Under-service (claimed, not yet completed) flags, vectors 31:0 (read-only). Debug and recovery visibility: a stuck bit here means a hart claimed the vector and never completed it; any hart can recover by writing the vector number to CLAIM.

Bits	Field	Type	Reset	Description
31:0	INSVCL	r	0x00000000	Under-service flags for vectors 31:0.

33.8.11 INSVCM: Under-service flags, vectors 63:32 (read-only)

Offset 0x824 **Reset** 0x00000000 **Width** 32 **Address** 0x7824

Bits	Field	Type	Reset	Description
31:0	INSVCM	r	0x00000000	Under-service flags for vectors 63:32.

33.8.12 INSVCU: Under-service flags, vectors 95:64 (read-only)

Offset 0x828 **Reset** 0x00000000 **Width** 32 **Address** 0x7828

Bits	Field	Type	Reset	Description
31:0	INSVCU	r	0x00000000	Under-service flags for vectors 95:64.

33.8.13 INSVCX

Offset 0x82C **Reset** 0x00000000 **Width** 32 **Address** 0x782C

Under-service flags, vectors 124:96 (bits 28:0, read-only; upper bits read as 0). Completes the under-service readback for the fourth enable word.

Bits	Field	Type	Reset	Description
31:29	-	-	-	Reserved. Reads zero; write zero.
28:0	INSVCX	r	0x00000000	Under-service flags for vectors 124:96.

34 PINMUX

34.1 Overview

The pin multiplexer (PINMUX) owns the nineteen spare digital pads pmx0 through pmx18 and gives each one a register-selected function. A pad's PMXSELn nibble selects one entry of *that pad's* menu, so the same code means a different function on a different pad; the menus are tabulated below. The dedicated pads of every other block are untouched: this block adds pins, it does not re-route existing ones. The main features are listed below:

- One 4-bit select field per pad, eight pads per select word (PMXCFG0 through PMXCFG2), so a byte-lane store reconfigures exactly two pads and disturbs no other
- High impedance at reset: every select field resets to 0, which means disabled, with the pad output driver off and the pull resistor disabled, so an unconfigured pad floats rather than fighting whatever is wired to it
- A refused code is a disabled pad: a value above the pad's own menu length, or above PMXMENU, reads back as written but leaves the pad high impedance, so a pad can never be pointed at a function it does not carry
- Per-function output-enable policy in hardware, not in a register bit: a transmit function drives push-pull, a receive function never drives, and an open-drain function drives low and releases the pad for the external pull-up
- A relocated input really reaches its peripheral: the pad level is returned to the selected function, and a function no pad selects sees its idle level, which is high for a receive line or a released open-drain bus
- Pad observability independent of configuration (PMXIN0): the level of every pad, whatever that pad is selecting and whichever way it is driven
- A capability register (PMXCAP) publishing the pad count, the menu depth and the block revision, so firmware walks the block without a compiled-in copy of the configuration

34.2 Instances and signals

Table 34.1: PINMUX instances

Instance	Base address	Interrupt vector(s)	Pins (AF)
PINMUX	0x7C00	none	none

34.3 Functional description

34.3.1 The Per-Pad Menu

A pad's menu is a short ordered list of functions, at most PMXMENU of them, fixed at build time from the chip configuration. Select code 0 is disabled on every pad; codes 1 upward index that pad's list. A function

whose peripheral the configuration does not instantiate is removed from every menu and the remaining entries close up, so a pad's codes are always 1 through the length of its live menu and no code reaches a tie-off. Spreading one function across several pads' menus is normal and is what makes the block a multiplexer rather than a pin table: a function may be offered on many pads, and selecting it on two pads at once drives both, which is legal for an output and a wiring error for an input.

The select fields are nibbles, eight to a word, so PMXSEL0 through PMXSEL7 live in PMXCFG0, PMXSEL8 through PMXSEL15 in PMXCFG1, and so on. A select word always carries eight nibbles; at a pad count that is not a multiple of eight the top nibbles of the last word are writable and read back but control no pad, and PMXNPADS says where the real pads stop.

34.3.2 The Pad Menus

Code 1 is a GPIOx port-bit mirror on every pad, and codes 2 through 8 repeat one of four relocatable classes, so the nineteen pads between them reach every UART, SPI, I2C and QSPI pin function the configuration instantiates. The authority for both tables is platform/common/config/pinmux.json, which the generator reads directly; a function whose peripheral the configuration drops is removed and the pad's remaining codes close up, so a pad's codes are always 1 through the length of its live menu.

Pad	Code 1	Class	Pad side
pmx0	GPIO0_0	A	W
pmx1	GPIO1_0	B	W
pmx2	GPIO2_0	C	W
pmx3	GPIO3_0	D	W
pmx4	GPIO4_0	A	W
pmx5	GPIO5_0	B	W
pmx6	GPIO0_1	C	S
pmx7	GPIO1_1	D	S
pmx8	GPIO2_1	A	S
pmx9	GPIO3_1	B	S
pmx10	GPIO4_1	C	S
pmx11	GPIO5_1	D	S
pmx12	GPIO0_2	A	E
pmx13	GPIO1_2	B	N
pmx14	GPIO2_2	C	N
pmx15	GPIO3_2	D	N
pmx16	GPIO4_2	A	N
pmx17	GPIO5_2	B	N
pmx18	GPIO0_3	C	N

Table 34.2: Pin-mux pad menus: the code-1 mirror and the class carried by codes 2 and up.

34.3.3 What the Pad Does

Which way the pad is driven is a property of the selected function and not of any register bit, which is what stops a configuration mistake from shorting a bus. Three policies exist. A push-pull function drives the pad with its own output level under its own output enable, which covers a transmitter, a GPIO mirror and a bidirectional memory data line. A receive function never drives the pad at all, whatever it asks for;

Class	Code 2	Code 3	Code 4	Code 5	Code 6	Code 7	Code 8
A	UART0_TX	UART0_RX	UART1_TX	UART1_RX	I2C0_SDA	I2C0_SCL	QSPI0_SCK
B	SPI0_SCK	SPI0_MOSI	SPI0_MISO	SPI0_CS	I2C1_SDA	I2C1_SCL	QSPI0_CS
C	SPI1_SCK	SPI1_MOSI	SPI1_MISO	QSPI0_IO0	QSPI0_IO1	UART0_TX	UART0_RX
D	QSPI0_IO2	QSPI0_IO3	UART1_TX	UART1_RX	I2C0_SDA	I2C0_SCL	SPI0_MISO

Table 34.3: The four pin-mux classes: what codes 2 and up select on a pad of each class.

the pad stays an input and only the pull resistor request reaches it. An open-drain function drives the pad low on a logic 0 and releases it on a logic 1, and its own output enable is not consulted, because on a wired-AND bus the data is the enable. The pull resistor follows the selected function, either because the function asks for it or because its policy does; a disabled pad has it off.

34.3.4 The Input Path

The level of a pad selecting a function is returned to that function directly, with no synchronizer in the way: a serial input that crosses a clock domain is synchronized by the peripheral that receives it, and a second chain here would only add latency to a path that is already timed. A function sees the idle level of its slot on every pad that is not selecting it, and the per-function view is the combination of all of them, so an unselected pin-mux pad contributes nothing: a receive line reads high and a released open-drain bus reads high. Each peripheral keeps its existing dedicated-pad input path, and the pin mux's contribution joins it, so the peripheral reads whichever pad is actually presenting the signal.

`PMXIN0` is a separate, slower path meant for bring-up: it reports every pad through a two-flop synchronizer on the peripheral clock, so a level appears about two clock edges after it settles, a pulse shorter than one clock period is not captured, and nothing is captured while the clock is stopped. It is independent of the selection, so a pad is readable before it is configured and after it is disabled.

34.3.5 The GPIO Mirrors

Each pad's menu begins with a mirror of one `GPIOx` port bit. The mirror repeats that bit's output level, direction and resistor enable onto the pin-mux pad while the port keeps reading its own dedicated pad, so no port input is contended and a mirror costs nothing in the `GPIOx` block. A software-controlled extra pin is therefore one register write away, and `PMXIN0` is how its level is read back.

34.4 Interrupts and events

`PINMUX` has *no interrupt vector* and is not an event-fabric source or sink. A pin-mux pad raises no pin interrupt of its own: pin interrupts are a `GPIOx` feature of the port's own dedicated pads, and the `GPIOx` menu entries mirror a port bit's output rather than handing it a second input. A relocated peripheral keeps its own interrupts, which arrive on that peripheral's vector exactly as they do from its dedicated pad; an edge on a pin-mux pad that no peripheral is reading is visible only by polling `PMXIN0`.

34.5 Programming

Read `PMXCAP` first: `PMXNPADS` gives the pad count, `PMXMENU` the largest code any pad accepts, and `PMXREV` the block revision. Then write a pad's nibble with the code its menu row gives in the pad-menu tables

above. A byte-lane store to PMXCFGn covers two pads and leaves the other six nibbles of the word alone, which is the safe way to change one pad without a read-modify-write.

A select write takes effect on the bus cycle that lands it: the pad multiplexer is combinational and there is no commit, no busy bit and no handshake. The consequence is that a function in the middle of a transfer loses its pad immediately, so deselect only an idle function, and configure a pad before enabling the peripheral that is to use it. Setting a nibble to 0 returns the pad to high impedance with its pull disabled, which is also what a reset does to every pad.

Two configuration errors the hardware cannot refuse are worth stating. Selecting one input function on two pads at once combines both pads into that function's input, which reads as a stuck level if only one pad is driven; select an input function on exactly one pad. And a code above the pad's live menu length is accepted by the register and ignored by the pad, so firmware that writes a code without consulting the pad's own menu row gets a silently disabled pin rather than a fault.

34.6 Usage notes

In configurations that include it, PINMUX lives in the shared peripheral window behind the multi-core arbiter (see Section 5) in the top quarter of the interrupt-router page at 0x7C00, so any hart can use it. Nothing arbitrates between harts for a pad: two harts writing the same PMXCFGn word race as they would on any other register, and the per-pad nibble packing is what lets them own separate pads without a lock.

The pads are 1.0 V bidirectional pads on the digital I/O rail, the same cell the GPIOx ports use. Their positions in the pad ring are a property of the ring and not of this block; the block's own view is the pad index alone.

34.7 Registers

Register offsets in this chapter are relative to the instance base address in Table 34.1 (PINMUX 0x7C00); the generated header names each base PINMUX_BASE.

Table 34.4: PINMUX register summary

Offset	Name	Access	Reset	Description
0x00	PMXCAP	r	0x00000000	Pin-mux capability register, read-only
0x04	PMXIN0	r	0x00000000	
0x08	PMXCFG0	rw	0x00000000	
0x0C	PMXCFG1	rw	0x00000000	
0x10	PMXCFG2	rw	0x00000000	

34.7.1 PMXCAP: Pin-mux capability register, read-only

Offset 0x00 **Reset** 0x00000000 **Width** 32 **Address** 0x7C00
Pin-mux capability register, read-only. Publishes the three numbers firmware needs to walk the block without a compiled-in copy of the configuration: how many pmx pads the ring carries, how deep each pad's function menu is, and the block revision. Writes are ignored.

Bits	Field	Type	Reset	Description
31:16	-	-	-	Reserved. Reads zero; write zero.
15:12	REV	r	0b0000	Block revision. 1 is the first silicon revision of this pin mux.

Bits	Field	Type	Reset	Description
11:8	MENU	r	0b0000	Menu depth: the largest select code any pad accepts, and therefore the number of functions in a pad's menu. Codes above this value are reserved and are treated as DISABLED by the hardware, as is a code above the pad's own menu length.
7:0	NPADS	r	0x00	Pad count: how many pmx pads exist, pmx0 through pmx(PMXNPADS-1). Select nibbles at or above this index are writable and read back but control no pad.

34.7.2 PMXIN0

Offset 0x04 **Reset** 0x00000000 **Width** 32 **Address** 0x7C04

Pin-mux pad input register 0, read-only: the logic level of pads 32*0+31 down to 32*0, whatever function the pad is selecting and whichever way it is driven. The pad is sampled through the block's two-flop synchroniser on the peripheral clock, so a level appears here two clock edges after it settles on the pad, a pulse shorter than one clock period is not captured, and nothing is captured while the clock is stopped. Bits above the pad count read 0. Writes are ignored.

Bits	Field	Type	Reset	Description
31:0	PIN0	r	0x00000000	Pad input levels for pads 32*0+31 down to 32*0, synchronised to the peripheral clock. Bit y reads the level of pad 32*0+y; bits above the pad count read 0.

34.7.3 PMXCFG0

Offset 0x08 **Reset** 0x00000000 **Width** 32 **Address** 0x7C08

Pin-mux pad select register 0: one 4-bit select field per pad for pads 8*0+7 down to 8*0. A field at 0 (the reset value) leaves its pad DISABLED, meaning the pad output driver is off, the pull resistor is disabled and the pad is a floating input; a field at 1 through PMXMENU selects that entry of the PAD'S OWN function menu, which is per pad and is tabulated in the pin configuration table of the Pin Multiplexer chapter. A code above the pad's menu length, or above PMXMENU, is treated as DISABLED, so a pad can never be pointed at a function it does not carry. The function's own output-enable policy decides whether the pad then drives: a transmit function drives push-pull, a receive function never drives and only returns the pad level to the peripheral, and an I2C function drives low only and leaves the pad released for the external pull-up. Changing a select field takes effect on the bus cycle that writes it, so a function in mid-transfer sees its pad disappear: deselect only an idle function.

Bits	Field	Type	Reset	Description
31:28	SEL7	rw	0b0000	Function select for pad pmx7. 0 = disabled (high-Z, pull off), 1 to PMXMENU = that entry of the pad's menu.
27:24	SEL6	rw	0b0000	Function select for pad pmx6. 0 = disabled (high-Z, pull off), 1 to PMXMENU = that entry of the pad's menu.
23:20	SEL5	rw	0b0000	Function select for pad pmx5. 0 = disabled (high-Z, pull off), 1 to PMXMENU = that entry of the pad's menu.
19:16	SEL4	rw	0b0000	Function select for pad pmx4. 0 = disabled (high-Z, pull off), 1 to PMXMENU = that entry of the pad's menu.
15:12	SEL3	rw	0b0000	Function select for pad pmx3. 0 = disabled (high-Z, pull off), 1 to PMXMENU = that entry of the pad's menu.

Bits	Field	Type	Reset	Description
11:8	SEL2	rw	0b0000	Function select for pad pmx2. 0 = disabled (high-Z, pull off), 1 to PMXMENU = that entry of the pad's menu.
7:4	SEL1	rw	0b0000	Function select for pad pmx1. 0 = disabled (high-Z, pull off), 1 to PMXMENU = that entry of the pad's menu.
3:0	SEL0	rw	0b0000	Function select for pad pmx0. 0 = disabled (high-Z, pull off), 1 to PMXMENU = that entry of the pad's menu.

34.7.4 PMXCFG1

Offset 0x0C **Reset** 0x00000000 **Width** 32 **Address** 0x7C0C

Pin-mux pad select register 1: one 4-bit select field per pad for pads 8*1+7 down to 8*1. A field at 0 (the reset value) leaves its pad DISABLED, meaning the pad output driver is off, the pull resistor is disabled and the pad is a floating input; a field at 1 through PMXMENU selects that entry of the PAD'S OWN function menu, which is per pad and is tabulated in the pin configuration table of the Pin Multiplexer chapter. A code above the pad's menu length, or above PMXMENU, is treated as DISABLED, so a pad can never be pointed at a function it does not carry. The function's own output-enable policy decides whether the pad then drives: a transmit function drives push-pull, a receive function never drives and only returns the pad level to the peripheral, and an I2C function drives low only and leaves the pad released for the external pull-up. Changing a select field takes effect on the bus cycle that writes it, so a function in mid-transfer sees its pad disappear: deselect only an idle function.

Bits	Field	Type	Reset	Description
31:28	SEL15	rw	0b0000	Function select for pad pmx15. 0 = disabled (high-Z, pull off), 1 to PMXMENU = that entry of the pad's menu.
27:24	SEL14	rw	0b0000	Function select for pad pmx14. 0 = disabled (high-Z, pull off), 1 to PMXMENU = that entry of the pad's menu.
23:20	SEL13	rw	0b0000	Function select for pad pmx13. 0 = disabled (high-Z, pull off), 1 to PMXMENU = that entry of the pad's menu.
19:16	SEL12	rw	0b0000	Function select for pad pmx12. 0 = disabled (high-Z, pull off), 1 to PMXMENU = that entry of the pad's menu.
15:12	SEL11	rw	0b0000	Function select for pad pmx11. 0 = disabled (high-Z, pull off), 1 to PMXMENU = that entry of the pad's menu.
11:8	SEL10	rw	0b0000	Function select for pad pmx10. 0 = disabled (high-Z, pull off), 1 to PMXMENU = that entry of the pad's menu.
7:4	SEL9	rw	0b0000	Function select for pad pmx9. 0 = disabled (high-Z, pull off), 1 to PMXMENU = that entry of the pad's menu.
3:0	SEL8	rw	0b0000	Function select for pad pmx8. 0 = disabled (high-Z, pull off), 1 to PMXMENU = that entry of the pad's menu.

34.7.5 PMXCFG2

Offset 0x10 **Reset** 0x00000000 **Width** 32 **Address** 0x7C10

Pin-mux pad select register 2: one 4-bit select field per pad for pads 8*2+7 down to 8*2. A field at 0 (the reset value) leaves its pad DISABLED, meaning the pad output driver is off, the pull resistor is disabled and the pad is a floating input; a field at 1 through PMXMENU selects that entry of the PAD'S OWN function menu, which is per pad and is tabulated in the pin configuration table of the Pin Multiplexer chapter. A code above the pad's menu length, or above PMXMENU, is treated as DISABLED, so a pad can never be

pointed at a function it does not carry. The function's own output-enable policy decides whether the pad then drives: a transmit function drives push-pull, a receive function never drives and only returns the pad level to the peripheral, and an I2C function drives low only and leaves the pad released for the external pull-up. Changing a select field takes effect on the bus cycle that writes it, so a function in mid-transfer sees its pad disappear: deselect only an idle function.

Bits	Field	Type	Reset	Description
31:28	SEL23	rw	0b0000	Function select for pad pmx23. 0 = disabled (high-Z, pull off), 1 to PMXMENU = that entry of the pad's menu.
27:24	SEL22	rw	0b0000	Function select for pad pmx22. 0 = disabled (high-Z, pull off), 1 to PMXMENU = that entry of the pad's menu.
23:20	SEL21	rw	0b0000	Function select for pad pmx21. 0 = disabled (high-Z, pull off), 1 to PMXMENU = that entry of the pad's menu.
19:16	SEL20	rw	0b0000	Function select for pad pmx20. 0 = disabled (high-Z, pull off), 1 to PMXMENU = that entry of the pad's menu.
15:12	SEL19	rw	0b0000	Function select for pad pmx19. 0 = disabled (high-Z, pull off), 1 to PMXMENU = that entry of the pad's menu.
11:8	SEL18	rw	0b0000	Function select for pad pmx18. 0 = disabled (high-Z, pull off), 1 to PMXMENU = that entry of the pad's menu.
7:4	SEL17	rw	0b0000	Function select for pad pmx17. 0 = disabled (high-Z, pull off), 1 to PMXMENU = that entry of the pad's menu.
3:0	SEL16	rw	0b0000	Function select for pad pmx16. 0 = disabled (high-Z, pull off), 1 to PMXMENU = that entry of the pad's menu.

Part IV Analog front end

Part V Boot ROM and software

35 ROM monitor

When the chip leaves reset with the P0.7 strap low (Section 6.2), hart 0 runs the ROM monitor: an interactive Forth interpreter stored in the boot ROM that communicates over UART0 at 115,200 baud, 8 data bits, no parity, one stop bit. The monitor is a bring-up console and the flash programming path of Chapter 36; it is not a development environment.

On start-up the monitor prints a one-line banner ending in rv4th-rom!, an empty line, and the prompt >, and is then ready to receive characters. The leading word of the banner is the ASIC_NAME string compiled into the ROM source and does not name this chip.

35.1 Interpreter model

Forth is a stack language. Every value is a 32-bit signed integer on one data stack, the top of which is called TOS. Typing a number pushes it: a plain decimal integer, or a hexadecimal value with the prefix 0x in either case and without leading zeros (0xA4). Typing a word (a function name) executes it; a word pops its arguments from the stack and pushes its results. Nothing is executed until a line feed is received, at which point the whole line is executed left to right. An input the interpreter does not recognise as a number or a word prints ?. Stack underflow is not an error: a pop from an empty stack yields 0.

By default the monitor echoes every received character back to the host, which suits a terminal emulator; 0 echo switches the echo off for a programmatic host.

35.2 Word list

The words below are listed with their stack effect in the usual Forth notation: the items left of the double dash are the arguments as they are typed (the rightmost is the TOS when the word runs), and the items right of it are the results, rightmost on top. Arguments are popped before the word executes and results are pushed after it finishes.

35.2.1 Memory Access Functions

- `@` `( addr -- )`

  Read directly from the memory address and push the value to the TOS

  Arguments: (in order from TOS down, reverse text order):

  1. addr: (TOS) The address to read from (must be aligned to a 4-byte (32-bit) boundary)

  Return values: none

- `!` `( x addr -- )`

  Write directly to memory address

  Arguments: (in order from TOS down, reverse text order):

  1. addr: (TOS) The address to write to (must be aligned to a 4-byte (32-bit) boundary)
  2. x: The value to write to the memory address

  Return values: none

- `+!` `( x addr -- )`

  Add to value, direct memory access (*addr += x)

  Arguments: (in order from TOS down, reverse text order):

  1. `addr`: (TOS) The address to write to (must be aligned to a 4-byte (32-bit) boundary)
  2. `x`: The value to add to the value at the memory address

  Return values: none

- `sbi` `( bitnum addr -- )`

  Set bit, direct memory access (*addr |= (1 « bitnum))

  Arguments: (in order from TOS down, reverse text order):

  1. `addr`: (TOS) The address to write to (must be aligned to a 4-byte (32-bit) boundary)
  2. `bitnum`: The bit number

  Return values: none

- `cbi` `( bitnum addr -- )`

  Clear bit, direct memory access (*addr &= ~(1 « bitnum))

  Arguments: (in order from TOS down, reverse text order):

  1. `addr`: (TOS) The address to write to (must be aligned to a 4-byte (32-bit) boundary)
  2. `bitnum`: The bit number

  Return values: none

- `mask` `( bitmask x addr -- )`

  Change only masked bits, direct memory access (*addr = (*addr & ~bitmask) | (x & bitmask))

  Arguments: (in order from TOS down, reverse text order):

  1. `addr`: (TOS) The address to write to (must be aligned to a 4-byte (32-bit) boundary)
  2. `x`: The value to write (only masked bits will be written)
  3. `bitmask`: The bit mask. Only bits with 1s can be changed

  Return values: none

- `mr` `( mode length start_address -- )`

  Reads a block of memory and sends it over UART0, followed by a CRC16/CDMA2000 of the bytes sent. Bytes go out in increasing address order, so a 32-bit word arrives least-significant byte first. Byte granular: no alignment is required.

  Arguments: (in order from TOS down, reverse text order):

  1. `start_address`: (TOS) The address of the first byte to read
  2. `length`: The number of bytes to read
  3. `mode`: Nonzero sends a raw binary payload and the CRC as two bytes, least significant first; zero sends an ASCII hex string and the CRC as four hex characters

Return values: none

- `mw` `( bin_payload length start_address -- )`

  Receives a block of data over UART0 and writes it to memory. The monitor sends a single dollar sign when it is ready for the payload, and sends the CRC16/CDMA2000 of what it received as four hex characters when the last byte has been stored. Byte granular: no alignment is required.

  Arguments: (in order from TOS down, reverse text order):

  1. `start_address`: (TOS) The address the first received byte is stored at
  2. `length`: The number of bytes to write
  3. `bin_payload`: Nonzero receives a raw binary payload of length bytes; zero receives an ASCII hex string of exactly twice that many characters, each byte most significant nibble first

  Return values: none

- `ms` `( set_byte length start_address -- )`

  Fills a block of memory with one byte value. Byte granular, and prints nothing.

  Arguments: (in order from TOS down, reverse text order):

  1. `start_address`: (TOS) The address of the first byte to set
  2. `length`: The number of bytes to set
  3. `set_byte`: The value written to every byte; only the low 8 bits are used

  Return values: none

35.2.2 Print and Input Functions

- `.` `( x -- )`

  Prints the TOS as an integer

  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) The value to print

  Return values: none

- `h.` `( x -- )`

  Prints the TOS as a hex string

  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) The value to print

  Return values: none

- `echo` `( bool -- )`

  Change the echo state, which controls whether the Forth interpreter repeats all of its received characters

  Arguments: (in order from TOS down, reverse text order):

1. `bool`: (TOS) If 0, does not echo. Otherwise, it does

Return values: none

- `emit ( x -- )`
  Prints the char at the TOS (integers are mapped to their ASCII counterparts)
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) The ASCII value of the char to print

  Return values: none
- `key ( -- char )`
  Waits for a char over UART and pushes the ASCII value of the char to the TOS
  Arguments: none
  Return values: (in order from TOS down, reverse text order):
  1. `char`: (TOS) The char that was received over UART
- `list ( -- )`
  Prints all currently available functions
  Arguments: none
  Return values: none

35.2.3 Arithmetic Functions

- `+ ( y x -- ret )`
  Adds two numbers (y + x)
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value
- `- ( y x -- ret )`
  Subtracts two numbers (y - x)
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value

- `*` `( y x -- retprodhi retprodlo )`
  Multiplies two numbers (y * x)
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `retprodlo`: (TOS) Lower 32 bits of the return product
  2. `retprodhi`: Upper 32 bits of the return product
- `/%` `( y x -- retdiv retrem )`
  Divides two numbers with remainder (y / x and y % x)
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `retrem`: (TOS) The remainder (or modulo) result
  2. `retdiv`: The integer division result
- `neg` `( x -- ret )`
  Returns the negative (twos complement) of x
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value
- `abs` `( x -- ret )`
  Absolute value (|x|)
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value

35.2.4 Bitwise Logic Functions

- `~` `( x -- ret )`
  Bitwise complement/inversion (~x)
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value
- `|` `( y x -- ret )`
  Bitwise OR (y | x)
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value
- `&` `( y x -- ret )`
  Bitwise AND (y & x)
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value
- `^` `( y x -- ret )`
  Bitwise XOR (y ^ x)
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value
- `*2` `( x -- ret )`
  Shift left 1 bit (x « 1)
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer

Return values: (in order from TOS down, reverse text order):

1. `ret`: (TOS) Return value

- `/2 ( x -- ret )`
  Shift right 1 bit (x » 1)
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value
- `sll ( x bits -- ret )`
  Shift left logical (x « bits)
  Arguments: (in order from TOS down, reverse text order):
  1. `bits`: (TOS) Signed integer
  2. `x`: Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value
- `srl ( x bits -- ret )`
  Shift right logical (x » bits), no sign extension
  Arguments: (in order from TOS down, reverse text order):
  1. `bits`: (TOS) Signed integer
  2. `x`: Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value

35.2.5 Comparison Functions

- `> ( y x -- ret )`
  Greater than. Returns 1 if y > x, returns 0 otherwise.
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) 1 if true, 0 if false

- `<` `( y x -- ret )`

  Less than. Returns 1 if y < x, returns 0 otherwise.

  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):

  1. `ret`: (TOS) 1 if true, 0 if false

- `==` `( y x -- ret )`

  Equals. Returns 1 if y == x, returns 0 otherwise.

  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):

  1. `ret`: (TOS) 1 if true, 0 if false

35.2.6 Boolean Functions

- `not` `( x -- ret )`

  Logical not (!x). Returns 1 if x is equal to 0, returns 0 otherwise

  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) Signed integer

  Return values: (in order from TOS down, reverse text order):

  1. `ret`: (TOS) Return value

- `and` `( y x -- ret )`

  Logical and (y && x). Returns 1 if y and x are both true (nonzero), returns 0 otherwise

  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):

  1. `ret`: (TOS) Return value

- `or` `( y x -- ret )`

  Logical or (y || x). Returns 1 if y or x are true (nonzero), returns 0 otherwise

  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) Signed integer

2. `y`: Signed integer

Return values: (in order from TOS down, reverse text order):

1. `ret`: (TOS) Return value

35.2.7 Stack Manipulation Functions

- `dup ( x -- x x )`
  Duplicates the TOS (pops the TOS and then pushes the value to the TOS twice)
  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) Value to duplicate

  Return values: (in order from TOS down, reverse text order):

  1. `x`: (TOS) The duplicated value
  2. `x`: The original TOS

- `drop ( x -- )`
  Pops the TOS and sets it as the internal Forth `i` value. Usually used to delete the TOS.
  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) The value that is dropped

  Return values: none

- `swap ( y x -- x y )`
  Swaps the TOS with the value below the TOS
  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) Old TOS
  2. `y`: Old value below the TOS

  Return values: (in order from TOS down, reverse text order):

  1. `y`: (TOS) Old value below the TOS
  2. `x`: Old TOS

- `over ( y x -- y x y )`
  Copies the value below the TOS and pushes it to the TOS
  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) Signed Integer
  2. `y`: Signed Integer

  Return values: (in order from TOS down, reverse text order):

  1. `y`: (TOS) Signed Integer
  2. `x`: Signed Integer

3. `y`: Signed Integer

- `tuck` `( y x -- x y x )`

  Copies the TOS and inserts it two after the TOS

  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) Signed Integer
  2. `y`: Signed Integer

  Return values: (in order from TOS down, reverse text order):

  1. `x`: (TOS) Signed Integer
  2. `y`: Signed Integer
  3. `x`: Signed Integer

- `roll` `( n -- )`

  Removes value at index n of the stack and pushes it to the TOS (zero indexed)

  Arguments: (in order from TOS down, reverse text order):

  1. `n`: (TOS) Stack index (zero indexed)

  Return values: none

- `pick` `( n -- )`

  Copies value at index n of the stack and pushes it to the TOS (zero indexed)

  Arguments: (in order from TOS down, reverse text order):

  1. `n`: (TOS) Stack index (zero indexed)

  Return values: none

- `depth` `( -- ret )`

  Gets size of stack and pushes it to the top of the stack (making the new stack size the value of TOS + 1)

  Arguments: none

  Return values: (in order from TOS down, reverse text order):

  1. `ret`: (TOS) Return value

35.2.8 SPI Flash R/W and Programming Functions

- `fr` `( bin length addr -- )`

  Reads data from the SPI flash.

  Arguments: (in order from TOS down, reverse text order):

  1. `addr`: (TOS) The address in the SPI flash to begin reading from
  2. `length`: The number of bytes to read
  3. `bin`: If true (nonzero), transmits binary data, otherwise transmits ASCII hex string

Return values: none

- `fe ( conf addr -- eraseSuccessful )`

  Erases a 256-byte page from the SPI flash memory. The start address must be 256-byte aligned.

  Arguments: (in order from TOS down, reverse text order):

  1. `addr`: (TOS) The start address of the page to erase in the SPI flash. Must be 256-byte aligned.
  2. `conf`: If equal to 123, erases the page and returns 1, otherwise does not erase and returns 0

  Return values: (in order from TOS down, reverse text order):

  1. `eraseSuccessful`: (TOS) Nonzero if successful, zero if failed

- `fw ( bin addr -- )`

  Writes data to the SPI flash. Must write exactly 256 bytes at a time. The start address must be 256-byte aligned.

  Arguments: (in order from TOS down, reverse text order):

  1. `addr`: (TOS) The address in the SPI flash to begin writing to. Must be 256-byte aligned.
  2. `bin`: If true (nonzero), expects to receive binary data, otherwise expects to receive an ASCII hex string

  Return values: none

35.2.9 Miscellaneous Functions

- `rst ( -- )`

  Performs a soft reset (turns on all memory cells and jumps to address 0 in the ROM, does not reset clocks or hardware)

  Arguments: none

  Return values: none

- `clk ( meastime clksel -- freq )`

  Measures selected clock frequency

  Arguments: (in order from TOS down, reverse text order):

  1. `clksel`: (TOS) Clock select. 0 = SMCLK; 1 = MCLK; 2 = LFXT; 3 = HFXT
  2. `meastime`: Measurement time. 0 = 1 s; 1 = 0.5 s; 2 = 0.25 s; 3 = 0.125 s

  Return values: (in order from TOS down, reverse text order):

  1. `freq`: (TOS) Selected clock frequency in Hz

- `min ( y x -- ret )`

  Returns minimum value of y and x

  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) Signed integer

2. `y`: Signed integer

Return values: (in order from TOS down, reverse text order):

1. `ret`: (TOS) Return value

- `max ( y x -- ret )`

  Returns maximum value of y and x

  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):

  1. `ret`: (TOS) Return value

- `swpb ( x -- ret )`

  Swap bits 15 downto 8 with bits 7 downto 0 of TOS, also causes bits 31 downto 16 to all be 0

  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) Signed integer

  Return values: (in order from TOS down, reverse text order):

  1. `ret`: (TOS) Return value

- `swphw ( x -- ret )`

  Swap upper 16 bits with lower 16 bits of TOS

  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) Signed integer

  Return values: (in order from TOS down, reverse text order):

  1. `ret`: (TOS) Return value

- `call0 ( func -- ret )`

  Calls a compiled routine at the address on the TOS with no arguments and pushes its 32-bit return value. The routine is entered with the standard calling convention, so any address that is not a valid function entry point will crash the monitor.

  Arguments: (in order from TOS down, reverse text order):

  1. `func`: (TOS) The address of the routine to call

  Return values: (in order from TOS down, reverse text order):

  1. `ret`: (TOS) The value the routine returned

- `call1` `( a func -- ret )`

  Calls a compiled routine at the address on the TOS with one argument and pushes its 32-bit return value. The routine is entered with the standard calling convention, so any address that is not a valid function entry point will crash the monitor.

  Arguments: (in order from TOS down, reverse text order):

  1. `func`: (TOS) The address of the routine to call
  2. `a`: Argument 1, passed to the function

  Return values: (in order from TOS down, reverse text order):

  1. `ret`: (TOS) The value the routine returned

- `call2` `( a b func -- ret )`

  Calls a compiled routine at the address on the TOS with two arguments and pushes its 32-bit return value. The routine is entered with the standard calling convention, so any address that is not a valid function entry point will crash the monitor.

  Arguments: (in order from TOS down, reverse text order):

  1. `func`: (TOS) The address of the routine to call
  2. `b`: Argument 2, passed to the function
  3. `a`: Argument 1, passed to the function

  Return values: (in order from TOS down, reverse text order):

  1. `ret`: (TOS) The value the routine returned

- `call3` `( a b c func -- ret )`

  Calls a compiled routine at the address on the TOS with three arguments and pushes its 32-bit return value. The routine is entered with the standard calling convention, so any address that is not a valid function entry point will crash the monitor.

  Arguments: (in order from TOS down, reverse text order):

  1. `func`: (TOS) The address of the routine to call
  2. `c`: Argument 3, passed to the function
  3. `b`: Argument 2, passed to the function
  4. `a`: Argument 1, passed to the function

  Return values: (in order from TOS down, reverse text order):

  1. `ret`: (TOS) The value the routine returned

- `call4` `( a b c d func -- ret )`

  Calls a compiled routine at the address on the TOS with four arguments and pushes its 32-bit return value. The routine is entered with the standard calling convention, so any address that is not a valid function entry point will crash the monitor.

  Arguments: (in order from TOS down, reverse text order):

1. `func`: (TOS) The address of the routine to call
2. `d`: Argument 4, passed to the function
3. `c`: Argument 3, passed to the function
4. `b`: Argument 2, passed to the function
5. `a`: Argument 1, passed to the function

Return values: (in order from TOS down, reverse text order):

1. `ret`: (TOS) The value the routine returned

35.3 Interpreter limits

The interpreter keeps all of its mutable state in hart 0's TCM, of which the interrupt vector table takes the first 125 words and the top 1 KiB is reserved for the C stack; the interpreter's arrays are sized to what is left. The capacities are fixed by the ROM and are not configurable at run time:

- Data (math) stack: 128 cells of 32 bits.
- Return (address) stack: 64 cells of 32 bits. Each active `do loop` uses three of them, so loop nesting is bounded well below that.
- Dictionary: 64 user-defined words, 512 bytes total of word names including the separating space after each, and 1024 opcodes of compiled body shared by all of them.
- Input line buffer: 128 characters.
- Word buffer: 64 characters, so a single word name is at most 63 characters.

Only three of these are checked at run time: math stack underflow, input line overflow and word buffer overflow. Exceeding the dictionary capacities or underflowing the return stack is not detected and corrupts interpreter state. A program that outgrows these limits belongs in the SPI flash boot mode of Chapter 36.

Caution: The monitor's receive path is a polled loop with no flow control, and it echoes each character as it takes it. A host that delivers a line back to back at 115,200 baud can lose characters; a host that leaves one character time of idle line between frames does not.

35.4 Defining words

A new word is defined with `: name body ;`, where body is any sequence of numbers and words. The new word may be used as soon as the semicolon has been read, even later on the same line. A user-defined word can only push and pop the stack and call other words. For example, a word that increments the TOS:

```
: inc 1 + ;
```

35.5 Conditionals

`if` and `then` form a conditional and may only be used inside a word definition. `if` pops the TOS and executes the words up to the matching `then` only when the popped value is nonzero. An `else` clause is optional:

```
cond if wordsIfTrue else wordsIfFalse then
```

For example, a word that prints `cool` when the TOS equals 5:

```
: cool? 5 == if 108 111 111 99 emit emit emit emit then ;
```

35.6 Loops

Two loop forms exist, and both may only be used inside a word definition. `begin ... until` executes its body, then pops the TOS at `until` and repeats the body while the popped value is zero. `stop start do ... loop` is a counted loop: it runs its body with the index taking the values `start` to `stop - 1`. Inside a counted loop the word `i` pushes the index of the innermost loop, and `j` and `k` the indices of the two enclosing loops. For example, a word that prints the numbers 0 to 9:

```
: looptest 10 0 do i . loop ;
```

36 Boot image format

When the P0.7 strap is high at reset (Section 6.2), hart 0's boot loader reads a boot image from the SPI flash on SPI0, copies its segments into RAM and enters it. The image is a stream of 32-bit words beginning at flash offset 0x000000, read with one continuous high-frequency read command. Table 36.1 is the authoritative description of the stream.

Table 36.1: Boot image stream, as read by the boot ROM from flash offset 0x000000.

Command word	Following words	Effect
0x10ADBEEF	START, END, then (END − START)/4 data words	Load segment: the data words are stored to RAM at START, START + 4, …, END − 4. END is exclusive.
0xDEADBEEF	START, END, VALUE	Fill segment: every word from START to END − 4 is set to VALUE.
0xCAFEBABE	none	Execute: the loader ends the flash read, restores the SPI0 pins, selects the boot clocks, clears the registers and enters the image.

Every segment must lie inside the load window: 0x8000 ≤ START < END ≤ 0x1FFFF + 1, which spans the private TCM, the NPU staging RAM and the shared RAM. A segment outside the window, or any other command word, traps the boot loader. The entry address is the word at 0x10640 in shared RAM if the image has loaded a nonzero value there, and 0x8200 otherwise; the loader zeroes the shared-RAM mailbox region before reading the flash, so an image that publishes nothing enters at 0x8200.

Words are transferred most-significant byte first: the flash byte at offset $4k$ is bits 31:24 of word k. A tool that writes the flash from a little-endian memory image must therefore reverse the byte order within every word.

A standard image, as produced by the build scripts, consists of a fill or load segment that zeroes the vector table region 0x8000 to 0x8200, one load segment per contiguous run of nonzero program words (a run of eight or more zero words separates segments, and each segment is padded by two trailing zero words), and the execute word. The image's first instruction is at 0x8200 in the TCM unless it publishes an entry address; an image larger than the TCM executes from the shared RAM and publishes its entry there. The vector table itself is written by the image's startup code.

36.1 Programming the flash from the ROM monitor

The ROM monitor (Chapter 35) writes, erases and reads the flash in 256-byte pages, and tools/flash_programmer/forthpr in the repository (Appendix A) drives it to program a whole image from an Intel hex file. The image occupies flash pages from offset 0x000000 upward, as many as its length requires.

bin addr fw
Writes one page at flash address addr, which must be a multiple of 256. If bin is nonzero the monitor expects 256 bytes of binary data; if zero, a 512-character ASCII hex string. After the line feed the monitor prints $ when it is ready for the data. When the data has arrived it prints the CRC of the page as four hex characters (CRC16/CDMA2000: polynomial 0xC857, initial value 0xFFFF, no final XOR, no reflection). The host answers with a single Y to commit the write; any other character aborts it and leaves the flash unchanged.

`conf addr fe`
Erases the page at `addr` to `0xFF` when `conf` equals 123 and pushes 1; otherwise does nothing and pushes 0. Erasing is faster than writing and is the right way to handle a blank page.

`fr`
Reads the flash back; see Section 35.2.8.

Note: The flash device specifies a page program time of up to 25 ms (typically 10 ms) and a page erase time of up to 25 ms (typically 6 ms). The handshake of `fw` is long enough that no extra delay is needed between writes; consecutive erases at a high baud rate may need one.

After the image is written, the chip runs it after a reset with the `P0.7` strap high. The `rst` word restarts hart 0 at the boot ROM without a pin reset, so a host that controls the strap can switch modes without toggling `RESETN`.

A Getting started and software development

The RTL of this chip, its boot ROM, its firmware applications, the generated headers and this manual are one repository, `https://github.com/maxxseminario/VestaRV`. Every command in this appendix is run from the root of a clone of it and needs nothing installed first: the build provisions its own Python, its host and RISC-V compilers, its VHDL simulator and its TEX distribution. Synthesis, place-and-route and physical signoff run on licensed tools that are not part of the repository and are not described here.

A.1 First run

1. Clone the repository.

   ```
   git clone https://github.com/maxxseminario/VestaRV.git
   cd VestaRV
   ```

2. Fetch the build driver. The script writes one launcher, `tools/bin/bazel`, and touches nothing outside the clone.

   ```
   sh tools/get_bazel.sh
   ```

3. Run the gate set.

   ```
   tools/bin/bazel test //...
   ```

   That is 337 tests: the generated RTL against the tracked RTL, the register descriptions against both the VHDL and the C headers, every firmware image against its tracked golden, and a license-free simulation regression of the instruction set. The first run fetches about 1 GB of toolchains and builds the VHDL simulator from source, 10 to 20 minutes; every later run is served from the cache.

4. Emit the artifacts of this configuration.

   ```
   make -C platform/common generate
   ```

   The run writes the RTL to `platform/common/out/hdl/`, the C headers to `out/software/include/`, the linker script fragments to `out/linker-scripts/`, the pad ring to `out/config/PadRing.json` and the sources of this manual to `platform/common/latex/TRM/`. `CONFIG=config/<name>.json` selects another configuration. The hermetic equivalent, which writes only into `bazel-bin`, is `tools/bin/bazel build //platform/common:chip_artifacts_<name>`, one target per tracked configuration.

Table A.1 lists what the build fetches. None of it is installed on the host, and none of it has to be on `PATH`.

Table A.1: Tools the build provisions on first use.

Tool	Version and origin
RISC-V C compiler	`riscv-none-elf-gcc` 13.2.0, fetched and hash-pinned in `MODULE.bazel`. The mask-ROM image has to reproduce byte for byte, so this compiler is never substituted.
Build driver	Bazel 9.2.0, run through the `bazelisk` launcher that `tools/get_bazel.sh` installs.

Tool	Version and origin
Python	3.11, provisioned by `rules_python` for every generator and test.
Host C/C++ compiler	Hermetic `zig`-based toolchain, so a host with no system compiler still builds.
VHDL simulator	GHDL 6.0.0, built from source with the mcode backend.
TEX	TeX Live 2022, for this manual.

A.2 Building a firmware image

Every application under `software/` is built by one macro, `myshkin_app` in `toolchains/riscv/defs.bzl`: the sources are compiled `-march=rv32ima -mabi=ilp32 -O2`, linked against `tools/build/linker-scripts/MCU.ld` and the generated memory-map fragments, and imaged. One application builds the whole artifact set of Table A.2.

```
tools/bin/bazel build //software/blinky:blinky_flashed_rcf
tools/bin/bazel build //software/...
tools/bin/bazel build //software/bootrom_mp:rom_rcf
```

Table A.2: Artifacts of one firmware target, for a package `//software/NAME`.

Target	What it is
`:NAME_elf`	Linked ELF with debug symbols, and the map file beside it. The file a debugger loads.
`:NAME_bin`	Raw binary of the loaded image.
`:NAME_hex`	Intel hex image, the input to the flash programmer of Section A.5.
`:NAME_dump`	Disassembly listing.
`:NAME_rcf`	One 32-bit word of the RAM image per line, for the VHDL benches.
`:NAME_flashed_rcf`	The same image as the boot-image stream of Chapter 36, command words and all, ready for the SPI flash.
`:NAME_flashed_rcf_test`	Gate: the built image is byte-identical to the tracked golden under `NAME/testdata/`. A firmware change is committed with its regenerated golden, so the diff shows which bytes moved.

The applications in the repository are `blinky` (`TIMER0` drives `P3.0` at 1 Hz with no processor in the loop), `gpiotoggle` (the same pad toggled from a CPU loop), `slowblink` (a CPU delay loop at a human-visible rate), `looptest` (an infinite loop that traps on nothing, which is how a load that worked is told from one that did not) and `traptest` (an all-zeros word, which traps). `//software/bootrom_mp:rom_rcf` is the mask-ROM content of the fabricated part; `:rom_rcf_reproducibility_test` is the gate that it cannot drift.

A.3 A program of your own

1. Create the package and take the startup file from an existing one.

   ```
   mkdir -p software/myapp/src
   cp software/blinky/src/start.S software/myapp/src/start.S
   ```

   `start.S` must be the first source in link order: the linker script places its first instruction at `0x8200`, the address the boot loader enters (Chapter 36). It sets the stack pointer, clears `.bss` and calls `main`.

2. Write `src/main.c` against the generated headers. `software/include/regs/` holds one header per peripheral block, emitted from the SystemRDL descriptions in `hdl/common/regs/rdl/`, and `castalia_regs.h` includes all of them and adds, for each instance of this build, a base address `<INST>_BASE_ADDR`, a typed pointer `<INST>_REGS` and, where the instance owns one, an interrupt vector `<INST>_IRQ_VECTOR`. Within a block the names are `<REG>_T__<FIELD>_bm` for the mask, `_bp` for the position, `_bw` for the width, `_reset` for the reset value, and `<REG>_T__<FIELD>__<VALUE>` for an enumerated field value. Every number in Part III is in these headers under those names.

```
#include "castalia_regs.h"

#define HFXT_HZ   24000000u
#define BAUD      115200u

static void uart0_putc(char c)
{
    while (UART0_REGS->UARTxSR & UARTXSR_T__TXBF_bm) { }
    UART0_REGS->UARTxTX = (uint8_t)c;
}

int main(void)
{
    SYSTEM_REGS->SYSCLKCR =
        (SYSCLKCR_T__MCLKSEL__HFXT  << SYSCLKCR_T__MCLKSEL_bp) |
        (SYSCLKCR_T__SMCLKSEL__HFXT << SYSCLKCR_T__SMCLKSEL_bp);

    UART0_REGS->UARTxBR = (HFXT_HZ / (16u * BAUD)) - 1u;
    UART0_REGS->UARTxCR = UARTXCR_T__UEN_bm;

    GPIO2_REGS->PxSEL = 0x00;
    GPIO2_REGS->PxDIR = 0x01;

    for (;;) {
        uart0_putc('.');
        GPIO2_REGS->PxOUTT = 0x01;
    }
    return 0;
}
```

   The block struct carries the template spelling of the register, not the instance spelling: the instance is in the pointer, so `UART0_REGS->UARTxCR` is `UART0`'s control register. The boot ROM leaves the clock system in a state of its own (Section 6.2), which is why the first write selects the SMCLK source before the baud divider is programmed. The `TX0` pad is already at its alternate function after reset (Table 3.2), so the program configures no pin for the UART and only the GPIO pad it drives itself.

3. Add `software/myapp/BUILD.bazel`. The `hdrs` and `copts` lines are what put the generated headers inside the sandbox; without them the build fails rather than silently compiling against a stale copy.

```
load("//toolchains/riscv:defs.bzl", "myshkin_app")

package(default_visibility = ["//visibility:public"])
```

```
myshkin_app(
    name = "myapp",
    copts = ["-Isoftware/include/regs"],
    hdrs = ["//software/include:regs_headers"],
)
```

4. Build the image.

   ```
   tools/bin/bazel build //software/myapp:myapp_flashed_rcf
   ```

5. Lock it against drift by adding a `firmware_image_test` beside the application, as `software/blinky/BUILD.bazel` does, and committing the golden it grades.

`myshkin_app` compiles for `rv32ima`, which harts 1 to 4 do not implement: this build's tiles are `rv32iac` (Section 7.1), and the multiply instructions trap there. An image that runs on a tile is built with `rv32_firmware` directly and `march = "rv32iac"`, which is the same macro one level down with every flag exposed.

A.4 Simulating the image before silicon

The simulation tier needs no license: the simulator is built from source by the same command that runs the test. Table A.3 lists the targets.

Table A.3: Simulation targets, all license-free.

Target	What it proves
`//opensource_sim:isa_regression`	The whole nine-suite instruction-set regression on the core. Run it after any change to the core or to the extensions a program compiles for.
`//opensource_sim/isa:rv32ui-p-add`	One image on the bare-core bench, which is the per-image debug loop. That bench is one hart, one flat RAM at `0x8000` and no peripherals, so an image of your own runs there only if it touches no peripheral window; the rule to copy is the `ghdl_test` in `opensource_sim/isa/BUILD.bazel`, with `image` naming your `:NAME_rcf`.
`//opensource_sim/asic_default:asic_default_elaborate`	The generated MCU binds and its memory-size assertions hold.
`//opensource_sim/asic_default:asic_default_boot`	Reset is released on the real mask-ROM image and the banner the ROM monitor prints on `UART0` is graded. It proves the part fetched through the arbiter, ran C out of its TCM and drove a pad.
`//opensource_sim/rv4th:rv4th_forth`	The ROM monitor answers its own bench over `UART0`, word by word.

A.5 Loading a program onto a chip

Both host paths talk to the ROM monitor (Chapter 35) over `UART0` at 115,200 baud, 8N1, with the `P0.7` strap low at reset (Section 6.2). The scripts are in `tools/flash_programmer/` and need `pyserial`; `forthprog.py` also needs `IntelHex` and `progressbar2`.

Into the SPI flash, which survives a power cycle. Program the flash through the monitor, then strap `P0.7` high and reset. The boot loader reads the image from flash offset `0x000000`, copies its segments into RAM and enters it (Chapter 36). The monitor's `rst` word restarts hart 0 without a pin reset, so a host that controls the strap needs no power cycle.

```
python3 tools/flash_programmer/forthprog.py -p /dev/ttyUSB0 -v \
    bazel-bin/software/myapp/myapp.hex
```

Straight into RAM, which does not touch the flash. The image is written word by word through the monitor's memory-write word and entered with `call0`. This is the path when the flash is unpopulated or its boot path is under suspicion, and it is the faster edit-and-run loop. The image's basename is padded with leading `x` to 22 characters, which is a fixed-length path contract of the VHDL benches and not a typing error.

```
python3 tools/flash_programmer/forthram.py -p /dev/ttyUSB0 --verify \
    bazel-bin/software/myapp/xxxxxxxxxxxxxmyapp.rcf
```

`rcf2forth.py` turns either image into a plain list of monitor commands with no hardware attached, `forthrun.py` streams such a list, and `tools/debug/rv4th_terminal.py` is an interactive terminal on the same port. The page-write handshake and the flash words those scripts drive are described in Section 36.1.

A.6 Debugging on the part

The debug port is a 5-pin JTAG TAP in front of a RISC-V Debug Module; Chapter 11 is the reference and Section 11.6 is the list of behaviours that will surprise a debugger. The part has been brought up against unmodified OpenOCD 0.12.0 and `riscv-none-elf-gdb` 13.2, which are host tools and are not provisioned by the build.

1. Start OpenOCD on `tools/debug/riscv-tests-debug/targets/VestaRV/ vesta_castalia.cfg`. It declares the TAP (IDCODE `0x1CA57EEF`, 5-bit IR), creates one target per hart, and forces every memory access through the program buffer, which is required because the module implements no system bus access. The file's transport lines drive a simulation bridge; on silicon replace them with the adapter's own driver and `adapter speed 3000` or slower, for the reason in Section 11.6.
2. Attach the debugger to the port OpenOCD reports, and raise its timeout: a memory read through the program buffer is far slower than the 2-second default assumes.

   ```
   riscv-none-elf-gdb bazel-bin/software/myapp/myapp.elf
   (gdb) set remotetimeout 120
   (gdb) target extended-remote localhost:3333
   ```

3. Halt, step, read state, resume. Registers follow the selected thread; memory follows the connection's target, which is hart 0, so per-tile memory work uses the independent-target configuration `vesta_castalia_nosmp.cfg` and one debugger per hart.

   ```
   (gdb) monitor halt
   (gdb) info registers pc mstatus mcause
   (gdb) x/8xw 0x10640
   (gdb) stepi
   (gdb) continue
   ```

4. Set breakpoints with `break`, never `hbreak`: there are no hardware triggers, so a breakpoint is an `EBREAK` written over the instruction and cannot be placed in the boot ROM or any other read-only region.

A program image cannot be loaded over the debug port. `abstractauto` is unimplemented, so a burst write lands only its first word; use the ROM monitor of Section A.5 to load, and the debug port to inspect.

B Register index

This appendix lists every peripheral instance of this build with its base address and every register with its absolute address, offset and size. Each register name links to its description in Part III.

Table B.1: Register index

Register	Address	Offset	Size	Access	Reset
GPIO0 base 0x4000 (shared window)					
P0IN	0x4000	0x00	1	r	0x00
P0OUT	0x4004	0x04	1	rw	0x01
P0OUTS	0x4008	0x08	1	rw1	0x00
P0OUTC	0x400C	0x0C	1	rw1	0x00
P0OUTT	0x4010	0x10	1	rw1	0x00
P0DIR	0x4014	0x14	1	rw	0x41
P0IF	0x4018	0x18	1	rw1	0x00
P0IES	0x401C	0x1C	1	rw	0x00
P0IE	0x4020	0x20	1	rw	0x00
P0SEL	0x4024	0x24	1	rw	0x4E
P0REN	0x4028	0x28	1	rw	0x80
P0AFS	0x402C	0x2C	4	rw	0x00000000
P0TASK	0x4030	0x30	1	rw	0x00
GPIO1 base 0x4100 (shared window)					
P1IN	0x4100	0x00	1	r	0x00
P1OUT	0x4104	0x04	1	rw	0x00
P1OUTS	0x4108	0x08	1	rw1	0x00
P1OUTC	0x410C	0x0C	1	rw1	0x00
P1OUTT	0x4110	0x10	1	rw1	0x00
P1DIR	0x4114	0x14	1	rw	0x10
P1IF	0x4118	0x18	1	rw1	0x00
P1IES	0x411C	0x1C	1	rw	0x00
P1IE	0x4120	0x20	1	rw	0x00
P1SEL	0x4124	0x24	1	rw	0x30
P1REN	0x4128	0x28	1	rw	0x00
P1AFS	0x412C	0x2C	4	rw	0x00000000
P1TASK	0x4130	0x30	1	rw	0x00
SPI0 base 0x4200 (shared window)					
SPI0CR	0x4200	0x00	4	rw	0x00000000
SPI0SR	0x4204	0x04	1	r/rw1	0x00
SPI0TX	0x4208	0x08	4	rw	0x00000000
SPI0RX	0x420C	0x0C	4	r	0x00000000
SPI0FOS	0x4210	0x10	4	rw	0x00000000
SPI1 base 0x4300 (shared window)					
SPI1CR	0x4300	0x00	4	rw	0x00000000
SPI1SR	0x4304	0x04	1	r/rw1	0x00
SPI1TX	0x4308	0x08	4	rw	0x00000000
SPI1RX	0x430C	0x0C	4	r	0x00000000
SPI1FOS	0x4310	0x10	4	rw	0x00000000
UART0 base 0x4400 (shared window)					
UART0CR	0x4400	0x00	1	rw	0x00
UART0SR	0x4404	0x04	1	r/rw1	0x00

Register	Address	Offset	Size	Access	Reset
UART0BR	0x4408	0x08	2	rw	0x0000
UART0RX	0x440C	0x0C	1	r	0x00
UART0TX	0x4410	0x10	1	rw	0x00
UART1 base 0x4500 (shared window)					
UART1CR	0x4500	0x00	1	rw	0x00
UART1SR	0x4504	0x04	1	r/rw1	0x00
UART1BR	0x4508	0x08	2	rw	0x0000
UART1RX	0x450C	0x0C	1	r	0x00
UART1TX	0x4510	0x10	1	rw	0x00
TIMER0 base 0x4600 (shared window)					
TIM0CR	0x4600	0x00	4	rw	0x00000000
TIM0SR	0x4604	0x04	1	r/rw1	0x00
TIM0VAL	0x4608	0x08	4	rw	0x00000000
TIM0CMP0	0x460C	0x0C	4	rw	0x00000000
TIM0CMP1	0x4610	0x10	4	rw	0x00000000
TIM0CMP2	0x4614	0x14	4	rw	0x00000000
TIM0CAPn (n = 0..1)	0x4618 + 4n	0x18 + 4n	4	r	0x00000000
TIMER1 base 0x4700 (shared window)					
TIM1CR	0x4700	0x00	4	rw	0x00000000
TIM1SR	0x4704	0x04	1	r/rw1	0x00
TIM1VAL	0x4708	0x08	4	rw	0x00000000
TIM1CMP0	0x470C	0x0C	4	rw	0x00000000
TIM1CMP1	0x4710	0x10	4	rw	0x00000000
TIM1CMP2	0x4714	0x14	4	rw	0x00000000
TIM1CAPn (n = 0..1)	0x4718 + 4n	0x18 + 4n	4	r	0x00000000
GPIO2 base 0x4800 (shared window)					
P2IN	0x4800	0x00	1	r	0x00
P2OUT	0x4804	0x04	1	rw	0x00
P2OUTS	0x4808	0x08	1	rw1	0x00
P2OUTC	0x480C	0x0C	1	rw1	0x00
P2OUTT	0x4810	0x10	1	rw1	0x00
P2DIR	0x4814	0x14	1	rw	0x00
P2IF	0x4818	0x18	1	rw1	0x00
P2IES	0x481C	0x1C	1	rw	0x00
P2IE	0x4820	0x20	1	rw	0x00
P2SEL	0x4824	0x24	1	rw	0x00
P2REN	0x4828	0x28	1	rw	0x00
P2AFS	0x482C	0x2C	4	rw	0x00000000
P2TASK	0x4830	0x30	1	rw	0x00
SYSTEM base 0x4900 (shared window)					
SYSCLKCR	0x4900	0x00	2	rw	0x0000
CLKDIVCR	0x4904	0x04	1	rw	0x00
BLOCKPWR	0x4908	0x08	1	rw	0x00
CRCDATA	0x490C	0x0C	1	rw	0x00
CRCSTATE	0x4910	0x10	2	rw	0x0000
WDTPASS	0x4930	0x30	4	w	0x00000000
WDTCR	0x4934	0x34	1	rw	0x00
WDTSR	0x4938	0x38	1	rw1	0x00
WDTVAL	0x493C	0x3C	4	r	0x00000000
DCOnBIAS (n = 0..1)	0x4940 + 4n	0x40 + 4n	2	rw	0x0800
NPU base 0x4A00 (shared window)					
NPUCR	0x4A00	0x00	4	rw	0x00000000
NPUIVSAR	0x4A04	0x04	4	rw	0x00000000

Register	Address	Offset	Size	Access	Reset
NPUWVSAR	0x4A08	0x08	4	rw	0x00000000
NPUOVSAR	0x4A0C	0x0C	4	rw	0x00000000
NPUSR	0x4A10	0x10	4	rw1	0x00000000
NPUCFG1	0x4A14	0x14	4	rw	0x00000000
NPUCFG2	0x4A18	0x18	4	rw	0x00000000
PWRCTRL base 0x4B00 (shared window)					
PWRCR	0x4B00	0x00	4	rw/r	0x00000000
PWRSR	0x4B04	0x04	4	r	0x00000000
PWRWAKE	0x4B14	0x14	4	rw	0x00000000
PWRSTS	0x4B18	0x18	4	r	0x00000000
TASKWKM	0x4B1C	0x1C	4	rw	0x00000000
QSPI0 base 0x4C00 (shared window)					
QSPI0CR	0x4C00	0x00	4	rw	0x00000000
QSPI0CMD	0x4C04	0x04	4	rw	0x00000000
QSPI0ADR	0x4C08	0x08	4	rw	0x00000000
QSPI0TX	0x4C0C	0x0C	4	rw	0x00000000
QSPI0RX	0x4C10	0x10	4	r	0x00000000
QSPI0SR	0x4C14	0x14	4	r/rw1	0x00000000
GPIO3 base 0x4D00 (shared window)					
P3IN	0x4D00	0x00	1	r	0x00
P3OUT	0x4D04	0x04	1	rw	0x00
P3OUTS	0x4D08	0x08	1	rw1	0x00
P3OUTC	0x4D0C	0x0C	1	rw1	0x00
P3OUTT	0x4D10	0x10	1	rw1	0x00
P3DIR	0x4D14	0x14	1	rw	0x00
P3IF	0x4D18	0x18	1	rw1	0x00
P3IES	0x4D1C	0x1C	1	rw	0x00
P3IE	0x4D20	0x20	1	rw	0x00
P3SEL	0x4D24	0x24	1	rw	0x00
P3REN	0x4D28	0x28	1	rw	0x80
P3AFS	0x4D2C	0x2C	4	rw	0x00000000
P3TASK	0x4D30	0x30	1	rw	0x00
I2C0 base 0x4E00 (shared window)					
I2C0CR	0x4E00	0x00	4	rw	0x00000000
I2C0FCR	0x4E04	0x04	1	w1	0x00
I2C0SR	0x4E08	0x08	2	r/rw1	0x0000
I2C0MTX	0x4E0C	0x0C	1	rw	0x00
I2C0MRX	0x4E10	0x10	1	r	0x00
I2C0STX	0x4E14	0x14	1	rw	0x00
I2C0SRX	0x4E18	0x18	1	r	0x00
I2C0AR	0x4E1C	0x1C	1	rw	0x79
I2C0AMR	0x4E20	0x20	1	rw	0x00
I2C1 base 0x4F00 (shared window)					
I2C1CR	0x4F00	0x00	4	rw	0x00000000
I2C1FCR	0x4F04	0x04	1	w1	0x00
I2C1SR	0x4F08	0x08	2	r/rw1	0x0000
I2C1MTX	0x4F0C	0x0C	1	rw	0x00
I2C1MRX	0x4F10	0x10	1	r	0x00
I2C1STX	0x4F14	0x14	1	rw	0x00
I2C1SRX	0x4F18	0x18	1	r	0x00
I2C1AR	0x4F1C	0x1C	1	rw	0x23
I2C1AMR	0x4F20	0x20	1	rw	0x00

Register	Address	Offset	Size	Access	Reset
CLINT base 0x5000 (shared window)					
MSIPn (n = 0..4)	0x5000 + 4n	0x00 + 4n	4	rw	0x00000000
MTIMEL	0x5020	0x20	4	rw	0x00000000
MTIMEH	0x5024	0x24	4	rw	0x00000000
MTIMECMPnL (n = 0..4)	0x5030 + 8n	0x30 + 8n	4	rw	0xFFFFFFFF
MTIMECMPnH (n = 0..4)	0x5034 + 8n	0x34 + 8n	4	rw	0xFFFFFFFF
MUTEX base 0x6000 (shared window)					
MUTEXn (n = 0..15)	0x6000 + 4n	0x00 + 4n	4	rw	0x00000000
I3C0 base 0x6100 (shared window)					
I3C0CR	0x6100	0x00	4	rw	0x00000004
I3C0CMD	0x6104	0x04	4	rw	0x00000000
I3C0TX	0x6108	0x08	4	rw	0x00000000
I3C0RX	0x610C	0x0C	4	r	0x00000000
I3C0SR	0x6110	0x10	4	r/rw1	0x00000000
I3C0DAT	0x6114	0x14	4	rw	0x00000000
I3C0DATPID	0x6118	0x18	4	rw	0x00000000
I3C0DATINFO	0x611C	0x1C	4	rw	0x00000000
I3C0IBI	0x6120	0x20	4	r	0x00000000
NFC0 base 0x6200 (shared window)					
NFC0CR	0x6200	0x00	4	rw/w1	0x00001000
NFC0SR	0x6204	0x04	4	r/rw1	0x00000000
NFC0UID	0x6208	0x08	4	rw	0x00000000
NFC0CFG	0x620C	0x0C	4	rw	0x00000044
NFC0TIM	0x6210	0x10	4	rw	0x088004D4
NFC0RXST	0x6214	0x14	4	r	0x00000000
NFC0IDX	0x6218	0x18	4	rw	0x00000000
NFC0DATA	0x621C	0x1C	4	rw	0x00000000
NFC0TXCTL	0x6220	0x20	4	rw	0x00000000
NFC0DBG	0x6224	0x24	4	r	0x00000000
GPIO4 base 0x6300 (shared window)					
P4IN	0x6300	0x00	1	r	0x00
P4OUT	0x6304	0x04	1	rw	0x00
P4OUTS	0x6308	0x08	1	rw1	0x00
P4OUTC	0x630C	0x0C	1	rw1	0x00
P4OUTT	0x6310	0x10	1	rw1	0x00
P4DIR	0x6314	0x14	1	rw	0x00
P4IF	0x6318	0x18	1	rw1	0x00
P4IES	0x631C	0x1C	1	rw	0x00
P4IE	0x6320	0x20	1	rw	0x00
P4SEL	0x6324	0x24	1	rw	0x00
P4REN	0x6328	0x28	1	rw	0xC0
P4AFS	0x632C	0x2C	4	rw	0x00000000
P4TASK	0x6330	0x30	1	rw	0x00
GPIO5 base 0x6400 (shared window)					
P5IN	0x6400	0x00	1	r	0x00
P5OUT	0x6404	0x04	1	rw	0x00
P5OUTS	0x6408	0x08	1	rw1	0x00
P5OUTC	0x640C	0x0C	1	rw1	0x00
P5OUTT	0x6410	0x10	1	rw1	0x00
P5DIR	0x6414	0x14	1	rw	0x00
P5IF	0x6418	0x18	1	rw1	0x00
P5IES	0x641C	0x1C	1	rw	0x00
P5IE	0x6420	0x20	1	rw	0x00

Register	Address	Offset	Size	Access	Reset
P5SEL	0x6424	0x24	1	rw	0x00
P5REN	0x6428	0x28	1	rw	0xC0
P5AFS	0x642C	0x2C	4	rw	0x00000001
P5TASK	0x6430	0x30	1	rw	0x00
RTC0 base 0x6500 (shared window)					
RTC0CR	0x6500	0x00	4	rw	0x00000000
RTC0SEC	0x6504	0x04	4	rw	0x00000000
RTC0SUB	0x6508	0x08	4	rw	0x00000000
RTC0ALM	0x650C	0x0C	4	rw	0x00000000
RTC0PER	0x6510	0x10	4	rw	0x00000000
RTC0SR	0x6514	0x14	4	r/rw1	0x00000000
RTC0TRIM	0x6518	0x18	4	r0	0x00000000
PWM0 base 0x6600 (shared window)					
PWM0CR	0x6600	0x00	4	rw/w1	0x00000000
PWM0PER	0x6604	0x04	4	rw	0x00000000
PWM0DTY0	0x6608	0x08	4	rw	0x00000000
PWM0DTY1	0x660C	0x0C	4	rw	0x00000000
PWM0DTY2	0x6610	0x10	4	r0	0x00000000
PWM0DTY3	0x6614	0x14	4	r0	0x00000000
PWM0POL	0x6618	0x18	4	rw	0x00000000
PWM0DT	0x661C	0x1C	4	r0	0x00000000
PWM0SR	0x6620	0x20	4	r/rw1	0x00000000
OW0 base 0x6700 (shared window)					
OW0CR	0x6700	0x00	4	rw	0x00000000
OW0CMD	0x6704	0x04	4	rw	0x00000000
OW0TX	0x6708	0x08	4	rw	0x00000000
OW0RX	0x670C	0x0C	4	r	0x00000000
OW0DIV	0x6710	0x10	4	rw	0x00000000
OW0SR	0x6714	0x14	4	r/rw1	0x00000000
OW0SPU	0x6718	0x18	4	r0	0x00000000
DMA0 base 0x6800 (shared window)					
DMA0CR	0x6800	0x00	4	rw/w	0x00000000
DMA0SR	0x6804	0x04	4	r/rw1	0x00000000
DMA0C0SRC	0x6808	0x08	4	rw	0x00000000
DMA0C0DST	0x680C	0x0C	4	rw	0x00000000
DMA0C0LEN	0x6810	0x10	4	rw	0x00000000
DMA0C0CFG	0x6814	0x14	4	rw	0x00000000
DMA0C1SRC	0x6818	0x18	4	rw	0x00000000
DMA0C1DST	0x681C	0x1C	4	rw	0x00000000
DMA0C1LEN	0x6820	0x20	4	rw	0x00000000
DMA0C1CFG	0x6824	0x24	4	rw	0x00000000
DMA0C2SRC	0x6828	0x28	4	rw	0x00000000
DMA0C2DST	0x682C	0x2C	4	rw	0x00000000
DMA0C2LEN	0x6830	0x30	4	rw	0x00000000
DMA0C2CFG	0x6834	0x34	4	rw	0x00000000
DMA0C3SRC	0x6838	0x38	4	rw	0x00000000
DMA0C3DST	0x683C	0x3C	4	rw	0x00000000
DMA0C3LEN	0x6840	0x40	4	rw	0x00000000
DMA0C3CFG	0x6844	0x44	4	rw	0x00000000
DMA0CRC	0x6848	0x48	4	rw	0x0000FFFF
DMA0DESC	0x684C	0x4C	4	r0	0x00000000
TRNG0 base 0x6900 (shared window)					
TRNG0CR	0x6900	0x00	4	rw	0x00000000

Register	Address	Offset	Size	Access	Reset
TRNG0SR	0x6904	0x04	4	r/rw1	0x00000000
TRNG0DR	0x6908	0x08	4	r	0x00000000
TRNG0HT	0x690C	0x0C	4	rw/r	0x00000000
I2CT0 base 0x6A00 (shared window)					
I2CT0CR	0x6A00	0x00	4	rw	0x00000000
I2CT0SR	0x6A04	0x04	4	r/rw1	0x00000000
I2CT0TX	0x6A08	0x08	4	rw	0x00000000
I2CT0RX	0x6A0C	0x0C	4	r	0x00000000
I2CT0WDG	0x6A10	0x10	4	rw	0x00000000
EVFAB base 0x6B00 (shared window)					
EVFCR	0x6B00	0x00	4	rw	0x00000000
EVFSR	0x6B04	0x04	4	r	0x00000000
EVFIE	0x6B08	0x08	4	r0	0x00000000
EVFCAP	0x6B0C	0x0C	4	r	0x010A1008
EVFCHEN	0x6B10	0x10	4	rw	0x00000000
EVFCHENSET	0x6B14	0x14	4	rw1	0x00000000
EVFCHENCLR	0x6B18	0x18	4	rw1	0x00000000
EVFCHTRIG	0x6B1C	0x1C	4	w	0x00000000
EVFFIRED	0x6B20	0x20	4	rw1	0x00000000
EVFOVR	0x6B24	0x24	4	rw1	0x00000000
EVFEVSTAT	0x6B28	0x28	4	rw1	0x00000000
EVFEVTRIG	0x6B2C	0x2C	4	w	0x00000000
EVFGPIOMASK	0x6B3C	0x3C	4	rw	0x00000000
EVFCH0CFG	0x6B40	0x40	4	rw/r	0x00000000
EVFCH1CFG	0x6B44	0x44	4	rw/r	0x00000000
EVFCH2CFG	0x6B48	0x48	4	rw/r	0x00000000
EVFCH3CFG	0x6B4C	0x4C	4	rw/r	0x00000000
EVFCH4CFG	0x6B50	0x50	4	rw/r	0x00000000
EVFCH5CFG	0x6B54	0x54	4	rw/r	0x00000000
EVFCH6CFG	0x6B58	0x58	4	rw/r	0x00000000
EVFCH7CFG	0x6B5C	0x5C	4	rw/r	0x00000000
EVFCH8CFG	0x6B60	0x60	4	r0	0x00000000
EVFCH9CFG	0x6B64	0x64	4	r0	0x00000000
EVFCH10CFG	0x6B68	0x68	4	r0	0x00000000
EVFCH11CFG	0x6B6C	0x6C	4	r0	0x00000000
EVFCH12CFG	0x6B70	0x70	4	r0	0x00000000
EVFCH13CFG	0x6B74	0x74	4	r0	0x00000000
EVFCH14CFG	0x6B78	0x78	4	r0	0x00000000
EVFCH15CFG	0x6B7C	0x7C	4	r0	0x00000000
AFE0 base 0x6C00 (shared window)					
AFE0CR	0x6C00	0x00	4	rw/w1	0x00000700
AFE0SR	0x6C04	0x04	4	r/rw1	0x00000000
AFE0DATA	0x6C08	0x08	4	r	0x00000000
AFE0TIA	0x6C0C	0x0C	4	rw	0x00000000
AFE0DACVP	0x6C10	0x10	4	rw	0x00000000
AFE0DACVCM	0x6C14	0x14	4	rw	0x00000000
AFE0BIAS	0x6C18	0x18	4	rw	0x00000000
AFE0MUX	0x6C1C	0x1C	4	rw	0x000000F0
AFE0SWAP	0x6C20	0x20	4	rw	0x00000000
AFE0BIASG0	0x6C24	0x24	4	rw	0x00000000
AFE0BIASG1	0x6C28	0x28	4	rw	0x00000000
AFE0BIASG2	0x6C2C	0x2C	4	rw	0x00000000
AFE0BIASG3	0x6C30	0x30	4	rw	0x00000000

Register	Address	Offset	Size	Access	Reset
AFE0BIASGCR	0x6C34	0x34	4	rw	0x00000000
AFE1 base 0x6D00 (shared window)					
AFE1CR	0x6D00	0x00	4	rw/w1	0x00000700
AFE1SR	0x6D04	0x04	4	r/rw1	0x00000000
AFE1DATA	0x6D08	0x08	4	r	0x00000000
AFE1TIA	0x6D0C	0x0C	4	rw	0x00000000
AFE1DACVP	0x6D10	0x10	4	rw	0x00000000
AFE1DACVCM	0x6D14	0x14	4	rw	0x00000000
AFE1BIAS	0x6D18	0x18	4	rw	0x00000000
AFE1MUX	0x6D1C	0x1C	4	rw	0x000000F0
AFE1SWAP	0x6D20	0x20	4	rw	0x00000000
AFE1BIASG0	0x6D24	0x24	4	rw	0x00000000
AFE1BIASG1	0x6D28	0x28	4	rw	0x00000000
AFE1BIASG2	0x6D2C	0x2C	4	rw	0x00000000
AFE1BIASG3	0x6D30	0x30	4	rw	0x00000000
AFE1BIASGCR	0x6D34	0x34	4	rw	0x00000000
AFE2 base 0x6E00 (shared window)					
AFE2CR	0x6E00	0x00	4	rw/w1	0x00000700
AFE2SR	0x6E04	0x04	4	r/rw1	0x00000000
AFE2DATA	0x6E08	0x08	4	r	0x00000000
AFE2TIA	0x6E0C	0x0C	4	rw	0x00000000
AFE2DACVP	0x6E10	0x10	4	rw	0x00000000
AFE2DACVCM	0x6E14	0x14	4	rw	0x00000000
AFE2BIAS	0x6E18	0x18	4	rw	0x00000000
AFE2MUX	0x6E1C	0x1C	4	rw	0x000000F0
AFE2SWAP	0x6E20	0x20	4	rw	0x00000000
AFE2BIASG0	0x6E24	0x24	4	rw	0x00000000
AFE2BIASG1	0x6E28	0x28	4	rw	0x00000000
AFE2BIASG2	0x6E2C	0x2C	4	rw	0x00000000
AFE2BIASG3	0x6E30	0x30	4	rw	0x00000000
AFE2BIASGCR	0x6E34	0x34	4	rw	0x00000000
AFE3 base 0x6F00 (shared window)					
AFE3CR	0x6F00	0x00	4	rw/w1	0x00000700
AFE3SR	0x6F04	0x04	4	r/rw1	0x00000000
AFE3DATA	0x6F08	0x08	4	r	0x00000000
AFE3TIA	0x6F0C	0x0C	4	rw	0x00000000
AFE3DACVP	0x6F10	0x10	4	rw	0x00000000
AFE3DACVCM	0x6F14	0x14	4	rw	0x00000000
AFE3BIAS	0x6F18	0x18	4	rw	0x00000000
AFE3MUX	0x6F1C	0x1C	4	rw	0x000000F0
AFE3SWAP	0x6F20	0x20	4	rw	0x00000000
AFE3BIASG0	0x6F24	0x24	4	rw	0x00000000
AFE3BIASG1	0x6F28	0x28	4	rw	0x00000000
AFE3BIASG2	0x6F2C	0x2C	4	rw	0x00000000
AFE3BIASG3	0x6F30	0x30	4	rw	0x00000000
AFE3BIASGCR	0x6F34	0x34	4	rw	0x00000000
IRQROUTER base 0x7000 (shared window)					
HnENL (n = 0..4)	0x7000 + 16n	0x00 + 16n	4	rw	0x00000000
HnENM (n = 0..4)	0x7004 + 16n	0x04 + 16n	4	rw	0x00000000
HnENU (n = 0..4)	0x7008 + 16n	0x08 + 16n	4	rw	0x00000000
HnENX (n = 0..4)	0x700C + 16n	0x0C + 16n	4	rw	0x00000000
CLAIM	0x7800	0x800	4	rw	0x00000000
PENDL	0x7810	0x810	4	r	0x00000000

Register	Address	Offset	Size	Access	Reset
PENDM	0x7814	0x814	4	r	0x00000000
PENDU	0x7818	0x818	4	r	0x00000000
PENDX	0x781C	0x81C	4	r	0x00000000
INSVCL	0x7820	0x820	4	r	0x00000000
INSVCM	0x7824	0x824	4	r	0x00000000
INSVCU	0x7828	0x828	4	r	0x00000000
INSVCX	0x782C	0x82C	4	r	0x00000000
PINMUX base 0x7C00 (shared window)					
PMXCAP	0x7C00	0x00	4	r	0x00000000
PMXIN0	0x7C04	0x04	4	r	0x00000000
PMXCFG0	0x7C08	0x08	4	rw	0x00000000
PMXCFG1	0x7C0C	0x0C	4	rw	0x00000000
PMXCFG2	0x7C10	0x10	4	rw	0x00000000

C Errata

This appendix lists known deviations of the hardware from this manual. Each entry gives the affected silicon revision, a reference number, a description of the fault, the circumstances in which it occurs, its effect, and the recommended software workaround. There are no known errata for the PentaWoundPt2mm MCU at the time of this revision.

Table C.1: Errata.

Ref.	Affects	Description	Conditions	Effect	Workaround
none	all	None known.			

D Revision history

Table D.1: Revision history of this manual.

Revision	Date	Summary
1	2026-10-13	Restructured into parts and chapters with a front matter chapter (conventions, access types, register anatomy, glossary). Register descriptions reformatted with summary tables, per-register reset values and field tables. Per-peripheral instance, signal and interrupt tables generated from the chip configuration. Reset, clocks and power consolidated into one chapter; the boot image format corrected from the ROM source; the hand-written instruction listing replaced by the ISA and CSR summary; software material moved to Appendix A.